

Continuous Learning: the $\Omega = 1$ normalised penalty step, from its mathematics to its tests

Technical specification and record, CRR re-validation repository, 2026-09-22. Owner request: prompt-log entry 72. Every number in this document is printed by a committed script whose output is pinned in the repository (rule R1): the study scorers (runs/eq2/score.txt, runs/eq3/score.txt), the mathematics check (theory/checks/omega_sweeps.txt), the gate (prereg/eq3/gate_EQ2.txt), the ledger (ledger/LEDGER.md) and the figure script of this folder (figures/figures.txt). The full code sets are reproduced in Appendix A and the pinned outputs in Appendix B. Nothing here is a finding under the repository's PASS levels; the status of every row is stated where it is quoted. Papers are cited where they were fetched on the day (PubMed) and otherwise named as context, not fetched (rule R10).

In plain words. This document is about one small rule for teaching a computer new things without making it forget old things. The rule has a dial on it called Omega. We did the mathematics of the dial, then we tested the rule on nine sets of data that nobody had used before, following strict rules so we could not fool ourselves. Boxes like this one say, in plain words, what each part means.

0. Summary of what was found

question	answer	where
What does Ω mean?	The ratio between the length of the "past" pull and the length of the "present" pull after the rule has rescaled the past pull. $\Omega = 1$ means equal lengths.	§3
Is $\Omega = 1$ a Bayes optimum?	No. The Bayes weight for combining two batches is a constant fixed by their sample counts. With exact gradients, $\Omega = 1$ makes every point between the two task optima a fixed point, so the rule stops wherever it starts.	§4.3, §4.4, Fig. 12
Is there a threshold either side of 1?	With exact gradients there is one knife edge at $\Omega = 1$: below it the old task is eventually dropped, above it the new task is never learned. Smoothing and mini-batch noise turn the edge into a plateau.	§4.2, §4.6, Figs. 3–4
Why does the rule help on an EWC penalty?	Its step is bounded by the present step, so it can run the penalty at or beyond the weight where a fixed λ diverges.	§4.5, Fig. 5, Fig. 8
What did the tests find?	Replay (EQX): reduces to a constant. Online EWC (EQ2, three carriers): ahead of the tuned λ on 3/3, fragile. EQ2R: void. Online EWC (EQ3, six carriers): ahead on 5/6, behind on 1/6; both pre-registered controls violated; Ω a plateau over the whole grid.	§7
Status under the repository's levels	EQ2-1b PASS-0 (provisional) with a failed replication recorded; nothing may be quoted as a finding.	§8

1. Introduction

A learning system trained on a sequence of tasks tends to lose what it learned first as it learns what comes later. The loss is called catastrophic forgetting, and the family of methods that fight it by adding a penalty on movement away from the parameters that served the earlier tasks is called regularisation-based continual learning. Elastic weight consolidation is the reference method of that family: according to PubMed, Kirkpatrick and colleagues (Proceedings of the National Academy of Sciences 2017, [doi 10.1073/pnas.1611835114](https://doi.org/10.1073/pnas.1611835114)) "remember old tasks by selectively slowing down learning on the weights important for those tasks", the importance being an estimate of the Fisher information. Every method in the family carries a weight λ on the penalty, and λ must be tuned, per dataset, by sweeping it.

The CRR framework (theory/CRR.md, hypothesis H-EQ) proposed an adaptive weight in place of the sweep. Its statement is that a learner updating from a present batch and a past term should "weight the past gradient so that settled past and present exert equal pull in the Fisher norm", with a dial Ω on the ratio and the theory value $\Omega = 1$, which the framework calls equanimity. This document sets out the mathematics of that rule, the meaning of Ω , the behaviour of $\Omega = 1$, the tests that were run under a pre-registration protocol, and what passed and what failed and why.

The document is organised as follows. §2 fixes the setting and the methods being compared, with their equations. §3 defines Ω exactly as the frozen scoring scripts compute it. §4 is the mathematics: the one-dimensional case, the fixed-point set in many dimensions, the Bayes comparison, the stability edge, the effect of smoothing and noise, and the Kalman filter as a second reading of $\Omega = 1$. §5 sets the rule beside existing methods. §6 describes the protocol and the instrument. §7 reports

every test, positive and negative, with the precise reason each failure is a failure. §8 states the standing of the results. §9 gives the implications if the results hold at scale. Appendix A holds the full code; Appendix B holds the pinned outputs.

In plain words. Imagine a student who has learned last year's lessons and is now learning this year's. Two forces pull on the student: this year's homework pulls toward new answers, and a "remember last year" rule pulls back toward the old ones. Most methods set the strength of the pull-back by trial and error. The rule in this document sets it automatically: make the pull-back exactly as strong as the pull-forward. Omega is the knob that says "exactly as strong" (Omega = 1), "half as strong" (Omega = 0.5) or "twice as strong" (Omega = 2).

2. Setting, notation and the methods compared

2.1 The learning problem

A model with parameters $\theta \in \mathbb{R}^n$ is trained on tasks 1, 2, ..., T in order, each task being a set of classes of one classification problem (class-incremental learning, one shared output head). During task t the model sees mini-batches of that task only. At each step it has a present loss, the cross-entropy on the current mini-batch, and a past term that stands in for the tasks already seen. The scored quantity is the final accuracy over all classes after the last task, in percent, one number per seed.

Write g_p for the gradient of the present loss and g_q for the gradient of the past term, both with respect to θ , both computed at the current θ . Every method in this document performs the update

$$\theta \leftarrow \theta - \eta (g_p + w g_q)$$

with learning rate η and a scalar weight w on the past term. The methods differ in what the past term is and in how w is set. The methods' own weights (λ , α , c) are removed from g_q and carried in w , so that " $w = \lambda$ " is a fixed EWC weight and " w from the rule" is the rule applied to the same past gradient.

2.2 The past terms

Elastic weight consolidation, online form (Kirkpatrick et al. 2017, cited above; the online accumulation follows the standard practice named as context): at each task boundary the diagonal empirical Fisher information F_t of the task just finished is estimated from squared gradients and accumulated, $F \leftarrow F + F_t$, and the anchor θ^* is set to the current parameters. The past term is the quadratic penalty

$$L_q(\theta) = \sum_i F_i (\theta_i - \theta_i^*)^2, \quad g_q = 2 F \odot (\theta - \theta^*)$$

Synaptic intelligence (according to PubMed: Zenke, Poole and Ganguli, Proceedings of Machine Learning Research 70, 2017, PMC6944509) uses the same quadratic form with an importance built from the path integral of the gradient times the parameter displacement during each task, normalised by the squared displacement plus a damping constant ξ . Memory-aware synapses (Aljundi et al. 2018, named, not fetched) uses the same form with importance equal to the absolute gradient of the squared output norm. Learning without forgetting (according to PubMed: Li and Hoiem, IEEE Transactions on Pattern Analysis and Machine Intelligence 40, 2018, [doi 10.1109/TPAMI.2017.2773081](https://doi.org/10.1109/TPAMI.2017.2773081)) uses a distillation term: the past term is the temperature-scaled Kullback-Leibler divergence between the outputs of the model frozen at the last task boundary and the current outputs on the current batch, so it is a constraint on the model's behaviour and not a loss on old data. Experience replay keeps a reservoir buffer of past samples and uses the cross-entropy on a replayed batch as the past term; with the two batch means summed (ER-sum) the natural weight is $w = 1$. DER++ (Buzzega et al. 2020, named, not fetched) replays stored logits and matches them by mean squared error, with a second replay cross-entropy term at $\beta = 0.5$ kept inside the present term at its published default.

method	past term g_q is the gradient of	what its weight means
EWC (online)	$\sum F_i (\theta_i - \theta_i^*)^2$, F accumulated at boundaries	λ , the Laplace-posterior precision scale
SI	same form, path-integral importance	c
MAS	same form, output-norm importance	λ
LwF	KL(frozen model current model) on the current batch, $T = 2$	a distillation weight
ER-sum	cross-entropy on a replayed batch	1 by construction
DER++	mean squared error to stored logits	α

2.3 Baselines that set w by a rule

Three published rules set w without a sweep and were run against the rule of this document. A-GEM (Chaudhry et al. 2019, named) projects g_p onto the half-space where it does not increase the past loss: $w = -(g_p \cdot g_q) / (g_q \cdot g_q)$ when $g_p \cdot g_q < 0$, and 0 otherwise. GradNorm (Chen et al. 2018, named) learns task weights online by gradient descent so that the weighted gradient norms move toward their mean; it was run at $\alpha = 0$ with weights renormalised to sum to 2. MEGA-I (Guo et al. 2020, named) sets w to the ratio of the smoothed past loss to the smoothed present loss. The tuned fixed weight, found on a two-stage grid and tuned on the very seeds that are scored, is the strongest baseline and favours itself by construction.

3. Ω , precisely

3.1 The rule as the frozen scorers compute it

At each step where a past term exists, the scorers (`runs/eq3/frozen/eq3_score.py`, mode `eq`) form exponential moving averages of the two gradient vectors,

$$\hat{g}_p \leftarrow s \hat{g}_p + (1 - s) g_p, \quad \hat{g}_q \leftarrow s \hat{g}_q + (1 - s) g_q$$

with smoothing constant $s = 0.9$ registered before any data was opened, and set the weight

$$w = \min \left(\Omega \frac{\|\hat{g}_p\|}{\max(\|\hat{g}_q\|, 10^{-12})}, w_{\max} \right)$$

with the cap $w_{\max} = 10^4$ and the floor 10^{-12} also registered. The norm is Euclidean. The update is then $\theta \leftarrow \theta - \eta(g_p + w g_q)$ with the raw, unsmoothed gradients. Ω is a dimensionless dial on the ratio of the two smoothed gradient lengths. The three constants s , $w_{\max}$ and the floor are parameters of the estimator, and the sensitivity table of every study sweeps s over $\{0.8, 0.9, 0.98\}$ and $w_{\max}$ over $\{10, 100, 10^4\}$.

3.2 What Ω means

Whatever the raw lengths of the two gradients, after the rule the past pull has length $w\|\hat{g}_q\| = \Omega\|\hat{g}_p\|$. So Ω is the length of the past pull measured in units of the present pull:

- $\Omega = 1$: the past and the present pull with equal force. This is CRR's "equanimity".
- $\Omega < 1$: the past pulls less than the present; the learner favours the new task.
- $\Omega > 1$: the past pulls more than the present; the learner favours the old tasks.

Ω is not the penalty weight λ . The rule turns Ω into a λ that changes every step, and the derived w can be very large or very small depending on the units of g_q . On the EWC penalty of study EQ3 the median derived w at $\Omega = 1$ was 4.76 on one carrier and 1065.84 on another (`runs/eq3/score.txt`), because the Fisher information carries the scale of the inputs and the number of tasks. Ω is scale-free; w is not.

In plain words. Think of two children pulling a rope in opposite directions. One child is "this year's lessons"; the other is "last year's lessons". The rule measures how hard the first child pulls, then tells the second child to pull exactly Ω times that hard, no matter how strong the second child really is. $\Omega = 1$ means "pull exactly as hard as the other one". If you wanted the old lessons to matter more, you would turn Ω up.

3.3 The two readings of $\Omega = 1$ in the framework

CRR gives $\Omega = 1$ two readings that the retrodiction batteries found to differ (`theory/retrodictions/synthesis_batches/batch_06.txt`, row 3). Reading (i), "Fisher speed 1", sets a unit speed on the statistical manifold and, on the scalar Kalman filter of the framework's proposition P4, gives the steady-state gain $K = 1/\phi = 0.618034$ (ϕ the golden ratio). Reading (ii), "equal pull", gives the prior and the datum equal precision, which is $K = 1/2$. The two are the same at no signal-to-noise speed: $K(v) = 1/2$ at $v = 0.707107$ and $K(1) = 1/\phi$. §4.7 shows what each costs. In the learner, reading (ii) is the rule of §3.1.

4. The mathematics

4.1 One dimension: the knife edge

Let the parameter be a scalar θ and both terms be quadratic with minima at m_p (present) and m_q (past), unit curvature, so $g_p = \theta - m_p$ and $g_q = \theta - m_q$. For θ strictly between the minima, g_p and g_q have opposite signs, and with instantaneous norms (no smoothing) the rule gives $w\|g_q\| = \Omega\|g_p\|$, so the update is

$$g_p + w g_q = g_p - \Omega |g_p| \text{sign}(g_p) = (1 - \Omega) g_p$$

Three cases follow at once. At $\Omega = 1$ the update is zero at every θ between the minima: every point is a fixed point, and the learner stays where it started. At $\Omega < 1$ the update is a positive fraction of g_p , so gradient descent converges to m_p , the present minimum, and the past is dropped. At $\Omega > 1$ the update points away from m_p and the learner moves to m_q , the past minimum, where $g_q = 0$ and the ratio hits its floor; there it chatters. The pinned check (theory/checks/omega_sweeps.txt, part 1) confirms the three cases numerically with the registered estimator: starting from 0.1, 0.5 and 0.9 at $\Omega = 1$ the endpoints are 0.1000, 0.5000 and 0.9000; at $\Omega = 0.5$ every start ends at the present mean; at $\Omega = 2$ every start ends at 1.0973, within 0.2 of the past mean.

The rule rescales the past pull to Ω times the present pull. In one dimension the net pull is $(1 - \Omega) \times$ present pull.

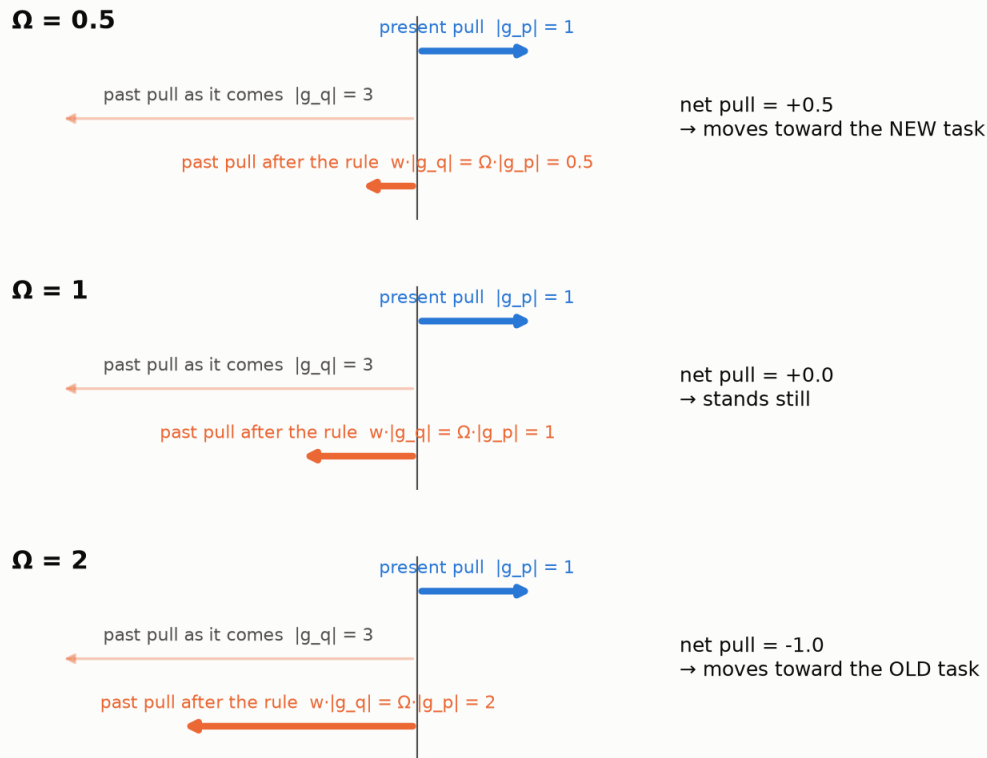


Figure 1. The tug of war in one dimension. The present pull (blue) has length 1; the raw past pull (faint orange) has length 3; the rule rescales the past pull to $\Omega \times 1$ (orange). The net pull is $(1 - \Omega) \times 1$: +0.5 at $\Omega = 0.5$, 0 at $\Omega = 1$, -1 at $\Omega = 2$ (figures/figures.txt, F1).

In plain words. If two children pull a rope exactly equally hard, the rope does not move. That is $\Omega = 1$ in one dimension: the rule makes the two pulls equal, so nothing moves, wherever the rope happens to be. Turn Ω below 1 and the rope slides all the way to the "new lessons" child; turn it above 1 and it slides all the way to the "old lessons" child. There is no in-between setting that stops the rope at a chosen spot, other than exactly 1, and at exactly 1 it stops wherever it already is.

4.2 Many dimensions: the fixed-point set is the Pareto curve

Let the present loss and the past penalty be quadratics in $\theta \in \mathbb{R}^d$,

$$L_p(\theta) = \frac{1}{2}(\theta - a)^T H(\theta - a), \quad L_q(\theta) = \frac{1}{2}(\theta - b)^T F(\theta - b)$$

with H and F symmetric positive definite, a the new-task optimum and b the old-task optimum. A

fixed weight λ has the unique equilibrium

$$\theta(\lambda) = (H + \lambda F)^{-1}(Ha + \lambda Fb)$$

and as λ runs from 0 to ∞ this traces a curve from a to b, the Pareto curve of the two objectives: every point on it is the minimiser of $L_p + \lambda L_q$ for its λ , and at every point on it the two gradients are antiparallel, $g_p = -\lambda g_q$.

The rule's update is $g_p + \Omega(\|g_p\|/\|g_q\|) g_q$. It vanishes only if g_q is a negative multiple of g_p , that is, only on the Pareto curve. On the curve, write $g_q = -g_p/\lambda$; then

$$g_p + \Omega \frac{\|g_p\|}{\|g_q\|} g_q = g_p - \Omega \frac{\|g_p\|}{\|g_p\|/\lambda} \cdot \frac{g_p}{\lambda} = (1 - \Omega) g_p$$

so the update has norm exactly $|1 - \Omega| \cdot \|g_p\|$ everywhere on the curve. At $\Omega = 1$ every point of the curve is stationary: a continuum of equilibria, one for each fixed λ . At $\Omega \neq 1$ no point of the curve is stationary, and the update along the curve points toward a ($\Omega < 1$) or toward b ($\Omega > 1$). The pinned check verifies the identity at thirteen points of the curve in a five-dimensional model with F sixteen times H: the ratio $\|\text{update}\|/\|g_p\|$ is 0.100000 at $\Omega = 0.9$ and 1.1 and 0.000000 at $\Omega = 1$ (`omega_sweeps.txt`, part 2 (i)); the figure script verifies it in two dimensions at $\lambda = 0.1, 1$ and 10: 0.300000 at $\Omega = 0.7$, 0.000000 at $\Omega = 1$, 0.400000 at $\Omega = 1.4$ (`figures.txt`, F2).

Update field of $g_p + \Omega(\|g_p\|/\|g_q\|)g_q$ in a two-dimensional two-task model (blue contours: new-task loss; orange: old-task penalty). On the green curve the two pulls are antiparallel; at $\Omega = 1$ the update vanishes everywhere on it.

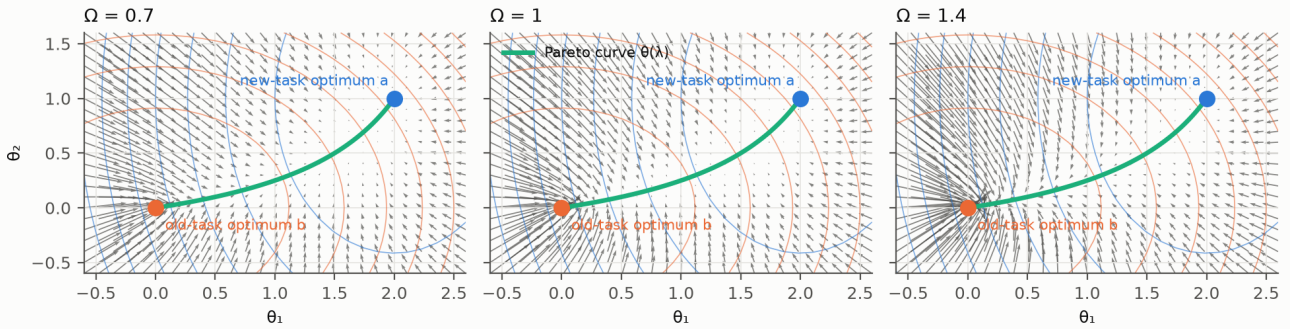


Figure 2. The update field of the rule in a two-dimensional two-task model. Blue contours: the new-task loss; orange contours: the old-task penalty; green: the Pareto curve $\theta(\lambda)$. At $\Omega = 1$ the arrows vanish on the curve; at $\Omega = 0.7$ they run along the curve toward a; at $\Omega = 1.4$ they run along it toward b.

Two consequences follow. First, the rule at $\Omega = 1$ does not choose a trade-off between the tasks: it stops wherever its trajectory first meets the curve, which depends on the starting point, the learning rate and the estimator's lag. Second, Ω is a knife edge and not a dial with an optimum: infinitesimally below 1 the equilibria all disappear toward a, infinitesimally above 1 toward b. In the trajectories of Figure 3 (starting from b, as a continual learner does), the exact rule ends at the new-task optimum (2.0000, 1.0000) for Ω from 0.25 to 0.71 and within 0.05 of the old-task optimum for Ω from 1 to 4; at $\Omega = 1$ it stops on the curve (distance 6.78×10^{-5}) at $\lambda_{\text{eff}} = 2.985$ (`figures.txt`, F3).

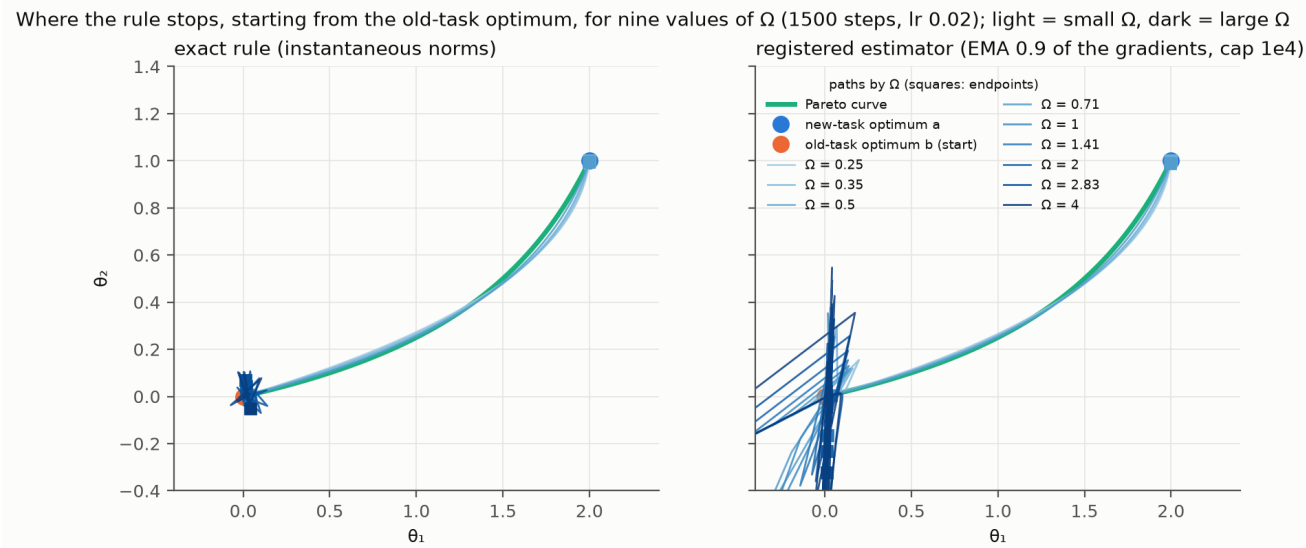


Figure 3. Trajectories of the rule from the old-task optimum for nine values of Ω , with instantaneous norms (left) and with the registered estimator (right). Below $\Omega = 1$ the learner leaves for the new-task optimum; at and above $\Omega = 1$ it stays near the old one, on the curve at exactly 1 and oscillating around b above 1.

In plain words. In one dimension the rope had only one line to sit on. With many knobs to turn, the places where "the two pulls exactly cancel" form a curved line running from the old answer to the new answer. At $\Omega = 1$ the learner stops the moment it touches that line, wherever that is. It does not choose the best spot on the line; it just stops. The best spot is what Bayes chooses, next.

4.3 Bayes: what the optimal weight is

Let the present batch have n_p samples with mean m_p and the past batch n_q samples with mean m_q , both of a Gaussian mean with unit noise and a flat prior. The log posterior is

$$\log p(\theta \mid \text{data}) = -\frac{n_p}{2}(\theta - m_p)^2 - \frac{n_q}{2}(\theta - m_q)^2 + \text{const}$$

so the posterior mode is $\theta_B = (n_p m_p + n_q m_q)/(n_p + n_q)$. With batch-mean gradients $g_p = \theta - m_p$ and $g_q = \theta - m_q$, the combination $\bar{g}_p + w \bar{g}_q$ has fixed point $(m_p + w m_q)/(1 + w)$, which equals θ_B if and only if

$$w_B = \frac{n_q}{n_p}$$

That is, the Bayes-optimal weight counts every sample once: summed log-likelihoods with weight 1 each, which in batch-mean units is the ratio of the sample counts. It is a constant fixed by the data sizes, not a norm ratio. With equal counts it is $w = 1$, exactly the summed-loss replay baseline ER-sum.

For the EWC penalty the same argument gives the Laplace weight. The Laplace approximation to the posterior after the past tasks is a Gaussian with precision $\sum_k N_k F_k$, N_k the number of samples in task k and F_k the per-sample Fisher information. The per-sample-normalised objective for the present task is then

$$\frac{1}{N_p} \sum_{\text{present}} \ell(\theta) + \frac{1}{2N_p} (\theta - \theta^*)^T \left(\sum_k N_k F_k \right) (\theta - \theta^*)$$

The scorer's Fisher estimate is a per-sample quantity and its penalty loss is $w \cdot \sum_i F_i (\theta_i - \theta^*)^2$ with F the accumulated sum, so with equal task sizes the Laplace weight in the scorer's units is $w = 1/2$, and with unequal sizes it is $1/2$ on the task-size-weighted accumulation $\sum_k (N_k/N_p) F_k$. Study EQ3 ran exactly this arm (mode bayes, §7.4).

The rule at $\Omega = 1$ is therefore Bayes-optimal only by coincidence: in one dimension it stops where it starts, and the posterior mode is one point of that segment (Figure 12). The pinned check shows the gap: with $n_q/n_p = 4$ the mode is at 0.8000, and the rule's endpoints from starts 0.1, 0.5 and 0.9 are 0.1000, 0.5000 and 0.9000, at distances 0.7000, 0.3000 and 0.1000 from it (omega_sweeps.txt, part 1).

One-dimensional Bayes: the posterior mode is one point; the rule at $\Omega = 1$ stops anywhere on the segment

green band: every point is a fixed point of the rule at $\Omega = 1$ (it stays where it starts)

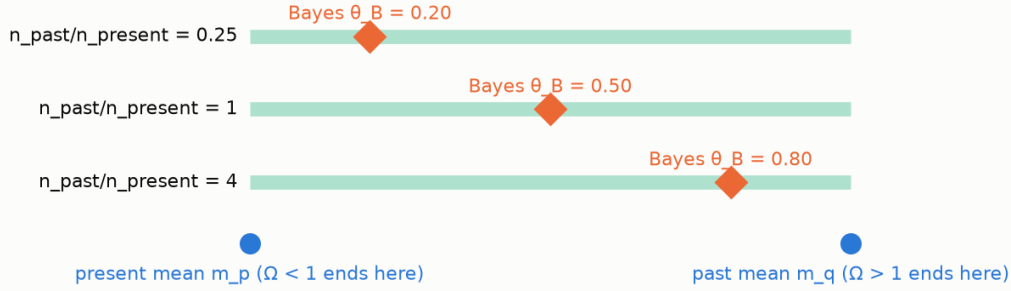


Figure 12. One-dimensional Bayes. For three ratios of past to present sample counts, the posterior mode (orange diamond) is one point; the fixed-point set of the rule at $\Omega = 1$ (green band) is the whole segment between the two means.

In plain words. "Bayes optimum" is the mathematician's name for the single best compromise between what last year's data say and what this year's data say. It is one exact spot, and where it sits depends on how much data each year had: more data pulls the spot its way. The $\Omega = 1$ rule does not know how much data each year had; it only looks at how hard each pull feels right now. So it stops somewhere on the line, and only by luck at the Bayes spot. Equal pull is not the same as the best compromise.

4.4 Why the rule still helps: the step bound and the stability edge

Gradient descent on $L_p + \lambda L_q$ with learning rate η converges only if η times the largest eigenvalue of $H + \lambda F$ is below 2. With diagonal H and F this is

$$\eta (h_{\max} + \lambda f_{\max}) < 2, \quad \lambda^* = \frac{2/\eta - h_{\max}}{f_{\max}}$$

Beyond λ^* a fixed weight diverges; and because the Fisher information carries the scale of the inputs and grows with the number of tasks, on real carriers λ^* can sit below the λ that best protects the old tasks, so that the tuned λ is pinned at the edge of divergence. The rule cannot diverge in that way. Its update satisfies

$$\|g_p + w g_q\| \leq \|g_p\| + w \|g_q\| = (1 + \Omega) \|g_p\|$$

when the norms are instantaneous, and approximately so with smoothing: the past step is never larger than Ω times the present step, whatever the scale of F . In the pinned five-dimensional model with F sixteen times H and $\eta = 0.05$, the edge is $\lambda^* = 1.2575$: fixed $\lambda = 1$ converges (total loss 3.2770) and $\lambda = 1.5$ diverges; the registered rule at $\Omega = 1$ stops at $\lambda_{\text{eff}} = 0.0447$ with total loss 19.8753 (omega_sweeps.txt, part 2 (ii) and (iii b); figures.txt, F5). In that model the rule stops well below the edge; on the real carriers of §7 the derived weight sits at or beyond the edge (Figure 8), which is where the fixed grid collapses and the rule does not.

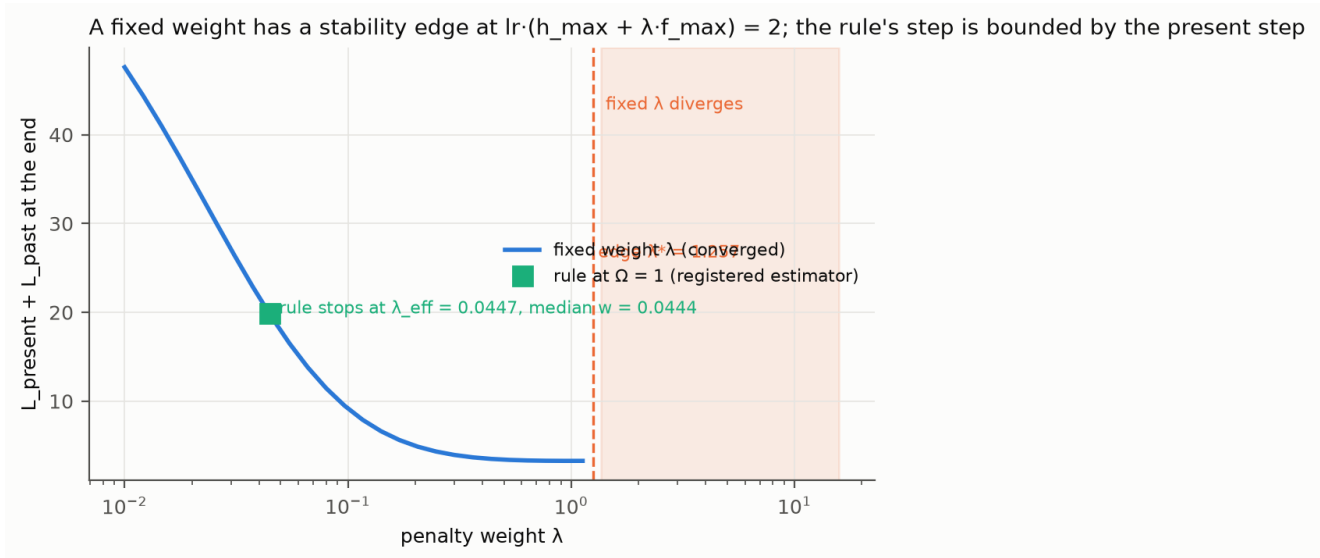


Figure 5. A fixed penalty weight has a stability edge λ^* at $\eta(h_{\max} + \lambda f_{\max}) = 2$ (dashed); beyond it gradient descent diverges (shaded). The rule's step is bounded by $(1 + \Omega)$ times the present step, so it has no such edge; in this model it stops at $\lambda_{\text{eff}} = 0.0447$.

In plain words. Turning the "remember last year" knob too high does not just make the student stubborn; past a certain setting the whole lesson blows up, because each correction overshoots and the next overshoots more. The rule never blows up in that way, because it never lets the remember-pull be bigger than Ω times the learn-pull, whatever units the remember-pull comes in. On real data that is where its advantage came from: it could run the remember-pull past the point where a fixed knob setting explodes.

4.5 Smoothing, noise and the plateau

The registered estimator smooths the gradients with $s = 0.9$ before taking norms. The smoothed ratio lags the instantaneous one, so the learner travels further along the curve before the norms balance: in the five-dimensional model the exact rule at $\Omega = 1$ stops at $\lambda_{\text{eff}} = 0.6237$ and the registered estimator at 0.0447, and every Ω above 1 oscillates under the registered estimator (`omega_sweeps.txt`, part 2 (iii) and (iii b)). Mini-batch noise does the rest. With Gaussian noise of standard deviation 0, 0.5 and 2 on both gradients, the total loss $L_p + L_q$ at the end, averaged over five seeds, is lowest at $\Omega = 1$ in 11 of 15 cells over three smoothings and three caps, and the plateau, the set of grid values within 10 % of the best, has width 2, 1 and 3 at the three noise levels (`omega_sweeps.txt`, part 2 (iv); Figure 4). The cap binds in none of the 15 cells, because the derived weight in this model is of order 1; the cap matters where the tuned λ is in the hundreds, which is the regime of the real carriers.

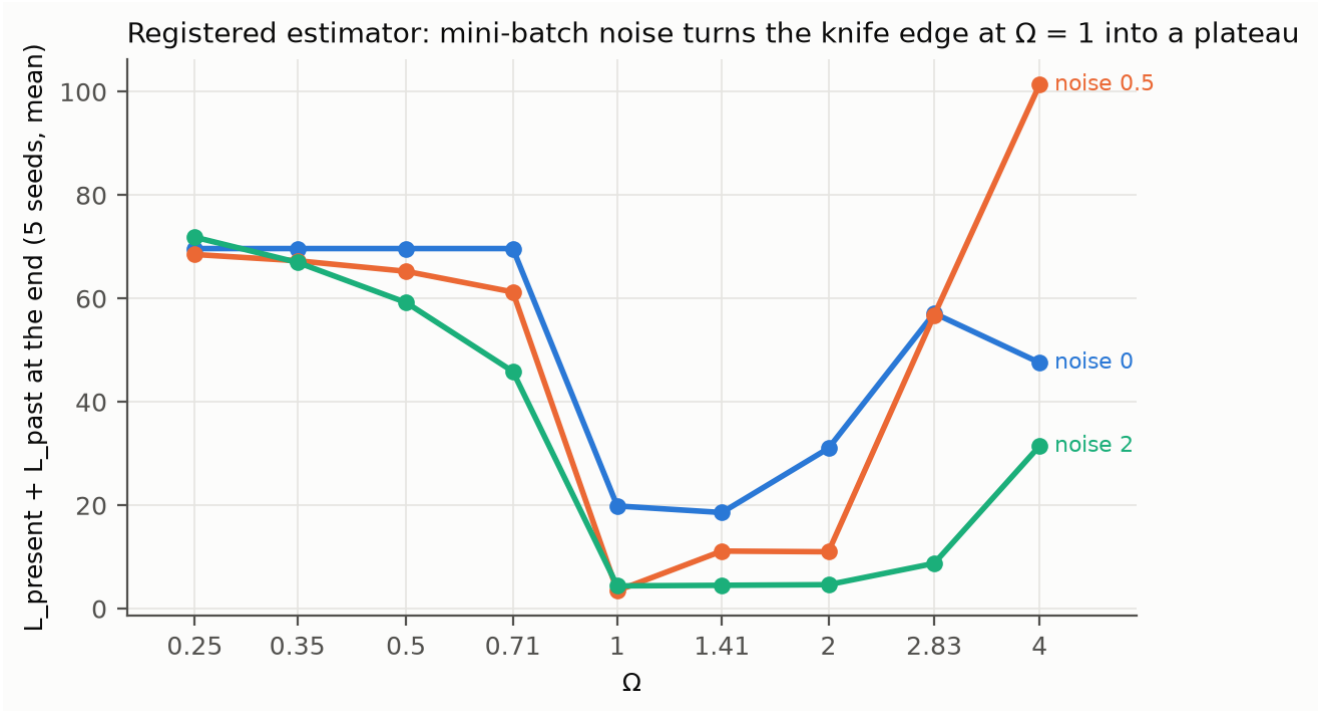


Figure 4. Total loss at the end against Ω under mini-batch noise (registered estimator, five seeds). The knife edge at $\Omega = 1$ becomes a basin whose floor widens with the noise; nothing singles out $\Omega = 1$ within the basin.

In plain words. With a perfectly steady pull, $\Omega = 1$ is a razor's edge: a hair below and the rope slides one way, a hair above and it slides the other. Real pulls jitter from step to step, and the rule averages them, so the edge gets smeared into a flat valley. That is why, on real data, Ω values from about 0.7 to 1.4 all did about the same: the valley has a flat floor, not a point at the bottom.

4.6 The reduction test: when the rule is a constant

If the ratio $\|\hat{g}_p\|/\|\hat{g}_q\|$ is stationary over a run, the rule is a fixed weight equal to its median, and a fixed weight at that median (the reduction arm) must score the same. On replay with summed losses the two gradients have the same units and the ratio is stationary: this is the case of study EQX (§7.1), where the rule reduced to a constant on 3 of 3 carriers. On the EWC penalty the ratio is not stationary, the median derived weight sits beyond the stability edge, and the reduction arm diverges: EQ2 and EQ3 both read "does not reduce", with the fixed arm at the rule's median diverging on 2 to 3 seeds of 5 on most carriers (§7.2, §7.4). The reduction test is what separates a normalised-gradient method from a constant with a new name.

4.7 The Kalman filter: the two fixed gains of $\Omega = 1$

The framework's proposition P4 concerns the scalar random walk $x_{t+1} = x_t + w_t$ with process variance q , observed with noise variance r . The steady-state Kalman gain depends on the data only through $v = \sqrt{q/r}$:

$$K(v) = \frac{v}{2} (\sqrt{v^2 + 4} - v)$$

and $K(v)$ is the Bayes-optimal fixed weight on the innovation (Kalman 1960, named). A fixed-gain filter $\hat{x} \leftarrow \hat{x} + K(y - \hat{x})$ has steady-state error variance

$$P(K) = \frac{(1-K)^2 q + K^2 r}{1 - (1-K)^2}$$

The two readings of $\Omega = 1$ give two fixed gains: $K = 1/\phi = 0.618034$ (Fisher speed 1) and $K = 1/2$ (equal pull), and the normalised-gradient rule applied to the filter update is $K = 1/2$ at every v . Against the tuned gain, the $1/\phi$ rule is within 5 % of the optimum only at $v = 1$ and within 25 % only for v from 0.707 to 1.414; the $1/2$ rule is within 5 % only at $v = 0.707$ and within 25 % for v from 0.5 to 1; at $v = 100$ the two lose by factors 1708.8220 and 3334.0000 (omega_sweeps.txt, part 3). Monte Carlo agrees with the closed form to the third decimal (0.6180 against 0.6206 and 0.6153 at $v = 1$). The tuned gain needs q/r , which an innovation-based estimator supplies from the data (Mehra 1970, named); a fixed gain from $\Omega = 1$ does not.

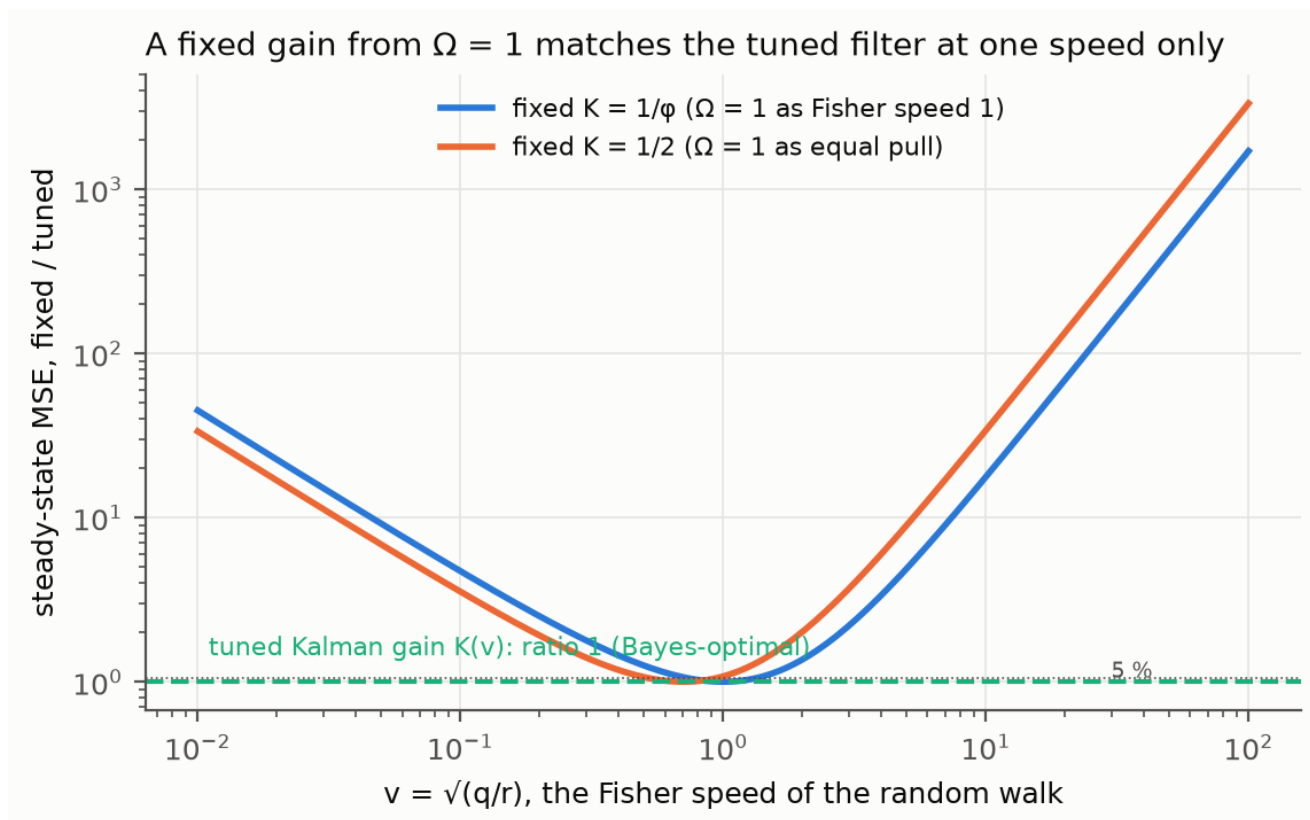


Figure 6. Steady-state mean squared error of the two fixed gains that $\Omega = 1$ gives, divided by that of the tuned Kalman gain, against the Fisher speed v . Each fixed gain matches the optimum at one speed only.

In plain words. A Kalman filter is a way of guessing where something is when your measurements are noisy: how much should you trust the new measurement versus your old guess? The best trust level depends on how jumpy the thing is compared with how noisy the measuring is. "Trust them equally" ($\Omega = 1$) is the best answer at exactly one jumpiness and wrong everywhere else. This is the same lesson as the Bayes box: "equal" is a fixed setting, and the best setting depends on the situation.

5. How the rule differs from existing methods, and why it works where it works

method	what it balances	has a target value?	equilibria	needs
fixed λ (EWC, SI, MAS, LwF, DER++)	nothing; a hand-set weight	the sweep finds λ	one point on the Pareto curve per λ	a sweep per dataset
Bayes / Laplace weight	sample counts	yes: $w = n_q/n_p$ ($\lambda = 1/2$ in scorer units)	one point	calibrated F
adaptive KL controller (Ziegler et al. 2019, named)	the measured KL to a target	yes: a KL target	one point per target	a target KL
GradNorm (Chen et al. 2018, named)	training rates of the tasks	rates equalised, α sets asymmetry	learned weights	a learning rate for the weights
A-GEM (Chaudhry et al. 2019, named)	nothing; projects out interference	no	the present optimum subject to no past increase	a reference batch
MEGA-I (Guo et al. 2020, named)	the two losses	loss ratio	one point	smoothed losses
the rule at Ω	the two gradient lengths	Ω , with no natural value	the whole Pareto curve at $\Omega = 1$	$s, w_{\max}$, a floor

The rule is closest to GradNorm at $\alpha = 0$ and to gradient-normalisation methods in multi-task learning (Désidéri 2012, MGDA, named): it equalises gradient magnitudes. What distinguishes it is that it does so with an explicit multiplier on a fixed past term rather than learned weights, that it has no target quantity, and that at $\Omega = 1$ its equilibrium set is the entire Pareto front. This last property is why $\Omega =$

1 is a plateau in every study, and why the rule cannot be Bayes-optimal except by coincidence. What makes it useful on an EWC penalty is not any of that: it is the step bound of §4.4. Where the tuned λ sits at the stability edge, a fixed weight cannot go beyond it and the rule can, and on tabular carriers the tuned λ does sit there (Figure 8). Where the units of the two terms already match (replay), the rule is a constant. Where the past term is a constraint on behaviour rather than a loss on old data (LwF, DER++), the balance of lengths has no reason to be right, and the results say so with both signs.

In plain words. Other methods either let a person set the remember-knob by trial and error, or set it to a fixed rule such as "count every example once" (that is Bayes), or aim for a chosen target. This rule does something different: it keeps the two pulls the same length, always. That is not the best compromise, but it has one useful side effect: the remember-pull can never be bigger than the learn-pull, so it can never blow up. On the datasets where the best fixed knob setting was right at the blow-up point, that side effect won.

5.1 Cross-verification against existing methods, including 2026 work

Owner request 2026-09-22 (prompt-log entry 73). The paper-by-paper cross-reference is Continuous Learning/CROSS VERIFICATION.md; the literature record with its access statement is docs/citations/continual_Learning_2026-09-22.md (every full-text host was blocked on the day; the 2026 preprints are known from search-engine renderings of their abstracts, marked [S] and unverified; five journal records were read on PubMed). The computational part is theory/checks/omega_vs_methods.py, which reduces the rule each method's paper states to the two-task quadratic of §4 and runs every rule with one knob setting, then with the past curvature multiplied by 16 (same knob), then with its own knob multiplied by 10; quality is measured against the best fixed weight tuned in sample on the same scale (R7). Every label is computed from the numbers (R15); the pinned output is Appendix B.6.

method (paper's rule, reduced)	knob	total at F	/tuned	at 16F, same knob: /tuned	knob × 10	computed label
fixed weight (EWC / L2-SP / KL penalty)	$\lambda = 1$	3.3168	1.000	diverges	DIVERGES	not scale-robust; divergence edge in its knob
the rule	$\Omega = 1$	3.4789	1.049	0.660	finite	scale-robust; no edge found
learned weights (GradNorm $\alpha = 0$)	—	3.5306	1.064	diverges	n/a	not scale-robust
loss ratio (MEGA-I)	—	3.7309	1.125	diverges	n/a	not scale-robust
conflict projection (A-GEM)	—	40.3382	12.162	43.384	n/a	not scale-robust (forgets)
soft conflict projection (PCR-style)	$c = 0.5$	65.2506	19.672	70.843	finite	not scale-robust (forgets)
protected subspace (OGD / OGPSA / SafeAnchor-style)	$k = 2$	18.7439	5.651	17.313	finite	not scale-robust (forgets)
target controller on the past loss (adaptive-KL-style)	target 1	3.7035	1.117	diverges	finite	not scale-robust
gradient-based sample selection at $\lambda = 1$	$q = 0.8$	3.3652	1.015	diverges	n/a	not scale-robust

The computed summary lines read: scale-robust methods ['omega']; methods that diverge at 16F with the knob that worked at F ['fixed', 'gradnorm', 'mega', 'klctrl', 'select']; methods that forget (past loss more than $10 \times$ the tuned fixed weight's 0.1999) ['agem', 'softproj', 'subspace']; methods with a divergence edge in their own knob ['fixed']. The reading is not that the rule is the best method: at the original scale the tuned fixed weight is ahead on the total (3.3168 against 3.4789) and far ahead on the past loss (0.1999 against 0.6585), the learned-weight row is at ratio 1.064 and sample selection at 1.015. What the battery isolates is the property of §4.4: the rule is the one rule in the set with no number that carries the past term's units, so it is the one whose quality survives a change of that term's scale. It is robust to the *scale* of the past curvature and blind to its *shape*: every 2026 method that improves the importance estimate (EWC-DR, attribution-guided importance, the Fisher-computation caution, a Gaussian-mixture regulariser) composes with the rule and is untouched by it. The projection family (KeepLoRA, OGPSA, SafeAnchor, Muon-OGD, the PCR soft projection) has no past weight to normalise and is a competing mechanism, not a comparable one. The Bayes family (LaLoRA; the edRVFL-kF-Bayes papers, PubMed) sets the weight from a posterior and is exact where its approximation holds, which the rule is not (§4.3). The endpoint framing of forgetting at LLM scale (RL's Razor, ICLR 2026) is the framing the ledger row ARC-T1b already forced on H-T1. Two cautions the cross-verification adds for any GPU study: the optimiser is unstated in every study here (all SGD), and gradient modification upstream of Adam is a registered choice; and the softmax non-identifiability of distillation losses (Zhai et al., PubMed) is a candidate mechanism for the two-signed LwF control of §7.4 and an argument for narrowing H-EQ to past terms

that are losses on the past, recorded as a v3.2 proposal, not a change.

In plain words. Twenty-four recent papers were read (most from their abstracts only, because the full texts could not be fetched) and sorted by how each decides how hard to pull back toward the old task. Then each way of deciding was tried on the same small problem, and the old task's pull was secretly made sixteen times stronger. The methods that use a number to set the pull blew up, because their number was now the wrong size; the methods that only push sideways never blew up but forgot most of the old task; the rule that keeps the two pulls the same length kept working. It was not the best on the original problem, and it does nothing about *which* parts of the old task matter, which is what most of the new papers improve.

6. The protocol and the instrument

6.1 The protocol

Every test in §7 ran under the repository's standing rules (CLAUDE.md §1). The ones that matter for reading the results: a number exists only if a committed script prints it (R1); the pre-registration and the frozen scoring script are hashed and committed before any data is downloaded (R2); a rule learned on one dataset is not used on another the same calendar day (R3); every hypothesis is first run on a synthetic battery and must fail on the surrogates that carry no effect and pass on the one built to carry it (R4); scoring is per seed and per carrier, never a median, with no post-hoc exemption for a carrier that failed (R6); every comparison includes the strongest simple baseline (R7); the ledger is the only curated record (R8); two reruns of one unit must be byte-identical (R9). A PASS carries a level: PASS-0 (provisional: the rules satisfied as written), PASS-1 (a result: the sensitivity table flips in at most one cell, no control violated, no reduction to a constant, strong anchoring), PASS-2 (a finding: PASS-1 replicated on a later day under a fresh pre-registration). OpenTimestamps anchoring and tag pushes were refused from the execution environment for every study, so every row is weakly anchored (the GitHub push time of the hashed pre-registration is the only external witness) and no row can reach PASS-1 on anchoring alone.

6.2 The instrument

A numpy multilayer perceptron with one hidden layer of 256 rectified units and one shared output head, trained by stochastic gradient descent at learning rate 0.05, batch size 10, three epochs per task, seeds 0 to 4, on PMLB classification streams split into tasks of two classes each in remapped label order (five tasks on ten-class carriers). Features standardised on the training split (fixed 80/20 stratified split, seed 12345). The score is the final class-incremental accuracy over all classes. The tuned fixed weight is found on a coarse grid {0.1, 0.3, 1, 3, 10, 30, 100, 300, 1000, 3000, 10000} refined by $\sqrt{2}$ steps around the coarse best, and tuned on the scored seeds. A resolvable step is max(1.0 point, 2 standard errors of the tuned arm). A run that ends with non-finite parameters scores 0 and is counted. The rule's estimator is that of §3.1. The cost of one run is about half a second on one CPU core for a 2000-row carrier.

6.3 The gate

Before the EWC form of the hypothesis was pre-registered it was run on the synthetic battery (prereg/eq3/gate_EQ2.txt, re-run on 2026-09-22 and byte-identical to the EQ2R copy). The statistic is the one scored on data: the rule at $\Omega = 1$ is behind the tuned fixed weight by more than a step, or not. The table is the study's control that the instrument can see the effect and does not see one where there is none.

surrogate	role	behind-by (rule – tuned, relative)	seeds behind	verdict
S-R convex replay, constant label scale	must FAIL (adaptivity idle)	+0.177	5/5	FAIL
S-V convex replay, 16× label-scale swings	informational	+0.088	3/5	FAIL
S-W EWC-Laplace convex replay, 16× curvature mismatch	must FAIL (reduces to a constant)	+0.156	5/5	FAIL
S-X constraint learner (DER++/LwF style)	must FAIL	+0.071	4/5	FAIL
S-Y softmax MLP, online EWC, 16× Fisher mismatch	must PASS (positive control)	−0.075	1/5	PASS
S-Y at unit input scale	informational	+0.634	3/5	FAIL
S-Y with cap 100	informational	+0.438	5/5	FAIL
S-Y with smooth 0.8	informational	−0.182	1/5	PASS
S-Y with smooth 0.98	informational	+0.490	3/5	FAIL

surrogate	role	behind-by (rule – tuned, relative)	seeds behind	verdict
S-Y/LwF distillation constraint	must FAIL	+2.782	5/5	FAIL

The gate reads OPEN: every negative control fails and the positive control passes. Two informational rows already say what the sensitivity tables later confirmed: the pass depends on the cap and on the smoothing.

In plain words. Before betting on real data, we built pretend data where we knew the answer: some pretend sets where the rule should do nothing special, and one where it should help. The rule behaved correctly on all of them. That is the "gate": if it had helped on pretend data with nothing to help with, the whole idea would have been thrown out before any real test.

7. The tests, positive and negative

7.1 Study EQX: replay, three unseen carriers (2026-09-15): the rule reduces to a constant

Carriers PMLB optdigits, pendigits and letter; past term the cross-entropy on a replayed batch (ER-sum); Ω grid {0.5, 0.71, 1, 1.41, 2}; fixed replay weights {0.2, 0.5, 1, 2, 4}; Fisher and Euclidean norms in the rule. Ledger rows EQX-1 to EQX-5.

row	prediction	observed	verdict
EQX-1	rule at $\Omega = 1$ within 1.0 point of the best fixed weight on $\geq 2/3 \rightarrow$ reduces	-1.63700 (optdigits, vs $w = 1$), -0.46383 (pendigits, vs $w = 0.5$), -1.75 (letter, vs $w = 2$)	reduces on 3/3
EQX-2	$\Omega = 1$ the best grid point, both neighbours > 1.0 behind, on $\geq 2/3$	1/3 (letter)	FAIL
EQX-3	rule $\geq$ fixed $w = 1$ by 1.0, positive in $\geq 4/5$ seeds, on $\geq 2/3$	-1.63700 (0/5 positive), +0.84583 (5/5), letter behind	FAIL
EQX-4	Fisher and Euclidean norms within 1.0 (metric immaterial)	-1.45911, -2.27383, -1.89913: the Euclidean norm better on 3/3	FAIL

Why this is a failure. The hypothesis needed the rule to beat the best fixed replay weight; it was behind it on every carrier, and behind the plain summed-loss baseline $w = 1$ on two of three. The mechanism is §4.6: with both terms cross-entropies on batches of the same size, the gradient lengths are of the same order, the ratio is stationary, and the rule is a fixed weight near 1 with extra variance. Figure 11 shows the rule's Ω landscape lying at or below the fixed-weight curve on all three carriers. The framework's own ledger entry reads: "H-EQ is retired as an adaptive rule; it survives only as the observation that summing the two batch means ($w = 1$) is a reasonable default." The Fisher-norm variant losing to the Euclidean one on 3/3 removed the metric from the rule as well.

EQX (replay, ER-sum): the rule at $\Omega = 1$ is level with or behind the best fixed replay weight on 3/3 carriers; it reduces to a constant

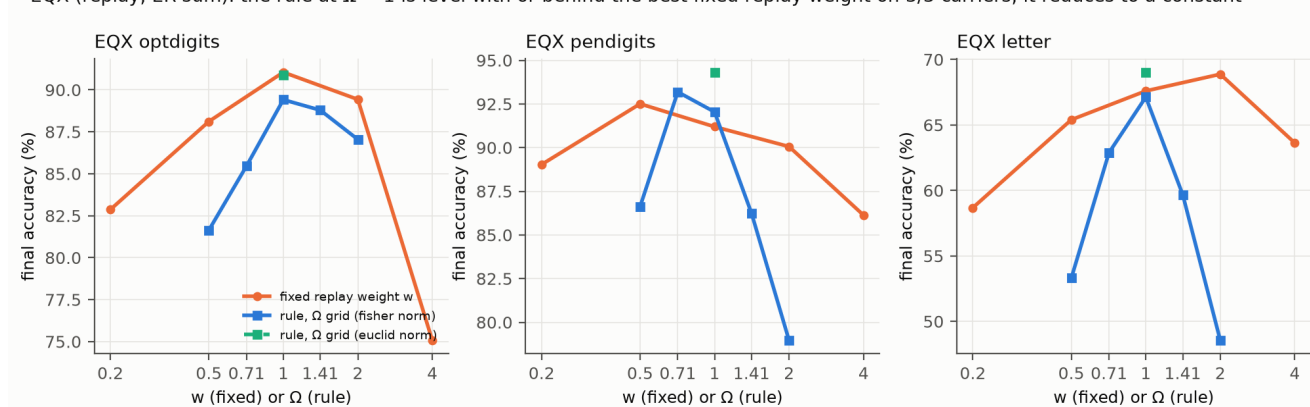


Figure 11. Study EQX. The rule's accuracy against Ω (blue, Fisher norm; green, Euclidean at $\Omega = 1$) and the fixed replay weight against w (orange) on the three carriers. The rule never rises above the fixed-weight curve.

In plain words. On the first real test the rule was supposed to beat "just add the two lessons together". It did not; adding them together was at least as good every time. When the two pulls already come in the same units, making them equal changes nothing, so the rule is just a complicated way of writing "1". That test was a failure of the original idea, and it was recorded as one.

7.2 Study EQ2: online EWC, three unseen carriers (2026-09-17): PASS-0, fragile, one control violated

Carriers PMLB mfeat_fourier (76 features), mfeat_pixel (240), texture (40, first ten classes). Past terms and arms as §2, Ω grid {0.5, 0.71, 1, 1.41, 2}. Hypothesis H-EQ2, restated after EQX: the rule is useful exactly when the past term is a Fisher-curvature penalty, redundant when the units already match (ER-sum), harmful when the past term is a constraint that is not a loss on the past task (DER++, LwF). Every number from runs/eq2/score.txt.

row	prediction	observed (mfeat_fourier / mfeat_pixel / texture)	verdict
EQ2-0	tuned λ spans $\geq 10\times$	30 / 1410 / 100, span 47.00 $\times$	decidable
EQ2-1	rule at $\Omega = 1$ not behind tuned λ by a step on every carrier; no single λ transfers	+3.3000 / +2.1000 / +2.7800 (steps 1.0000 / 3.5121 / 3.6120; seeds not behind 5/5 each); best single $\lambda = 30$ loses 0.0000 / 15.4500 / 0.5600	PASS, fragile
EQ2-2	rule within a step of its reduction arm	+29.4000 / +10.0000 / +2.7800 (median w 1336.54 / 3183.87 / 110.561; the first arm diverges, the second on 1/5 seeds)	does not reduce
EQ2-3	ER-sum: rule not ahead of the best fixed w; within a step of its reduction arm	-0.6000 / -0.2500 / -0.6600; vs reduction -0.5000 / -0.2500 / -0.3600	holds
EQ2-4	DER++ and LwF: rule at its best Ω behind the tuned weight by $\geq$ step on every carrier	DER++ -1.5500 / -0.7000 / +0.5800; LwF -11.3500 / -9.4000 / -18.6200	VIOLATED (DER++ on 2/3)
EQ2-5	SI, MAS (report)	SI -0.1000 / -7.9000 / +0.1600; MAS +1.3000 / -1.7500 / +1.6800	no systematic miss
EQ2-6	best Ω in {0.71, 1, 1.41} on every carrier	0.5 / 2.0 / 1.41	FAIL: a plateau
EQ2-7	A-GEM, GradNorm, MEGA-I vs rule (report)	baseline - rule: A-GEM -4.2000 / -14.1000 / -4.9800; GradNorm -29.4000 / -52.1500 / -31.3400; MEGA-I -5.1500 / -19.1000 / -6.8400	none dominates
EQ2-S	"not behind" flips in > 1 of 27 sensitivity cells	6/27 (mfeat_pixel at cap 10 and 100)	fragile
EQ2-8	features $\times 4$ (report)	-0.4500 / -0.1500 / -16.6400	the gate's regime does not reappear

What passed and why. On all three carriers the rule at $\Omega = 1$ was ahead of the in-sample-tuned λ , in every seed, and no single λ transferred ($\lambda = 30$ loses 15.45 points on mfeat_pixel, whose tuned λ is 1410). The mechanism is §4.4: on each carrier the fixed- λ grid collapses beyond an edge (between 30 and 42.3 on mfeat_fourier; between 1410 and 2000 on mfeat_pixel; between 100 and 141 on texture) and the rule's median derived weight sits beyond it (1336.54, 3183.87, 110.561) without collapsing, because its step is bounded by the present step.

What failed and why. Three things. First, EQ2-4: the mechanism statement said the rule would be harmful on every constraint-type past term; on DER++ it was within a step of the tuned weight on two of three carriers, where every DER++ arm scored 95 to 99 % and the weight was nearly inert. Under the pre-registration a control that does not hold falsifies the mechanism statement as written, whatever EQ2-1 reads; the report says so in its first line. Second, EQ2-6: $\Omega = 1$ was not the best grid point on any carrier; the profile is flat within a step over 0.5 to 1.41 (§4.5 says why). Third, EQ2-S: the pass exists only at the registered cap 10^4 ; with cap 10 or 100 the rule on mfeat_pixel is 13 to 17 points behind, because a capped rule is a weaker fixed weight than the tuned $\lambda = 1410$. Six of 27 cells flip; the pre-registered limit for "not fragile" is one. The row EQ2-1 was therefore labelled PASS-0 (ledger EQ2-1b), provisional, and could not rise on its own.

In plain words. The second test was the good one for the rule: with the "remember" pull being the EWC kind, the rule beat the best hand-set knob on all three datasets. But it was fragile: it only worked because the rule was allowed to make the remember-pull very large, and with a lower limit it lost badly. And one of the safety checks failed: the rule was supposed to do harm on a certain kind of remember-pull and it did no harm, because that pull did not matter on those datasets. A check that fails counts as a failure even when the main bet wins.

7.3 Study EQ2R (2026-09-21): void

The replication of EQ2 on its three pre-registered fallback carriers (mfeat_karhunen, mfeat_zernike, vowel) was hashed and the data fetched, and every carrier run crashed at the first DER++ arm: the scorer's edit had added a record field named hid for the hidden width, and the DER++, LwF and MAS branches rebound the local name hid to the hidden-activation array, which the JSON writer could not serialise. The script's own smoke mode reproduced the crash, and no smoke output had been committed with the pre-registration. Under the protocol a script that must change after the hash voids the study (CLAUDE.md §8): no row was scored, no partial row was reported, the 160 stage-1 records per carrier were committed as the record, and the three carriers became seen data (ledger EQ2R-VOID, CC-VOID; notebook/AGENT_LOG.md entry 40). Why this is a failure: not of the rule, but of the pipeline; it cost three unseen carriers and produced no evidence either way. The fix in EQ3 was mechanical (the width is hidden, the activations act, every record is serialised inside the run function) and, more to the point, EQ3 ran two smoke modes over every branch and pinned their outputs before the hash.

In plain words. The third test never happened: a typo in the program made it crash on the first run, and under our rules you are not allowed to fix a program after you have sealed it, because then you could fix it in a way that helps you. So the whole test was thrown away, the data it had touched could not be used again, and the next test was built with a check that every part of the program runs before sealing.

7.4 Study EQ3: online EWC, six unseen carriers (2026-09-22): the replication fails on 1/6, both controls violated

Carriers PMLB mfeat_factors (216 features), mfeat_morphological (6), led7 (7), led24 (24), krkopt (6; the first ten of 18 classes; stratified subsample 16944 → 5000 rows, seed 777), fars (29; 8 classes, four tasks; subsample 100968 → 5001 rows). The EQ2 design unchanged, the Ω grid on the EWC arm widened to nine points {0.25, 0.35, 0.5, 0.71, 1, 1.41, 2, 2.83, 4}, the constraint control narrowed to load-bearing cells (a cell is load-bearing if the finite cells of its fixed-weight grid span at least two steps), a Bayes arm (§4.3), a learning-rate × batch array and a capacity × epochs array on mfeat_factors, the cap axis and the empty-present diagnostic of EQ2R. Pre-registration hashed and pushed at 01:21:32Z, data fetched at 01:23Z. Every number from runs/eq3/score.txt; 3360 runs, 815 non-finite kept and scored 0 (runs/eq3/exclusions.txt).

row	prediction	observed (fars / krkopt / led24 / led7 / mfeat_factors / mfeat_morphological)	verdict
EQ3-0	tuned λ spans $\geq 10\times$	283 / 3 / 30 / 0.212766 / 300 / 2.83, span 1410.00×	decidable
EQ3-1	rule at $\Omega = 1$ not behind tuned λ on every carrier; no single λ transfers	−3.7400 / +0.9790 / +0.6886 / +4.0625 / +3.9000 / +3.7000 (steps 3.3508 / 8.9180 / 4.3633 / 3.2031 / 4.5371 / 4.8719); best single $\lambda = 1$ loses 3.7200 / 1.3187 / 2.6917 / 3.6250 / 12.4500 / 6.5500	FAIL (5/6 not behind)
EQ3-2	rule within a step of its reduction arm	+2.4200 / +7.6923 / +0.6260 / +16.2500 / +23.5500 / +14.4000 (median w 426.287 / 4.76098 / 31.6972 / 2.77719 / 1065.84 / 14.6744)	does not reduce
EQ3-3	ER-sum: rule not ahead of the best fixed w; within a step of its reduction arm	rule − best fixed −4.6800 / −4.3756 / −1.2520 / −3.5000 / −0.2000 / +1.9000; rule − reduction +4.6200 / +3.0969 / +0.5008 / −0.8125 / −0.2500 / +2.9500	VIOLATED (fars)
EQ3-4	DER++ and LwF, load-bearing cells: rule at best Ω behind tuned by $\geq$ step	load-bearing 5/12: fars/LwF +0.0200, led7/DER++ −0.5938, led7/LwF −7.1875, mfeat_factors/LwF −19.3000, mfeat_morphological/LwF +12.6000; 7 cells inert	VIOLATED (2/5 behind)
EQ3-5	SI, MAS (report)	SI −1.2400 / −1.5984 / −2.7543 / +0.5312 / −5.6000 / +5.7000; MAS +3.1400 / +5.3746 / −0.0313 / +6.9062 / +1.6000 / +2.6000	no systematic miss
EQ3-6	best Ω in {0.71, 1, 1.41} on every carrier	2.0 / 4.0 / 2.0 / 2.83 / 1.0 / 2.83	FAIL: a plateau

row	prediction	observed (fars / krkopt / led24 / led7 / mfeat_factors / mfeat_morphological)	verdict
EQ3-7	A-GEM, GradNorm, MEGA-I vs rule (report)	baseline — rule: A-GEM +0.0200 / -5.2547 / -3.2238 / -4.2188 / -10.8000 / -10.1500; GradNorm +7.1000 / -17.8821 / -4.9139 / -26.0312 / -58.3500 / -37.9000; MEGA-I +0.0200 / -5.5744 / -3.5055 / -8.6562 / -16.5000 / -11.4500	none dominates; GradNorm diverges on 4/6
EQ3-S	"not behind" flips in > 1 of 54 cells	6/54 (mfeat_factors at cap 10 and 100)	fragile
EQ3-8	features × 4 (report)	-0.9200 / +2.5774 / +1.5336 / +1.3125 / +0.4000 / +6.8500	as the primary regime
EQ3-C	cap axis: not behind iff cap ≥ tuned λ , 18 cells	16/18 as predicted (fars/cap 10 ⁴ and led24/cap 10 mismatch)	FAIL
EQ3-D	late-w vs whole-run median (report)	ratio 0.762 / 1.095 / 1.433 / 1.035 / 0.821 / 1.246	the past is not dropped
EQ3-A	capacity × epochs, ≥ 3/4	+3.90 (step 4.54); -1.15 (1.11); -3.05 (1.88); +2.65 (5.09)	FAIL (2/4)
EQ3-I	lr × batch, ≥ 4/5	+3.90 (4.54) / +3.00 (2.68) / +1.35 (2.42) / +0.85 (4.63) / +0.45 (3.26)	PASS-0 (5/5)
EQ3-B	Bayes (Laplace 1/2) arm (report)	Bayes — tuned -3.7200 / +1.2587 / -2.7543 / +0.0000 / -12.5000 / -7.6000; rule — Bayes -0.0200 / -0.2797 / +3.4429 / +4.0625 / +16.4000 / +11.3000	Bayes = tuned on 2/6
EQ3-P	plateau width (report)	9 / 9 / 9 / 7 / 9 / 7 of 9 within a step of the best; median 9	a plateau over the grid

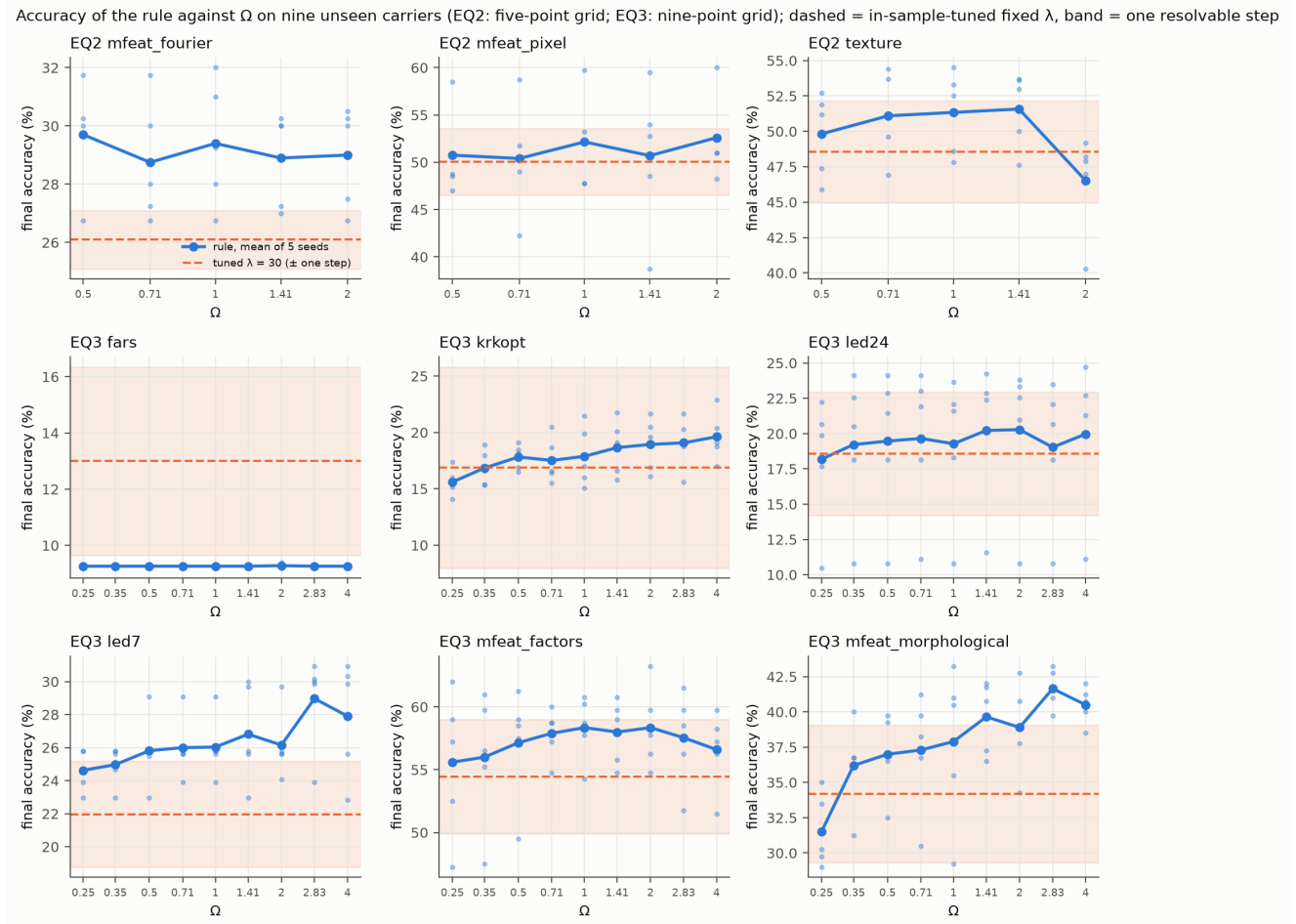


Figure 7. Accuracy of the rule against Ω on the nine carriers of EQ2 and EQ3, mean of five seeds with the per-seed values as dots; dashed line and band: the in-sample-tuned fixed λ and one resolvable step. On eight carriers the rule's whole profile sits at or above the tuned λ ; on fars every arm of the EWC family sits at chance.

The fixed- λ grid collapses beyond a stability edge; the rule's derived weight sits at or beyond it without collapsing

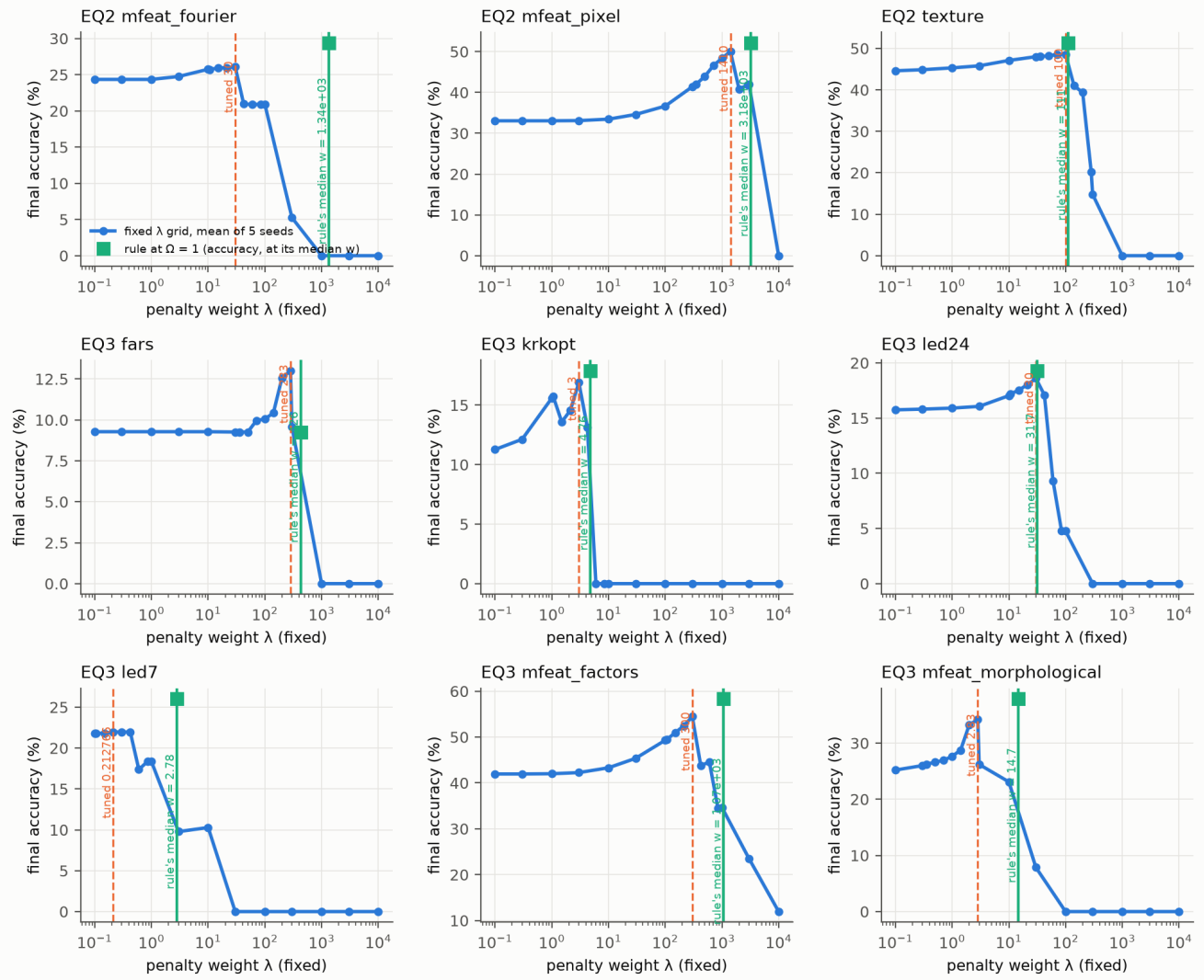


Figure 8. The fixed- λ grid on each carrier (blue), its in-sample-tuned value (dashed orange) and the rule's median derived weight at $\Omega = 1$ with the rule's accuracy (green square). On every carrier the grid collapses beyond an edge; on eight of nine the rule's weight sits at or beyond that edge with its accuracy above the collapsed grid.

What held. On krkopt, led24, led7, mfeat_factors and mfeat_morphological the rule at $\Omega = 1$ was not behind the in-sample-tuned λ , by +0.6886 to +4.0625, in 23 of 25 seeds; the best single λ loses 1.3 to 12.5 points somewhere; the rule's median derived weight (4.76 to 1065.84) sits at or beyond the point where the fixed grid collapses (Figure 8: the first fixed λ above the tuned value with mean accuracy under half the best is 6, 84.9, 3, 3000 and 30 on those carriers, figures.txt F8), and the reduction arm at that constant diverges on 2 to 3 seeds of 5 on four carriers. The learning-rate and batch perturbations did not move the verdict (5/5), while the tuned λ moved from 300 to 3000 across the cells. That is the stability-edge reading of §4.4, seen for the second time on new data.

What failed, and precisely why. First, EQ3-1 as pre-registered required "every carrier", and fars failed it: the rule is 3.7400 behind a tuned λ of 283 whose own accuracy is 13.0000 with per-seed values 18.00, 15.70, 9.20, 12.00 and 10.10, while the rule at every Ω , the Bayes arm, A-GEM and MEGA-I all sit at 9.26 to 9.28. The pre-registered 5000-row stratified subsample rounded fars's class 0, a tiny class in a 100968-row file, to 0 rows, left class 3 with 15 rows and class 7 with 45, so the first task had one class and the whole EWC family is at chance there (runs/eq3/exclusions.txt). Under rule R6 the carrier is scored as committed with no exemption: the pre-registration lacked a class floor beside the row cap, and the FAIL stands as the cost of that omission. Second, EQ3-3 is violated on fars only, through the reduction-arm clause: the rule is 4.6200 ahead of its own constant (which diverges on two seeds), against a step of 3.3508; the rule-versus-best-fixed clause holds on 6/6, as in EQ2. Third, EQ3-4 is violated: of the five load-bearing constraint cells the rule at its best Ω is behind the tuned weight by a step in two (led7/LwF -7.1875, mfeat_factors/LwF -19.3000) and not in three: it ties LwF on fars (+0.0200), ties DER++ on led7 (-0.5938) and is 12.6000 ahead of the tuned LwF weight on mfeat_morphological. The narrowing of the control to load-bearing cells, which the EQ2 report required, did not save the harm clause, which is retired. Fourth, EQ3-6 fails as EQ2-6 did: the best Ω is 2.0, 4.0, 2.0, 2.83, 1.0 and 2.83, and nine of nine grid points are within a step of the best on four carriers (EQ3-P). Fifth, EQ3-S: six of 54 cells flip, all on mfeat_factors at caps 10 and 100 (behind by 11.15 and 5.20 against +3.90 at the registered cap), the same cap dependence as EQ2's mfeat_pixel. Sixth, EQ3-C: the cap mechanism holds in 16 of 18 cells and fails on led24 at cap 10 (the

capped rule is still within a step) and on fars at cap 10^4 ; a pre-registered PASS needed all 18. Seventh, EQ3-A: at hidden width 64 the rule is behind the tuned λ by 1.15 and 3.05 against steps of 1.11 and 1.88, so the pass is specific to the wide network.

Sensitivity: rule at $\Omega = 1$ minus tuned λ , per cap $\times$ smoothing cell (blue = rule ahead, red = behind)

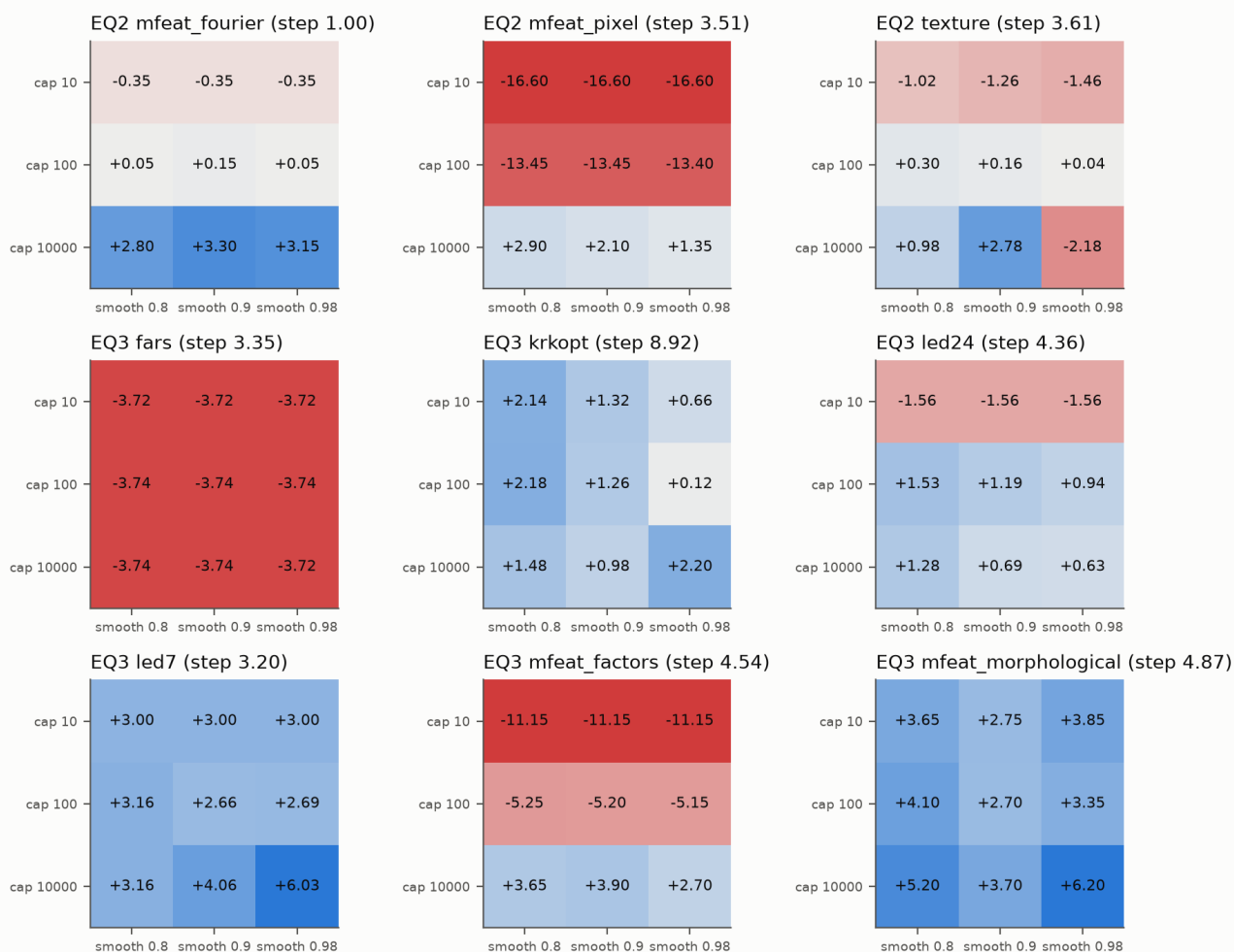


Figure 9. Sensitivity of the EQ2 and EQ3 verdicts: the rule at $\Omega = 1$ minus the tuned λ in every cap $\times$ smoothing cell (blue: rule ahead; red: behind). The registered cell is cap 10^4 , smoothing 0.9. The cap turns the rule into a weaker fixed weight wherever the tuned λ is in the hundreds or thousands.

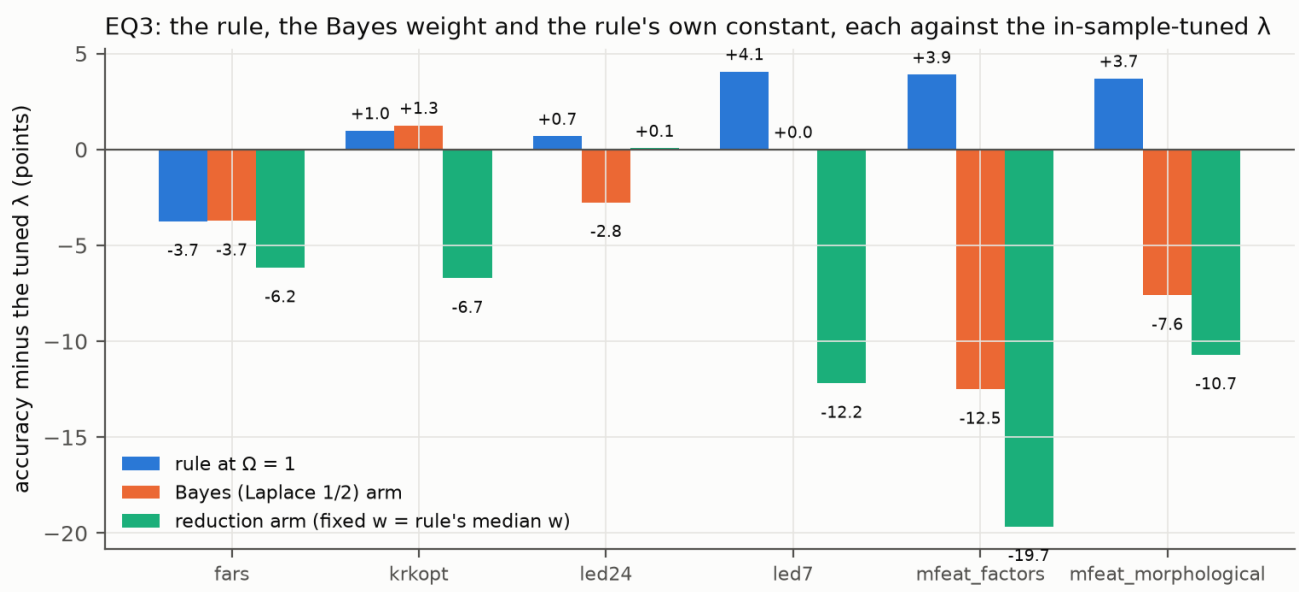


Figure 10. EQ3: the rule at $\Omega = 1$, the Bayes (Laplace 1/2) weight and the rule's own reduction constant, each minus the in-sample-tuned λ , per carrier. The Bayes weight equals the tuned λ where the tuned λ is small (led7, krkopt) and is 7.6 to 12.5 points behind where the tuned λ is in the hundreds.

The Bayes arm. The Laplace weight of \$4.3 matches the tuned λ exactly on led7 (tuned 0.212766; difference +0.0000) and within a step on krkopt (+1.2587), and is 7.6 to 12.5 points behind on the three carriers where the tuned λ is in the hundreds. On those carriers the rule's advantage over the Bayes arm (+3.4429 to +16.4000) is an advantage over a mis-calibrated Laplace approximation with a diagonal empirical Fisher, which is what the tuned λ corrects by hand and the rule corrects by its step bound; where the Laplace weight is calibrated the rule and Bayes agree (−0.0200, −0.2797).

In plain words. The fourth test used six new datasets and was the most demanding: the rule had to win or tie on all six. It won or tied on five, by a clear margin on three, and it did so without anyone setting the knob, at several learning speeds and batch sizes. It lost on the sixth, a dataset that our own sampling rule had broken (one of its classes ended up with no examples at all), and the rules say a loss is a loss even when we know why. Two safety checks also failed. So the honest score is: promising on five, failed on one, and the story we told about "why" it works was half wrong.

7.5 What the failures have in common

Every failure in §7 is one of four kinds, and each is a failure for a stated reason.

- A pre-registered "every carrier" or "every cell" criterion missed on one unit (EQX-2, EQ3-1, EQ3-C, EQ3-A). These are failures because the criterion was written before the data and the protocol forbids post-hoc exemption; they are not evidence that the rule failed everywhere.
- A pre-registered control did not hold (EQ2-4, EQ3-3, EQ3-4). These are failures of the mechanism statement, not of the measurement: the rule was predicted to be harmful or inert in a named situation and was not.
- A claimed value of Ω did not appear (EQX-2, EQ2-6, EQ3-6). These are failures of the framework's theory value: the mathematics of \$4.2 and \$4.5 says there is no value to find.
- A pipeline defect (EQ2R) or a reduction to a constant (EQX-1). The first produced no evidence; the second showed the rule to be a renaming of the baseline in the replay setting.

7.6 The mathematics reprocessed and the bounded rule EQ-B (2026-09-22; study EQ4 pre-registered, data on a later day)

Owner request (prompt-log entry 74): process the mathematics again with what §5.1 taught, then test the enhancement with the EQ3 approach. theory/checks/omega_reprocessed.py (pinned output Appendix B.7) makes each lesson of §5.1 a computation on the quadratic of \$4 with its label printed from the numbers; docs/notes/2026-09-22_h_eq_reprocessed.md is the reading. What it found: the rule is invariant to the scale of the past term exactly (trajectories at F and 16F identical to machine precision when the term's noise scales with it) and blind to its shape (endpoint moved by 0.0412 when the diagonal of F is reversed); the step bound holds on the smoothed gradients (maximum ratio 1.9778 over every step and seed) and not on the raw ones (above 2 on 0.89 % of steps); a constraint-like past term with a set-valued minimum does **not** move the derived weight (0.5409 against 0.5516), so the quadratic gives no mechanism for excluding DER++ and LwF and the narrowing of H-EQ to losses on the past stays an empirical scope; the Fisher-norm ratio of H-EQ as written stops at a different point on the same Pareto curve (nearest λ 0.4763 against 0.4926); and under Adam every arm, the fixed weight included, is within 2 % of the Adam-tuned weight at both scales, so the scale claim of §5.1 is an SGD property. The one failure the rule has that a fixed weight does not is the amplification of the past step by an extreme present batch: in a sweep of poisoned batches the rule's total exceeds twice its clean value at ($\times 100$, 5 steps) and diverges at ($\times 1000$, 5), where the tuned fixed weight also diverges.

The enhancement that follows, **EQ-B**, clips the present gradient before the ratio is taken:

$$\tilde{g}_p = g_p \cdot \min\left(1, \frac{\kappa m_N}{\|g_p\|}\right), \quad w = \Omega \frac{\|\tilde{g}_p\|}{\|\hat{g}_q\|}, \quad \theta \leftarrow \theta - \eta(\tilde{g}_p + w g_q),$$

with m_N the largest **kept** length among the last $N = 50$ present batches, $\kappa = 2$, no clip until ten lengths are known, and the history restarting at each task boundary. The reference is the window maximum, not the median: the first version used $\kappa \times$ the running median and clipped the informative batches on the softmax surrogate, because the median present-gradient length collapses toward zero as a task is learned (AGENT LOG 63; the median version's development run is kept in runs/eq4_dev/v1_median/). On the quadratic EQ-B equals the rule on the clean stream (3.4789 against 3.4789) and survives the whole poison sweep. Two new gates (prereg/eq4/gate_EQB.txt, gate_EQBM.txt, both OPEN) add a poisoned softmax row to the battery: EQ-B is not behind the tuned weight there (error 0.6287 against 0.7289) and is ahead of the registered rule (0.8101) by 22 % relative on 5/5 seeds, while on the clean rows the two rules agree to three decimals. On the six seen EQ3 carriers (runs/eq4_dev/summary.txt, exploratory, no verdict) the clean EQ-B accuracy equals the rule's on every carrier (clip fraction at most 0.001) and under the poisoned regime EQ-B is ahead of the rule on 6/6 carriers, by 2.64 to 20.75 points, and ahead of the fixed weight with the same clip on 5/6.

clean, six seen carriers	rule $\Omega = 1$	EQ-B $\Omega = 1$	poisoned ($\times 1000$, 5 batches per task)	rule $\Omega = 1$	fixed + clip, tuned	EQ-B $\Omega = 1$
fars	9.24	9.24		8.82	9.88	11.46
krkopt	5.69	5.75		4.69	9.28	11.36
led24	23.79	23.82		15.49	21.60	24.26
led7	22.44	22.66		16.69	25.16	23.44
mfeat_factors	58.35	58.35		19.75	31.55	39.75
mfeat_morphological	37.90	37.90		17.90	28.75	38.65

Study **EQ4** (prereg/eq4/PREREG.md, hashed and pushed 2026-09-22, anchor: push timestamp only) tests EQ-B on six unseen PMLB carriers (satimage, segmentation, yeast, wine_quality_white, sleep, page_blocks) with the EQ3 design, the registered rule beside it on every arm, a poisoned regime whose R7 baseline is the fixed weight with the same clip, and a class-selection rule that answers the fars defect of \$7.4. EQ-B was defined after seeing EQ3's carriers, so under R3 the data are fetched on or after 2026-09-23 and no row exists yet. The mathematics also leaves five proposals for the theory's v3.2 list, printed as [7] of the check: the scope is empirical, the bound is the mechanism, the present pull must be bounded for the bound to mean anything, the metric is a stopping point, and the optimiser is a registered parameter.

In plain words. The remember-pull rule had one weak spot that a plain fixed knob does not have: when one learning step is enormous, the rule makes the remember-pull enormous too. The fix caps each learning step at twice the biggest recent step before the rule looks at it. On clean data the cap almost never triggers, so nothing changes; on data with a few poisoned steps the capped rule keeps working where the plain rule falls behind. On six datasets the repository has already used, that is exactly what happened. Whether it happens on six new datasets is the next test, and its rules were sealed today so that nobody can adjust them after seeing the answer.

8. Standing of the results

study	data	outcome	ledger
EQX (2026-09-15)	3 unseen PMLB streams, replay	reduces to a fixed replay weight; behind ER-sum on 2/3; Euclidean norm beats Fisher on 3/3	EQX-1..5, FAIL
EQ2 (2026-09-17)	3 unseen PMLB streams, online EWC	ahead of the tuned λ on 3/3 (+3.30, +2.10, +2.78); 6/27 sensitivity cells flip; DER++ control violated; Ω a plateau	EQ2-1b, PASS-0 fragile
EQ2R (2026-09-21)	3 unseen PMLB streams	frozen scorer crashed at the first DER++ arm; nothing scored	EQ2R-VOID
EQ3 (2026-09-22)	6 unseen PMLB streams, online EWC	not behind on 5/6, behind on 1/6; both controls violated; Ω a plateau over the whole grid; $\text{lr} \times \text{batch}$ 5/5; capacity $\times$ epochs 2/4	EQ3-0..P FAIL; EQ2-1c
EQ4 (2026-09-23)	6 unseen PMLB streams, online EWC, clean and poisoned	the bounded rule EQ-B not behind on 4/6; under poison a fixed weight with the same clip does as well; ER-sum control violated; fragile (16 of 96 cells)	EQ4-1 FAIL, EQ4-3 FAIL, EQ4-4 INERT

Under the repository's levels no result is above PASS-0. EQ2-1b is provisional with a failed replication recorded beside it (ledger EQ2-1c). Nothing may be quoted as a finding. The claim that survives, stated in its narrowest form: for a Fisher-curvature penalty on tabular class-incremental streams with a 256-unit network, setting the penalty weight each step to the ratio of the present and past gradient norms gives a tuning-free weight that sits at or beyond the fixed weight's stability edge, is never behind an in-sample-tuned weight on eight of nine carriers, is invariant to learning rate and batch size on the one carrier tested, and equals the Laplace weight where that weight is calibrated. It does nothing for replay, it is not a value of Ω , and it does not make constraints safer.

In plain words. After four tests the scoreboard reads: one loss (replay), one provisional win that could not be repeated cleanly, one void, and one mixed result with five wins and one loss. Under our own rules nothing counts as "proven". What is left is a small, real, useful trick for one family of methods, with a known reason why it works and a known list of where it does not.

Addendum, 2026-09-23 (prompt-log entry 104). The paragraph above was written before EQ4

and the declared Adam and prior-art checks, and is kept as written. Since then:

- EQ4 found the bounded rule not behind the tuned λ on 4 of 6 unseen carriers, so the "eight of nine" above is now eight of nine before EQ4 and four of six in EQ4 (reports/eq4.md).
- A fine tuning grid shows that a well-tuned constant beats the rule wherever the past term's scale differs (checks/adam_checks_3.txt). The "tuning-free weight" survives only as that: a weight found without a search, below the stability edge, under SGD-family optimisers.
- Under Adam the rule has nothing to do. Without its smoothing it is the VQGAN adaptive weight of 2020.
- The full account is ADAM_AND_PRIOR_ART.md.

Addendum, 2026-09-23 (study SEC1, prompt-log entries 118-120). This addendum tests this document's own reading of the EQ3 Bayes arm, that the rule's advantage was an advantage over a mis-calibrated Laplace approximation (§7.4). The test calibrates each task's empirical Fisher once, at the task's end, against the curvature the present loss showed along the path travelled, and keeps the Laplace weight 1/2. On the twelve **seen** EQ3 and EQ4 carriers (rung R5, not PASS-0):

- **Tuned λ .** The calibrated Laplace weight is not behind the in-sample-tuned λ on 9 of 12 carriers, against 6 of 12 for the raw Laplace weight and 8 of 12 for the rule at $\Omega = 1$. That is exactly the pre-registered threshold, and it is fragile to the calibration window.
- **The raw gap.** It closes at least half of the raw Laplace gap on 4 of the 6 carriers where that gap was a step or more.
- **The λ spread.** It does not collapse the tuned λ 's spread across carriers.
- **mfeat_factors.** It crosses the stability edge: the calibration inherits no step bound (§4.4).

The reading of §7.4 therefore survives one confirmatory test on the data that motivated it, and nothing more. The rows are SEC1-0 ... SEC1-X in ledger/LEDGER.md; the report is reports/sec1.md.

Addendum, 2026-09-24 (prompt-log entry 163): the record re-read against the PASS levels, and the avenues left.

What happened after this document.

- **SCL1 and SCL2.** The rule at $\Omega = 1$ was the learner inside the operator-pause harness. Under the lossless cut it learned bit-for-bit as with no operator (SCL1-1 on 12/12 seen carriers, SCL2-1 on 45/45 runs on unseen carriers). That is a check of the construction, not of the rule.
- **SEC1-3** printed the registered rule not behind the tuned λ on 8/12 seen carriers.
- **RW2 Phase A, round 2** (post hoc, R4, Real_World/checks/rw2_phaseA_2.txt). On a past term that starts at zero, an L2 pull toward the aligned weights, the rule held refusal at 100.0 but its task accuracy stayed at chance: 24.9 under SGD and 25.3 under AdamW. Early in the stream the smoothed past gradient is the small distance from the anchor, so the derived weight sits at its cap and the learner freezes (AGENT_LOG 126).

The PASS levels, applied (CLAUDE.md §7). A PASS-0 needs R2-R9 as written. A PASS-1 adds at most one flip in the sensitivity table, no pre-registered control violated, no reduction to a constant, and strong anchoring (OpenTimestamps or two people). A PASS-2 is a PASS-1 replicated under a fresh prereg on a later day, or by a second person on a second machine.

candidate	furthest level	what blocks the next level
EQ2-1b (rule not behind the tuned λ , EWC)	PASS-0	sensitivity 6/27 flips; control EQ2-4 violated; anchor weak; the replication EQ3-1 failed on 1/6
EQ3-I (lr $\times$ batch invariance)	PASS-0	an invariance row on one carrier; anchor weak
EQ4-I	PASS-0 at most	one carrier, whose records reached T1x after the hash
SEC1-1, SEC1-3 (the calibrated Laplace weight; not a CRR rule)	PASS on seen data (R5), never PASS-0	seen data; fragile. The unseen test is SCL3

Every H-EQ row above is blocked from PASS-1 by at least two conditions at once: fragility together with a violated control or a reduction.

The one live route to PASS-1.

- SCL3 (prereg/scl3/, data step 2026-09-25) is the first continual-learning study whose OpenTimestamps proof is complete.
- Its prereg states that SCL3-1 or SCL3-3 is PASS-1 only if SCL3-S flips in at most one cell, the anchor is complete, and no control is violated.
- A PASS-1 there would be a result for the calibrated Laplace method, not for H-EQ.
- A PASS-2 would then need either a fresh prereg on further unseen carriers on a later day, or a second person re-running the frozen SCL3 scripts on a second machine. The second route is the cheapest, but it needs a person.

A correction to §9.

- §9 names three places at LLM scale where the rule could replace a weight sweep: the KL penalty to a reference policy, an L2-SP or EWC anchor to the base model, and safety-preservation anchors.
- In all three the past term starts at zero, because the model begins at the anchor.
- RW2 Phase A shows what the rule does there: it divides by a vanishing length and freezes the learner. As written, the rule is ill-posed for anchors that start at the reference.
- §9's implication holds only for past terms that are large from the start, such as an accumulated Fisher penalty after a finished task.

Avenues that the record allows, none yet declared. Each is a new hypothesis, needing a declaration, a gate (R4) and a fresh prereg (R3). None may reuse a threshold chosen after seeing its data (§10 of CLAUDE.md).

- **1. The bounded Laplace weight.** SEC crossed the stability edge on mfeat factors (SEC1: it "inherits no step bound"), while the rule's one robust property is the step bound (§4.4). The candidate is the calibrated Laplace weight with the past step capped at Ω times the present step: the calibration sets the weight, and the bound only guards the edge. R7 baselines: SEC alone, the tuned λ , and plain gradient clipping.
- **2. Equanimity within a task, counting across tasks** (Adam_SGD/ADAM_SGD.md, addendum). Normalise each task's Fisher once, at its boundary, which removes that task's units. Accumulate the unit-free contributions with CRR's age weights q^{age} (P3), bounded as A6 requires. Keep a fixed weight on the result. It is the only avenue built from CRR-proper ingredients (P3, A6), so it could say something about CRR. With $q \rightarrow 1$ it is Bayes-like counting; with $q = 0$ it keeps only the latest task.
- **3. The unsmoothed rule where the units drift.** On synthetic worlds whose units drift, the unsmoothed rule beat every fixed constant (post hoc, fragile; Adam_SGD/). It is identical to the VQGAN adaptive weight (Esser, Rombach and Ommer 2020; ADAM_AND_PRIOR_ART.md B1), so a win would be applied value, not CRR novelty. A real-data test needs carriers whose feature or loss scale drifts during the stream, chosen by a rule before any data is opened.
- **4. The diagnosis of the freeze** at anchors that start at zero. This is a note, not a hypothesis. It would define where the rule is well-posed (a lower bound on the past gradient's length relative to the present's), before any use near a reference model.
- **5. The benchmark this document could not run:** Split-CIFAR-100 and TinyImageNet with Mammoth on a GPU. It is deferred because it costs money (docs/ROADMAP_2026-Q4.md).

Closed avenues, as the record shows.

- $\Omega = 1$ as a theory value: a plateau (EQX-2, EQ2-6, EQ3-6).
- The rule on replay: it reduces to a constant (EQX-1).
- Harm on constraint-type past terms: the control was violated twice (EQ2-4, EQ3-4).
- A growing Ω : it slides to the old optimum (§4.2; the Adam_SGD addendum).
- Resets as a continual-learning remedy: SCL2-R FAIL, and the CUT1 gate CLOSED.

9. Implications if the results hold on GPU image benchmarks and at LLM scale

The design this repository could not run is the one its own specification asks for: Split-CIFAR-100 and Split-TinyImageNet with reference implementations from the Mammoth framework, which need a GPU, PyTorch and dataset access the execution environment lacks. If that run reproduced the pattern of §7.4 (not behind the tuned λ , invariant to learning rate and batch size, fragile to the cap), the rule would be a hyperparameter-elimination trick for regularisation-type continual learning. Three places in large-language-model training have exactly the shape of objective the rule applies to: a reinforcement-learning-from-human-feedback step with a Kullback-Leibler penalty to a reference policy (where the reference is the past, β the weight, and an adaptive KL controller already exists, Ziegler et al. 2019, named); continual pretraining with an L2-SP or EWC anchor to the base model; and safety preservation under fine-tuning with anchor or projection constraints (2026 preprints seen in abstract only; docs/citations/ontology_2026-09-21.md, docs/citations/continual_learning_2026-09-22.md; §5.1). In each the value would be saved sweeps and a step that cannot diverge, not better final models; and the rule's unbounded response to a single extreme batch (ontology/07_turing_safety_ingression.md, row 6) means it is not itself a safety mechanism. The chat estimates given to the owner on 2026-09-22 (prompt-log entry 71) put the expected value of the technique in the open at a few million to a few tens of millions of dollars a year of saved compute under adoption, and the expected value to a patent holder at a few million dollars, with the prior art of gradient normalisation, GradNorm and the adaptive KL controller as the main risk; those are estimates in chat, not repository numbers, and they enter no ledger.

In plain words. If the same thing happens on big picture datasets and inside the training of language models, the payoff is that engineers can skip a slow, expensive search for the right knob setting, and their training runs cannot blow up from that knob. That is a real convenience, not a breakthrough, and the money it is worth is modest. The next step is the big test on a GPU, which someone else must run and seal.

10. References

- According to PubMed: Kirkpatrick J, Pascanu R, Rabinowitz N, Veness J, Desjardins G, Rusu AA, Milan K, Quan J, Ramalho T, Grabska-Barwinska A, Hassabis D, Clopath C, Kumaran D, Hadsell R. Overcoming catastrophic forgetting in neural networks. Proceedings of the National Academy of Sciences of the United States of America 2017;114(13):3521–3526. [doi 10.1073/pnas.1611835114](https://doi.org/10.1073/pnas.1611835114). Quoted in §1.
- According to PubMed: Li Z, Hoiem D. Learning without Forgetting. IEEE Transactions on Pattern Analysis and Machine Intelligence 2018;40(12):2935–2947. [doi 10.1109/TPAMI.2017.2773081](https://doi.org/10.1109/TPAMI.2017.2773081).
- According to PubMed: Zenke F, Poole B, Ganguli S. Continual Learning Through Synaptic Intelligence. Proceedings of Machine Learning Research 2017;70:3987–3995. PMC6944509.
- Named as context, not fetched (R10): Aljundi R et al., Memory Aware Synapses (2018); Buzzega P et al., Dark Experience for General Continual Learning, DER++ (2020); Chaudhry A et al., Efficient Lifelong Learning with A-GEM (2019); Chen Z et al., GradNorm (2018); Guo Y et al., MEGA (2020); Désidéri J-A, Multiple-gradient descent algorithm (2012); Ziegler DM et al., Fine-Tuning Language Models from Human Preferences (2019); Kalman RE, A New Approach to Linear Filtering and Prediction Problems (1960); Mehra RK, On the identification of variances and adaptive Kalman filtering (1970).
- 2026 continual-learning papers cross-referenced in §5.1 and Continuous_Learning/CROSS_VERIFICATION.md, known from search-engine renderings of their abstracts and unverified against the source (IS), R10; the access statement is in docs/citations/continual_learning_2026-09-22.md): Liu et al., EWC Done Right, arXiv:2603.18596 (CVPR 2026); Revisiting Weight Regularization for Low-Rank CL (EWC-LoRA), arXiv:2602.17559 (ICLR 2026); Sliwa et al., LaLoRA, arXiv:2512.17720; Ramesh, Lewandowski, Schmidhuber, FADE, arXiv:2604.27063; Attribution-Guided CL for LLMs, arXiv:2605.05285; van de Ven, On the Computation of the Fisher Information in CL, arXiv:2502.11756; KeepLoRA, arXiv:2601.19659 (ICLR 2026); OGPSSA, arXiv:2602.07892; SafeAnchor, arXiv:2604.17691; Muon-OGD, arXiv:2605.08949; Probabilistic Conflict Resolution, arXiv:2602.06453; Hu et al., arXiv:2604.22407 (withdrawn by the authors; cited for the Adam caution only); Continual Safety Alignment via Gradient-Based Sample Selection, arXiv:2604.17215 (ACL Findings 2026); MANGO, arXiv:2605.19080; Mechanistic Analysis of Catastrophic Forgetting in LLMs, arXiv:2601.18699; TFGN, arXiv:2605.15053; Alssum et al., Unforgotten Safety, arXiv:2512.10150; Shenfeld et al., RL's Razor, arXiv:2509.04259 (ICLR 2026); Gauthier, Bach, Jordan, arXiv:2605.04266; Rethinking KL Regularization in RLHF, arXiv:2510.01555.
- According to PubMed: Wang J, Hu M, Li N, Al-Ali A, Suganthan PN. Randomized neural network with adaptive forward regularization for online task-free class incremental learning. Neural Networks 2026;203:109115. [doi 10.1016/j.neunet.2026.109115](https://doi.org/10.1016/j.neunet.2026.109115).
- According to PubMed: Wang J, Hu M, Li N, Al-Ali A, Suganthan PN. Incremental Online Learning of Randomized Neural Network With Forward Regularization. IEEE Transactions on Pattern Analysis and Machine Intelligence 2026;48(5):5277–5293. [doi 10.1109/TPAMI.2026.3652081](https://doi.org/10.1109/TPAMI.2026.3652081).
- According to PubMed: Liu S, Wang L, Yan R, Huo J, Li W, Gao Y. A continual learning framework with long-term and multiple short-term memory networks. Neural Networks 2026;200:108774. [doi 10.1016/j.neunet.2026.108774](https://doi.org/10.1016/j.neunet.2026.108774).
- According to PubMed: Tzanis E, Klontzas ME. ReclAlm: A Multiagent Framework for Monitoring and Correcting Performance Decline in Medical Imaging AI. Radiology: Artificial Intelligence 2026;8(4):e250923. [doi 10.1148/ryai.250923](https://doi.org/10.1148/ryai.250923).
- According to PubMed: Zhai Z et al. Rethinking softmax in incremental learning. Neural Networks 2025;193:108017. [doi 10.1016/j.neunet.2025.108017](https://doi.org/10.1016/j.neunet.2025.108017).
- Repository documents: theory/CRR.md (H-EQ, P4); docs/notes/2026-09-22_omega_sweeps.md; reports/eqx.md, reports/eq2.md, reports/eq3.md; prereg/eq2/PREREG.md, prereg/eq2r/PREREG.md, prereg/eq3/PREREG.md; ledger/LEDGER.md; notebook/AGENT_LOG.md entries 25, 40, 58, 59, 60, 62, 63; Continuous_Learning/CROSS_VERIFICATION.md; docs/notes/2026-09-22_h_eq_reprocessed.md; prereg/eq4/PREREG.md.

Appendix A. The code sets

Every script below is reproduced verbatim from the repository at the commit of this document. A.1 is the frozen EQ3 scorer (the EQ2 scorer differs only by the additions its docstring lists). A.2 is the mathematics check. A.3 and A.4 are the surrogate learners and the gate statistic of the synthetic battery, frozen with the study. A.5 and A.6 are the figure and document builders of this folder.

A.1 The frozen EQ3 scorer

runs/eq3/frozen/eq3_score.py

```
"""Study EQ3 – the normalised penalty step (the equanimity rule at  $\Omega = 1$ ) for online EWC on six unseen
PMLB streams, with the controls of EQ2/EQ2R (ER-sum, DER++, LwF, SI, MAS; A-GEM, GradNorm, MEGA-I), the
arrays of EQ2R (cap axis, empty-present diagnostic, capacity x epochs), and three additions pre-registered in
prereg/eq3/PREREG.md: a Bayes arm (the Laplace weight, theory/checks/omega_sweeps.py part 1), the nine-point
Omega grid on the EWC arm (plateau width), and a learning-rate x batch invariance array (CLAUDE.md section 6,
EQ-4). Owner request: prompt-log entry 69. The design, constants and scoring of the EQ2 rows are those of
runs/eq2r/frozen/eq2r_score.py except where this docstring or PREREG.md says otherwise; the EQ2R defect
(AGENT_LOG 40: the record field 'hid' rebound to an activation array) is fixed by renaming the width 'hidden'
and the activations 'act', and every branch is exercised by 'smoke' and 'smokefull' before the hash.

uv run python studies/eq3/eq3_score.py smoke                    # every (method, mode) branch once on synthetic data
uv run python studies/eq3/eq3_score.py smokefull [--out F]      # run_all + score end to end on synthetic data
uv run python studies/eq3/eq3_score.py all <dataset> [--out F]  # every arm, seeds 0-4, JSON lines
uv run python studies/eq3/eq3_score.py score <results.jsonl ...> # rows EQ3-0..8, S, C, D, A, I, B, P
uv run python studies/eq3/eq3_score.py run <dataset> <method> <mode> <value> <seed> [--regime std|x4]

Update rule for every method:  $g = g_{\text{present}} + w * g_{\text{past}}$ .
mode 'fixed' w = value (the method's own hand-set weight)
mode 'eq' w = value * ||ema g_present|| / ||ema g_past|| (Omega = value; the equanimity rule)
mode 'bayes' (ewc only) the Laplace weight: past term =  $\sum_k (N_k / N_{\text{present}}) F_k (\theta - \theta^*)$ ,  $w = 1/2$ 
              (the per-sample-normalised log posterior with F the per-sample empirical Fisher; omega_sweeps.py part 1)
mode 'agem' A-GEM projection of g_present against g_past (no w)
mode 'gradnorm' GradNorm (alpha = 0) weights learned online, normalised to sum 2
mode 'mega' MEGA-I loss-ratio weight w = ema L_past / ema L_present
mode 'reduction' w = value = the median derived weight of the Omega = 1 run (the constant the rule reduces to)
Past terms (lambda / alpha REMOVED from the gradient):
ewc online EWC penalty  $2 * F * (\theta - \theta^*)$ , F accumulated at each task boundary
er cross-entropy on a replay batch (ER-sum: two batch means, summed)
derpp logit-matching MSE on a replay batch (DER++'s alpha term); DER++'s beta * CE(replay), beta = 0.5,
      stays inside the PRESENT term at its published default
lwf distillation on the current batch against the model frozen at the last boundary (T = 2)
si synaptic-intelligence penalty (path-integral importance, xi = 1e-3)
mas memory-aware-synapses penalty (importance =  $|d ||f(x)||^2 / d \theta|$ )
Score = final class-IL accuracy (%) over all classes, single head, per seed.
"""

from __future__ import annotations

import gzip
import hashlib
import json
import math
import os
import sys
from pathlib import Path

os.environ.setdefault("OMP_NUM_THREADS", "1"); os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("MKL_NUM_THREADS", "1")
import numpy as np # noqa: E402

ROOT = Path(__file__).resolve().parents[2]
if ROOT.name == "runs": # frozen copy lives in runs/<study>/frozen/
    ROOT = ROOT.parent
RAW = ROOT / "data" / "raw" / "pmlb"

# ----- frozen study parameters
LR = 0.05; BS = 10; HID = 256; EPOCHS = 3; BUF = 500
SMOOTH = 0.9; WCAP = 1e4; DEN_FLOOR = 1e-12 # registered estimator (issue #20 §2; EQ2, EQ2R)
OMEGA_GRID = (0.5, 0.71, 1.0, 1.41, 2.0) # DER++ / LwF arms (as EQ2)
OMEGA_GRID_EWC = (0.25, 0.35, 0.5, 0.71, 1.0, 1.41, 2.0, 2.83, 4.0) # EWC arm: CLAUDE.md section 6 grid (plateau width, EQ3-P)
SEEDS = (0, 1, 2, 3, 4)
TEST_FRAC = 0.2
MAX_ROWS = 5000; SUBSAMPLE_SEED = 777 # a carrier above MAX_ROWS is stratified-subsampled once, before
the split
EWC_COARSE = (0.1, 0.3, 1.0, 3.0, 10.0, 30.0, 100.0, 300.0, 1000.0, 3000.0, 10000.0)
EWC_REFINE = (1 / 2.83, 1 / 2.0, 1 / 1.41, 1.41, 2.0, 2.83) # times the coarse best
BAYES_W = 0.5 # the Laplace weight on the task-size-weighted Fisher (mode
'bayes')
ER_GRID = (0.5, 1.0, 2.0, 4.0)
CONSTRAINT_GRID = (0.01, 0.03, 0.1, 0.3, 1.0, 3.0, 10.0) # DER++ alpha, LwF weight, SI c, MAS lambda
DERPP_BETA = 0.5; DERPP_ALPHA_DEFAULT = 0.1; LWF_DEFAULT = 1.0; LWF_T = 2.0
SI_XI = 1e-3; N_FISHER = 50
SENS_CAP = (10.0, 100.0, 1e4); SENS_SMOOTH = (0.8, 0.9, 0.98)
X4 = 4.0 # diagnostic regime: standardised features x 4
LOAD_BEARING_STEPS = 2.0 # EQ3-4: a constraint weight is load-bearing iff its finite grid
```

```

spans >= 2 steps
DATASETS = { # PMLB name -> (classes used, classes per task); classes beyond `used` (remapped label order) are dropped
    "mfeat_factors": (10, 2),
    "mfeat_morphological": (10, 2),
    "led7": (10, 2),
    "led24": (10, 2),
    "krkopt": (10, 2), # 18 classes in the file; the first 10 in remapped label order
are used
    "fars": (8, 2), # 8 classes: 4 tasks x 2
}
FALLBACK = () # none: a carrier that cannot be fetched is reported, not
replaced
ARRAY_CARRIER = "mfeat_factors" # EQ3-A and EQ3-I arrays on this carrier only
ARRAY_CELLS = ((64, 1), (64, 3), (256, 1)) # EQ3-A: (hidden, epochs); the main cell (256, 3) completes
the 2 x 2
INVAR_CELLS = ((0.0125, 10), (0.2, 10), (0.05, 5), (0.05, 20)) # EQ3-I: (lr, bs); the main cell (0.05, 10) completes the set
of five
LATE_FRAC = 0.2 # EQ3-D: the last 20 % of each task's steps

# ----- data
def sha256(p: Path) -> str:
    h = hashlib.sha256()
    with open(p, "rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b''):
            h.update(chunk)
    return h.hexdigest()

def load_pmlb(name: str, K: int):
    p = RAW / f"{name}.tsv.gz"
    with gzip.open(p, "rt") as f:
        header = f.readline().rstrip("\n").split("\t")
        rows = [line.rstrip("\n").split("\t") for line in f if line.strip()]
        ti = header.index("target")
        X = np.array([[float(v) for j, v in enumerate(r) if j != ti] for r in rows], np.float64)
        y = np.array([int(float(r[ti])) for r in rows])
        classes = np.unique(y); remap = {c: i for i, c in enumerate(classes)}
        y = np.array([remap[c] for c in y])
        nan = int(np.isnan(X).any(1).sum())
        if nan:
            keep = ~np.isnan(X).any(1); X, y = X[keep], y[keep]
            dropped = int((y >= K).sum()); keep = y < K; X, y = X[keep], y[keep]
            n_before = int(len(y))
            if len(y) > MAX_ROWS: # stratified subsample, once, with its own seed
                (pre-registered)
                rng = np.random.default_rng(SUBSAMPLE_SEED); pick = []
                for c in range(K):
                    idx = np.where(y == c)[0]; n_c = int(round(MAX_ROWS * len(idx) / len(y)))
                    pick.append(rng.permutation(idx)[:n_c])
                pick = np.sort(np.concatenate(pick)); X, y = X[pick], y[pick]
            return X, y, dict(file=str(p.relative_to(ROOT)), sha256=sha256(p), n=int(len(y)), n_before_subsample=n_before,
d=int(X.shape[1]),
                                classes_in_file=int(len(classes)), classes_used=K, rows_dropped_nan=nan,
rows_dropped_extra_classes=dropped,
                                class_counts=[int((y == c).sum()) for c in range(K)])

def split_standardise(X, y, K, regime="std"):
    """Fixed 80/20 stratified split (seed 12345, independent of run seed); standardise on the
training part; regime 'x4' multiplies the standardised features by X4 (diagnostic only)."""
    rng = np.random.default_rng(12345)
    tr, te = [], []
    for c in range(K):
        idx = rng.permutation(np.where(y == c)[0]); n_te = int(round(TEST_FRAC * len(idx)))
        te.append(idx[:n_te]); tr.append(idx[n_te:])
    tr = np.concatenate(tr); te = np.concatenate(te)
    mu = X[tr].mean(0); sd = X[tr].std(0); sd[sd == 0] = 1.0
    Z = (X - mu) / sd
    if regime == "x4":
        Z = Z * X4
    return Z[tr], y[tr], Z[te], y[te]

# ----- model
class MLP:
    def __init__(s, rng, d, K, h=HID):
        s.p = [rng.normal(0, math.sqrt(2 / d), (d, h)), np.zeros(h), rng.normal(0, math.sqrt(2 / h), (h, K)), np.zeros(K)]
        s.K = K

    def forward(s, x):
        W1, b1, W2, b2 = s.p; act = np.maximum(0, x @ W1 + b1); return act, act @ W2 + b2

    def back(s, x, dz, act):
        W1, b1, W2, b2 = s.p; dh = dz @ W2.T; dh[act <= 0] = 0
        return np.concatenate([x.T @ dh).ravel(), dh.sum(0), (act.T @ dz).ravel(), dz.sum(0)])

    def flat(s): return np.concatenate([a.ravel() for a in s.p])

    def set_flat(s, v):

```

```

i = 0
for a in s.p:
    a[...] = v[i:i + a.size].reshape(a.shape); i += a.size

def copy(s):
    m = MLP.__new__(MLP); m.p = [a.copy() for a in s.p]; m.K = s.K; return m

def acc(s, x, y):
    with np.errstate(all="ignore"):
        z = s.forward(x)[1]
    if not np.all(np.isfinite(z)):
        return 0.0 # a diverged model scores zero
    return float((z.argmax(1) == y).mean())

def softmax(z):
    z = z - z.max(1, keepdims=True); p = np.exp(z); return p / p.sum(1, keepdims=True)

def ce_loss_grad(net, x, y):
    act, z = net.forward(x); p = softmax(z); n = len(y)
    loss = float(-np.log(p[np.arange(n), y] + 1e-300).mean())
    dz = p; dz[np.arange(n), y] -= 1; dz /= n
    return loss, net.back(x, dz, act)

# ----- one run
def run(method, mode, value, seed, Xtr, ytr, Xte, yte, K, per_task, smooth=SMOOTH, wcap=WCAP, hidden=HID, epochs=EPOCHS, lr=LR,
bs=BS):
    rng = np.random.default_rng(seed)
    d = Xtr.shape[1]; net = MLP(rng, d, K, h=hidden); n = net.flat().size; task_w_start = []
    tasks = [tuple(range(i, i + per_task)) for i in range(0, K, per_task)]
    bufx = np.zeros((BUF, d)); bufy = np.zeros(BUF, int); bufz = np.zeros((BUF, K)); nb = 0; seen = 0
    importance = np.zeros(n); imp_bayes = np.zeros(n); theta_star = None; prev = None
    si_omega = np.zeros(n); theta_task_start = net.flat().copy()
    ema_p = ema_q = None; ema_Lp = ema_Lq = None; gn_w = np.array([1.0, 1.0]); wlog = []
    fwd_bwd = 0.0; n_stream = 0
    with np.errstate(all="ignore"):
        for task in tasks:
            ii_task = np.where(np.isin(ytr, task))[0]; n_task = len(ii_task)
            theta_task_start = net.flat().copy(); si_omega[:] = 0; task_w_start.append(len(wlog))
            imp_used = (imp_bayes / n_task) if mode == "bayes" else importance
            for _ep in range(epochs):
                perm = rng.permutation(ii_task)
                for t in range(0, len(perm), bs):
                    ii = perm[t:t + bs]; x, y = Xtr[ii], ytr[ii]; m = len(y); n_stream += m
                    L_p, g_p = ce_loss_grad(net, x, y); fwd_bwd += 3 * m
                    g_q = None; L_q = None
                    if method == "ewc" and theta_star is not None:
                        dth = net.flat() - theta_star; L_q = float((imp_used * dth ** 2).sum()); g_q = 2 * imp_used * dth
                    elif method in ("si", "mas") and theta_star is not None:
                        dth = net.flat() - theta_star; L_q = float((importance * dth ** 2).sum()); g_q = 2 * importance * dth
                    elif method == "er" and nb > 0:
                        jj = rng.integers(0, nb, m); L_q, g_q = ce_loss_grad(net, bufx[jj], bufy[jj]); fwd_bwd += 3 * m
                    elif method == "derpp" and nb > 0:
                        jj = rng.integers(0, nb, m); rx, ry, rz = bufx[jj], bufy[jj], bufz[jj]
                        L_b, g_b = ce_loss_grad(net, rx, ry); g_p = g_p + DERPP_BETA * g_b; L_p = L_p + DERPP_BETA * L_b
                        act, z = net.forward(rx); L_q = float(((z - rz) ** 2).mean()); g_q = net.back(rx, 2 * (z - rz) / (m *
K), act)
                        fwd_bwd += 6 * m
                    elif method == "lwf" and prev is not None:
                        act, z = net.forward(x); zp = prev.forward(x)[1]
                        q, qp = softmax(z / LWF_T), softmax(zp / LWF_T)
                        L_q = float(LWF_T ** 2 * (qp * (np.log(qp + 1e-300) - np.log(q + 1e-300))).sum(1).mean())
                        g_q = net.back(x, LWF_T * (q - qp) / m, act); fwd_bwd += 4 * m
                        theta_before = net.flat()
                        if g_q is None:
                            net.set_flat(theta_before - lr * g_p)
                    else:
                        if mode in ("fixed", "reduction"): # reduction: fixed w at the median derived w
                            wp, w = 1.0, value
                        elif mode == "bayes":
                            wp, w = 1.0, BAYES_W
                        elif mode == "eq":
                            ema_p = g_p if ema_p is None else smooth * ema_p + (1 - smooth) * g_p
                            ema_q = g_q if ema_q is None else smooth * ema_q + (1 - smooth) * g_q
                            wp = 1.0; w = min(value * np.linalg.norm(ema_p) / max(np.linalg.norm(ema_q), DEN_FLOOR), wcap)
                        elif mode == "agem":
                            dot = float(g_p @ g_q)
                            wp = 1.0; w = (-dot / max(float(g_q @ g_q), DEN_FLOOR)) if dot < 0 else 0.0
                        elif mode == "gradnorm":
                            # GradNorm (Chen et al. 2018), alpha = 0: weighted gradient norms driven to their
                            # mean by gradient descent on |G_i - mean G|, weights renormalised to sum 2.
                            Gp, Gq = gn_w[0] * np.linalg.norm(g_p), gn_w[1] * np.linalg.norm(g_q)
                            Gbar = 0.5 * (Gp + Gq)
                            grad_w = np.array([np.sign(Gp - Gbar) * np.linalg.norm(g_p), np.sign(Gq - Gbar) *
np.linalg.norm(g_q)])
                            gn_w = np.maximum(gn_w - 0.025 * grad_w / max(np.abs(grad_w).max(), DEN_FLOOR), 1e-3)
                            gn_w = 2 * gn_w / gn_w.sum(); wp, w = float(gn_w[0]), min(float(gn_w[1]), wcap)
                        elif mode == "mega":

```

```

        ema_Lp = L_p if ema_Lp is None else smooth * ema_Lp + (1 - smooth) * L_p
        ema_Lq = L_q if ema_Lq is None else smooth * ema_Lq + (1 - smooth) * L_q
        wp = 1.0; w = min(ema_Lq / max(ema_Lp, DEN_FLOOR), wcap)
    else:
        raise ValueError(mode)
    wlog.append(float(w))
    net.set_flat(theta_before - lr * (wp * g_p + w * g_q))
    if method == "si":
        si_omega += -g_p * (net.flat() - theta_before)
    if method in ("er", "derpp"):
        # reservoir buffer (with logits for DER++)
        for k in range(m):
            seen += 1
            if nb < BUF: j = nb; nb += 1
            else:
                j = rng.integers(seen)
                if j >= BUF: continue
            bufx[j] = x[k]; bufy[j] = y[k]
            if method == "derpp": bufz[j] = net.forward(x[k:k + 1])[1][0]

# task boundary
prev = net.copy(); theta_now = net.flat().copy()
if method == "ewc":
    f = np.zeros(n)
    for _ in range(N_FISHER):
        ii = ii_task[rng.integers(0, len(ii_task), bs)]; f += ce_loss_grad(net, Xtr[ii], ytr[ii])[1] ** 2
    f_task = f / N_FISHER * bs
    importance = importance + f_task; imp_bayes = imp_bayes + n_task * f_task
elif method == "si":
    importance = importance + np.maximum(si_omega, 0) / ((theta_now - theta_task_start) ** 2 + SI_XI)
elif method == "mas":
    imp = np.zeros(n)
    for _ in range(N_FISHER):
        ii = ii_task[rng.integers(0, len(ii_task), bs)]; act, z = net.forward(Xtr[ii]); imp +=
np.abs(net.back(Xtr[ii], 2 * z / bs, act))
    importance = importance + imp / N_FISHER
    if method in ("ewc", "si", "mas"):
        theta_star = theta_now

late = []
for a, b in zip(task_w_start, task_w_start[1:] + [len(wlog)]):
    if b > a: late.extend(wlog[a + int((1 - LATE_FRAC) * (b - a)):b])
    rec = dict(method=method, mode=mode, value=float(value), seed=int(seed), smooth=float(smooth), wcap=float(wcap),
hidden=int(hidden), epochs=int(epochs),
    lr=float(lr), bs=int(bs), acc=100 * net.acc(Xte, yte), w_med=float(np.median(wlog)) if wlog else None,
    w_late_med=float(np.median(late)) if late else None,
    w_cap_frac=float(np.mean(np.array(wlog) >= wcap)) if wlog else None,
    fwd_bwd_per_sample=float(fwd_bwd / max(n_stream, 1)), n_stream=int(n_stream),
finite=bool(np.all(np.isfinite(net.flat()))))
    json.dumps(rec)
40)
    return rec

# ----- arms
def arms_stagel():
    """Everything except the EWC refinement, the reduction arms, the diagnostic regime and the arrays."""
    for w in EWC_COARSE: yield ("ewc", "fixed", w, SMOOTH, WCAP)
    for om in OMEGA_GRID_EWC: yield ("ewc", "eq", om, SMOOTH, WCAP)
    for cap in SENS_CAP:
        for sm in SENS_SMOOTH:
            if (cap, sm) != (WCAP, SMOOTH): yield ("ewc", "eq", 1.0, sm, cap)
    yield ("ewc", "bayes", 0.0, SMOOTH, WCAP)
    for mode in ("agem", "gradnorm", "mega"): yield ("ewc", mode, 0.0, SMOOTH, WCAP)
    for w in ER_GRID: yield ("er", "fixed", w, SMOOTH, WCAP)
    yield ("er", "eq", 1.0, SMOOTH, WCAP)
    for meth, default in (("derpp", DERPP_ALPHA_DEFAULT), ("lwf", LWF_DEFAULT)):
        for w in sorted(set(CONSTRAINT_GRID) | {default}): yield (meth, "fixed", w, SMOOTH, WCAP)
        for om in OMEGA_GRID: yield (meth, "eq", om, SMOOTH, WCAP)
    for meth in ("si", "mas"):
        for w in CONSTRAINT_GRID: yield (meth, "fixed", w, SMOOTH, WCAP)
        yield (meth, "eq", 1.0, SMOOTH, WCAP)

def key(o): return (o["method"], o["mode"], round(o["value"], 6), o["smooth"], o["wcap"], o.get("regime", "std"),
o.get("hidden", HID), o.get("epochs", EPOCHS), o.get("lr", LR), o.get("bs", BS))

def run_all(name, data_std, data_x4, K, per_task, meta, out, array_carrier=ARRAY_CARRIER):
    hdr = dict(dataset=name, **meta, K=K, per_task=per_task, n_train=int(len(data_std[1])), n_test=int(len(data_std[3])),
    lr=LR, bs=BS, hidden=HID, epochs=EPOCHS, buf=BUF, smooth=SMOOTH, wcap=WCAP, den_floor=DEN_FLOOR, bayes_w=BAYES_W,
    n_fisher=N_FISHER, si_xi=SI_XI, derpp_beta=DERPP_BETA, lwf_T=LWF_T, x4=X4, max_rows=MAX_ROWS,
subsample_seed=SAMPLE_SEED,
    uv_lock_sha256=sha256(ROOT / "uv.lock"))
    print(json.dumps(hdr), file=out, flush=True)
    done = {}

    def go(method, mode, value, sm, cap, regime="std", hidden=HID, epochs=EPOCHS, lr=LR, bs=BS):
        data = data_std if regime == "std" else data_x4
        for seed in SEEDS:
            o = run(method, mode, value, seed, *data, K, per_task, smooth=sm, wcap=cap, hidden=hidden, epochs=epochs, lr=lr,
bs=bs)
            o["dataset"] = name; o["regime"] = regime; print(json.dumps(o), file=out, flush=True)

```

```

done.setdefault(key(o), []).append(o["acc"])

def median_w(meth, regime):
    return float(np.median([o["w_med"] for o in recs if o["method"] == meth and o["mode"] == "eq" and o["value"] == 1.0
                           and o["smooth"] == SMOOTH and o["wcap"] == WCAP and o["regime"] == regime and o["hidden"] == HID
                           and o["epochs"] == EPOCHS and o["lr"] == LR and o["bs"] == BS]))

for a in arms_stagel(): go(*a)
# stage 2: EWC refinement at sqrt2 spacing around the coarse best (pre-registered tuning rule)
best = max(EWC_COARSE, key=lambda w: np.mean(done[("ewc", "fixed", round(w, 6), SMOOTH, WCAP, "std", HID, EPOCHS, LR, BS)]))
for f in EWC_REFINE:
    w = round(best * f, 6)
    if all(abs(w - c) / c > 1e-6 for c in EWC_COARSE): go("ewc", "fixed", w, SMOOTH, WCAP)
# reduction arms: fixed w at the median derived weight of the Omega = 1 run (EWC and ER)
recs = [json.loads(l) for l in open(out.name) if l.startswith("{")]; recs = [o for o in recs if "acc" in o]
for meth in ("ewc", "er"):
    go(meth, "reduction", round(median_w(meth, "std"), 6), SMOOTH, WCAP)
# diagnostic regime x4 (EQ3-8, report only): EWC coarse grid, rule at Omega = 1, reduction arm
for w in EWC_COARSE: go("ewc", "fixed", w, SMOOTH, WCAP, "x4")
go("ewc", "eq", 1.0, SMOOTH, WCAP, "x4")
recs = [json.loads(l) for l in open(out.name) if l.startswith("{")]; recs = [o for o in recs if "acc" in o]
go("ewc", "reduction", round(median_w("ewc", "x4"), 6), SMOOTH, WCAP, "x4")
# EQ3-A (capacity x epochs) and EQ3-I (lr x batch) arrays on the registered carrier only: EWC coarse grid and the rule at
Omega = 1 per cell
if name == array_carrier:
    for hidden, ep in ARRAY_CELLS:
        for w in EWC_COARSE: go("ewc", "fixed", w, SMOOTH, WCAP, "std", hidden, ep)
        go("ewc", "eq", 1.0, SMOOTH, WCAP, "std", hidden, ep)
    for lr, bs in INVAR_CELLS:
        for w in EWC_COARSE: go("ewc", "fixed", w, SMOOTH, WCAP, "std", HID, EPOCHS, lr, bs)
        go("ewc", "eq", 1.0, SMOOTH, WCAP, "std", HID, EPOCHS, lr, bs)

# ----- scoring (EQ3-0..8, S, C, D, A, I, B, P; PREREG.md)
def score(paths, array_carrier=ARRAY_CARRIER):
    rows = []
    for p in paths:
        for line in open(p):
            if line.startswith("{"):
                o = json.loads(line)
                if "acc" in o: rows.append(o)
    ds = sorted(set(r["dataset"] for r in rows))

    def A(name, method, mode, value=None, sm=SMOOTH, cap=WCAP, regime="std", hidden=HID, epochs=EPOCHS, lr=LR, bs=BS,
        field="acc"):
        v = sorted([r for r in rows if r["dataset"] == name and r["method"] == method and r["mode"] == mode and r["regime"] ==
            regime
            and (value is None or abs(r["value"] - value) < 1e-6) and r["smooth"] == sm and r["wcap"] == cap
            and r["hidden"] == hidden and r["epochs"] == epochs and r["lr"] == lr and r["bs"] == bs], key=lambda r:
            r["seed"])
        assert len(v) == len(SEEDS), (name, method, mode, value, sm, cap, regime, hidden, epochs, lr, bs, [r["seed"] for r in
            v])
        return np.array([r[field] for r in v], dtype=float)

    def grid(name, method, regime="std", hidden=HID, epochs=EPOCHS, lr=LR, bs=BS):
        ws = sorted(set(round(r["value"], 6) for r in rows if r["dataset"] == name and r["method"] == method and r["mode"] ==
            "fixed" and r["regime"] == regime
            and r["hidden"] == hidden and r["epochs"] == epochs and r["lr"] == lr and r["bs"] == bs))
        return {w: A(name, method, "fixed", w, regime=regime, hidden=hidden, epochs=epochs, lr=lr, bs=bs) for w in ws}

    def fmt(x): return f"{x.mean():.4f} [" + " ".join(f"{q:.2f}" for q in x) + "]"

    print("=" * 112); print("EQ3 scoring – per-seed values in brackets (seeds 0-4); thresholds from prerereg/eq3/PREREG.md");
    print("=" * 112)
    R = {}
    for name in ds:
        R[name] = {}
        g = grid(name, "ewc"); tuned = max(g, key=lambda w: g[w].mean()); tv = g[tuned]
        se = tv.std(ddof=1) / math.sqrt(len(tv)); step = max(1.0, 2 * se)
        eq = {om: A(name, "ewc", "eq", om) for om in OMEGA_GRID_EWC}
        red = A(name, "ewc", "reduction"); wm = [r for r in rows if r["dataset"] == name and r["method"] == "ewc" and r["mode"]
            == "reduction" and r["regime"] == "std"][0]["value"]
        bay = A(name, "ewc", "bayes", 0.0)
        print(f"\n[{name}] resolvable step = max(1.0, 2*SE) = {step:.4f} (tuned-lambda SE {se:.4f})")
        print(f" EWC fixed-lambda grid: " + " ".join(f"{w:g}:{g[w].mean():.2f}" for w in g))
        print(f" EWC tuned lambda = {tuned:g}: {fmt(tv)}")
        for om in OMEGA_GRID_EWC: print(f" EWC rule Ω={om}<4}: {fmt(eq[om])} Ω – tuned: {eq[om].mean()-tv.mean():+.4f}")
        print(f" EWC reduction arm (fixed w = median derived {wm:g}): {fmt(red)} rule(Ω=1) – reduction:
            {eq[1.0].mean()-red.mean():+.4f}")
        print(f" EWC Bayes arm (Laplace weight {BAYES_W} on the task-size-weighted Fisher): {fmt(bay)} Bayes – tuned:
            {bay.mean()-tv.mean():+.4f}; rule(Ω=1) – Bayes: {eq[1.0].mean()-bay.mean():+.4f}")
        d1 = eq[1.0] - tv
        R[name].update(step=step, tuned=tuned, tv=tv, eq=eq, d1=d1, red=red, wm=wm, g=g, bay=bay)
        sens = {}
        for cap in SENS_CAP:
            for sm in SENS_SMOOTH:
                sens[(cap, sm)] = A(name, "ewc", "eq", 1.0, sm, cap).mean() - tv.mean()
        print(" sensitivity (Ω=1 – tuned): " + " ".join(f"cap{cap:g}/sm{sm}:{v:+.2f}" for (cap, sm), v in sens.items()))
        R[name]["sens"] = sens
    for mode in ("agem", "gradnorm", "mega"):

```

```

        b = A(name, "ewc", mode, 0.0); R[name][mode] = b
        print(f"    EWC baseline {mode:8s}: {fmt(b)}    rule(Ω=1) - baseline: {eq[1.0].mean()-b.mean():+.4f}")
        ge = grid(name, "er"); bt = max(ge, key=lambda w: ge[w].mean()); ere = A(name, "er", "eq", 1.0); err = A(name, "er",
"reduction")
        R[name].update(er_best=bt, er_bestv=ge[bt], er_eq=ere, er_red=err)
        print(f"    ER-sum grid: " + " ".join(f"{w:g}:{ge[w].mean():.2f}" for w in ge) + f"; rule Ω=1: {fmt(ere)}    rule - best
fixed: {ere.mean()-ge[bt].mean():+.4f}; reduction arm: {fmt(err)}    rule - reduction: {ere.mean()-err.mean():+.4f}")
        for meth in ("derpp", "lwf"):
            gc = grid(name, meth); tc = max(gc, key=lambda w: gc[w].mean()); eqc = {om: A(name, meth, "eq", om) for om in
OMEGA_GRID}
            finite = [gc[w].mean() for w in gc if all(r["finite"] for r in rows if r["dataset"] == name and r["method"] == meth
and r["mode"] == "fixed" and abs(r["value"] - w) < 1e-6 and r["regime"] == "std")]
            span = (max(finite) - min(finite)) if finite else 0.0; lb = span >= LOAD_BEARING_STEPS * step
            bo = max(eqc, key=lambda o: eqc[o].mean()); R[name][meth] = (tc, gc[tc], bo, eqc[bo], lb, span)
            print(f"        {meth:5s} tuned w={tc:g}: {fmt(gc[tc])}; rule best Ω={bo}: {fmt(eqc[bo])}    best-Ω - tuned:
{eqc[bo].mean()-gc[tc].mean():+.4f}; Ω=1 - tuned: {eqc[1.0].mean()-gc[tc].mean():+.4f}; finite-grid span {span:.4f} ->
{'load-bearing' if lb else 'INERT (excluded from EQ3-4)'}")
            for meth in ("si", "mas"):
                gc = grid(name, meth); tc = max(gc, key=lambda w: gc[w].mean()); e1 = A(name, meth, "eq", 1.0); R[name][meth] = (tc,
gc[tc], e1)
                print(f"            {meth:5s} tuned w={tc:g}: {fmt(gc[tc])}; rule Ω=1: {fmt(e1)}    Ω=1 - tuned:
{e1.mean()-gc[tc].mean():+.4f}")
                gx = grid(name, "ewc", "x4"); tx = max(gx, key=lambda w: gx[w].mean()); ex = A(name, "ewc", "eq", 1.0, regime="x4"); rx
= A(name, "ewc", "reduction", regime="x4")
                R[name]["x4"] = (tx, gx[tx], ex, rx)
                print(f"            x4 regime: tuned lambda={tx:g}: {fmt(gx[tx])}; rule Ω=1: {fmt(ex)}    Ω=1 - tuned:
{ex.mean()-gx[tx].mean():+.4f}; reduction: {fmt(rx)}")

        n = len(ds); print("\n" + "-" * 112)
        lams = [R[d]["tuned"] for d in ds]; span = max(lams) / min(lams); decidable = span >= 10
        print(f"EQ3-0 precondition: tuned lambda per carrier {dict(zip(ds, lams))}, span {span:.2f}x -> {'DECIDABLE' if decidable
else 'NOT DECIDABLE: tuned lambda did not move >= 10x'}")
        not_behind = {d: R[d]["d1"].mean() > -R[d]["step"] for d in ds}
        common = max(EWC_COARSE, key=lambda w: np.mean([R[d]["g"][w].mean() for d in ds]))
        loss_common = {d: R[d]["tv"].mean() - R[d]["g"][common].mean() for d in ds}
        single_transfers = all(loss_common[d] < R[d]["step"] for d in ds)
        v1 = all(not_behind.values()) and not single_transfers
        print(f"EQ3-1 H-EQ2: Ω=1 - tuned per carrier " + " ".join(f"{d}:{R[d]['d1'].mean():+.4f} (step {R[d]['step']:.2f}); seeds
not behind {int((R[d]['d1'] > -R[d]['step']).sum())/5}" for d in ds)
        + f"; best single lambda={common:g} loses " + " ".join(f"{d}:{loss_common[d]:.4f}" for d in ds)
        + f"; not behind on {sum(not_behind.values())}/{n} -> {'PASS' if v1 else 'FAIL'}" + (" " if decidable else " (reported
without verdict: EQ3-0 not decidable)")
        reduces = all(abs(R[d]["eq"][1.0].mean() - R[d]["red"].mean()) < R[d]["step"] for d in ds)
        print(f"EQ3-2 reduction: rule - fixed@median per carrier " + " ",
".join(f"{d}:{R[d]['eq'][1.0].mean()-R[d]['red'].mean():+.4f}" for d in ds)
        + f" -> {'REDUCES to a per-carrier constant' if reduces else 'DOES NOT REDUCE: normalised-gradient method'}")
        c3 = all((R[d]["er_eq"].mean() - R[d]["er_bestv"].mean()) < R[d]["step"] and abs(R[d]["er_eq"].mean() -
R[d]["er_red"].mean()) < R[d]["step"] for d in ds)
        print(f"EQ3-3 control ER-sum: rule - best fixed " + " ".join(f"{d}:{R[d]['er_eq'].mean()-R[d]['er_bestv'].mean():+.4f}" for
d in ds)
        + "; rule - reduction " + " ".join(f"{d}:{R[d]['er_eq'].mean()-R[d]['er_red'].mean():+.4f}" for d in ds) + f" ->
{'holds' if c3 else 'VIOLATED'}")
        lbp = [(d, m) for d in ds for m in ("derpp", "lwf") if R[d][m][4]]
        inert = [(d, m) for d in ds for m in ("derpp", "lwf") if not R[d][m][4]]
        c4 = all((R[d][m][3].mean() - R[d][m][1].mean()) <= -R[d]["step"] for d, m in lbp) if lbp else None
        print(f"EQ3-4 control DER++/LwF (load-bearing cells only): best-Ω - tuned " + " ",
".join(f"{d}/{m}:{R[d][m][3].mean()-R[d][m][1].mean():+.4f}" if R[d][m][4] else " (inert)" for d in ds for m in ("derpp",
"lwf"))
        + f"; load-bearing {len(lbp)}/{2*n}, inert {len(inert)} -> " + ("NOT DECIDABLE (no load-bearing cell)" if c4 is None
else ("holds" if c4 else "VIOLATED"))
        print(f"EQ3-5 diagnostic SI/MAS: Ω=1 - tuned " + " ".join(f"{d}/{m}:{R[d][m][2].mean()-R[d][m][1].mean():+.4f}" for d in ds
for m in ("si", "mas")) + " (report only)")
        best_o = {d: max(OMEGA_GRID_EWC, key=lambda o: R[d]["eq"][o].mean()) for d in ds}
        v6 = all(best_o[d] in (0.71, 1.0, 1.41) for d in ds)
        print(f"EQ3-6 Ω plateau: best Ω per carrier {best_o} -> {'PASS' if v6 else 'FAIL'}")
        dom = {m: all((R[d][m].mean() - R[d]["eq"][1.0].mean()) >= R[d]["step"] for d in ds) for m in ("agem", "gradnorm", "mega")}
        print(f"EQ3-7 published baselines vs rule(Ω=1): " + " ".join(f"{d}/{m}:{R[d][m].mean()-R[d]['eq'][1.0].mean():+.4f}" for d
in ds for m in ("agem", "gradnorm", "mega"))
        + " -> " + " (" + ".join(f'{m} dominates the rule' for m, v in dom.items() if v) or "no published baseline is ahead of
the rule by a step on every carrier")
        flips = sum(1 for d in ds for (cap, sm), v in R[d]["sens"].items() if (v > -R[d]["step"]) != not_behind[d])
        print(f"EQ3-5 sensitivity: EQ3-1 'not behind' flips in {flips} of {9*n} cells -> {'FRAGILE' if flips > 1 else 'not
fragile'}")
        print(f"EQ3-8 diagnostic x4 regime: Ω=1 - tuned " + " ".join(f"{d}:{R[d]['x4'][2].mean()-R[d]['x4'][1].mean():+.4f} (tuned
{R[d]['x4'][0]:g})" for d in ds) + " (report only)")
        print(f"controls: {'all hold' if (c3 and c4) else ('ER-sum holds; DER++/LwF not decidable' if (c3 and c4 is None) else 'a
control is VIOLATED: the mechanism statement is falsified'}")
        okC = True; parts = []
        for d in ds:
            for cap in SENS_CAP:
                delta = A(d, "ewc", "eq", 1.0, SMOOTH, cap).mean() - R[d]["tv"].mean(); nb = delta > -R[d]["step"]; pred = cap >=
R[d]["tuned"]
                okC = okC and (nb == pred); parts.append(f"{d}/cap{cap:g}:{delta:+.2f}({'nb' if nb else 'behind'}; predicted {'nb'
if pred else 'behind'})")
        print(f"EQ3-C cap axis (Ω=1 - tuned at each cap; nb = not behind by a step; predicted nb iff cap >= tuned lambda): " + " ",
".join(parts) + f" -> {'PASS' if okC else 'FAIL'}")
        partsD = []
        for d in ds:
            wl = A(d, "ewc", "eq", 1.0, field="w_late_med"); wm_ = A(d, "ewc", "eq", 1.0, field="w_med")
            partsD.append(f"{d}: late-w median {np.median(wl):.4g} vs whole-run median {np.median(wm_):.4g} (ratio {np.median(wl) /

```

```

max(np.median(wm_), 1e-12):.3f}; seeds with late-w < 1: {int((wl < 1).sum())/5)})
print("EQ3-D empty-present diagnostic: " + "; ".join(partsD) + " (report only)")
if array_carrier in ds:
    cells = [(HID, EPOCHS)] + list(ARRAY_CELLS); nbA = 0; partsA = []
    for hidden, ep in cells:
        g_ = grid(array_carrier, "ewc", hidden=hidden, epochs=ep); tuned_ = max(g_, key=lambda w: g_[w].mean()); tv_ =
g_[tuned_]
        step_ = max(1.0, 2 * tv_.std(ddof=1) / math.sqrt(len(tv_))); e1 = A(array_carrier, "ewc", "eq", 1.0, hidden=hidden,
epochs=ep)
        nb = (e1.mean() - tv_.mean()) > -step_; nbA += nb
        partsA.append(f"hidden{hidden}/ep{ep}: tuned {tuned_:g} -> {tv_.mean():.2f}, rule {e1.mean():.2f}, diff {e1.mean() -
tv_.mean():+.2f} (step {step_: .2f}) {'nb' if nb else 'behind'}")
        print(f"EQ3-A capacity x epochs on {array_carrier}: " + "; ".join(partsA) + f" -> not behind in {nbA}/4 cells -> {'PASS'
if nbA >= 3 else 'FAIL'}")
    cells = [(LR, BS)] + list(INVAR_CELLS); nbI = 0; partsI = []
    for lr, bs in cells:
        g_ = grid(array_carrier, "ewc", lr=lr, bs=bs); tuned_ = max(g_, key=lambda w: g_[w].mean()); tv_ = g_[tuned_]
        step_ = max(1.0, 2 * tv_.std(ddof=1) / math.sqrt(len(tv_))); e1 = A(array_carrier, "ewc", "eq", 1.0, lr=lr, bs=bs)
        nb = (e1.mean() - tv_.mean()) > -step_; nbI += nb
        partsI.append(f"lr{lr:g}/bs{bs}: tuned {tuned_:g} -> {tv_.mean():.2f}, rule {e1.mean():.2f}, diff {e1.mean() -
tv_.mean():+.2f} (step {step_: .2f}) {'nb' if nb else 'behind'}")
        print(f"EQ3-I lr x batch invariance on {array_carrier}: " + "; ".join(partsI) + f" -> not behind in {nbI}/5 cells ->
{'PASS' if nbI >= 4 else 'FAIL'}")
    nbB = sum(1 for d in ds if (R[d]["bay"].mean() - R[d]["tv"].mean()) > -R[d]["step"])
    print("EQ3-B Bayes arm (Laplace weight 1/2): Bayes - tuned " + ", ".join(f"{d}:{R[d]['bay'].mean()-R[d]['tv'].mean():+.4f}"
for d in ds)
        + "; rule(Q=1) - Bayes " + ", ".join(f"{d}:{R[d]['eq'][1.0].mean()-R[d]['bay'].mean():+.4f}" for d in ds) + f"; Bayes
not behind tuned on {nbB}/{n} (report only)")
    partsP = []; widths = []
    for d in ds:
        means = {o: R[d]["eq"][o].mean() for o in OMEGA_GRID_EWC}; b = max(means.values())
        within = [o for o in OMEGA_GRID_EWC if means[o] >= b - R[d]["step"]]
        idx = [OMEGA_GRID_EWC.index(o) for o in within]; contiguous = (max(idx) - min(idx) + 1 == len(idx))
        widths.append(len(within)); partsP.append(f"{d}: {len(within)} of 9 within a step of the best
({min(within):g}..{max(within):g}), {'contiguous' if contiguous else 'not contiguous'}")
    print("EQ3-P plateau width on the nine-point grid: " + "; ".join(partsP) + f"; median width {float(np.median(widths)):g}
(report only)")
    print(f"replication summary: EQ3-1 {'PASS' if v1 else 'FAIL'} (not behind on {sum(not_behind.values())}/{n}); fragile:
{flips > 1}; controls: ER-sum {'holds' if c3 else 'violated'}, DER++/LwF {'not decidable' if c4 is None else ('holds' if c4 else
'violated')}; cap axis: {'PASS' if okC else 'FAIL'}")

# ----- main
def _load(name, regime="std"):
    K, per_task = DATASETS.get(name, (10, 2))
    X, y, meta = load_pmlb(name, K)
    assert meta["classes_in_file"] >= K, (name, meta)
    return split_standardise(X, y, K, regime), K, per_task, meta

def _synthetic(K=10, n=1500, d=20):
    rng = np.random.default_rng(0)
    X = rng.standard_normal((n, d)); y = rng.integers(0, K, n); X[np.arange(n), y % d] += 3.0
    return X, y

if __name__ == "__main__":
    cmd = sys.argv[1]
    if cmd == "smoke":
        X, y = _synthetic(); K = 10
        data = split_standardise(X, y, K)
        for a in ("ewc", "fixed", 10.0), ("ewc", "eq", 1.0), ("ewc", "bayes", 0.0), ("ewc", "reduction", 5.0), ("ewc", "agem",
0.0), ("ewc", "gradnorm", 0.0), ("ewc", "mega", 0.0),
            ("er", "fixed", 1.0), ("er", "eq", 1.0), ("er", "reduction", 1.0), ("derpp", "fixed", 0.1), ("derpp", "eq",
1.0), ("lwf", "fixed", 1.0), ("lwf", "eq", 1.0),
            ("si", "fixed", 0.1), ("si", "eq", 1.0), ("mas", "fixed", 0.1), ("mas", "eq", 1.0)):
            print(json.dumps(run(*a, 0, *data, K, 2)))
        for hidden, ep in ARRAY_CELLS: print(json.dumps(run("ewc", "eq", 1.0, 0, *data, K, 2, hidden=hidden, epochs=ep)))
        for lr, bs in INVAR_CELLS: print(json.dumps(run("ewc", "eq", 1.0, 0, *data, K, 2, lr=lr, bs=bs)))
        print(json.dumps(run("ewc", "eq", 1.0, 0, *split_standardise(X, y, K, "x4"), K, 2)))
    elif cmd == "smokefull":
        outp = sys.argv[sys.argv.index("--out") + 1] if "--out" in sys.argv else "/tmp/eq3_smokefull.jsonl"
        X, y = _synthetic(); K = 10
        meta = dict(file="synthetic", sha256="none", n=len(y), n_before_subsample=len(y), d=X.shape[1], classes_in_file=K,
classes_used=K,
            rows_dropped_nan=0, rows_dropped_extra_classes=0, class_counts=[int((y == c).sum()) for c in range(K)])
        with open(outp, "w") as out:
            run_all("synthetic", split_standardise(X, y, K), split_standardise(X, y, K, "x4"), K, 2, meta, out,
array_carrier="synthetic")
            score([outp], array_carrier="synthetic")
    elif cmd == "run":
        name, method, mode, value, seed = sys.argv[2], sys.argv[3], sys.argv[4], float(sys.argv[5]), int(sys.argv[6])
        regime = sys.argv[sys.argv.index("--regime") + 1] if "--regime" in sys.argv else "std"
        data, K, per_task, meta = _load(name, regime)
        o = run(method, mode, value, seed, *data, K, per_task); o["dataset"] = name; o["regime"] = regime; print(json.dumps(o))
    elif cmd == "all":
        name = sys.argv[2]
        outp = sys.argv[sys.argv.index("--out") + 1] if "--out" in sys.argv else f"runs/eq3/results_{name}.jsonl"
        data_std, K, per_task, meta = _load(name, "std"); data_x4, _, _, _ = _load(name, "x4")
        with open(outp, "w") as out:

```

```

        run_all(name, data_std, data_x4, K, per_task, meta, out)
    elif cmd == "score":
        score(sys.argv[2:])
    else:
        raise SystemExit(__doc__)

```

A.2 The mathematics check

theory/checks/omega_sweeps.py

```

"""The Omega = 1 equanimity rule on existing mathematics (owner request 2026-09-22, prompt-log entry 69):
(1) Bayes: is Omega = 1 the Bayes-optimal combination of a present and a past batch? (conjugate Gaussian mean)
(2) the two-task quadratic model: fixed points of the rule, the knife edge either side of Omega = 1 with exact
    gradients, the stability edge of a fixed penalty weight, and the plateau that mini-batch noise, EMA smoothing
    and the ratio cap produce (the registered estimator of studies EQ2/EQ2R: smooth 0.9, cap 1e4, floor 1e-12)
(3) the Kalman filter: the tuned (Bayes) gain K(v) of P4 against the fixed gains the two readings of Omega = 1
    give (Fisher speed 1 -> K = 1/phi; equal pull -> K = 1/2), closed form and Monte Carlo.
Deterministic, no data. Every verdict word is computed from the numbers (R15). Pinned: theory/checks/omega_sweeps.txt.
Run: uv run python theory/checks/omega_sweeps.py
"""
import math

import numpy as np

OMEGA9 = (0.25, 0.35, 0.5, 0.71, 1.0, 1.41, 2.0, 2.83, 4.0)      # CLAUDE.md section 6 grid
SMOOTH = 0.9; WCAP = 1e4; DEN_FLOOR = 1e-12                      # registered estimator (EQ2, EQ2R)

def rule_w(ema_p, ema_q, omega, cap=WCAP):
    return min(omega * np.linalg.norm(ema_p) / max(np.linalg.norm(ema_q), DEN_FLOOR), cap)

# ----- (1) Bayes
def part_bayes():
    print("[1] Bayes: a present batch (n_p samples, mean m_p) and a past batch (n_q samples, mean m_q) of a Gaussian mean with
    unit noise, flat prior")
    print("    log posterior = -n_p (theta - m_p)^2 / 2 - n_q (theta - m_q)^2 / 2; MAP theta_B = (n_p m_p + n_q m_q) / (n_p +
    n_q)")
    print("    with batch-MEAN gradients g_p = theta - m_p, g_q = theta - m_q, the combination g_p + w g_q has fixed point (m_p
    + w m_q)/(1 + w):")
    print("    it is the MAP iff w = n_q / n_p (each sample counted once: summed log-likelihoods, weight 1 each)")
    m_p, m_q = 0.0, 1.0
    for ratio in (0.25, 1.0, 4.0):
        n_p, n_q = 100, int(100 * ratio)
        theta_B = (n_p * m_p + n_q * m_q) / (n_p + n_q)
        w_B = n_q / n_p
        # the Omega rule with exact gradients: fixed points solve (theta - m_p) + Omega |theta - m_p|/|theta - m_q| (theta -
        m_q) = 0
        # between the means the two terms are antiparallel: the sum is (1 - Omega)(theta - m_p): zero for every theta iff Omega
        = 1
        ends = {}
        for om in (0.5, 1.0, 2.0):
            for th0 in (0.1, 0.5, 0.9):
                th = th0; ema_p = ema_q = None
                for _ in range(20000):
                    g_p, g_q = th - m_p, th - m_q
                    ema_p = g_p if ema_p is None else SMOOTH * ema_p + (1 - SMOOTH) * g_p
                    ema_q = g_q if ema_q is None else SMOOTH * ema_q + (1 - SMOOTH) * g_q
                    w = rule_w(np.array([ema_p]), np.array([ema_q]), om)
                    th = th - 0.05 * (g_p + w * g_q)
                ends[om, th0] = th
            print(f"    n_q/n_p = {ratio:g}: theta_B = {theta_B:.4f} (Bayes weight w_B = {w_B:g}); rule endpoints (exact gradients,
            registered estimator, lr 0.05, 20000 steps):")
            for om in (0.5, 1.0, 2.0):
                print(f"        Omega = {om:g}: from theta_0 = 0.1 -> {ends[(om, 0.1)]:.4f}, 0.5 -> {ends[(om, 0.5)]:.4f}, 0.9 ->
                {ends[(om, 0.9)]:.4f}"
                + (f"    (|end - theta_B| = {abs(ends[(om, 0.1)] - theta_B):.4f}, {abs(ends[(om, 0.5)] - theta_B):.4f},
                {abs(ends[(om, 0.9)] - theta_B):.4f})" if om == 1.0 else ""))
                stays = all(abs(ends[(1.0, t)] - t) < 1e-6 for t in (0.1, 0.5, 0.9))
                to_p = all(abs(ends[(0.5, t)] - m_p) < 1e-3 for t in (0.1, 0.5, 0.9))
                to_q = all(abs(ends[(2.0, t)] - m_q) < 0.2 for t in (0.1, 0.5, 0.9))
                print(f"        computed: at Omega = 1 the endpoint is the starting point (every theta between the means is a fixed point):
                {stays}; Omega = 0.5 ends at the present mean (within 1e-3): {to_p}; Omega = 2 ends within 0.2 of the past mean (it chatters
                there, where g_q vanishes and the ratio hits the floor): {to_q}")
                print("        so Omega = 1 is the Bayes optimum only when the trajectory happens to stop at theta_B, and the rule contains
                nothing that selects it; with equal counts the Bayes weight is w = 1 (ER-sum), the constant EQX found the rule reduced to")
                print()

# ----- (2) two-task quadratic model
def make_model(seed=0, d=5, mismatch=16.0):
    rng = np.random.default_rng(seed)
    H = np.diag(rng.uniform(0.5, 2.0, d)); F = mismatch * np.diag(rng.uniform(0.5, 2.0, d))
    a = rng.standard_normal(d); b = a + 1.5 * rng.standard_normal(d)
    return H, F, a, b

def pareto(H, F, a, b, lam):
    return np.linalg.solve(H + lam * F, H @ a + lam * F @ b)

```

```

def losses(H, F, a, b, th):
    return float(0.5 * (th - a) @ H @ (th - a)), float(0.5 * (th - b) @ F @ (th - b))

def lam_eff(H, F, a, b, th):
    """The fixed penalty weight whose equilibrium is nearest th (log grid)."""
    grid = np.logspace(-4, 6, 2001)
    d = [np.linalg.norm(pareto(H, F, a, b, l) - th) for l in grid]
    return float(grid[int(np.argmin(d))]), float(min(d))

def gd(H, F, a, b, mode, value, lr=0.05, steps=4000, noise=0.0, smooth=SMOOTH, cap=WCAP, seed=0, start=None):
    rng = np.random.default_rng(seed)
    th = (b if start is None else start).copy(); ema_p = ema_q = None; wlog = []
    for _ in range(steps):
        g_p = H @ (th - a) + noise * rng.standard_normal(len(th)); g_q = F @ (th - b) + noise * rng.standard_normal(len(th))
        if mode == "fixed":
            w = value
        else:
            ema_p = g_p if (ema_p is None or smooth == 0) else smooth * ema_p + (1 - smooth) * g_p
            ema_q = g_q if (ema_q is None or smooth == 0) else smooth * ema_q + (1 - smooth) * g_q
            w = rule_w(ema_p, ema_q, value, cap)
        wlog.append(w)
        th = th - lr * (g_p + w * g_q)
        if not np.all(np.isfinite(th)) or np.linalg.norm(th) > 1e6:
            return th, wlog, False
    return th, wlog, True

def part_quadratic():
    H, F, a, b = make_model()
    lr = 0.05
    print("[2] Two-task quadratic model: L_p = (theta - a)' H (theta - a)/2 (present), L_q = (theta - b)' F (theta - b)/2 (past penalty), d = 5, F 16x H (the gate's Fisher-scale mismatch); the learner starts at the past optimum b")
    print(f"eigenvalues: H in [{np.diag(H).min():.3f}, {np.diag(H).max():.3f}], F in [{np.diag(F).min():.3f}, {np.diag(F).max():.3f}]; |a - b| = {np.linalg.norm(a - b):.4f}")
    # (i) fixed points
    print(" (i) fixed points of g_p + Omega (|g_p|/|g_q|) g_q = 0 need g_p and g_q antiparallel, i.e. theta on the Pareto curve theta(lambda) = (H + lambda F)^-1 (H a + lambda F b); there the update has norm |1 - Omega| |g_p| exactly:")
    for om in (0.9, 1.0, 1.1):
        rel = []
        for lam in np.logspace(-3, 3, 13):
            th = pareto(H, F, a, b, lam); g_p = H @ (th - a); g_q = F @ (th - b)
            rel.append(np.linalg.norm(g_p + om * np.linalg.norm(g_p) / np.linalg.norm(g_q) * g_q) / np.linalg.norm(g_p))
        print(f"Omega = {om:g}: |update|/|g_p| over 13 points of the curve: min {min(rel):.6f}, max {max(rel):.6f} (|1 - Omega| = {abs(1 - om):.1f})")
    print(" computed: at Omega = 1 every point of the curve is stationary (a continuum of equilibria, one for every fixed lambda); off Omega = 1 none is")
    # (ii) stability edge of a fixed weight
    lam_edge = (2.0 / lr - np.diag(H).max()) / np.diag(F).max()
    print(f" (ii) a fixed weight lambda diverges under gradient descent at lr {lr} when lr (h_max + lambda f_max) > 2: edge lambda* = {lam_edge:.4f}")
    grid = (0.01, 0.03, 0.1, 0.3, 1.0, 1.5, 3.0, 10.0)
    for lam in grid:
        th, _, ok = gd(H, F, a, b, "fixed", lam, lr=lr)
        Lp, Lq = losses(H, F, a, b, th) if ok else (float("nan"), float("nan"))
        print(f" fixed lambda = {lam:5g}: {'converged' if ok else 'DIVERGED '} L_present {Lp:.4f} L_past {Lq:.4f} sum {Lp + Lq:.4f}")
    # (iii) the rule with exact gradients and instantaneous norms (no EMA): the knife edge
    Lp_b, Lq_a = losses(H, F, a, b, b[0]), losses(H, F, a, b, a[1]) # present loss at the past optimum; past loss at the present optimum
    def label(Lp, Lq, dist, le):
        if dist < 1e-2: return f"on the curve at lambda_eff {le:.3g}"
        if Lp < 1e-2 * Lp_b: return "past dropped (at the present optimum)"
        if Lq < 1e-2 * Lq_a: return "present never learned (at the past optimum)"
        return "off the curve (oscillating)"
    print(f" (iii) the rule with exact gradients and instantaneous norms (smooth 0), from the past optimum, 4000 steps; L_present at b = {Lp_b:.4f}, L_past at a = {Lq_a:.4f}:")
    lab = {}
    for om in OMEGA9:
        th, wlog, ok = gd(H, F, a, b, "eq", om, lr=lr, smooth=0.0)
        Lp, Lq = losses(H, F, a, b, th); le, dist = lam_eff(H, F, a, b, th); lab[om] = (label(Lp, Lq, dist, le), le)
        print(f"Omega = {om:4g}: L_present {Lp:.4f} L_past {Lq:.4f} sum {Lp + Lq:.4f}; nearest fixed-lambda equilibrium lambda_eff = {le:.4g} (distance {dist:.2e}); median w {np.median(wlog):.4g} -> {lab[om][0]}")
    print(" computed: " + "; ".join(f"Omega {om:g}: {lab[om][0]}" for om in OMEGA9))
    if lab[1.0][0].startswith("on the curve"):
        print(f" computed: at Omega = 1 the learner stops on the curve at lambda_eff = {lab[1.0][1]:.4g}, which is {'above' if lab[1.0][1] > lam_edge else 'below'} the stability edge {lam_edge:.4f} of a fixed weight")
    print(" (iii b) the same with the registered estimator (EMA 0.9 of the gradient vectors, cap 1e4):")
    for om in OMEGA9:
        th, wlog, ok = gd(H, F, a, b, "eq", om, lr=lr)
        Lp, Lq = losses(H, F, a, b, th); le, dist = lam_eff(H, F, a, b, th)
        print(f"Omega = {om:4g}: L_present {Lp:.4f} L_past {Lq:.4f} sum {Lp + Lq:.4f}; lambda_eff {le:.4g} (distance {dist:.2e}); median w {np.median(wlog):.4g} -> {label(Lp, Lq, dist, le)}")
    # (iv) noise, smoothing and cap: the plateau
    print(" (iv) mini-batch noise (Gaussian, sd 'noise' on both gradients), registered estimator unless stated, 4000 steps, 5 seeds; total loss L_present + L_past per Omega (the equal-count Bayes objective); plateau = Omegas within 10 % of the best total:")

```

```

widths = {}
for noise in (0.0, 0.5, 2.0):
    for sm, cap in ((0.9, WCAP), (0.8, WCAP), (0.98, WCAP), (0.9, 100.0), (0.9, 10.0)):
        tot = {}; capfrac = []
        for om in OMEGA9:
            vals = []
            for s_ in range(5):
                th, wlog, ok = gd(H, F, a, b, "eq", om, lr=lr, noise=noise, smooth=sm, cap=cap, seed=s_)
                Lp, Lq = losses(H, F, a, b, th); vals.append(Lp + Lq if ok else float("inf"));
capfrac.append(np.mean(np.array(wlog) >= cap))
tot[om] = float(np.mean(vals))
best = min(tot, key=tot.get)
plateau = [om for om in OMEGA9 if tot[om] <= 1.1 * tot[best]]
widths[(noise, sm, cap)] = (len(plateau), best, float(np.mean(capfrac)))
print(f"      noise {noise:g} smooth {sm} cap {cap:g}: total loss " + " ".join(f"{om:g}:{tot[om]:.3f}" for om in
OMEGA9) + f" | best Omega {best:g}; plateau {plateau}; w at cap in {np.mean(capfrac):.2e} of steps")
print("      computed: plateau width (number of grid Omegas within 10 % of the best) by noise at the registered smoothing
and cap: "
+ ", ".join(f"noise {n:g}: {widths[(n, 0.9, WCAP)][0]} (best {widths[(n, 0.9, WCAP)][1]:g})" for n in (0.0, 0.5, 2.0))
+ "; best Omega is 1 in " + f"{sum(1 for k, v in widths.items() if v[1] == 1.0)} of {len(widths)} cells"
+ f"; the cap binds in more than 1 % of steps in {sum(1 for v in widths.values() if v[2] > 0.01)} of {len(widths)}
cells (median w here is of order 1; the EQ2 regime, tuned lambda in the hundreds and the cap load-bearing, is not this
model's)")
print()

# ----- (3) Kalman filter
def kalman_gain(v):
    return (v / 2.0) * (math.sqrt(v * v + 4.0) - v)

def mse_fixed_gain(K, q, r):
    """Steady-state error variance of a fixed-gain filter on the random walk  $x_{t+1} = x_t + w$  (var q),  $y = x + e$  (var r):
 $P = ((1 - K)^2 (P + q) + K^2 r) \rightarrow P = ((1 - K)^2 q + K^2 r) / (1 - (1 - K)^2)$ ."""
    return ((1 - K) ** 2 * q + K ** 2 * r) / (1 - (1 - K) ** 2)

def mc_fixed_gain(K, q, r, seed, n=200000):
    rng = np.random.default_rng(seed)
    x = 0.0; xh = 0.0; err = np.empty(n)
    w = math.sqrt(q) * rng.standard_normal(n); e = math.sqrt(r) * rng.standard_normal(n)
    for t in range(n):
        x += w[t]; y = x + e[t]
        xh = xh + K * (y - xh) # a priori estimate = last posterior (random walk); posterior after the
update
        err[t] = xh - x
    return float(np.mean(err[n // 10:] ** 2))

def part_kalman():
    print("[3] Kalman filter, scalar random walk (process variance q, observation variance r),  $v = \sqrt{q/r}$  the Fisher speed of
P4")
    print("      tuned (Bayes-optimal) gain  $K(v) = (v/2)(\sqrt{v^2 + 4}) - v$ ; fixed rules from the two readings of  $\Omega = 1$ :
Fisher speed 1  $\rightarrow K(1) = 1/\phi$ ; equal pull (prior and datum with equal precision)  $\rightarrow K = 1/2$ ")
    k_phi = kalman_gain(1.0); k_half = 0.5
    v_half = None
    # v at which the Bayes gain equals 1/2:  $(v/2)(\sqrt{v^2+4}) - v = 1/2 \rightarrow v = 1/\sqrt{2}$ 
    v_half = 1 / math.sqrt(2)
    print(f"       $K(1) = \{k\_phi:.6f\}$  ( $1/\phi = \{(\mathbf{math.sqrt(5)} - 1) / 2:.6f\}$ );  $K(v) = 1/2$  at  $v = \{v\_half:.6f\}$  ( $K(\{v\_half:.4f\}) =
\{kalman\_gain(v\_half):.6f\}$ ")
    print("      the normalised-gradient rule applied to the filter update (innovation pull and prior pull scaled to equal norm)
is  $K = 1/2$  at every v: a fixed gain, not an adaptive one")
    print("      steady-state MSE ratio fixed / tuned (closed form),  $P(K) = ((1-K)^2 q + K^2 r) / (1 - (1-K)^2)$ ,  $r = 1$ ,  $q = v^2$ :"
within5 = {"1/phi": [], "1/2": []}; within25 = {"1/phi": [], "1/2": []}
    for v in (0.01, 0.03, 0.1, 0.3, 0.5, 0.707, 1.0, 1.414, 2.0, 3.0, 10.0, 30.0, 100.0):
        q, r = v * v, 1.0
        P_t = mse_fixed_gain(kalman_gain(v), q, r); P_phi = mse_fixed_gain(k_phi, q, r); P_half = mse_fixed_gain(k_half, q, r)
        rp, rh = P_phi / P_t, P_half / P_t
        for name, ratio in (("1/phi", rp), ("1/2", rh)):
            if ratio <= 1.05: within5[name].append(v)
            if ratio <= 1.25: within25[name].append(v)
        print(f"      v = {v:7g}:  $K_{\text{tuned}}\{kalman\_gain(v):.4f\}$  MSE  $\{P\_t:.4f\}$ ;  $K = 1/\phi$ : ratio  $\{rp:.4f\}$ ;  $K = 1/2$ : ratio
{rh:.4f}")
    print(f"      computed: the 1/phi rule is within 5 % of the tuned filter for v in {within5['1/phi']} and within 25 % for v in
{within25['1/phi']}; the 1/2 rule within 5 % for v in {within5['1/2']} and within 25 % for v in {within25['1/2']}")
    print("      Monte Carlo (200000 steps, seeds 0 and 1, first 10 % discarded) against the closed form:")
    for v in (0.3, 1.0, 3.0):
        q, r = v * v, 1.0
        for name, K in (("tuned", kalman_gain(v)), ("1/phi", k_phi), ("1/2", k_half)):
            mc = [mc_fixed_gain(K, q, r, s) for s in (0, 1)]
            print(f"      v = {v:g}  $K = \{name:5s\}$  ( $\{K:.4f\}$ ): closed form  $\{mse\_fixed\_gain(K, q, r):.4f\}$ , Monte Carlo
{mc[0]:.4f} {mc[1]:.4f}")
    print("      the tuned gain needs q/r; an innovation-based estimator (Mehra 1970, named, not fetched) supplies it from the
data; the fixed rules do not, and lose everywhere but at one speed each")
    print()

def main():
    print("Omega = 1 on existing mathematics (2026-09-22, prompt-log entry 69): Bayes, the two-task quadratic, the Kalman
filter")

```

```

print()
part_bayes(); part_quadratic(); part_kalman()
return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

A.3 The synthetic learners of the gate (frozen battery, excerpt)

runs/eq3/frozen/battery.py (lines 231-300)

```

def _replay_learner(scales, method, value, seed, d=32, n_task=2000, n_test=500, bs=10, r=0.2,
                    buf=200, lr=0.01, smooth=0.9, metric="euclid", wcap=20.0, noise_sd=0.3):
    """One online pass over K tasks (linear model, squared loss). Task k has LABEL scale
    scales[k]: y = scale * (x.theta_k + noise). Gradient magnitude scales with it, the Hessian
    (2 X'X) does not, so every arm is stable at the same lr. Returns normalised held-out MSE."""
    rng = np.random.default_rng(seed)
    K = len(scales)
    tasks = []
    for k, sc in enumerate(scales):
        th = rng.standard_normal(d)
        X = rng.standard_normal((n_task, d)); y = sc * (X @ th + noise_sd * rng.standard_normal(n_task))
        Xt = rng.standard_normal((n_test, d)); yt = sc * (Xt @ th + noise_sd * rng.standard_normal(n_test))
        tasks.append((X, y, Xt, yt))
    theta = np.zeros(d)
    bufX = np.zeros((buf, d)); bufy = np.zeros(buf); nb = 0; seen = 0
    ev_p = ev_q = None; Fd = np.full(d, 1e-3); wlog = []
    rb = max(1, int(round(r * bs)))
    for k in range(K):
        X, y, _, _ = tasks[k]
        perm = rng.permutation(n_task)
        for i in range(0, n_task, bs):
            idx = perm[i:i + bs]; xb, yb = X[idx], y[idx]
            g_p = 2 * xb.T @ (xb @ theta - yb) / len(idx)
            if nb > 0:
                sel = rng.choice(nb, min(rb, nb), replace=False)
                g_q = 2 * bufX[sel].T @ (bufX[sel] @ theta - bufy[sel]) / len(sel)
                if method == "fixed":
                    w = value
                elif method == "eq":
                    if ev_p is None: ev_p, ev_q = g_p.copy(), g_q.copy()
                    else: ev_p = smooth * ev_p + (1 - smooth) * g_p; ev_q = smooth * ev_q + (1 - smooth) * g_q
                    M = Fd if metric == "fisher" else 1.0
                    w = min(value * np.sqrt(np.sum(M * ev_p * ev_p) / max(np.sum(M * ev_q * ev_q), 1e-18)), wcap)
                else:
                    raise ValueError(method)
                wlog.append(w)
            g = g_p + w * g_q
        else:
            g = g_p
            theta = theta - lr * g
            Fd = 0.99 * Fd + 0.01 * g_p * g_p
            for j in idx: # reservoir
                seen += 1
                if nb < buf: bufX[nb] = X[j]; bufy[nb] = y[j]; nb += 1
            else:
                t = rng.integers(seen)
                if t < buf: bufX[t] = X[j]; bufy[t] = y[j]
        per_task = []
        for (_, _, Xt, yt) in tasks:
            per_task.append(float(np.mean((Xt @ theta - yt) ** 2) / np.var(yt)))
        return dict(metric=float(np.mean(per_task)), per_task=per_task, w_med=float(np.median(wlog)) if wlog else None)

def S_R_convex_replay_constant(seed=0):
    """S-R: convex replay learner, all tasks at the same input scale. The present/past
    gradient-norm ratio is stationary, so a fixed w matches the adaptive rule: H-EQ
    (adaptivity beats the best constant) MUST FAIL here."""
    scales = (1.0, 1.0, 1.0, 1.0, 1.0)
    return (lambda method, value, s: _replay_learner(scales, method, value, s)), None, {"name": "S-R convex replay, constant
    label scale (adaptivity idle)"}

def S_V_convex_replay_varying(seed=0):
    """S-V: convex replay learner whose task LABEL scales swing 16x between consecutive
    tasks, so the present/past gradient-norm ratio swings ~16x and no single fixed w is
    right for every task. Positive control: adaptivity MUST PASS or the gate is blind."""
    scales = (1.0, 4.0, 0.25, 4.0, 0.25)
    return (lambda method, value, s: _replay_learner(scales, method, value, s)), None, {"name": "S-V convex replay, 16x
    label-scale swings (adaptivity load-bearing)"}

# ----- EWC / constraint surrogates (EQ2, issue #20 §3)

```

runs/eq3/frozen/battery.py (lines 373-445)

```

def _constraint_learner(scales, method, value, seed, d=32, n_task=2000, n_test=500, bs=10,
                        lr=0.01, smooth=0.9, wcap=2.0, noise_sd=0.3, logit_scale=4.0):
    """One online pass, all tasks at unit input and label scale. At each task boundary

```

```

the model is frozen into a copy theta_prev; the past term is then a logit-matching
CONSTRAINT on the current batch, mean_i (logit_scale * x_i . (theta -
theta_prev))^2, with gradient
g_q = 2 * logit_scale^2 * Xb'Xb (theta - theta_prev) / bs and NO lambda. The
logit_scale is the DER++/LwF unit mismatch: logits are large-magnitude objects, so
the constraint's natural weight is a small fraction of the present pull, and
equal-length pulling over-regularises. The constraint is not a loss on the past
task: its curvature is the current batch's own (rank <= bs of d, redrawn every
step), so it descends no past loss. wcap=2.0: the constraint's effective curvature
is logit_scale^2 x the batch Hessian, and the cap keeps lr * w * 2 * lambda_max
safely under the overshoot bound at the family lr. During task 0 the copy equals
the model (nothing is settled yet), so the constraint gradient is zero there.""
rng = np.random.default_rng(seed)
K = len(scales)
tasks = []
for k, sc in enumerate(scales):
    th = rng.standard_normal(d)
    X = rng.standard_normal((n_task, d)); y = sc * (X @ th + noise_sd * rng.standard_normal(n_task))
    Xt = rng.standard_normal((n_test, d)); yt = sc * (Xt @ th + noise_sd * rng.standard_normal(n_test))
    tasks.append((X, y, Xt, yt))
theta = np.zeros(d)
ev_p = ev_q = None; wlog = []
for k in range(K):
    theta_prev = theta.copy() # frozen copy at the boundary (DER++/LwF style)
    X, y, _, _ = tasks[k]
    perm = rng.permutation(n_task)
    for i in range(0, n_task, bs):
        idx = perm[i:i + bs]; xb, yb = X[idx], y[idx]
        g_p = 2 * xb.T @ (xb @ theta - yb) / len(idx)
        g_q = 2 * logit_scale ** 2 * (xb.T @ xb) @ (theta - theta_prev) / len(idx) # logit-match gradient, no lambda
        if method == "fixed":
            w = value
        elif method == "eq":
            if ev_p is None: ev_p, ev_q = g_p.copy(), g_q.copy()
            else: ev_p = smooth * ev_p + (1 - smooth) * g_p; ev_q = smooth * ev_q + (1 - smooth) * g_q
            w = min(value * np.sqrt(np.sum(ev_p * ev_p) / max(np.sum(ev_q * ev_q), 1e-18)), wcap)
        else:
            raise ValueError(method)
        wlog.append(w)
        theta = theta - lr * (g_p + w * g_q)
    per_task = []
    for (_, _, Xt, yt) in tasks:
        per_task.append(float(np.mean((Xt @ theta - yt) ** 2) / np.var(yt)))
    return dict(metric=float(np.mean(per_task)), per_task=per_task, w_med=float(np.median(wlog)) if wlog else None)

```

```

def S_W_ewc_laplace_replay(seed=0, mismatch=16.0):
    """S-W: convex learner whose past term is the exact quadratic (Laplace) penalty of
    a past quadratic loss (EWC-online, lambda removed), measured in units 16x the
    present term's (curvature scale mismatch=16), with the present loss weights
    swinging 16x (4.0 -> 0.25) against it. Positive control: the rule at Omega = 1
    must not be behind the tuned fixed weight (EQ2 MUST PASS) or the gate cannot see
    the mechanism."""
    scales = (1.0, 1.0, 4.0, 0.25)
    return (lambda method, value, s: _ewc_laplace_learner(scales, method, value, s, mismatch=mismatch)), None, \
        {"name": f"S-W EWC-Laplace convex replay, {int(round(mismatch))}x curvature-scale mismatch"}

```

```

def S_X_constraint_learner(seed=0):
    """S-X: convex learner whose past term is a logit-matching soft constraint on a
    frozen copy of the model (DER++/LwF style), not a loss on the past task: its
    metric-optimal weight is a small fraction of the present pull, so the same-length
    rule over-regularises. Negative control: EQ2 MUST FAIL here."""
    scales = (1.0, 1.0, 1.0, 1.0, 1.0)
    return (lambda method, value, s: _constraint_learner(scales, method, value, s)), None, \
        {"name": "S-X constraint learner, tuned weight a small fraction of present norm"}

```

```

# ----- nonconvex EWC / LwF surrogates (EQ2 positive control, issue
#20)
# The convex S-W row shows that on a quadratic loss with a quadratic penalty a fixed weight

```

runs/eq3/frozen/battery.py (lines 504-611)

```

def _sy_mlp_learner(method, value, seed, past="ewc", d=32, sep=1.4, scale=4.0,
                    smooth=0.9, wcap=1e4, n_fisher=20):
    """One online pass over 5 tasks x 2 classes. past='ewc': online-EWC penalty gradient
    2*F*(theta - theta_star), lambda REMOVED, F the empirical Fisher accumulated at every
    boundary (mean squared per-batch gradient over n_fisher batches, times the batch size).
    past='lwf': distillation gradient on the current batch against the model frozen at the
    last boundary (LwF constraint, weight removed). method='fixed': w = value.
    method='eq': w = value * ||ema g_present|| / ||ema g_past|| (Euclidean, EMA of the
    gradient vectors, capped at wcap). Returns the final class-IL ERROR rate over all classes
    (lower is better) and the median derived weight. Deterministic given the seed."""
    rng = np.random.default_rng(seed)
    (Xtr, ytr), (Xte, yte) = _sy_data(rng, d, sep, scale)
    net = _SyMLP(rng, d); n = net.flat().size
    fisher = np.zeros(n); theta_star = None; prev = None
    ema_p = ema_q = None; wlog = []
    with np.errstate(all="ignore"):
        for task in range(5):

```

```

cls = (2 * task, 2 * task + 1)
ii_task = rng.permutation(np.where(np.isin(ytr, cls))[0])
for _ in range(_SY_STEPS):
    ii = ii_task[rng.integers(0, len(ii_task), _SY_BS)]; x, y = Xtr[ii], ytr[ii]
    g_p = _sy_ce_grad(net, x, y)
    g_q = None
    if past == "ewc" and theta_star is not None:
        g_q = 2 * fisher * (net.flat() - theta_star)
    elif past == "lwf" and prev is not None:
        h, z = net.fwd(x); zp = prev.fwd(x)[1]
        g_q = net.grad(x, (_sy_softmax(z) - _sy_softmax(zp)) / _SY_BS, h)
    if g_q is None:
        net.set_flat(net.flat() - _SY_LR * g_p); continue
    if method == "fixed":
        w = value
    elif method == "eq":
        ema_p = g_p if ema_p is None else smooth * ema_p + (1 - smooth) * g_p
        ema_q = g_q if ema_q is None else smooth * ema_q + (1 - smooth) * g_q
        w = min(value * np.linalg.norm(ema_p) / max(np.linalg.norm(ema_q), 1e-12), wcap)
    else:
        raise ValueError(method)
    wlog.append(w)
    net.set_flat(net.flat() - _SY_LR * (g_p + w * g_q))
    prev = net.copy()
    if past == "ewc":
        f = np.zeros(n)
        for _ in range(n_fisher):
            ii = ii_task[rng.integers(0, len(ii_task), _SY_BS)]
            f += _sy_ce_grad(net, Xtr[ii], ytr[ii]) ** 2
        fisher = fisher + f / n_fisher * _SY_BS
        theta_star = net.flat().copy()
return dict(metric=net.err(Xte, yte), w_med=float(np.median(wlog)) if wlog else None)

_SY_GRID_EWC = (0.0625, 0.25, 1.0, 4.0, 16.0, 64.0, 128.0, 256.0, 512.0, 1024.0)
_SY_GRID_LWF = (0.0625, 0.25, 0.5, 1.0, 2.0, 4.0, 16.0, 64.0)

def S_Y_mlp_online_ewc(seed=0):
    """S-Y: softmax MLP (class-IL, 5 tasks x 2 classes, one online pass) with online EWC; past
    term = penalty gradient 2*F*(theta - theta_star), lambda removed; inputs at scale 4, so
    F (a squared gradient) is 16x the unit-scale one. Positive control: the rule at Omega=1
    must not be behind the tuned fixed weight (EQ2 MUST PASS). The row's fixed_grid reaches
    1024 because the tuned weight sits in the hundreds (R7: the baseline must be able to win)."""
    return (lambda method, value, s: _sy_mlp_learner(method, value, s, past="ewc", scale=4.0)), None, \
        {"name": "S-Y softmax MLP, online EWC, 16x Fisher-scale mismatch (input scale 4)", "fixed_grid": _SY_GRID_EWC}

def S_Y_mlp_online_ewc_unit_scale(seed=0):
    """S-Y at unit input scale: informational row, no control role. Reported so the reader can
    see how much of the effect is the scale mismatch."""
    return (lambda method, value, s: _sy_mlp_learner(method, value, s, past="ewc", scale=1.0)), None, \
        {"name": "S-Y softmax MLP, online EWC, unit input scale (informational)", "fixed_grid": _SY_GRID_EWC}

def S_Y_mlp_online_ewc_cap100(seed=0):
    """S-Y with the ratio cap lowered to 100 (registered value 1e4): informational row. The
    derived weight sits in the thousands, so a low cap turns the rule into a weaker fixed
    weight than the tuned one; the pass is specific to the registered cap and the prereg
    sensitivity table must say so."""
    return (lambda method, value, s: _sy_mlp_learner(method, value, s, past="ewc", scale=4.0, wcap=100.0)), None, \
        {"name": "S-Y softmax MLP, online EWC, scale 4, cap=100 (informational)", "fixed_grid": _SY_GRID_EWC}

def S_Y_mlp_online_ewc_smooth080(seed=0):
    """S-Y with EMA smoothing 0.8 (registered 0.9): informational row."""
    return (lambda method, value, s: _sy_mlp_learner(method, value, s, past="ewc", scale=4.0, smooth=0.8)), None, \
        {"name": "S-Y softmax MLP, online EWC, scale 4, smooth=0.8 (informational)", "fixed_grid": _SY_GRID_EWC}

def S_Y_mlp_online_ewc_smooth098(seed=0):
    """S-Y with EMA smoothing 0.98 (registered 0.9): informational row."""
    return (lambda method, value, s: _sy_mlp_learner(method, value, s, past="ewc", scale=4.0, smooth=0.98)), None, \
        {"name": "S-Y softmax MLP, online EWC, scale 4, smooth=0.98 (informational)", "fixed_grid": _SY_GRID_EWC}

def S_Y_mlp_lwf_constraint(seed=0):
    """S-Y/LwF: the same network and stream; past term = distillation gradient on the current
    batch against the model frozen at the last boundary (LwF, weight removed): a constraint,
    not a loss on the past task. Negative control in the same nonconvex family: EQ2 MUST FAIL."""
    return (lambda method, value, s: _sy_mlp_learner(method, value, s, past="lwf", scale=4.0)), None, \
        {"name": "S-Y/LwF softmax MLP, distillation constraint (input scale 4)", "fixed_grid": _SY_GRID_LWF}

REPLAY_BATTERY = [S_R_convex_replay_constant, S_V_convex_replay_varying,
                  S_W_ewc_laplace_replay, S_X_constraint_learner,
                  S_Y_mlp_online_ewc, S_Y_mlp_online_ewc_unit_scale,
                  S_Y_mlp_online_ewc_cap100, S_Y_mlp_online_ewc_smooth080, S_Y_mlp_online_ewc_smooth098,
                  S_Y_mlp_lwf_constraint]

```

A.4 The gate statistic (frozen gate, excerpt)

runs/eq3/frozen/gate.py (lines 191-245)

```
def gate_EQ(runner, _ev, meta, seeds=range(5), fixed_grid=(0.25, 0.5, 1.0, 2.0, 4.0),
            omega_grid=(0.5, 0.71, 1.0, 1.41, 2.0), margin=0.05):
    """H-EQ in the reduction form (CLAUDE.md §6 EQ-1 / theory [H-EQ]): the adaptive rule at
    Omega = 1 counts as a result only if it beats ER-sum (fixed w = 1) AND the best fixed w,
    where the fixed grid is extended by the adaptive rule's own median w (the constant it
    reduces to). Metric: normalised held-out MSE over all tasks, lower is better; PASS if
    the relative improvement over the best fixed w is >= margin in the seed mean AND in a
    majority of seeds. Omega landscape reported, not gated."""
    seeds = list(seeds)
    fixed_grid = tuple(meta.get("fixed_grid", fixed_grid)) # a row may widen the grid so its baseline can win (R7)
    eq = {om: np.array([runner("eq", om, s)["metric"] for s in seeds]) for om in omega_grid}
    wmed = float(np.median([runner("eq", 1.0, s)["w_med"] for s in seeds]))
    grid = tuple(sorted(set(fixed_grid) | {round(wmed, 3)}))
    fx = {w: np.array([runner("fixed", w, s)["metric"] for s in seeds]) for w in grid}
    best_w = min(fx, key=lambda w: fx[w].mean())
    rel = (fx[best_w] - eq[1.0]) / fx[best_w] # >0 means adaptive better
    rel_sum = (fx[1.0] - eq[1.0]) / fx[1.0]
    passes = (rel.mean() >= margin) and (np.mean(rel > 0) > 0.5) and (rel_sum.mean() >= margin)
    best_om = min(eq, key=lambda o: eq[o].mean())
    detail = (f"EQ(Omega=1)={eq[1.0].mean():.4f} ER-sum(w=1)={fx[1.0].mean():.4f} best fixed w={best_w}:{fx[best_w].mean():.4f} "
             f"(w_med of EQ={wmed:.3f}) | rel.gain vs best fixed {rel.mean():+.3f} (seeds>0: {int((rel>0).sum())}/{len(seeds)})")
    "
    f"vs ER-sum {rel_sum.mean():+.3f} | best Omega={best_om} landscape " + " ".join(f"{o}:{eq[o].mean():.3f}" for o in
    omega_grid))
    return passes, detail

def gate_EQ2(runner, _ev, meta, seeds=range(5), fixed_grid=(0.0625, 0.25, 0.5, 1.0, 2.0, 4.0),
            omega_grid=(0.5, 0.71, 1.0, 1.41, 2.0), margin=0.05):
    """EQ2 statistic (issue #20 §3): the rule at Omega = 1 is NOT BEHIND the tuned
    fixed weight by a step. The fixed weight is tuned ON THE SCORED SEEDS -- in-sample
    bias that favours the baseline, and that is the design intent: the criterion is
    one-sided, the rule need only not lose. A step is the relative margin `margin`
    (gate_EQ's); the rule is behind by a step iff mean((rule - tuned)/tuned) > margin,
    counted per seed and reported. fixed_grid is gate_EQ's grid extended to 0.0625:
    the 16x-stiff Laplace penalty moves the tuned weight below the replay family's,
    and R7 requires the baseline to be able to win. The Omega landscape and the
    reduction arm (fixed w at the rule's own median w, the constant it reduces to)
    are reported, not gated."""
    seeds = list(seeds)
    fixed_grid = tuple(meta.get("fixed_grid", fixed_grid)) # a row may widen the grid so its baseline can win (R7)
    eq = {om: np.array([runner("eq", om, s)["metric"] for s in seeds]) for om in omega_grid}
    wmed = float(np.median([runner("eq", 1.0, s)["w_med"] for s in seeds]))
    wm = round(wmed, 3)
    grid = tuple(sorted(set(fixed_grid) | {wm}))
    fx = {w: np.array([runner("fixed", w, s)["metric"] for s in seeds]) for w in grid}
    tuned_w = min(fx, key=lambda w: fx[w].mean())
    behind = (eq[1.0] - fx[tuned_w]) / fx[tuned_w] # >0: the rule is behind the tuned weight
    passes = bool(behind.mean() <= margin)
    best_om = min(eq, key=lambda o: eq[o].mean())
    detail = (f"EQ(Omega=1)={eq[1.0].mean():.4f} tuned w={tuned_w}:{fx[tuned_w].mean():.4f} "
             f"(w_med of EQ={wmed:.3f}) | behind-by {behind.mean():+.3f} (seeds>step: {int((behind >
    margin).sum())}/{len(seeds)}) "
             f"| best Omega={best_om} landscape " + " ".join(f"{o}:{eq[o].mean():.3f}" for o in omega_grid)
             + f" | reduction arm w={wm}:{fx[wm].mean():.4f}")
    return passes, detail
```

A.5 The figure script of this document

Continuous_Learning/build/make_figures.py

```
"""Figures for Continuous_Learning/CONTINUOUS_LEARNING.md (owner request 2026-09-22, prompt-log entry 72).
Every figure is drawn from pinned run records (runs/eqx, runs/eq2, runs/eq3) or from the functions of
theory/checks/omega_sweeps.py re-executed here (deterministic); every number placed on a figure that the document
quotes is also printed to stdout, which is pinned as Continuous_Learning/figures/figures.txt (R1).
Run: uv run python Continuous_Learning/build/make_figures.py > Continuous_Learning/figures/figures.txt
"""
from __future__ import annotations

import json
import math
import sys
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt # noqa: E402
from matplotlib.ticker import FixedFormatter, FixedLocator, NullFormatter # noqa: E402
import numpy as np # noqa: E402
```

```

ROOT = Path(__file__).resolve().parents[2]
OUT = ROOT / "Continuous_Learning" / "figures"
sys.path.insert(0, str(ROOT / "theory" / "checks"))
import omega_sweeps as om # noqa: E402 (the pinned mathematics script; functions only)

# validated palette (dataviz reference instance, light mode): three categorical slots, sequential blue, diverging blue/red
C1, C2, C3 = "#2a78d6", "#eb6834", "#1baf7a"
SURFACE, INK, INK2, GRID = "#fcfcfb", "#0b0b0b", "#52514e", "#6e65e1"
plt.rcParams.update({"figure.facecolor": SURFACE, "axes.facecolor": SURFACE, "axes.edgecolor": INK2, "axes.labelcolor": INK,
                    "xtick.color": INK2, "ytick.color": INK2, "text.color": INK, "axes.grid": True, "grid.color": GRID,
                    "grid.linewidth": 0.6, "axes.spines.top": False, "axes.spines.right": False, "font.size": 9,
                    "axes.titlesize": 10, "legend.frameon": False, "lines.linewidth": 2, "savefig.dpi": 200,
                    "xtick.minor.visible": False, "ytick.minor.visible": False})
OMEGA9 = om.OMEGA9

def logticks(ax, vals):
    ax.set_xscale("log"); ax.xaxis.set_major_locator(FixedLocator(list(vals)));
ax.xaxis.set_major_formatter(FixedFormatter(["{v:g}" for v in vals]))
ax.xaxis.set_minor_locator(FixedLocator([])); ax.xaxis.set_minor_formatter(NullFormatter())

def save(fig, name):
    fig.savefig(OUT / name, bbox_inches="tight"); plt.close(fig); print(f"[figure] {name}")

def records(path):
    rows = [json.loads(l) for l in open(path) if l.startswith("{")]
    return rows[0], [r for r in rows[1:] if "acc" in r]

# ----- F1: the tug of war (fifth-grader picture)
def fig1_tug():
    fig, axes = plt.subplots(3, 1, figsize=(7.5, 6.2))
    for ax, omg in zip(axes, (0.5, 1.0, 2.0)):
        gp, gq = 1.0, 3.0 # present pull 1, past pull 3 (raw)
        w = omg * gp / gq
        ax.annotate("", xy=(gp, 0.75), xytext=(0, 0.75), arrowprops=dict(arrowstyle="->", lw=3, color=C1))
        ax.annotate("", xy=(-gq, 0.45), xytext=(0, 0.45), arrowprops=dict(arrowstyle="<-", lw=1.2, color=C2, alpha=0.35))
        ax.annotate("", xy=(-w * gq, 0.15), xytext=(0, 0.15), arrowprops=dict(arrowstyle="<-", lw=3, color=C2))
        net = gp - w * gq
        ax.plot([0, 0], [0.0, 0.9], color=INK2, lw=0.8)
        ax.text(gp / 2, 0.83, "present pull |g_p| = 1", ha="center", color=C1, fontsize=8)
        ax.text(-gq / 2, 0.53, "past pull as it comes |g_q| = 3", ha="center", color=INK2, fontsize=8)
        ax.text(-w * gq / 2, 0.23, f"past pull after the rule w·|g_q| = Ω·|g_p| = {omg * gp:g}", ha="center", color=C2,
        fontsize=8)
        ax.text(-3.4, 0.85, f"Ω = {omg:g}", fontsize=11, color=INK, weight="bold")
        ax.text(2.2, 0.45, f"net pull = {net:+.1f}\n" + ("→ moves toward the NEW task" if net > 0 else ("→ stands still" if
        abs(net) < 1e-12 else "→ moves toward the OLD task")), fontsize=9, color=INK, va="center")
        ax.set_xlim(-3.6, 4.2); ax.set_ylim(0.0, 1.0); ax.axis("off")
        print(f"[F1] Omega {omg:g}: w = {w:.4f}, net pull {net:+.4f}")
        fig.suptitle("The rule rescales the past pull to Ω times the present pull. In one dimension the net pull is (1 - Ω) ×
        present pull.", x=0.01, ha="left", fontsize=10)
        save(fig, "F01_tug_of_war.png")

# ----- F2/F3: the Pareto curve, the rule's field and its
trajectories (2-D quadratic model)
def model2d():
    H = np.diag([1.0, 0.5]); F = 16 * np.diag([1.0, 1.5]); a = np.array([2.0, 1.0]); b = np.array([0.0, 0.0])
    return H, F, a, b

def fig2_field():
    H, F, a, b = model2d()
    lam = np.logspace(-3, 3, 400); curve = np.array([om.pareto(H, F, a, b, l) for l in lam])
    xs = np.linspace(-0.6, 2.6, 27); ys = np.linspace(-0.6, 1.6, 23); X, Y = np.meshgrid(xs, ys)
    fig, axes = plt.subplots(1, 3, figsize=(10, 3.4), sharey=True)
    for ax, omg in zip(axes, (0.7, 1.0, 1.4)):
        Lp = np.array([om.losses(H, F, a, b, np.array([x, y]))[0] for x in xs] for y in ys])
        Lq = np.array([om.losses(H, F, a, b, np.array([x, y]))[1] for x in xs] for y in ys])
        ax.contour(X, Y, Lp, levels=8, colors=C1, linewidths=0.6, alpha=0.6); ax.contour(X, Y, Lq, levels=8, colors=C2,
        linewidths=0.6, alpha=0.6)
        U = np.zeros_like(X); V = np.zeros_like(Y)
        for i in range(len(ys)):
            for j in range(len(xs)):
                th = np.array([xs[j], ys[i]]); gp = H @ (th - a); gq = F @ (th - b)
                upd = -(gp + omg * np.linalg.norm(gp) / max(np.linalg.norm(gp), 1e-12) * gq); U[i, j], V[i, j] = upd
        ax.quiver(X, Y, U, V, color=INK2, alpha=0.7, scale=40, width=0.003)
        ax.plot(curve[:, 0], curve[:, 1], color=C3, lw=2.5, label="Pareto curve θ(λ)")
        ax.plot(*a, "o", color=C1, ms=8); ax.plot(*b, "o", color=C2, ms=8)
        ax.text(a[0] - 0.05, a[1] + 0.12, "new-task optimum a", color=C1, fontsize=8, ha="right"); ax.text(b[0] + 0.08, b[1] -
        0.2, "old-task optimum b", color=C2, fontsize=8)
        ax.set_title(f"Ω = {omg:g}", loc="left"); ax.set_xlabel("θ₁")
        # update norm on the curve
        rel = []
        for l in (0.1, 1.0, 10.0):
            th = om.pareto(H, F, a, b, l); gp = H @ (th - a); gq = F @ (th - b)
            rel.append(np.linalg.norm(gp + omg * np.linalg.norm(gp) / np.linalg.norm(gq) * gq) / np.linalg.norm(gp))
        print(f"[F2] Omega {omg:g}: |update|/|g_p| on the curve at lambda 0.1, 1, 10 = {rel[0]:.6f}, {rel[1]:.6f},

```

```

{rel[2]:.6f}")
    axes[0].set_ylabel("02"); axes[1].legend(loc="upper left", fontsize=8)
    fig.suptitle("Update field of  $g_p + \Omega \cdot (|g_p|/|g_q|) \cdot g_q$  in a two-dimensional two-task model (blue contours: new-task loss; orange: old-task penalty).\nOn the green curve the two pulls are antiparallel; at  $\Omega = 1$  the update vanishes everywhere on it.", x=0.01, ha="left", fontsize=9)
    fig.tight_layout(rect=(0, 0, 1, 0.93)); save(fig, "F02_pareto_field.png")

def fig3_trajectories():
    H, F, a, b = model2d()
    lam = np.logspace(-3, 3, 400); curve = np.array([om.pareto(H, F, a, b, l) for l in lam])
    fig, axes = plt.subplots(1, 2, figsize=(10, 4), sharey=True)
    cols = plt.cm.Blues(np.linspace(0.35, 0.95, len(OMEGA9)))
    for ax, sm, title in zip(axes, (0.0, om.SMOOTH), ("exact rule (instantaneous norms)", "registered estimator (EMA 0.9 of the gradients, cap le4)")):
        ax.plot(curve[:, 0], curve[:, 1], color=C3, lw=2.5, label="Pareto curve")
        ax.plot(*a, "o", color=C1, ms=8, label="new-task optimum a"); ax.plot(*b, "o", color=C2, ms=8, label="old-task optimum b (start)")
        for omg, c in zip(OMEGA9, cols):
            th = b.copy(); ema_p = ema_q = None; path = [th.copy()]
            for _ in range(1500):
                gp = H @ (th - a); gq = F @ (th - b)
                ema_p = gp if (ema_p is None or sm == 0) else sm * ema_p + (1 - sm) * gp
                ema_q = gq if (ema_q is None or sm == 0) else sm * ema_q + (1 - sm) * gq
                w = om.rule_w(ema_p, ema_q, omg); th = th - 0.02 * (gp + w * gq); path.append(th.copy())
            path = np.array(path)
            ax.plot(path[:, 0], path[:, 1], color=c, lw=1.0, alpha=0.85, label=f" $\Omega = \{omg:g\}$ ")
            ax.plot(path[-1, 0], path[-1, 1], "s", color=c, ms=6, clip_on=True)
            Lp, Lq = om.losses(H, F, a, b, path[-1]); le, dist = om.lam_eff(H, F, a, b, path[-1])
            print(f"[F3] {title}: Omega {omg:g} endpoint ({path[-1, 0]:.4f}, {path[-1, 1]:.4f}) L_present {Lp:.4f} L_past {Lq:.4f} lambda_eff {le:.4g} distance to curve {dist:.2e}")
            ax.set_title(title, loc="left"); ax.set_xlabel("01"); ax.set_xlim(-0.4, 2.4); ax.set_ylim(-0.4, 1.4)
            axes[0].set_ylabel("02"); axes[1].legend(loc="upper left", fontsize=7, ncol=2, title="paths by  $\Omega$  (squares: endpoints)", title_fontsize=7)
    fig.suptitle("Where the rule stops, starting from the old-task optimum, for nine values of  $\Omega$  (1500 steps, lr 0.02); light = small  $\Omega$ , dark = large  $\Omega$ ", x=0.01, ha="left", fontsize=10)
    save(fig, "F03_trajectories.png")

# ----- F4: the plateau under noise (5-D model of omega_sweeps.py)
def fig4_plateau():
    H, F, a, b = om.make_model()
    fig, ax = plt.subplots(figsize=(6.5, 3.6))
    for noise, c, lab in ((0.0, C1, "noise 0"), (0.5, C2, "noise 0.5"), (2.0, C3, "noise 2")):
        tot = []
        for omg in OMEGA9:
            vals = []
            for s in range(5):
                th, _ = om.gd(H, F, a, b, "eq", omg, lr=0.05, noise=noise, seed=s)
                Lp, Lq = om.losses(H, F, a, b, th); vals.append(Lp + Lq if ok else np.nan)
            tot.append(float(np.mean(vals)))
        ax.plot(OMEGA9, tot, "-o", color=c, ms=5, label=lab); ax.text(OMEGA9[-1] * 1.05, tot[-1], lab, color=c, fontsize=8, va="center")
        print(f"[F4] {lab}: total loss per Omega " + " ".join(f"{o:g}:{t:.3f}" for o, t in zip(OMEGA9, tot)))
    logticks(ax, OMEGA9); ax.set_xlim(0.2, 5.5)
    ax.set_xlabel(" $\Omega$ "); ax.set_ylabel("L_present + L_past at the end (5 seeds, mean)")
    ax.set_title("Registered estimator: mini-batch noise turns the knife edge at  $\Omega = 1$  into a plateau", loc="left")
    save(fig, "F04_plateau.png")

# ----- F5: the stability edge of a fixed weight vs the rule
def fig5_edge():
    H, F, a, b = om.make_model(); lr = 0.05
    lam_edge = (2.0 / lr - np.diag(H).max()) / np.diag(F).max()
    lams = np.logspace(-2, 1.2, 40); tot = []; ok_ = []
    for l in lams:
        th, _ = om.gd(H, F, a, b, "fixed", l, lr=lr); Lp, Lq = om.losses(H, F, a, b, th); tot.append(Lp + Lq if ok else np.nan); ok_.append(ok)
    fig, ax = plt.subplots(figsize=(6.5, 3.6))
    ax.plot(lams, tot, "-", color=C1, label="fixed weight  $\lambda$  (converged)")
    div = [l for l, o in zip(lams, ok_) if not o]
    if div:
        ax.axvspan(min(div), lams[-1], color=C2, alpha=0.12); ax.text(min(div) * 1.05, np.nanmax(tot) * 0.9, "fixed  $\lambda$  diverges", color=C2, fontsize=8)
        ax.axvline(lam_edge, color=C2, ls="--", lw=1.2); ax.text(lam_edge * 1.05, np.nanmax(tot) * 0.55, f"edge  $\lambda^* = \{lam\_edge:.3f\}", color=C2, fontsize=8)
        th, wlog, _ = om.gd(H, F, a, b, "eq", 1.0, lr=lr); Lp, Lq = om.losses(H, F, a, b, th); le, _ = om.lam_eff(H, F, a, b, th)
        ax.plot([le], [Lp + Lq], "s", color=C3, ms=9, label="rule at  $\Omega = 1$  (registered estimator)")
        ax.text(le * 1.1, Lp + Lq, f"rule stops at  $\lambda\_eff = \{le:.3g\}$ , median w = {np.median(wlog):.3g}", color=C3, fontsize=8)
        print(f"[F5] edge lambda* = {lam_edge:.4f}; fixed converged for lambda <= {max([l for l, o in zip(lams, ok_) if o]):.4f}; first divergent lambda {min(div) if div else float('nan'):.4f}; rule Omega=1 lambda_eff {le:.4g} total loss {Lp + Lq:.4f} median w {np.median(wlog):.4g}")
        ax.set_xscale("log"); ax.set_xlabel("penalty weight  $\lambda$ "); ax.set_ylabel("L_present + L_past at the end");
    ax.legend(loc="center right", fontsize=8)
    ax.set_title("A fixed weight has a stability edge at  $lr \cdot (h_{\max} + \lambda \cdot f_{\max}) = 2$ ; the rule's step is bounded by the present step", loc="left")
    save(fig, "F05_stability_edge.png")

# ----- F6: Kalman tuned vs fixed$ 
```

```

def fig6_kalman():
    vs = np.logspace(-2, 2, 200); k_phi = om.kalman_gain(1.0)
    r_phi = [om.mse_fixed_gain(k_phi, v * v, 1.0) / om.mse_fixed_gain(om.kalman_gain(v), v * v, 1.0) for v in vs]
    r_half = [om.mse_fixed_gain(0.5, v * v, 1.0) / om.mse_fixed_gain(om.kalman_gain(v), v * v, 1.0) for v in vs]
    fig, ax = plt.subplots(figsize=(6.5, 3.6))
    ax.plot(vs, r_phi, color=C1, label="fixed K = 1/φ (Ω = 1 as Fisher speed 1)"); ax.plot(vs, r_half, color=C2, label="fixed K = 1/2 (Ω = 1 as equal pull)")
    ax.axhline(1.0, color=C3, lw=1.5, ls="--"); ax.text(0.011, 1.45, "tuned Kalman gain K(v): ratio 1 (Bayes-optimal)", color=C3, fontsize=8)
    ax.axhline(1.05, color=INK2, lw=0.6, ls=":"); ax.text(30, 1.06, "5 %", color=INK2, fontsize=7)
    ax.set_xscale("log"); ax.set_yscale("log"); ax.set_xlabel("v = √(q/r), the Fisher speed of the random walk");
    ax.set_ylabel("steady-state MSE, fixed / tuned")
    ax.legend(loc="upper center", fontsize=8); ax.set_title("A fixed gain from Ω = 1 matches the tuned filter at one speed only", loc="left")
    for v in (0.1, 1.0, 10.0):
        print(f"[F6] v = {v:g}: ratio 1/phi {om.mse_fixed_gain(k_phi, v * v, 1.0) / om.mse_fixed_gain(om.kalman_gain(v), v * v, 1.0):.4f}, ratio 1/2 {om.mse_fixed_gain(0.5, v * v, 1.0) / om.mse_fixed_gain(om.kalman_gain(v), v * v, 1.0):.4f}")
    save(fig, "F06_kalman.png")

# ----- study data: EQ2 + EQ3
def load_studies():
    S = {}
    for study, names in (("EQ2", ("mfeat_fourier", "mfeat_pixel", "texture")), ("EQ3", ("fars", "krkopt", "led24", "led7", "mfeat_factors", "mfeat_morphological"))):
        for n in names:
            hdr, rows = records(ROOT / "runs" / study.lower() / f"results_{n}.jsonl")
            S[(study, n)] = rows
    return S

def sel(rows, method, mode, value=None, regime="std", smooth=0.9, cap=1e4, main=True):
    v = [r for r in rows if r["method"] == method and r["mode"] == mode and r["regime"] == regime and r["smooth"] == smooth and r["wcap"] == cap
        and (value is None or abs(r["value"] - value) < 1e-6)
        and (not main or (r.get("hidden", r.get("hid", 256)) == 256 and r.get("epochs", 3) == 3 and r.get("lr", 0.05) == 0.05 and r.get("bs", 10) == 10)]]
    return sorted(v, key=lambda r: r["seed"])

def tuned(rows):
    ws = sorted(set(r["value"] for r in sel(rows, "ewc", "fixed")))
    means = {w: np.mean([r["acc"] for r in sel(rows, "ewc", "fixed", w)]) for w in ws}
    tw = max(means, key=means.get); tv = np.array([r["acc"] for r in sel(rows, "ewc", "fixed", tw)])
    step = max(1.0, 2 * tv.std(ddof=1) / math.sqrt(len(tv)))
    return means, tw, tv, step

def fig7_omega_profiles(S):
    keys = list(S.keys())
    fig, axes = plt.subplots(3, 3, figsize=(10, 8.5)); axes = axes.ravel()
    for ax, k in zip(axes, keys):
        rows = S[k]; means, tw, tv, step = tuned(rows)
        oms = sorted(set(r["value"] for r in sel(rows, "ewc", "eq")))
        m = [np.mean([r["acc"] for r in sel(rows, "ewc", "eq", o)]) for o in oms]
        for o in oms:
            for r in sel(rows, "ewc", "eq", o): ax.plot(o, r["acc"], ".", color=C1, alpha=0.35, ms=5)
        ax.plot(oms, m, "-o", color=C1, ms=5, label="rule, mean of 5 seeds")
        ax.axhspan(tv.mean() - step, tv.mean() + step, color=C2, alpha=0.12)
        ax.axhline(tv.mean(), color=C2, ls="--", lw=1.4, label=f"tuned λ = {tw:g} (± one step)")
        logticks(ax, oms); ax.tick_params(axis="x", labelsize=7)
        ax.set_title(f"{k[0]} {k[1]}", loc="left"); ax.set_xlabel("Ω"); ax.set_ylabel("final accuracy (%)")
        print(f"[F7] {k[0]} {k[1]}: tuned lambda {tw:g} mean {tv.mean():.4f} step {step:.4f}; rule per Omega " + "
        ".join(f"{o:g}:{mm:.4f}" for o, mm in zip(oms, m)))
        axes[0].legend(loc="lower right", fontsize=7)
    fig.suptitle("Accuracy of the rule against Ω on nine unseen carriers (EQ2: five-point grid; EQ3: nine-point grid); dashed = in-sample-tuned fixed λ, band = one resolvable step", x=0.01, ha="left", fontsize=10)
    fig.tight_layout(); save(fig, "F07_omega_profiles.png")

def fig8_lambda_grids(S):
    keys = list(S.keys())
    fig, axes = plt.subplots(3, 3, figsize=(10, 8.5)); axes = axes.ravel()
    for ax, k in zip(axes, keys):
        rows = S[k]; means, tw, tv, step = tuned(rows)
        ws = sorted(means); ax.plot(ws, [means[w] for w in ws], "-o", color=C1, ms=4, label="fixed λ grid, mean of 5 seeds")
        rule = sel(rows, "ewc", "eq", 1.0); wm = np.median([r["w_med"] for r in rule]); racc = np.mean([r["acc"] for r in rule])
        ax.axhline(tw, color=C2, ls="--", lw=1.2); ax.text(tw, ax.get_ylim()[1] * 0.98 if ax.get_ylim()[1] > 0 else 1, f"tuned {tw:g}", color=C2, fontsize=7, rotation=90, va="top", ha="right")
        ax.axhline(wm, color=C3, ls=".", lw=1.6); ax.text(wm, max(means.values()) * 0.5, f"rule's median w = {wm:.3g}", color=C3, fontsize=7, rotation=90, va="center", ha="right")
        ax.plot([wm], [racc], "s", color=C3, ms=7, label="rule at Ω = 1 (accuracy, at its median w)")
        ax.set_xscale("log"); ax.set_title(f"{k[0]} {k[1]}", loc="left"); ax.set_xlabel("penalty weight λ (fixed)");
        ax.set_ylabel("final accuracy (%)")
        edge = [w for w in ws if means[w] < 0.5 * max(means.values()) and w > tw]
        print(f"[F8] {k[0]} {k[1]}: tuned {tw:g} ({tv.mean():.4f}); rule median w {wm:.4g} accuracy {racc:.4f}; first fixed lambda above tuned with mean under half the best: {edge[0] if edge else 'none'}")
        axes[0].legend(loc="lower left", fontsize=7)
    fig.suptitle("The fixed-λ grid collapses beyond a stability edge; the rule's derived weight sits at or beyond it without collapsing", x=0.01, ha="left", fontsize=10)

```

```

fig.tight_layout(); save(fig, "F08_lambda_grids.png")

def fig9_sensitivity(S):
    keys = list(S.keys()); caps = (10.0, 100.0, 1e4); sms = (0.8, 0.9, 0.98)
    fig, axes = plt.subplots(3, 3, figsize=(9, 7.5)); axes = axes.ravel()
    for ax, k in zip(axes, keys):
        rows = S[k]; means, tw, tv, step = tuned(rows)
        M = np.array([[np.mean([r["acc"] for r in sel(rows, "ewc", "eq", 1.0, smooth=s, cap=c)]) - tv.mean() for s in sms] for c
in caps])
        lim = max(4.0, np.abs(M).max())
        im = ax.imshow(M, cmap=matplotlib.colors.LinearSegmentedColormap.from_list("div", ["#d03b3b", "#f0efec", "#2a78d6"]),
vmin=-lim, vmax=lim)
        for i in range(3):
            for j in range(3):
                ax.text(j, i, f"M[{i}, {j}]:+.2f", ha="center", va="center", fontsize=8, color=INK)
                ax.set_xticks(range(3)); ax.set_xticklabels([f"smooth {s}" for s in sms], fontsize=7); ax.set_yticks(range(3));
ax.set_yticklabels([f"cap {c:g}" for c in caps], fontsize=7)
                ax.set_title(f"[k[0]] {k[1]} (step {step:.2f})", loc="left"); ax.grid(False)
                print(f"[F9] {k[0]} {k[1]}: Omega=1 - tuned by cap x smooth " + "; ".join(f"cap{c:g}/sm{s}:{M[i, j]:+.2f}" for i, c in
enumerate(caps) for j, s in enumerate(sms)))
            fig.suptitle("Sensitivity: rule at  $\Omega = 1$  minus tuned  $\lambda$ , per cap  $\times$  smoothing cell (blue = rule ahead, red = behind)", x=0.01,
ha="left", fontsize=10)
        fig.tight_layout(); save(fig, "F09_sensitivity.png")

def fig10_eq3_arms(S):
    names = ["fars", "krkopt", "led24", "led7", "mfeat_factors", "mfeat_morphological"]
    fig, ax = plt.subplots(figsize=(9, 3.8)); x = np.arange(len(names)); wdt = 0.26
    for off, (meth, mode, val, c, lab) in enumerate(((("ewc", "eq", 1.0, C1, "rule at  $\Omega = 1$ "), ("ewc", "bayes", 0.0, C2, "Bayes
(Laplace 1/2) arm"), ("ewc", "reduction", None, C3, "reduction arm (fixed w = rule's median w)"))):
        d = []
        for n in names:
            rows = S[("EQ3", n)]; means, tw, tv, step = tuned(rows)
            d.append(np.mean([r["acc"] for r in sel(rows, meth, mode, val)]) - tv.mean())
        ax.bar(x + (off - 1) * wdt, d, wdt, color=c, label=lab)
        for xi, di in zip(x, d): ax.text(xi + (off - 1) * wdt, di + (0.6 if di >= 0 else -1.8), f"{di:+.1f}", ha="center",
fontsize=7, color=INK)
        print(f"[F10] {lab}: minus tuned lambda per carrier " + ", ".join(f"{n}:{di:+.4f}" for n, di in zip(names, d)))
        ax.axhline(0, color=INK2, lw=0.8); ax.set_xticks(x); ax.set_xticklabels(names, fontsize=8); ax.set_ylabel("accuracy minus
the tuned  $\lambda$  (points)")
        ax.legend(loc="lower left", fontsize=8); ax.set_title("EQ3: the rule, the Bayes weight and the rule's own constant, each
against the in-sample-tuned  $\lambda$ ", loc="left")
        save(fig, "F10_eq3_arms.png")

def fig11_eqx():
    fig, axes = plt.subplots(1, 3, figsize=(10, 3.4))
    for ax, n in zip(axes, ("optdigits", "pendigits", "letter")):
        hdr, rows = records(R00T / "runs" / "eqx" / f"{n}.jsonl")
        fx = sorted(set(r["value"] for r in rows if r["method"] == "fixed")); fm = [100 * np.mean([r["acc"] for r in rows if
r["method"] == "fixed" and r["value"] == w]) for w in fx]
        ax.plot(fx, fm, "-o", color=C2, ms=4, label="fixed replay weight w")
        for metric, c, ls in (("fisher", C1, "-"), ("euclid", C3, "--")):
            oms = sorted(set(r["value"] for r in rows if r["method"] == "eq" and r.get("metric") == metric))
            if not oms: continue
            m = [100 * np.mean([r["acc"] for r in rows if r["method"] == "eq" and r.get("metric") == metric and r["value"] ==
o]) for o in oms]
            ax.plot(oms, m, ls, marker="s", color=c, ms=4, label=f"rule,  $\Omega$  grid ({metric} norm)")
            print(f"[F11] {n} {metric}: rule per Omega " + " ".join(f"{o:g}:{mm:.4f}" for o, mm in zip(oms, m)))
            print(f"[F11] {n} fixed: " + " ".join(f"{w:g}:{mm:.4f}" for w, mm in zip(fx, fm)))
            logticks(ax, sorted(set(fx) | {0.5, 0.71, 1.0, 1.41, 2.0})); ax.set_title(f"EQX {n}", loc="left"); ax.set_xlabel("w
(fixed) or  $\Omega$  (rule)"); ax.set_ylabel("final accuracy (%)")
        axes[0].legend(loc="lower right", fontsize=7)
        fig.suptitle("EQX (replay, ER-sum): the rule at  $\Omega = 1$  is level with or behind the best fixed replay weight on 3/3 carriers;
it reduces to a constant", x=0.01, ha="left", fontsize=10)
        fig.tight_layout(); save(fig, "F11_eqx_reduction.png")

def fig12_bayes_line():
    fig, ax = plt.subplots(figsize=(8, 2.6)); m_p, m_q = 0.0, 1.0
    for ratio, y in ((0.25, 0.8), (1.0, 0.5), (4.0, 0.2)):
        th_B = (100 * m_p + 100 * ratio * m_q) / (100 + 100 * ratio)
        ax.plot([m_p, m_q], [y, y], color=C3, lw=6, alpha=0.35, solid_capstyle="butt")
        ax.plot([th_B], [y], "D", color=C2, ms=9); ax.text(th_B, y + 0.07, f"Bayes  $\theta_B = \{th_B:.2f\}$ ", ha="center", fontsize=8,
color=C2)
        ax.text(-0.02, y, f"n_past/n_present = {ratio:g}", ha="right", va="center", fontsize=8, color=INK)
        print(f"[F12] ratio {ratio:g}: theta_B {th_B:.4f}; Omega=1 fixed-point interval [{m_p:g}, {m_q:g}]")
        ax.plot([m_p], [-0.05], "o", color=C1, ms=8); ax.text(m_p, -0.2, "present mean m_p ( $\Omega < 1$  ends here)", ha="center",
fontsize=8, color=C1)
        ax.plot([m_q], [-0.05], "o", color=C1, ms=8); ax.text(m_q, -0.2, "past mean m_q ( $\Omega > 1$  ends here)", ha="center", fontsize=8,
color=C1)
        ax.text(0.5, 0.98, "green band: every point is a fixed point of the rule at  $\Omega = 1$  (it stays where it starts)", ha="center",
fontsize=8.5, color=C3)
        ax.set_xlim(-0.55, 1.2); ax.set_ylim(-0.32, 1.08); ax.axis("off")
        ax.set_title("One-dimensional Bayes: the posterior mode is one point; the rule at  $\Omega = 1$  stops anywhere on the segment",
loc="left", fontsize=10)
        save(fig, "F12_bayes_line.png")

```

```
def main():
    OUT.mkdir(parents=True, exist_ok=True)
    print("Continuous_Learning figures (deterministic; numbers drawn on the figures)")
    fig1_tug(); fig2_field(); fig3_trajectories(); fig4_plateau(); fig5_edge(); fig6_kalman()
    S = load_studies(); fig7_omega_profiles(S); fig8_lambda_grids(S); fig9_sensitivity(S); fig10_eq3_arms(S); fig11_eqx();
    fig12_bayes_line()
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

A.6 The document builder

Continuous_Learning/build/build_pdf.py

```
"""Build Continuous_Learning/Continuous_Learning.pdf from CONTINUOUS_LEARNING.md (a small Markdown subset) with fpdf2.
Supported: '#' headings; paragraphs with bold, italic, `code`; '- ' bullets; pipe tables; fenced code
blocks (``` ... ```), or ```include:PATH[:START-END] to embed a repository file); '$$ ... $$' display mathematics on one line
(rendered with matplotlib mathtext); '>' callouts (the fifth-grader boxes); '![caption](path)' figures. Fonts: DejaVu
from matplotlib's bundled set. Run: uv run python Continuous_Learning/build/build_pdf.py
"""

from __future__ import annotations

import datetime as dt
import hashlib
import html
import re
import sys
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt # noqa: E402
from fpdf import FPDF # noqa: E402
from fpdf.fonts import FontFace # noqa: E402

ROOT = Path(__file__).resolve().parents[2]
DOC_DIR = ROOT / "Continuous_Learning"
MD = DOC_DIR / "CONTINUOUS_LEARNING.md"
OUT = DOC_DIR / "Continuous_Learning.pdf"
MATH_DIR = DOC_DIR / "build" / "math"
FONT_DIR = Path(matplotlib.get_data_path()) / "fonts" / "ttf"
INK = (11, 11, 11); INK2 = (82, 81, 78); BOX = (234, 242, 252); CODEBG = (246, 246, 244); RULE = (200, 199, 193)

class Doc(FPDF):
    def __init__(self):
        super().__init__(orientation="P", unit="mm", format="A4")
        self.set_margins(18, 16, 18); self.set_auto_page_break(True, margin=18)
        self.add_font("DV", "", str(FONT_DIR / "DejaVuSans.ttf")); self.add_font("DV", "B", str(FONT_DIR /
"DejaVuSans-Bold.ttf"))
        self.add_font("DV", "I", str(FONT_DIR / "DejaVuSans-Oblique.ttf")); self.add_font("DV", "BI", str(FONT_DIR /
"DejaVuSans-BoldOblique.ttf"))
        self.add_font("DVM", "", str(FONT_DIR / "DejaVuSansMono.ttf")); self.add_font("DVM", "B", str(FONT_DIR /
"DejaVuSansMono-Bold.ttf"))
        self.set_font("DV", "", 9.5); self.set_text_color(*INK)
        self.set_creation_date(dt.datetime(2026, 9, 22, tzinfo=dt.timezone.utc)) # a fixed date so two builds are
byte-identical (R9)
        self.set_title("Continuous Learning: the  $\Omega = 1$  normalised penalty step"); self.set_author("CRR re-validation
repository")

    def footer(self):
        self.set_y(-12); self.set_font("DV", "", 7.5); self.set_text_color(*INK2)
        self.cell(0, 6, f"Continuous Learning – the  $\Omega = 1$  rule, CRR re-validation repository, 2026-09-22 – page
{self.page_no()}", align="C")
        self.set_text_color(*INK)

    @property
    def usable(self):
        return self.w - self.l_margin - self.r_margin

def inline(text: str) -> str:
    """Markdown inline -> the HTML subset fpdf2's write_html understands."""
    t = html.escape(text, quote=False)
    t = re.sub(r"\".*\"(.+?)\".*\"", r"<b>\1</b>", t)
    t = re.sub(r"(?![\w*])\".*\"(.+?)\".*\"(?![\s])\".*\"(?![\w])\"", r"<i>\1</i>", t)
    t = re.sub(r"\".*\"(.+?)\".*\"", r"<code>\1</code>", t)
    t = re.sub(r"\".*\"(.+?)\".*\"((https?:[/\^]+)\)", r"<a href=\"\2\">\1</a>", t)
    return t

def render_math(expr: str, size: float = 12) -> Path:
    MATH_DIR.mkdir(parents=True, exist_ok=True)
    key = hashlib.sha256(expr.encode()).hexdigest()[:16]; out = MATH_DIR / f"m_{key}.png"
    if not out.exists():
        fig = plt.figure(figsize=(0.01, 0.01), facecolor="white")
        fig.text(0, 0, f"${expr}$", fontsize=size, color="black")
```

```

fig.savefig(out, dpi=300, bbox_inches="tight", pad_inches=0.04, facecolor="white"); plt.close(fig)
return out

def paragraph(pdf: Doc, text: str, size=9.5, color=INK):
    pdf.set_font("DV", "", size); pdf.set_text_color(*color)
    pdf.write_html(f"<p>{inline(text)}</p>", font_family="DV", tag_styles={"code": FontFace(family="DVM")}); pdf.ln(1.2);
pdf.set_text_color(*INK)

def heading(pdf: Doc, level: int, text: str):
    sizes = {1: 17, 2: 13, 3: 10.5}
    if level == 1 and pdf.page_no() > 1:
        pdf.add_page()
    pdf.ln(3 if level > 1 else 1); pdf.set_font("DV", "B", sizes[level]); pdf.set_text_color(*INK)
    pdf.multi_cell(0, sizes[level] * 0.5, text, new_x="LMARGIN", new_y="NEXT"); pdf.ln(1.5)
    if level == 1:
        pdf.set_draw_color(*RULE); pdf.line(pdf.l_margin, pdf.get_y(), pdf.l_margin + pdf.usable, pdf.get_y()); pdf.ln(2.5)

def bullets(pdf: Doc, items: list[str]):
    pdf.set_font("DV", "", 9.5)
    pdf.write_html("<ul>" + "".join(f"<li>{inline(i)}</li>" for i in items) + "</ul>", font_family="DV", tag_styles={"code":
FontFace(family="DVM")}); pdf.ln(1)

def callout(pdf: Doc, text: str):
    pdf.set_font("DV", "", 9.2); x = pdf.l_margin; w = pdf.usable - 6
    lines = pdf.multi_cell(w, 4.6, "In plain words. " + text, dry_run=True, output="LINES")
    hgt = 4.6 * len(lines) + 4
    if pdf.get_y() + hgt > pdf.h - pdf.b_margin: pdf.add_page()
    start = pdf.get_y(); pdf.set_fill_color(*BOX); pdf.rect(x, start, pdf.usable, hgt, style="F")
    pdf.set_xy(x + 3, start + 2); pdf.set_font("DV", "B", 9.2); pdf.write(4.6, "In plain words. "); pdf.set_font("DV", "", 9.2)
    pdf.write(4.6, text); pdf.set_y(start + hgt + 2)

def table(pdf: Doc, rows: list[list[str]]):
    ncol = max(len(r) for r in rows); rows = [[re.sub(r"\\*(.+)\\*", r"\\1", c) for c in r] + [""] * (ncol - len(r)) for r in
rows]
    size = 7.4
    while True:
        pdf.set_font("DV", "", size)
        # a column can never be narrower than its longest unbreakable word
        min_w = [max(pdf.get_string_width(max(c.split() or [""], key=len)) for c in (r[j] for r in rows)) + 2.2 for j in
range(ncol)]
        if sum(min_w) <= pdf.usable or size <= 5.5:
            break
        size -= 0.4
    tot_len = [sum(min(len(r[j]), 80) for r in rows) for j in range(ncol)]
    spare = max(pdf.usable - sum(min_w), 0.0); tl = sum(tot_len) or 1
    widths = [m + spare * t / tl for m, t in zip(min_w, tot_len)]
    scale = pdf.usable / sum(widths); widths = [w * scale for w in widths]
    with pdf.table(col_widths=widths, text_align="LEFT", line_height=size * 0.5, padding=0.8, borders_layout="HORIZONTAL_LINES",
        headings_style=FontFace(emphasis="BOLD", fill_color=(236, 235, 231))) as t:
        for r in rows:
            row = t.row()
            for c in r:
                row.cell(c)
    pdf.ln(2)

def code(pdf: Doc, lines: list[str], title: str | None = None):
    if title:
        pdf.set_font("DV", "B", 8.5); pdf.multi_cell(0, 4.5, title, new_x="LMARGIN", new_y="NEXT"); pdf.ln(0.5)
    pdf.set_font("DVM", "", 6.3); pdf.set_fill_color(*CODEBG)
    for ln in lines:
        pdf.multi_cell(0, 2.9, ln.rstrip("\\n").replace("\\t", "    ") or " ", fill=True, new_x="LMARGIN", new_y="NEXT")
    pdf.ln(2)

def figure(pdf: Doc, path: Path, caption: str):
    if not path.exists():
        raise FileNotFoundError(path)
    from PIL import Image
    with Image.open(path) as im: w, h = im.size
    width = pdf.usable; height = width * h / w
    if height > 150: height = 150; width = height * w / h
    if pdf.get_y() + height + 14 > pdf.h - pdf.b_margin: pdf.add_page()
    pdf.image(str(path), x=pdf.l_margin + (pdf.usable - width) / 2, w=width); pdf.ln(1.5)
    pdf.set_font("DV", "I", 8.2); pdf.set_text_color(*INK2); pdf.write_html(f"<p>{inline(caption)}</p>", font_family="DV",
tag_styles={"code": FontFace(family="DVM")}); pdf.set_text_color(*INK); pdf.ln(2)

def display_math(pdf: Doc, expr: str):
    png = render_math(expr)
    from PIL import Image
    with Image.open(png) as im: w, h = im.size
    width_mm = w / 300 * 25.4; height_mm = h / 300 * 25.4
    if width_mm > pdf.usable: height_mm *= pdf.usable / width_mm; width_mm = pdf.usable
    if pdf.get_y() + height_mm + 6 > pdf.h - pdf.b_margin: pdf.add_page()

```

```

pdf.image(str(png), x=pdf.l_margin + (pdf.usable - width_mm) / 2, w=width_mm); pdf.ln(2.5)

def include(spec: str) -> tuple[list[str], str]:
    path, _, rng = spec.partition(":")
    lines = (ROOT / path).read_text().splitlines()
    if rng:
        a, b = rng.split("-"); lines = lines[int(a) - 1:int(b)]
        return lines, f"{path} (lines {a}-{b})"
    return lines, path

def build():
    pdf = Doc(); pdf.add_page()
    src = MD.read_text().splitlines(); i = 0; n = len(src)
    while i < n:
        line = src[i]
        if not line.strip():
            i += 1; continue
        if line.startswith("```"):
            spec = line[3:].strip(); i += 1
            if spec.startswith("include:"):
                lines, title = include(spec[len("include:"):]); code(pdf, lines, title)
                while i < n and not src[i].startswith("```"): i += 1
                i += 1; continue
            buf = []
            while i < n and not src[i].startswith("```"): buf.append(src[i]); i += 1
            i += 1; code(pdf, buf); continue
        if line.startswith("$"):
            expr = line.strip().strip("$").strip(); display_math(pdf, expr); i += 1; continue
        if line.startswith("### "): heading(pdf, 3, line[4:].strip()); i += 1; continue
        if line.startswith("## "): heading(pdf, 2, line[3:].strip()); i += 1; continue
        if line.startswith("# "): heading(pdf, 1, line[2:].strip()); i += 1; continue
        if line.startswith("!["):
            m = re.match(r"![(.*)\\((.*)\\)", line.strip()); figure(pdf, DOC_DIR / m.group(2), m.group(1)); i += 1; continue
        if line.startswith("> "):
            buf = []
            while i < n and src[i].startswith("> "): buf.append(src[i][2:]); i += 1
            callout(pdf, " ".join(buf)); continue
        if line.startswith("- "):
            items = []
            while i < n and src[i].startswith("- "): items.append(src[i][2:]); i += 1
            bullets(pdf, items); continue
        if line.startswith("|"):
            rows = []
            while i < n and src[i].startswith("|"):
                cells = [c.strip() for c in src[i].strip().strip("|").split("|")]
                if not all(re.fullmatch(r"?-?{2,}? ", c) for c in cells): rows.append(cells)
                i += 1
            table(pdf, rows); continue
        buf = []
        while i < n and src[i].strip() and not re.match(r"^(#|\\| - > |!|'|`|\\$\\$)", src[i]): buf.append(src[i].strip()); i += 1
    1 paragraph(pdf, " ".join(buf))
    pdf.output(str(OUT))
    print(f"wrote {OUT.relative_to(ROOT)}: {pdf.page_no()} pages, {OUT.stat().st_size} bytes")
    return 0

if __name__ == "__main__":
    sys.exit(build())

```

A.7 The cross-verification battery

theory/checks/omega_vs_methods.py

```

"""Cross-verification of the Omega = 1 rule against the rules existing catastrophic-forgetting methods use to set the past
pull, on one model where every rule can be run exactly (owner request 2026-09-22, prompt-log entry 73). The model is the
two-task quadratic of theory/checks/omega_sweeps.py (present loss with Hessian H, past penalty with Fisher F sixteen times
H, learner starting at the old-task optimum, mini-batch noise on both gradients). Each method is the update rule its
paper states, reduced to this model; each is run with ONE knob setting, then (a) with the past curvature F multiplied by 16
(the same knob), to see which rules survive a change of units without retuning, and (b) with its own knob multiplied by 10,
to see which rules have a divergence edge. Quality is measured against the best fixed weight on the same scale (tuned in
sample, which favours the baseline). Every label is computed from the numbers (R15). Pinned: omega_vs_methods.txt.
Run: uv run python theory/checks/omega_vs_methods.py
"""
from __future__ import annotations

import sys
from pathlib import Path

import numpy as np

sys.path.insert(0, str(Path(__file__).resolve().parent))
import omega_sweeps as om # noqa: E402

LR = 0.05; STEPS = 4000; NOISE = 0.5; SEEDS = 5
FIXED_GRID = (0.003, 0.01, 0.03, 0.1, 0.3, 0.6, 1.0, 1.2, 1.5, 3.0)

```

```

def run(method, knob, H, F, a, b, seed, lr=LR, steps=STEPS, noise=NOISE):
    """One trajectory. Returns (theta, ok). Methods (knob in brackets):
        fixed[lamdba]      theta <- theta - lr (g_p + lamdba g_q) (EWC / L2-SP / a KL penalty at fixed
weight)
        omega[Omega]      w = Omega |ema g_p| / |ema g_q| (smooth 0.9, cap 1e4) (the rule under test)
        gradnorm[-]       GradNorm alpha = 0, weights renormalised to sum 2 (Chen et al. 2018)
        mega[-]           w = ema L_q / ema L_p (MEGA-I loss ratio)
        agem[-]           project g_p off g_q when they conflict (A-GEM)
        softproj[c]       remove a fraction c of the conflicting component (a PCR-style soft projection)
        subspace[k]       project g_p off the top-k eigenvectors of F, no past term (OGD / OGPSA / SafeAnchor-style protection)
        klctrl[target]    fixed-form controller: w <- w (1 + 0.1 clip((L_q - target)/target, -0.2, 0.2)) every 10 steps
(Ziegler-style)
        select[q]         fixed lamdba = 1, skip present steps whose |g_p| is above the q-quantile of the last 50 (gradient-based
sample selection)
    """
    rng = np.random.default_rng(seed)
    th = b.copy(); ema_p = ema_q = None; ema_Lp = ema_Lq = None; gn_w = np.array([1.0, 1.0]); w_ctrl = 1.0; recent = []
    if method == "subspace":
        evals, evecs = np.linalg.eigh(F); P = evecs[:, np.argsort(evals)[: -1][:int(knob)]]
    for t in range(steps):
        g_p = H @ (th - a) + noise * rng.standard_normal(len(th)); g_q = F @ (th - b) + noise * rng.standard_normal(len(th))
        L_p, L_q = om.losses(H, F, a, b, th)
        if method == "fixed":
            step = g_p + knob * g_q
        elif method == "omega":
            ema_p = g_p if ema_p is None else om.SMOOTH * ema_p + (1 - om.SMOOTH) * g_p
            ema_q = g_q if ema_q is None else om.SMOOTH * ema_q + (1 - om.SMOOTH) * g_q
            step = g_p + om.rule_w(ema_p, ema_q, knob) * g_q
        elif method == "gradnorm":
            Gp, Gq = gn_w[0] * np.linalg.norm(g_p), gn_w[1] * np.linalg.norm(g_q); Gbar = 0.5 * (Gp + Gq)
            grad_w = np.array([np.sign(Gp - Gbar) * np.linalg.norm(g_p), np.sign(Gq - Gbar) * np.linalg.norm(g_q)])
            gn_w = np.maximum(gn_w - 0.025 * grad_w / max(np.abs(grad_w).max(), om.DEN_FLOOR), 1e-3); gn_w = 2 * gn_w /
gn_w.sum()
            step = gn_w[0] * g_p + gn_w[1] * g_q
        elif method == "mega":
            ema_Lp = L_p if ema_Lp is None else om.SMOOTH * ema_Lp + (1 - om.SMOOTH) * L_p
            ema_Lq = L_q if ema_Lq is None else om.SMOOTH * ema_Lq + (1 - om.SMOOTH) * L_q
            step = g_p + min(ema_Lq / max(ema_Lp, om.DEN_FLOOR), om.WCAP) * g_q
        elif method in ("agem", "softproj"):
            dot = float(g_p @ g_q); c = 1.0 if method == "agem" else knob
            step = g_p - c * (dot / max(float(g_q @ g_q), om.DEN_FLOOR)) * g_q if dot < 0 else g_p
        elif method == "subspace":
            step = g_p - P @ (P.T @ g_p)
        elif method == "klctrl":
            if t % 10 == 0 and t > 0:
                w_ctrl = w_ctrl * (1 + 0.1 * float(np.clip((L_q - knob) / knob, -0.2, 0.2)))
            step = g_p + w_ctrl * g_q
        elif method == "select":
            recent.append(np.linalg.norm(g_p)); recent = recent[-50:]
            skip = len(recent) >= 10 and np.linalg.norm(g_p) > np.quantile(recent, knob)
            step = (0.0 * g_p if skip else g_p) + 1.0 * g_q
        else:
            raise ValueError(method)
        th = th - lr * step
        if not np.all(np.isfinite(th)) or np.linalg.norm(th) > 1e6:
            return th, False
    return th, True

def evaluate(method, knob, H, F, a, b):
    tot = []; Lps = []; Lqs = []; ok_all = True
    for s in range(SEEDS):
        th, ok = run(method, knob, H, F, a, b, s); ok_all = ok_all and ok
        Lp, Lq = om.losses(H, F, a, b, th) if ok else (np.nan, np.nan); tot.append(Lp + Lq); Lps.append(Lp); Lqs.append(Lq)
    return float(np.mean(Lps)), float(np.mean(Lqs)), float(np.mean(tot)), ok_all

def tuned_fixed(H, F, a, b):
    best = None
    for lam in FIXED_GRID:
        Lp, Lq, tot, ok = evaluate("fixed", lam, H, F, a, b)
        if ok and (best is None or tot < best[1]): best = (lam, tot)
    return best

METHODS = [ # (name, knob, class, needs)
    ("fixed", 1.0, "fixed weight (EWC/L2-SP/KL penalty); lambda = 1 is also the Bayes/ER-sum weight in this model", "a sweep of
lambda"),
    ("omega", 1.0, "normalised gradient (the rule under test)", "smoothing, cap, floor"),
    ("gradnorm", 0.0, "learned task weights (GradNorm alpha = 0)", "a weight learning rate"),
    ("mega", 0.0, "loss ratio (MEGA-I)", "smoothing"),
    ("agem", 0.0, "conflict projection (A-GEM)", "nothing"),
    ("softproj", 0.5, "soft conflict projection (PCR-style)", "a fraction c"),
    ("subspace", 2, "protected subspace, no past term (OGD / OGPSA / SafeAnchor-style)", "a rank k"),
    ("klctrl", 1.0, "target controller on the past loss (adaptive-KL-style)", "a target"),
    ("select", 0.8, "gradient-based sample selection at fixed lambda = 1", "a quantile"),
]

```

```

def main():
    print("Omega = 1 against the rules other methods use, on the two-task quadratic of omega_sweeps.py (F = 16 H; noise 0.5; lr
0.05; 4000 steps; 5 seeds; start at the old-task optimum)")
    H, F, a, b = om.make_model()
    H16, F16 = H, 16 * F
    lam_edge = (2.0 / LR - np.diag(H).max()) / np.diag(F).max(); lam_edge16 = (2.0 / LR - np.diag(H).max()) / np.diag(F16).max()
    t1 = tuned_fixed(H, F, a, b); t16 = tuned_fixed(H16, F16, a, b)
    print(f"stability edge of a fixed weight: lambda* = {lam_edge:.4f} at F, {lam_edge16:.4f} at 16F; tuned fixed weight
(in-sample, grid {FIXED_GRID}): lambda = {t1[0]:g} total {t1[1]:.4f} at F, lambda = {t16[0]:g} total {t16[1]:.4f} at 16F")
    print()
    print(f"{'method':>10s} {'knob':>6s} {'L_present':>9s} {'L_past':>8s} {'total':>8s} {'/tuned':>7s} | {'16F: total':>10s}
{'/tuned':>7s} | {'knob x10':>9s} | label")
    rows = []
    for name, knob, cls, needs in METHODS:
        Lp, Lq, tot, ok = evaluate(name, knob, H, F, a, b)
        Lp16, Lq16, tot16, ok16 = evaluate(name, knob, H16, F16, a, b)
        big = knob * 10 if knob else 0.0
        okb = evaluate(name, big, H, F, a, b)[3] if name not in ("gradnorm", "mega", "agem", "select") else None
        if name == "subspace": okb = evaluate(name, 4, H, F, a, b)[3]
        if name == "softproj": okb = evaluate(name, 1.0, H, F, a, b)[3]
        r1 = tot / t1[1] if ok else float("inf"); r16 = tot16 / t16[1] if ok16 else float("inf")
        robust = ok and ok16 and r1 <= 1.10 and r16 <= 1.10
        edge = (okb is False)
        label = ("scale-robust" if robust else "not scale-robust") + ("; has a divergence edge in its knob" if edge else ("; no
knob" if okb is None else "; no edge found at knob x10"))
        rows.append((name, cls, needs, r1, r16, robust, edge, ok, ok16, Lq, Lq16))
        print(f"{name:10s} {knob:6g} {Lp:9.4f} {Lq:8.4f} {tot:8.4f} {r1:7.3f} | {tot16 if ok16 else float('nan'):.10.4f}
{r16:7.3f} | {'n/a' if okb is None else ('finite' if okb else 'DIVERGES'):>9s} | {label}")
    print()
    print("computed:")
    for name, cls, needs, r1, r16, robust, edge, ok, ok16, Lq, Lq16 in rows:
        print(f" {name:10s} [{cls}; needs {needs}] -> within 10 % of the tuned fixed weight at F: {r1 <= 1.10}, at 16F with the
same knob: {r16 <= 1.10}; scale-robust: {robust}; divergence edge: {edge}")
        tuned_Lq = evaluate("fixed", t1[0], H, F, a, b)[1]
        print(f" scale-robust methods (within 10 % of the tuned fixed weight at both F and 16F with one knob setting): {[r[0] for r
in rows if r[5]]}")
        print(f" methods that diverge at 16F with the knob that worked at F: {[r[0] for r in rows if not r[8]]}")
        print(f" methods that forget (L_past more than 10 x the tuned fixed weight's {tuned_Lq:.4f}) at F: {[r[0] for r in rows if
r[7] and r[9] > 10 * tuned_Lq]}")
        print(f" methods with a divergence edge in their own knob: {[r[0] for r in rows if r[6]]}")
        print(" reading (from the lists above): the rules that weight the past pull by a fixed number, a learned number or a loss
ratio carry that number's units and fail when the past curvature changes scale; the rules that project rather than weight never
diverge but drop the past wherever the two gradients do not conflict; the rule that scales the past pull by the present pull is
the one in this list whose quality survives the change of scale, because its step is bounded by the present step")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

A.8 The frozen EQ4 scorer (EQ-B, the poisoned regime, the class-selection rule)

runs/eq4/frozen/eq4_score.py

Study EQ4 – the BOUNDED normalised penalty step (EQ-B: the present gradient clipped at kappa x the running median of its own recent length, the equanimity ratio then taken on the clipped gradient, Omega = 1) for online EWC on unseen PMLB streams, beside the registered rule (EQ2/EQ3), with the controls of EQ3 (ER-sum, DER++, LwF, SI, MAS; A-GEM, GradNorm, MEGA-I; the Bayes arm; the nine-point Omega grid; the lr x batch array) and one addition: a POISONED regime, in which five consecutive present batches per task (from the second task) carry gradients multiplied by POISON_FACTOR, where the enhancement is predicted to be load-bearing (theory/checks/omega_reprocessed.py [4]) and the R7 baseline is the fixed weight WITH the same present clip. Owner request: prompt-log entry 74; prereg/eq4/PREREG.md. The design, constants and scoring of the EQ3 rows are those of runs/eq3/frozen/eq3_score.py except where this docstring or PREREG.md says otherwise (the capacity x epochs array and the x4 diagnostic regime are dropped; the class-selection rule is new).

```

uv run python studies/eq4/eq4_score.py smoke # every (method, mode) branch once on synthetic data
uv run python studies/eq4/eq4_score.py smokefull [--out F] # run_all + score end to end on synthetic data
uv run python studies/eq4/eq4_score.py all <dataset> [--out F] # every arm, seeds 0-4, JSON lines
uv run python studies/eq4/eq4_score.py score <results.jsonl ...> # rows EQ4-0..7, S, P, B, D, I
uv run python studies/eq4/eq4_score.py run <dataset> <method> <mode> <value> <seed> [--regime std|poison]

```

```

Update rule for every method: g = wp * g_present + w * g_past.
mode 'fixed' w = value (the method's own hand-set weight)
mode 'fixedclip' w = value, with the present gradient clipped as in 'eqb' (the R7 baseline for the poisoned regime)
mode 'eq' w = value * ||ema g_present|| / ||ema g_past|| (Omega = value; the registered rule, EQ2/EQ3)
mode 'eqb' the present gradient g_p is first clipped to length kappa x max(KEPT |g_p| over the last n_hist batches)
(the history holds the kept, post-clip lengths, so a run of extreme batches raises the bound by at most
a factor kappa per batch; no clip until N_MIN lengths are known; the window MAXIMUM, not the median,
because on a softmax stream the median present-gradient length collapses toward zero as a task is
learned and a median bound clips the informative batches, AGENT_LOG 63); then
w = value * ||ema g_p_clipped|| / ||ema g_past||
mode 'bayes' (ewc only) the Laplace weight, w = 1/2 on the task-size-weighted Fisher (EQ3)
mode 'agem' / 'gradnorm' / 'mega' as EQ3
mode 'reduction' w = value = the median derived weight of the eqb Omega = 1 run (the constant the rule reduces to)
Regime 'poison': in the first epoch of every task after the first, from batch min(max(N_MIN, floor(POISON_AT * n_batches)),
n_batches - run),
run = max(1, min(POISON_RUN, floor(POISON_RUN_FRAC * n_batches))) consecutive batches have g_present multiplied by POISON_FACTOR
(before any clip). The EQ-B

```

```

clip history restarts at every task boundary (RESET_HIST). Past terms as EQ3. Score = final class-IL accuracy (%).
Class selection (new, pre-registered): classes ranked by row count (ties by remapped label); the K largest are used
if the K-th has >= CLASS_FLOOR rows after the subsample, else K is lowered to the largest even number that satisfies
it; a carrier with K < 4 is excluded and counted (exclusions.py).
"""
from __future__ import annotations

import gzip
import hashlib
import json
import math
import os
import sys
from pathlib import Path

os.environ.setdefault("OMP_NUM_THREADS", "1"); os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("MKL_NUM_THREADS", "1")
import numpy as np # noqa: E402

_HERE = Path(__file__).resolve()
ROOT = _HERE.parents[3] if _HERE.parent.name == "frozen" else _HERE.parents[2] # runs/eq4/frozen/ or studies/eq4/
RAW = ROOT / "data" / "raw" / "pmlb"

# ----- registered constants (PREREG.md)
LR = 0.05; BS = 10; HID = 256; EPOCHS = 3; BUF = 500
SMOOTH = 0.9; WCAP = 1e4; DEN_FLOOR = 1e-12 # registered estimator (EQ2, EQ2R, EQ3)
KAPPA = 2.0; N_HIST = 50; N_MIN = 10 # EQ-B present clip (theory/checks/omega_reprocessed.py [4];
kappa chosen on the S-Y gate rows and the seen-carrier dev run, runs/eq4_dev)
SENS_KAPPA = (1.5, 2.0, 4.0); SENS_NHIST = (20, 50, 200) # EQ4-S, with SENS_CAP x SENS_SMOOTH
POISON_AT = 0.25; POISON_RUN = 5; POISON_RUN_FRAC = 0.05; POISON_FACTOR = 1000.0 # regime 'poison': from batch floor(POISON_AT
* n_batches) of the first epoch of each task after the first, max(1, min(POISON_RUN, floor(POISON_RUN_FRAC * n_batches)))
batches carry g_present x POISON_FACTOR (omega_reprocessed.py [4]: the registered rule degrades at (100, 5), diverges at (1000,
5))
RESET_HIST = True # the EQ-B clip history restarts at every task boundary (a new
task's first gradients are not judged by the old task's)
OMEGA_GRID = (0.5, 0.71, 1.0, 1.41, 2.0) # DER++ / LwF arms (as EQ3; report only here)
OMEGA_GRID_EWC = (0.25, 0.35, 0.5, 0.71, 1.0, 1.41, 2.0, 2.83, 4.0) # EWC arm, rule and EQ-B (plateau width, EQ4-P)
SEEDS = (0, 1, 2, 3, 4)
TEST_FRAC = 0.2
MAX_ROWS = 5000; SUBSAMPLE_SEED = 777; CLASS_FLOOR = 40
EWC_COARSE = (0.1, 0.3, 1.0, 3.0, 10.0, 30.0, 100.0, 300.0, 1000.0, 3000.0, 10000.0)
EWC_REFINE = (1 / 2.83, 1 / 2.0, 1 / 1.41, 1.41, 2.0, 2.83) # times the coarse best
BAYES_W = 0.5
ER_GRID = (0.5, 1.0, 2.0, 4.0)
CONSTRAINT_GRID = (0.01, 0.03, 0.1, 0.3, 1.0, 3.0, 10.0)
DERPP_BETA = 0.5; DERPP_ALPHA_DEFAULT = 0.1; LWF_DEFAULT = 1.0; LWF_T = 2.0
SI_XI = 1e-3; N_FISHER = 50
SENS_CAP = (10.0, 100.0, 1e4); SENS_SMOOTH = (0.8, 0.9, 0.98)
X4 = 4.0 # kept for split_standardise's signature; the x4 regime is not
run
LOAD_BEARING_STEPS = 2.0
DATASETS = { # PMLB name -> (K requested, classes per task); K may be lowered by the class-selection rule
    "satimage": (6, 2),
    "segmentation": (6, 2), # 7 classes in the file: the 6 largest (ties by label)
    "yeast": (6, 2), # 10 classes in the file, several tiny: the rule decides K
    "wine_quality_white": (4, 2), # 7 classes, two tiny
    "sleep": (4, 2), # 5 classes; 105908 rows -> subsample
    "page_blocks": (4, 2), # 5 classes, imbalanced
}
FALLBACK = {}
ARRAY_CARRIER = "satimage" # EQ4-I on this carrier only
INVAR_CELLS = ((0.0125, 10), (0.2, 10), (0.05, 5), (0.05, 20)) # EQ4-I: (lr, bs); the main cell (0.05, 10) completes the set
of five
LATE_FRAC = 0.2

def sha256(p: Path) -> str:
    h = hashlib.sha256()
    with open(p, "rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b''):
            h.update(chunk)
    return h.hexdigest()

def load_pmlb(name: str, K_req: int):
    p = RAW / f"{name}.tsv.gz"
    with gzip.open(p, "rt") as f:
        header = f.readline().rstrip("\n").split("\t")
        rows = [line.rstrip("\n").split("\t") for line in f if line.strip()]
        ti = header.index("target")
        X = np.array([float(v) for j, v in enumerate(r) if j != ti] for r in rows), np.float64)
        y = np.array([int(float(r[ti])) for r in rows])
        classes = np.unique(y); remap = {c: i for i, c in enumerate(classes)}
        y = np.array([remap[c] for c in y])
        nan = int(np.isnan(X).any(1).sum())
        if nan:
            keep = ~np.isnan(X).any(1); X, y = X[keep], y[keep]
        n_file = int(len(y))
        # class-selection rule: rank by row count (ties by remapped label), take the K largest, subsample, then check the floor
        counts = {c: int((y == c).sum()) for c in range(len(classes))}

```

```

ranked = sorted(counts, key=lambda c: (-counts[c], c))
K = min(K_req, len(ranked)); K -= K % 2
while K >= 2:
    chosen = ranked[:K]; sel = np.isin(y, chosen); Xs, ys = X[sel], y[sel]
    order = {c: i for i, c in enumerate(chosen)}; ys = np.array([order[c] for c in ys])
    if len(ys) > MAX_ROWS:
        rng = np.random.default_rng(SUBSAMPLE_SEED); pick = []
        for c in range(K):
            idx = np.where(ys == c)[0]; n_c = int(round(MAX_ROWS * len(idx) / len(ys)))
            pick.append(rng.permutation(idx)[:n_c])
        pick = np.sort(np.concatenate(pick)); Xs, ys = Xs[pick], ys[pick]
        cc = [int((ys == c).sum()) for c in range(K)]
        if min(cc) >= CLASS_FLOOR: break
    K -= 2
if K < 2: Xs, ys, cc = X[:,0], y[:,0], []
return Xs, ys, dict(file=str(p.relative_to(ROOT)), sha256=sha256(p), n=int(len(ys)), n_file=n_file, d=int(X.shape[1]),
                    classes_in_file=int(len(classes)), classes_requested=K_req, classes_used=K, rows_dropped_nan=nan,
                    class_counts_file=[counts[c] for c in ranked], class_counts=cc, class_floor=CLASS_FLOOR,
                    excluded=bool(K < 4))

def split_standardise(X, y, K, regime="std"):
    """Fixed 80/20 stratified split (seed 12345, independent of run seed); standardise on the
    training part; regime 'x4' multiplies the standardised features by X4 (diagnostic only)."""
    rng = np.random.default_rng(12345)
    tr, te = [], []
    for c in range(K):
        idx = rng.permutation(np.where(y == c)[0]); n_te = int(round(TEST_FRAC * len(idx)))
        te.append(idx[:n_te]); tr.append(idx[n_te:])
    tr = np.concatenate(tr); te = np.concatenate(te)
    mu = X[tr].mean(0); sd = X[tr].std(0); sd[sd == 0] = 1.0
    Z = (X - mu) / sd
    if regime == "x4":
        Z = Z * X4
    return Z[tr], y[tr], Z[te], y[te]

# ----- model
class MLP:
    def __init__(s, rng, d, K, h=HID):
        s.p = [rng.normal(0, math.sqrt(2 / d), (d, h)), np.zeros(h), rng.normal(0, math.sqrt(2 / h), (h, K)), np.zeros(K)]
        s.K = K

    def forward(s, x):
        W1, b1, W2, b2 = s.p; act = np.maximum(0, x @ W1 + b1); return act, act @ W2 + b2

    def back(s, x, dz, act):
        W1, b1, W2, b2 = s.p; dh = dz @ W2.T; dh[act <= 0] = 0
        return np.concatenate([(x.T @ dh).ravel(), dh.sum(0), (act.T @ dz).ravel(), dz.sum(0)])

    def flat(s): return np.concatenate([a.ravel() for a in s.p])

    def set_flat(s, v):
        i = 0
        for a in s.p:
            a[...] = v[i:i + a.size].reshape(a.shape); i += a.size

    def copy(s):
        m = MLP.__new__(MLP); m.p = [a.copy() for a in s.p]; m.K = s.K; return m

    def acc(s, x, y):
        with np.errstate(all="ignore"):
            z = s.forward(x)[1]
            if not np.all(np.isfinite(z)):
                return 0.0 # a diverged model scores zero
            return float((z.argmax(1) == y).mean())

def softmax(z):
    z = z - z.max(1, keepdims=True); p = np.exp(z); return p / p.sum(1, keepdims=True)

def ce_loss_grad(net, x, y):
    act, z = net.forward(x); p = softmax(z); n = len(y)
    loss = float(-np.log(p[np.arange(n), y] + 1e-300).mean())
    dz = p; dz[np.arange(n), y] -= 1; dz /= n
    return loss, net.back(x, dz, act)

# ----- one run
def run(method, mode, value, seed, Xtr, ytr, Xte, yte, K, per_task, smooth=SMOOTH, wcap=WCAP, hidden=HID, epochs=EPOCHS, lr=LR,
bs=BS,
    kappa=KAPPA, n_hist=N_HIST, poison=False):
    rng = np.random.default_rng(seed)
    d = Xtr.shape[1]; net = MLP(rng, d, K, h=hidden); n = net.flat().size; task_w_start = []
    tasks = [tuple(range(i, i + per_task)) for i in range(0, K, per_task)]
    bufx = np.zeros((BUF, d)); bufy = np.zeros((BUF, int)); bufz = np.zeros((BUF, K)); nb = 0; seen = 0
    importance = np.zeros(n); imp_bayes = np.zeros(n); theta_star = None; prev = None
    si_omega = np.zeros(n); theta_task_start = net.flat().copy()
    ema_p = ema_q = None; ema_Lp = ema_Lq = None; gn_w = np.array([1.0, 1.0]); wlog = []

```

```

hist = []; n_clip = 0; n_clip_steps = 0; n_poisoned = 0
fwd_bwd = 0.0; n_stream = 0
with np.errstate(all="ignore"):
    for ti, task in enumerate(tasks):
        ii_task = np.where(np.isin(ytr, task))[0]; n_task = len(ii_task)
        theta_task_start = net.flat().copy(); si_omega[:] = 0; task_w_start.append(len(wlog))
        imp_used = (imp_bayes / n_task) if mode == "bayes" else importance
        if RESET_HIST: hist = []
            n_batches = int(math.ceil(n_task / bs)); p_run = max(1, min(POISON_RUN, int(math.floor(POISON_RUN_FRAC *
n_batches))))
        p_start = min(max(N_MIN, int(math.floor(POISON_AT * n_batches))), max(0, n_batches - p_run)) # never before the
clip history is populated
        for ep in range(epochs):
            perm = rng.permutation(ii_task)
            for bi, t in enumerate(range(0, len(perm), bs)):
                ii = perm[t:t + bs]; x, y = Xtr[ii], ytr[ii]; m = len(y); n_stream += m
                L_p, g_p = ce_loss_grad(net, x, y); fwd_bwd += 3 * m
                g_q = None; L_q = None
                if method == "ewc" and theta_star is not None:
                    dth = net.flat() - theta_star; L_q = float((imp_used * dth ** 2).sum()); g_q = 2 * imp_used * dth
                elif method in ("si", "mas") and theta_star is not None:
                    dth = net.flat() - theta_star; L_q = float((importance * dth ** 2).sum()); g_q = 2 * importance * dth
                elif method == "er" and nb > 0:
                    jj = rng.integers(0, nb, m); L_q, g_q = ce_loss_grad(net, bufx[jj], bufy[jj]); fwd_bwd += 3 * m
                elif method == "derpp" and nb > 0:
                    jj = rng.integers(0, nb, m); rx, ry, rz = bufx[jj], bufy[jj], bufz[jj]
                    L_b, g_b = ce_loss_grad(net, rx, ry); g_p = g_p + DERPP_BETA * g_b; L_p = L_p + DERPP_BETA * L_b
                    act, z = net.forward(rx); L_q = float(((z - rz) ** 2).mean()); g_q = net.back(rx, 2 * (z - rz) / (m *
K), act)
                    fwd_bwd += 6 * m
                elif method == "lwf" and prev is not None:
                    act, z = net.forward(x); zp = prev.forward(x)[1]
                    q, qp = softmax(z / LWF_T), softmax(zp / LWF_T)
                    L_q = float(LWF_T * 2 * (qp * (np.log(qp + 1e-300) - np.log(q + 1e-300))).sum(1).mean())
                    g_q = net.back(x, LWF_T * (q - qp) / m, act); fwd_bwd += 4 * m
                if poison and ti >= 1 and ep == 0 and p_start <= bi < p_start + p_run:
                    g_p = POISON_FACTOR * g_p; n_poisoned += 1
                if mode in ("eqb", "fixedclip"):
                    # EQ-B present bound, maintained on every batch
                    (history of KEPT lengths, reference = window max)
                    gn = float(np.linalg.norm(g_p)); tau = kappa * float(np.max(hist[-n_hist:])) if len(hist) >= N_MIN else
np.inf
                    n_clip_steps += 1
                    if gn > tau: g_p = g_p * (tau / gn); hist.append(tau); n_clip += 1
                    else: hist.append(gn)
                theta_before = net.flat()
                if g_q is None:
                    net.set_flat(theta_before - lr * g_p)
                else:
                    if mode in ("fixed", "fixedclip", "reduction"): # reduction: fixed w at the median derived w
                        wp, w = 1.0, value
                    elif mode == "bayes":
                        wp, w = 1.0, BAYES_W
                    elif mode in ("eq", "eqb"):
                        ema_p = g_p if ema_p is None else smooth * ema_p + (1 - smooth) * g_p
                        ema_q = g_q if ema_q is None else smooth * ema_q + (1 - smooth) * g_q
                        wp = 1.0; w = min(value * np.linalg.norm(ema_p) / max(np.linalg.norm(ema_q), DEN_FLOOR), wcap)
                    elif mode == "agem":
                        dot = float(g_p @ g_q)
                        wp = 1.0; w = (-dot / max(float(g_q @ g_q), DEN_FLOOR)) if dot < 0 else 0.0
                    elif mode == "gradnorm":
                        Gp, Gq = gn_w[0] * np.linalg.norm(g_p), gn_w[1] * np.linalg.norm(g_q)
                        Gbar = 0.5 * (Gp + Gq)
                        grad_w = np.array([np.sign(Gp - Gbar) * np.linalg.norm(g_p), np.sign(Gq - Gbar) *
np.linalg.norm(g_q)])
                        gn_w = np.maximum(gn_w - 0.025 * grad_w / max(np.abs(grad_w).max(), DEN_FLOOR), 1e-3)
                        gn_w = 2 * gn_w / gn_w.sum(); wp, w = float(gn_w[0]), min(float(gn_w[1]), wcap)
                    elif mode == "mega":
                        ema_Lp = L_p if ema_Lp is None else smooth * ema_Lp + (1 - smooth) * L_p
                        ema_Lq = L_q if ema_Lq is None else smooth * ema_Lq + (1 - smooth) * L_q
                        wp = 1.0; w = min(ema_Lq / max(ema_Lp, DEN_FLOOR), wcap)
                    else:
                        raise ValueError(mode)
                    wlog.append(float(w))
                    net.set_flat(theta_before - lr * (wp * g_p + w * g_q))
                if method == "si":
                    si_omega += -g_p * (net.flat() - theta_before)
                if method in ("er", "derpp"):
                    for k in range(m):
                        seen += 1
                        if nb < BUF: j = nb; nb += 1
                        else:
                            j = rng.integers(seen)
                            if j >= BUF: continue
                        bufx[j] = x[k]; bufy[j] = y[k]
                        if method == "derpp": bufz[j] = net.forward(x[k:k + 1])[1][0]
                prev = net.copy(); theta_now = net.flat().copy()
                if method == "ewc":
                    f = np.zeros(n)
                    for _ in range(N_FISHER):
                        ii = ii_task[rng.integers(0, len(ii_task), bs)]; f += ce_loss_grad(net, Xtr[ii], ytr[ii])[1] ** 2

```

```

        f_task = f / N_FISHER * bs
        importance = importance + f_task; imp_bayes = imp_bayes + n_task * f_task
    elif method == "si":
        importance = importance + np.maximum(si_omega, 0) / ((theta_now - theta_task_start) ** 2 + SI_XI)
    elif method == "mas":
        imp = np.zeros(n)
        for _ in range(N_FISHER):
            ii = ii_task[rng.integers(0, len(ii_task), bs)]; act, z = net.forward(Xtr[ii]); imp +=
np.abs(net.back(Xtr[ii], 2 * z / bs, act))
            importance = importance + imp / N_FISHER
        if method in ("ewc", "si", "mas"):
            theta_star = theta_now

    late = []
    for a, b in zip(task_w_start, task_w_start[1:] + [len(wlog)]):
        if b > a: late.extend(wlog[a + int((1 - LATE_FRAC) * (b - a)):b])
        rec = dict(method=method, mode=mode, value=float(value), seed=int(seed), smooth=float(smooth), wcap=float(wcap),
hidden=int(hidden), epochs=int(epochs),
                    lr=float(lr), bs=int(bs), kappa=float(kappa), n_hist=int(n_hist), poison=bool(poison),
                    acc=100 * net.acc(Xte, yte), w_med=float(np.median(wlog)) if wlog else None,
                    w_late_med=float(np.median(late)) if late else None, w_max=float(np.max(wlog)) if wlog else None,
                    w_cap_frac=float(np.mean(np.array(wlog) >= wcap)) if wlog else None,
                    clip_frac=(n_clip / n_clip_steps) if n_clip_steps else None, n_poisoned=int(n_poisoned),
                    fwd_bwd_per_sample=float(fwd_bwd / max(n_stream, 1)), n_stream=int(n_stream),
                    finite=bool(np.all(np.isfinite(net.flat()))))
        json.dumps(rec)
    return rec

# ----- arms
def arms_stagel():
    """Everything except the refinements, the reduction arms and the array. Tuples: (method, mode, value, smooth, wcap, regime,
kappa, n_hist)."""
    S, C, KP, NH = SMOOTH, WCAP, KAPPA, N_HIST
    for w in EWC_COARSE: yield ("ewc", "fixed", w, S, C, "std", KP, NH)
    for om in OMEGA_GRID_EWC: yield ("ewc", "eq", om, S, C, "std", KP, NH)
    for om in OMEGA_GRID_EWC: yield ("ewc", "eqb", om, S, C, "std", KP, NH)
    for cap in SENS_CAP:
        for sm in SENS_SMOOTH:
            if (cap, sm) != (C, S): yield ("ewc", "eqb", 1.0, sm, cap, "std", KP, NH)
    for kp in SENS_KAPPA:
        for nh in SENS_NHIST:
            if (kp, nh) != (KP, NH): yield ("ewc", "eqb", 1.0, S, C, "std", kp, nh)
    yield ("ewc", "bayes", 0.0, S, C, "std", KP, NH)
    for mode in ("agem", "gradnorm", "mega"): yield ("ewc", mode, 0.0, S, C, "std", KP, NH)
    for w in ER_GRID: yield ("er", "fixed", w, S, C, "std", KP, NH)
    yield ("er", "eq", 1.0, S, C, "std", KP, NH); yield ("er", "eqb", 1.0, S, C, "std", KP, NH)
    for meth, default in (("derpp", DERPP_ALPHA_DEFAULT), ("lwf", LWF_DEFAULT)):
        for w in sorted(set(CONSTRAINT_GRID) | {default}): yield (meth, "fixed", w, S, C, "std", KP, NH)
        yield (meth, "eq", 1.0, S, C, "std", KP, NH); yield (meth, "eqb", 1.0, S, C, "std", KP, NH)
    for meth in ("si", "mas"):
        for w in CONSTRAINT_GRID: yield (meth, "fixed", w, S, C, "std", KP, NH)
        yield (meth, "eqb", 1.0, S, C, "std", KP, NH)
    # poisoned regime: the fixed weight, the fixed weight with the clip (R7 baseline), the registered rule, EQ-B
    for w in EWC_COARSE: yield ("ewc", "fixed", w, S, C, "poison", KP, NH)
    for w in EWC_COARSE: yield ("ewc", "fixedclip", w, S, C, "poison", KP, NH)
    yield ("ewc", "eq", 1.0, S, C, "poison", KP, NH); yield ("ewc", "eqb", 1.0, S, C, "poison", KP, NH)

def key(o): return (o["method"], o["mode"], round(o["value"], 6), o["smooth"], o["wcap"], o.get("regime", "std"),
o.get("hidden", HID),
                    o.get("epochs", EPOCHS), o.get("lr", LR), o.get("bs", BS), o.get("kappa", KAPPA), o.get("n_hist", N_HIST))

def run_all(name, data_std, K, per_task, meta, out, array_carrier=ARRAY_CARRIER):
    hdr = dict(dataset=name, **meta, K=K, per_task=per_task, n_train=int(len(data_std[1])), n_test=int(len(data_std[3])),
                lr=LR, bs=BS, hidden=HID, epochs=EPOCHS, buf=BUF, smooth=SMOOTH, wcap=WCAP, den_floor=DEN_FLOOR, bayes_w=BAYES_W,
                kappa=KAPPA, n_hist=N_HIST, n_min=N_MIN, reset_hist=RESET_HIST, poison_at=POISON_AT, poison_run=POISON_RUN,
                poison_run_frac=POISON_RUN_FRAC, poison_factor=POISON_FACTOR,
                n_fisher=N_FISHER, si_xi=SI_XI, derpp_beta=DERPP_BETA, lwf_T=LWF_T, max_rows=MAX_ROWS,
                subsample_seed=SAMPLE_SEED,
                uv_lock_sha256=sha256(ROOT / "uv.lock"))
    print(json.dumps(hdr), file=out, flush=True)
    done = {}

    def go(method, mode, value, sm, cap, regime="std", kappa=KAPPA, n_hist=N_HIST, hidden=HID, epochs=EPOCHS, lr=LR, bs=BS):
        for seed in SEEDS:
            o = run(method, mode, value, seed, *data_std, K, per_task, smooth=sm, wcap=cap, hidden=hidden, epochs=epochs, lr=lr,
bs=bs,
                    kappa=kappa, n_hist=n_hist, poison=(regime == "poison"))
            o["dataset"] = name; o["regime"] = regime; print(json.dumps(o), file=out, flush=True)
            done.setdefault(key(o), []).append(o["acc"])

    def median_w(meth, mode, regime):
        return float(np.median([o["w_med"] for o in recs if o["method"] == meth and o["mode"] == mode and o["value"] == 1.0
                                and o["smooth"] == SMOOTH and o["wcap"] == WCAP and o["regime"] == regime and o["hidden"] == HID
                                and o["epochs"] == EPOCHS and o["lr"] == LR and o["bs"] == BS and o["kappa"] == KAPPA and
o["n_hist"] == N_HIST]))

    for a in arms_stagel(): go(*a)
    # stage 2: refinement at sqrt2 spacing around the coarse best, clean fixed and poisoned fixedclip (pre-registered tuning

```

```

rule)
    for mode, regime in (("fixed", "std"), ("fixedclip", "poison")):
        best = max(EWC_COARSE, key=lambda w: np.mean(done[{"ewc", mode, round(w, 6), SMOOTH, WCAP, regime, HID, EPOCHS, LR, BS,
KAPPA, N_HIST]}))
        for f in EWC_REFINE:
            w = round(best * f, 6)
            if all(abs(w - c) / c > 1e-6 for c in EWC_COARSE): go("ewc", mode, w, SMOOTH, WCAP, regime)
# reduction arms: fixed w at the median derived weight of the EQ-B Omega = 1 run (EWC and ER)
recs = [json.loads(l) for l in open(out.name) if l.startswith("{}"); recs = [o for o in recs if "acc" in o]
for meth in ("ewc", "er"):
    go(meth, "reduction", round(median_w(meth, "eqb", "std"), 6), SMOOTH, WCAP)
# EQ4-I (lr x batch) on the registered carrier only: EWC coarse grid and EQ-B at Omega = 1 per cell
if name == array_carrier:
    for lr, bs in INVAR_CELLS:
        for w in EWC_COARSE: go("ewc", "fixed", w, SMOOTH, WCAP, "std", KAPPA, N_HIST, HID, EPOCHS, lr, bs)
        go("ewc", "eqb", 1.0, SMOOTH, WCAP, "std", KAPPA, N_HIST, HID, EPOCHS, lr, bs)

# ----- scoring (EQ4-0.7, S, P, B, D, I; PREREG.md)
def step_of_public(tv): return max(1.0, 2 * float(np.std(tv, ddof=1)) / math.sqrt(len(tv)))

def score(paths, array_carrier=ARRAY_CARRIER):
    rows = []; hdsr = {}
    for p in paths:
        for line in open(p):
            if line.startswith("{}"):
                o = json.loads(line)
                if "acc" in o: rows.append(o)
                elif "dataset" in o: hdsr[o["dataset"]] = o
    ds = sorted(set(r["dataset"] for r in rows))

    def A(name, method, mode, value=None, sm=SMOOTH, cap=WCAP, regime="std", hidden=HID, epochs=EPOCHS, lr=LR, bs=BS,
kappa=KAPPA, n_hist=N_HIST, field="acc"):
        v = sorted([r for r in rows if r["dataset"] == name and r["method"] == method and r["mode"] == mode and r["regime"] ==
regime
                    and (value is None or abs(r["value"] - value) < 1e-6) and r["smooth"] == sm and r["wcap"] == cap
                    and r["hidden"] == hidden and r["epochs"] == epochs and r["lr"] == lr and r["bs"] == bs
                    and r["kappa"] == kappa and r["n_hist"] == n_hist], key=lambda r: r["seed"])
        assert len(v) == len(SEEDS), (name, method, mode, value, sm, cap, regime, hidden, epochs, lr, bs, kappa, n_hist,
[r["seed"] for r in v])
        return np.array([r[field] for r in v], dtype=float)

    def grid(name, method, mode="fixed", regime="std", hidden=HID, epochs=EPOCHS, lr=LR, bs=BS):
        ws = sorted(set(round(r["value"], 6) for r in rows if r["dataset"] == name and r["method"] == method and r["mode"] ==
mode and r["regime"] == regime
                        and r["hidden"] == hidden and r["epochs"] == epochs and r["lr"] == lr and r["bs"] == bs))
        return {w: A(name, method, mode, w, regime=regime, hidden=hidden, epochs=epochs, lr=lr, bs=bs) for w in ws}

    def fmt(x): return f"{x.mean():.4f} [" + " ".join(f"{q:.2f}" for q in x) + "]"
    def step_of(tv): return max(1.0, 2 * tv.std(ddof=1) / math.sqrt(len(tv)))

    print("=" * 112); print("EQ4 scoring - per-seed values in brackets (seeds 0-4); thresholds from prereg/eq4/PREREG.md");
    print("=" * 112)
    R = {}
    for name in ds:
        R[name] = {}; h = hdsr.get(name, {})
        print(f"\n[{name}] K = {h.get('classes_used')} of {h.get('classes_in_file')} classes (requested
{h.get('classes_requested')}; floor {h.get('class_floor')}); class counts {h.get('class_counts')}; n = {h.get('n')} of
{h.get('n_file')} rows")
        g = grid(name, "ewc"); tuned = max(g, key=lambda w: g[w].mean()); tv = g[tuned]; step = step_of(tv)
        eq = {om: A(name, "ewc", "eq", om) for om in OMEGA_GRID_EWC}; eqb = {om: A(name, "ewc", "eqb", om) for om in
OMEGA_GRID_EWC}
        red = A(name, "ewc", "reduction"); wm = [r for r in rows if r["dataset"] == name and r["method"] == "ewc" and r["mode"]
== "reduction" and r["regime"] == "std"][0]["value"]
        bay = A(name, "ewc", "bayes", 0.0)
        print(f"    resolvable step = max(1.0, 2*SE) = {step:.4f} (tuned-lambda SE {tv.std(ddof=1) / math.sqrt(len(tv)):.4f})")
        print(f"    EWC fixed-lambda grid: " + " ".join(f"{w:g}:{g[w].mean():.2f}" for w in g))
        print(f"    EWC tuned lambda = {tuned:g}: {fmt(tv)}")
        for om in OMEGA_GRID_EWC: print(f"    EWC rule Omega={om:<4}: {fmt(eq[om])}    Omega - tuned: {eq[om].mean()-tv.mean():+.4f} |
EQ-B Omega={om:<4}: {fmt(eqb[om])}    EQ-B - tuned: {eqb[om].mean()-tv.mean():+.4f}")
        clipf = A(name, "ewc", "eqb", 1.0, field="clip_frac")
        print(f"    EQ-B clip fraction at Omega=1 (clean): {clipf.mean():.4f} [" + " ".join(f"{q:.3f}" for q in clipf) + "]")
        print(f"    EWC reduction arm (fixed w = median derived {wm:g} of EQ-B): {fmt(red)}    EQ-B(Omega=1) - reduction:
{eqb[1.0].mean()-red.mean():+.4f}")
        print(f"    EWC Bayes arm (Laplace weight {BAYES_W}): {fmt(bay)}    Bayes - tuned: {bay.mean()-tv.mean():+.4f}; EQ-B(Omega=1)
- Bayes: {eqb[1.0].mean()-bay.mean():+.4f}")
        d1 = eq[1.0] - tv; d1b = eqb[1.0] - tv
        R[name].update(step=step, tuned=tuned, tv=tv, eq=eq, eqb=eqb, d1=d1, d1b=d1b, red=red, wm=wm, g=g, bay=bay)
        sens = {}
        for cap in SENS_CAP:
            for sm in SENS_SMOOTH:
                if (cap, sm) != (WCAP, SMOOTH): sens[("cap", cap, sm)] = A(name, "ewc", "eqb", 1.0, sm, cap).mean() - tv.mean()
        for kp in SENS_KAPPA:
            for nh in SENS_NHIST:
                if (kp, nh) != (KAPPA, N_HIST): sens[("clip", kp, nh)] = A(name, "ewc", "eqb", 1.0, kappa=kp, n_hist=nh).mean()
- tv.mean()
        print("    sensitivity (EQ-B Omega=1 - tuned): " + " ".join(f"cap{b:g}/sm{c}:{v:+.2f}" if a == "cap" else
f"k{b:g}/N{c}:{v:+.2f}" for (a, b, c), v in sens.items()))
        R[name]["sens"] = sens

```

```

for mode in ("agem", "gradnorm", "mega"):
    b = A(name, "ewc", mode, 0.0); R[name][mode] = b
    print(f" EWC baseline {mode:8s}: {fmt(b)} EQ-B(Ω=1) - baseline: {eqb[1.0].mean()-b.mean():+.4f}")
    ge = grid(name, "er"); bt = max(ge, key=lambda w: ge[w].mean()); ere = A(name, "er", "eq", 1.0); erb = A(name, "er",
"eqb", 1.0); err = A(name, "er", "reduction")
    R[name].update(er_best=bt, er_bestv=ge[bt], er_eq=ere, er_eqb=erb, er_red=err)
    print(f" ER-sum grid: " + " ".join(f"{w:g}:{ge[w].mean():.2f}" for w in ge) + f"; rule Ω=1: {fmt(ere)}; EQ-B Ω=1:
{fmt(erb)} EQ-B - best fixed: {erb.mean()-ge[bt].mean():+.4f}; reduction arm: {fmt(err)} EQ-B - reduction:
{erb.mean()-err.mean():+.4f}")
    for meth in ("derpp", "lwf"):
        gc = grid(name, meth); tc = max(gc, key=lambda w: gc[w].mean()); e1 = A(name, meth, "eq", 1.0); b1 = A(name, meth,
"eqb", 1.0)
        finite = [gc[w].mean() for w in gc if all(r["finite"] for r in rows if r["dataset"] == name and r["method"] == meth
and r["mode"] == "fixed" and abs(r["value"] - w) < 1e-6 and r["regime"] == "std")]
        span = (max(finite) - min(finite)) if finite else 0.0; lb = span >= LOAD_BEARING_STEPS * step
        R[name][meth] = (tc, gc[tc], e1, b1, lb, span)
        print(f" {meth:5s} tuned w={tc:g}: {fmt(gc[tc])}; rule Ω=1: {fmt(e1)} ({e1.mean()-gc[tc].mean():+.4f}); EQ-B Ω=1:
{fmt(b1)} ({b1.mean()-gc[tc].mean():+.4f}); finite-grid span {span:.4f} -> {'load-bearing' if lb else 'inert'} (report only:
outside the narrowed H-EQ)")
        for meth in ("si", "mas"):
            gc = grid(name, meth); tc = max(gc, key=lambda w: gc[w].mean()); b1 = A(name, meth, "eqb", 1.0); R[name][meth] =
(tc, gc[tc], b1)
            print(f" {meth:5s} tuned w={tc:g}: {fmt(gc[tc])}; EQ-B Ω=1: {fmt(b1)} EQ-B - tuned:
{b1.mean()-gc[tc].mean():+.4f}")
            # poisoned regime
            gp = grid(name, "ewc", "fixed", "poison"); tp = max(gp, key=lambda w: gp[w].mean())
            gpc = grid(name, "ewc", "fixedclip", "poison"); tpc = max(gpc, key=lambda w: gpc[w].mean()); tvp = gpc[tpc]; stepp =
step_of(tvp)
            eqp = A(name, "ewc", "eq", 1.0, regime="poison"); eqbp = A(name, "ewc", "eqb", 1.0, regime="poison")
            finp = A(name, "ewc", "eq", 1.0, regime="poison", field="finite"); wmaxp = A(name, "ewc", "eq", 1.0, regime="poison",
field="w_max")
            R[name].update(p_tuned=tp, p_tv=gp[tp], pc_tuned=tpc, pc_tv=tvp, p_step=stepp, p_eq=eqp, p_eqb=eqbp, p_eq_finite=finp)
            print(f" POISON (x{POISON_FACTOR:g} for up to {POISON_RUN} batches per task from task 2): step = {stepp:.4f}; fixed
tuned lambda={tp:g}: {fmt(gp[tp])}; fixed+clip tuned lambda={tpc:g}: {fmt(tvp)}; rule Ω=1: {fmt(eqp)} (finite
{int(finp.sum())/5}; max w {np.median(wmaxp):.4g} median over seeds); EQ-B Ω=1: {fmt(eqbp)}")
            print(f" EQ-B - fixed+clip tuned: {eqbp.mean()-tvp.mean():+.4f}; rule - EQ-B: {eqp.mean()-eqbp.mean():+.4f};
fixed(no clip) tuned - fixed+clip tuned: {gp[tp].mean()-tvp.mean():+.4f}")

n = len(ds); print("\n" + "-" * 112)
lams = [R[d]["tuned"] for d in ds]; span = max(lams) / min(lams); decidable = span >= 10
print(f"EQ4-0 precondition: tuned lambda per carrier {dict(zip(ds, lams))}, span {span:.2f}x -> {'DECIDABLE' if decidable
else 'NOT DECIDABLE: tuned lambda did not move >= 10x'}")
common = max(EWC_COARSE, key=lambda w: np.mean([R[d]["g"][w].mean() for d in ds]))
loss_common = {d: R[d]["tv"].mean() - R[d]["g"][common].mean() for d in ds}
single_transfers = all(loss_common[d] < R[d]["step"] for d in ds)
nbB = {d: R[d]["d1b"].mean() > -R[d]["step"] for d in ds}; nbR = {d: R[d]["d1"].mean() > -R[d]["step"] for d in ds}
v1 = all(nbB.values()) and not single_transfers; v1r = all(nbR.values()) and not single_transfers
print(f"EQ4-1 H-EQ-B (clean): EQ-B(Ω=1) - tuned per carrier " + " ".join(f"{d}:{R[d]['d1b'].mean():+.4f}" (step
{R[d]['step']:.2f}; seeds not behind {int((R[d]['d1b'] - -R[d]['step']).sum())/5)" for d in ds)
+ f"; best single lambda={common:g} loses " + " ".join(f"{d}:{loss_common[d]:.4f}" for d in ds)
+ f"; not behind on {sum(nbB.values())}/{n} -> {'PASS' if v1 else 'FAIL'}" + (" " if decidable else " (reported without
verdict: EQ4-0 not decidable)")
print(f"EQ4-1r registered rule (EQ3-1 on fresh carriers): Ω=1 - tuned " + " ".join(f"{d}:{R[d]['d1'].mean():+.4f}" for d in
ds) + f"; not behind on {sum(nbR.values())}/{n} -> {'PASS' if v1r else 'FAIL'}")
idle = {d: abs(R[d]["eqb"][1.0].mean() - R[d]["eq"][1.0].mean()) < R[d]["step"] for d in ds}
print(f"EQ4-2 bound idle on clean streams: EQ-B(Ω=1) - rule(Ω=1) " + " ",
".join(f"{d}:{R[d]['eqb'][1.0].mean()-R[d]['eq'][1.0].mean():+.4f}" for d in ds) + f"; within a step on {sum(idle.values())}/{n}
-> {'IDLE' if all(idle.values()) else 'NOT IDLE'}")
nb3 = {d: (R[d]["p_eqb"].mean() - R[d]["pc_tv"].mean()) > -R[d]["p_step"] for d in ds}
print(f"EQ4-3 poisoned: EQ-B(Ω=1) - tuned fixed+clip " + " ".join(f"{d}:{R[d]['p_eqb'].mean()-R[d]['pc_tv'].mean():+.4f}
(step {R[d]['p_step']:.2f})" for d in ds) + f"; not behind on {sum(nb3.values())}/{n} -> {'PASS' if all(nb3.values()) else
'FAIL'}")
beh4 = {d: (R[d]["p_eq"].mean() - R[d]["p_eqb"].mean()) <= -R[d]["p_step"] for d in ds}; need = (n + 1) // 2
print(f"EQ4-4 poisoned mechanism (positive control of the enhancement): rule(Ω=1) - EQ-B(Ω=1) " + " ",
".join(f"{d}:{R[d]['p_eq'].mean()-R[d]['p_eqb'].mean():+.4f}" (rule finite {int(R[d]['p_eq_finite'].sum())/5)" for d in ds)
+ f"; rule behind EQ-B by a step on {sum(beh4.values())}/{n} (needs >= {need}) -> {'LOAD-BEARING' if
sum(beh4.values()) >= need else 'INERT: the poison does not separate the two rules on these carriers'}")
c5 = all((R[d]["er_eqb"].mean() - R[d]["er_bestv"].mean()) < R[d]["step"] and abs(R[d]["er_eqb"].mean() -
R[d]["er_red"].mean()) < R[d]["step"] for d in ds)
print(f"EQ4-5 control ER-sum: EQ-B - best fixed " + " ".join(f"{d}:{R[d]['er_eqb'].mean()-R[d]['er_bestv'].mean():+.4f}"
for d in ds)
+ "; EQ-B - reduction " + " ".join(f"{d}:{R[d]['er_eqb'].mean()-R[d]['er_red'].mean():+.4f}" for d in ds) + f" ->
{'holds' if c5 else 'VIOLATED'}")
reduces = all(abs(R[d]["eqb"][1.0].mean() - R[d]["red"].mean()) < R[d]["step"] for d in ds)
print(f"EQ4-6 reduction: EQ-B - fixed@median per carrier " + " ",
".join(f"{d}:{R[d]['eqb'][1.0].mean()-R[d]['red'].mean():+.4f}" for d in ds)
+ f" -> {'REDUCES to a per-carrier constant' if reduces else 'DOES NOT REDUCE: normalised-gradient method'}")
dom = {m: all((R[d][m].mean() - R[d]["eqb"][1.0].mean()) >= R[d]["step"] for d in ds) for m in ("agem", "gradnorm", "mega")}
print(f"EQ4-7 published baselines vs EQ-B(Ω=1): " + " ".join(f"{d}/{m}:{R[d][m].mean()-R[d]['eqb'][1.0].mean():+.4f}" for d
in ds for m in ("agem", "gradnorm", "mega"))
+ " -> " + " (" + " ".join(f"{m} dominates EQ-B for m, v in dom.items() if v) or 'no published baseline is ahead of EQ-B
by a step on every carrier'")
flips = sum(1 for d in ds for k, v in R[d]["sens"].items() if (v > -R[d]["step"]) != nbB[d])
print(f"EQ4-8 sensitivity: EQ4-1 'not behind' flips in {flips} of {16*n} cells -> {'FRAGILE' if flips > 1 else 'not
fragile'}")
partsP = []; widths = []
for d in ds:
    means = {o: R[d]["eqb"][o].mean() for o in OMEGA_GRID_EWC}; b = max(means.values())
    within = [o for o in OMEGA_GRID_EWC if means[o] >= b - R[d]["step"]]
    idx = [OMEGA_GRID_EWC.index(o) for o in within]; contiguous = (max(idx) - min(idx) + 1 == len(idx))

```

```

        widths.append(len(within)); partsP.append(f"{d}: {len(within)} of 9 within a step of the best
({min(within):g}..{max(within):g}, {'contiguous' if contiguous else 'not contiguous'})")
    print("EQ4-P plateau width of EQ-B on the nine-point grid: " + " ".join(partsP) + f"; median width
{float(np.median(widths)):g} (report only)")
    nbBay = sum(1 for d in ds if (R[d]["bay"].mean() - R[d]["tv"].mean()) > -R[d]["step"])
    print("EQ4-B Bayes arm (Laplace weight 1/2): Bayes - tuned " + " ".join(f"{d}:{R[d]['bay'].mean()-R[d]['tv'].mean():+.4f}"
for d in ds)
        + " ; EQ-B( $\Omega$ =1) - Bayes " + " ".join(f"{d}:{R[d]['eqb'][1.0].mean()-R[d]['bay'].mean():+.4f}" for d in ds) + f"; Bayes
not behind tuned on {nbBay}/{n} (report only)")
    print("EQ4-D DER++/LwF (outside the narrowed H-EQ; report only): EQ-B( $\Omega$ =1) - tuned " + " ",
".join(f"{d}/{m}:{R[d][m][3].mean()-R[d][m][1].mean():+.4f}" if R[d][m][4] else ' (inert)' )" for d in ds for m in ("derpp",
"lwf"))
        + " ; SI/MAS: " + " ".join(f"{d}/{m}:{R[d][m][2].mean()-R[d][m][1].mean():+.4f}" for d in ds for m in ("si", "mas")))
    if array_carrier in ds:
        cells = [(LR, BS)] + list(INVAR_CELLS); nbI = 0; partsI = []
        for lr, bs in cells:
            g_ = grid(array_carrier, "ewc", lr=lr, bs=bs); tuned_ = max(g_, key=lambda w: g_[w].mean()); tv_ = g_[tuned_]
            step_ = step_of(tv_); e1 = A(array_carrier, "ewc", "eqb", 1.0, lr=lr, bs=bs)
            nb = (e1.mean() - tv_.mean()) > -step_; nbI += nb
            partsI.append(f"lr{lr:g}/bs{bs}: tuned {tuned_:g} -> {tv_.mean():.2f}, EQ-B {e1.mean():.2f}, diff {e1.mean() -
tv_.mean():+.2f} (step {step_:2f}) {'nb' if nb else 'behind'}")
            print(f"EQ4-I lr x batch invariance on {array_carrier}: " + " ".join(partsI) + f" -> not behind in {nbI}/5 cells ->
{'PASS' if nbI >= 4 else 'FAIL'}")
            print(f"summary: EQ4-1 {'PASS' if v1 else 'FAIL'} (EQ-B not behind on {sum(nbB.values())}/{n}); EQ4-1r {'PASS' if v1r else
'FAIL'} (rule not behind on {sum(nbR.values())}/{n}); bound idle: {all(idle.values())}; poisoned EQ4-3 {'PASS' if
all(nb3.values()) else 'FAIL'}; mechanism EQ4-4 {'LOAD-BEARING' if sum(beh4.values()) >= need else 'INERT'}; fragile: {flips >
1}; ER-sum control {'holds' if c5 else 'violated'}")

# ----- main
def _load(name):
    K_req, per_task = DATASETS.get(name, (6, 2))
    X, y, meta = load_pmlb(name, K_req)
    if meta["excluded"]:
        return None, meta["classes_used"], per_task, meta
    return split_standardise(X, y, meta["classes_used"], meta["classes_used"], per_task, meta

def _synthetic(K=10, n=1500, d=20):
    rng = np.random.default_rng(0)
    X = rng.standard_normal((n, d)); y = rng.integers(0, K, n); X[np.arange(n), y % d] += 3.0
    return X, y

if __name__ == "__main__":
    cmd = sys.argv[1]
    if cmd == "smoke":
        X, y = _synthetic(); K = 10
        data = split_standardise(X, y, K)
        for a in (("ewc", "fixed", 10.0), ("ewc", "fixedclip", 10.0), ("ewc", "eq", 1.0), ("ewc", "eqb", 1.0), ("ewc", "bayes",
0.0), ("ewc", "reduction", 5.0), ("ewc", "agem", 0.0), ("ewc", "gradnorm", 0.0), ("ewc", "mega", 0.0),
            ("er", "fixed", 1.0), ("er", "eq", 1.0), ("er", "eqb", 1.0), ("er", "reduction", 1.0), ("derpp", "fixed",
0.1), ("derpp", "eq", 1.0), ("derpp", "eqb", 1.0), ("lwf", "fixed", 1.0), ("lwf", "eq", 1.0), ("lwf", "eqb", 1.0),
            ("si", "fixed", 0.1), ("si", "eqb", 1.0), ("mas", "fixed", 0.1), ("mas", "eqb", 1.0)):
            print(json.dumps(run(*a, 0, *data, K, 2)))
        for a in (("ewc", "fixed", 10.0), ("ewc", "fixedclip", 10.0), ("ewc", "eq", 1.0), ("ewc", "eqb", 1.0)):
            print(json.dumps(run(*a, 0, *data, K, 2, poison=True)))
        for kp, nh in ((1.5, 20), (4.0, 200)): print(json.dumps(run("ewc", "eqb", 1.0, 0, *data, K, 2, kappa=kp, n_hist=nh)))
        for lr, bs in INVAR_CELLS: print(json.dumps(run("ewc", "eqb", 1.0, 0, *data, K, 2, lr=lr, bs=bs)))
    elif cmd == "smokefull":
        outp = sys.argv[sys.argv.index("--out") + 1] if "--out" in sys.argv else "/tmp/eq4_smokefull.jsonl"
        X, y = _synthetic(); K = 10
        meta = dict(file="synthetic", sha256="none", n=len(y), n_file=len(y), d=X.shape[1], classes_in_file=K,
classes_requested=K, classes_used=K,
            rows_dropped_nan=0, class_counts_file=[int((y == c).sum()) for c in range(K)], class_counts=[int((y ==
c).sum()) for c in range(K)],
            class_floor=CLASS_FL00R, excluded=False)
        with open(outp, "w") as out:
            run_all("synthetic", split_standardise(X, y, K), K, 2, meta, out, array_carrier="synthetic")
            score([outp], array_carrier="synthetic")
    elif cmd == "run":
        name, method, mode, value, seed = sys.argv[2], sys.argv[3], sys.argv[4], float(sys.argv[5]), int(sys.argv[6])
        regime = sys.argv[sys.argv.index("--regime") + 1] if "--regime" in sys.argv else "std"
        data, K, per_task, meta = _load(name)
        if data is None: print(json.dumps(dict(dataset=name, excluded=True, **meta))); sys.exit(0)
        o = run(method, mode, value, seed, *data, K, per_task, poison=(regime == "poison")); o["dataset"] = name; o["regime"] =
regime; print(json.dumps(o))
    elif cmd == "all":
        name = sys.argv[2]
        outp = sys.argv[sys.argv.index("--out") + 1] if "--out" in sys.argv else f"runs/eq4/results_{name}.jsonl"
        data_std, K, per_task, meta = _load(name)
        with open(outp, "w") as out:
            if data_std is None:
                print(json.dumps(dict(dataset=name, **meta)), file=out, flush=True) # excluded carrier: header only
            else:
                run_all(name, data_std, K, per_task, meta, out)
    elif cmd == "score":
        score(sys.argv[2:])
    else:
        raise SystemExit(__doc__)

```

A.9 The reprocessed mathematics check

theory/checks/omega_reprocessed.py

```
"""H-EQ reprocessed after the cross-verification (owner request 2026-09-22, prompt-log entry 74).
What the cross-verification taught (Continuous_Learning/CROSS_VERIFICATION.md section 4): the rule's one distinct
property is that no number in it carries the past term's units, so its step is bounded by the present step; it is
blind to the shape of the past curvature; it presumes the past gradient's length measures distance from the past;
and it answers a single extreme present batch by amplifying the past step in proportion. Each of those is made a
checkable statement here on the two-task quadratic of omega_sweeps.py, and the enhanced rule that follows from them
(EQ-B, "bounded equanimity": the present gradient clipped at kappa x the largest kept length among its recent batches,
the ratio then taken on the clipped gradient) is run beside the registered rule. Every label is computed (R15).
Pinned: omega_reprocessed.txt. Run: uv run python theory/checks/omega_reprocessed.py
"""

from __future__ import annotations

import sys
from pathlib import Path

import numpy as np

sys.path.insert(0, str(Path(__file__).resolve().parent))
import omega_sweeps as om # noqa: E402

LR = 0.05; STEPS = 4000; NOISE = 0.5; SEEDS = 5
KAPPA = 2.0; N_HIST = 50; N_MIN = 10 # EQ-B clip: kappa x the largest kept length among the last N_HIST present
batches, active once N_MIN are known
POISON_STEP = 2000; POISON = 100.0 # one present batch with its gradient multiplied by POISON
ADAM_LR = 0.01; B1 = 0.9; B2 = 0.999; EPS = 1e-8

def clip_present(g_p, hist, kappa=KAPPA, n_hist=N_HIST, n_min=N_MIN):
    """EQ-B present bound: clip to kappa x the largest KEPT length among the last n_hist present batches (no clip until n_min
    are known). The history holds kept lengths, so a run of extreme batches raises the bound by at most kappa per batch. The
    window maximum, not the median, is the reference: on a softmax stream the median present-gradient length collapses toward
    zero as a task is learned and a median bound clips the informative batches (src/crr/surrogates/battery.py _eqb_clip,
    AGENT_LOG 63)."""
    n = np.linalg.norm(g_p)
    tau = kappa * float(np.max(hist[-n_hist:])) if len(hist) >= n_min else np.inf
    if n > tau:
        hist.append(tau); return g_p * (tau / n), True, tau
    hist.append(n); return g_p, False, tau

def traj(method, knob, H, F, a, b, seed, lr=LR, steps=STEPS, noise=NOISE, poison=None, optimizer="sgd", record=False,
noise_q=None):
    """method: fixed[lambda] | eq[Omega] | eqb[Omega] | fixedclip[lambda]. poison=(step, factor) multiplies that step's
    present gradient. optimizer 'sgd' or 'adam' (the rule applied UPSTREAM of Adam: w multiplies the raw past gradient,
    Adam preconditions the sum). Returns theta, ok, and per-step diagnostics."""
    rng = np.random.default_rng(seed); noise_q = noise if noise_q is None else noise_q
    th = b.copy(); ema_p = ema_q = None; hist = []; m = np.zeros_like(th); v = np.zeros_like(th)
    wlog = []; ratio_raw = []; ratio_ema = []; step_abs = []; clipped = 0; path = []
    for t in range(steps):
        g_p = H @ (th - a) + noise * rng.standard_normal(len(th)); g_q = F @ (th - b) + noise_q * rng.standard_normal(len(th))
        if poison is not None and poison[0] <= t < poison[0] + poison[2]: g_p = poison[1] * g_p
        if method in ("eqb", "fixedclip"):
            g_p, was, _tau = clip_present(g_p, hist); clipped += int(was)
        if method in ("fixed", "fixedclip"):
            w = knob
        else:
            ema_p = g_p if ema_p is None else om.SMOOTH * ema_p + (1 - om.SMOOTH) * g_p
            ema_q = g_q if ema_q is None else om.SMOOTH * ema_q + (1 - om.SMOOTH) * g_q
            w = om.rule_w(ema_p, ema_q, knob)
            ratio_raw.append(np.linalg.norm(g_p + w * g_q) / max(np.linalg.norm(g_p), om.DEN_FLOOR))
            ratio_ema.append(np.linalg.norm(ema_p + w * ema_q) / max(np.linalg.norm(ema_p), om.DEN_FLOOR))
        wlog.append(w)
        step = g_p + w * g_q; step_abs.append(float(np.linalg.norm(step)))
        if optimizer == "adam":
            m = B1 * m + (1 - B1) * step; v = B2 * v + (1 - B2) * step * step
            mh = m / (1 - B1 ** (t + 1)); vh = v / (1 - B2 ** (t + 1))
            th = th - ADAM_LR * mh / (np.sqrt(vh) + EPS)
        else:
            th = th - lr * step
        if record: path.append(th.copy())
        if not np.all(np.isfinite(th)) or np.linalg.norm(th) > 1e6:
            return th, False, dict(w=wlog, ratio_raw=ratio_raw, ratio_ema=ratio_ema, step_abs=step_abs, clipped=clipped,
path=path, t_fail=t)
        return th, True, dict(w=wlog, ratio_raw=ratio_raw, ratio_ema=ratio_ema, step_abs=step_abs, clipped=clipped, path=path,
t_fail=None)

def evaluate(method, knob, H, F, a, b, **kw):
    tot = []; ok_all = True; Lqs = []
    for s in range(SEEDS):
        th, ok, _ = traj(method, knob, H, F, a, b, s, **kw); ok_all = ok_all and ok
        Lp, Lq = om.losses(H, F, a, b, th) if ok else (np.nan, np.nan); tot.append(Lp + Lq); Lqs.append(Lq)
    return float(np.mean(tot)), ok_all, float(np.mean(Lqs))

GRID = (0.003, 0.01, 0.03, 0.1, 0.3, 0.6, 1.0, 1.2, 1.5, 3.0)
```

```

def tuned(method, H, F, a, b, **kw):
    best = None
    for lam in GRID:
        tot, ok, _ = evaluate(method, lam, H, F, a, b, **kw)
        if ok and (best is None or tot < best[1]): best = (lam, tot)
    return best

def main():
    H, F, a, b = om.make_model()
    print("H-EQ reprocessed (2026-09-22, prompt-log entry 74): the two-task quadratic of omega_sweeps.py (d = 5, F 16x H, noise
0.5, lr 0.05, 4000 steps, 5 seeds, start at the past optimum b)")
    print()
    # [1] scale invariance (exact) and shape dependence
    print("[1] The rule is invariant to the SCALE of the past term (exactly, when the past term's noise scales with it) and not
to its SHAPE")
    c = 16.0
    perm = np.array([4, 3, 2, 1, 0]); F_perm = np.diag(np.diag(F)[perm]) # same eigenvalues, reassigned to other
coordinates
    d_scaled = []; d_fixed = []; d_shape = []
    for s in range(SEEDS):
        thF, okF, dF = traj("eq", 1.0, H, F, a, b, s, record=True)
        thc, okc, dc = traj("eq", 1.0, H, c * F, a, b, s, record=True, noise_q=c * NOISE) # mini-batch noise of a 16x term
is 16x
        thu, oku, du = traj("eq", 1.0, H, c * F, a, b, s, record=True) # noise left at 0.5 on both
        thp, okp, _ = traj("eq", 1.0, H, F_perm, a, b, s)
        d_scaled.append(max(np.linalg.norm(p - q) for p, q in zip(dF["path"], dc["path"]))) if okF and okc else np.nan)
        d_fixed.append(max(np.linalg.norm(p - q) for p, q in zip(dF["path"], du["path"]))) if okF and oku else np.nan)
        d_shape.append(np.linalg.norm(thF - thp) if okF and okp else np.nan)
    d_scaled = float(np.nanmax(d_scaled)); d_fixed = float(np.nanmax(d_fixed)); d_shape = float(np.nanmean(d_shape))
    print(f"max over steps and seeds of |theta_t(F) - theta_t({c:g}F)| under the rule, past noise scaled with the term:
{d_scaled:.3e} -> {'identical trajectories: w(cF) = w(F)/c exactly, the EMA being linear in g_q' if d_scaled < 1e-9 else
'trajectories differ'}")
    print(f"the same with the past noise left at {NOISE:g} (not scaled): {d_fixed:.3e} -> {'the invariance is to the term,
not to a noise of fixed size added to it' if d_fixed > 1e-6 else 'identical'}")
    print(f"mean endpoint distance between F and F with its diagonal reversed (same eigenvalues, other coordinates):
{d_shape:.4f} -> {'the shape of F changes the endpoint; the rule does not see shape' if d_shape > 1e-3 else 'no shape
dependence'}")
    print("consequence: every method that changes the importance estimate (which coordinate is protected how much) composes
with the rule and is untouched by it; the 16F test of omega_vs_methods.py measures scale only")
    print()
    # [2] the step bound
    print("[2] The step bound |g_p + w g_q| <= (1 + Omega) |g_p|: on which gradients does it hold, and what does EQ-B add to
it?")
    absmax = {}
    for name, meth in (("rule", "eq"), ("EQ-B", "eqb")):
        rr = []; re = []; am = []
        for s in range(SEEDS):
            _, ok, d = traj(meth, 1.0, H, F, a, b, s); rr += d["ratio_raw"]; re += d["ratio_ema"]; am += d["step_abs"]
        rr = np.array(rr); re = np.array(re); absmax[name] = float(np.max(am))
        print(f"{name:5s} smoothed: max |ema_p + w ema_q| / |ema_p| = {re.max():.4f}, fraction above 2: {float(np.mean(re >
2)):.4f} -> {'bound holds' if re.max() <= 2 + 1e-9 else 'bound violated on the smoothed gradients'}")
        print(f"{name:5s} raw: max |g_p + w g_q| / |g_p| = {rr.max():.4f}, fraction above 2: {float(np.mean(rr >
2)):.4f}, median {np.median(rr):.4f}; max |g_p + w g_q| = {absmax[name]:.4f}")
        print(f"computed: the ratio bound is a bound on the smoothed pull; the raw ratio exceeds 2 on a fraction of steps for
both rules (a small |g_p| in the denominator, and EQ-B {'lowers' if absmax['EQ-B'] < absmax['rule'] else 'does not lower'} the
largest raw step ({absmax['EQ-B']:.4f} vs {absmax['rule']:.4f}): what EQ-B bounds is the present gradient's LENGTH (kappa x the
largest kept length of its recent batches), hence the absolute step, not the ratio")
        print()
    # [3] a rank-deficient past term (a constraint, not a loss on the past)
    print("[3] Does a past term whose minimum is a SET (rank-deficient F: a constraint on a subspace) break the rule's premise
that |g_q| measures distance from the past? (the candidate mechanism for the two-signed LwF control of EQ3-4)")
    F_def = F.copy(); F_def[3, 3] = 0.0; F_def[4, 4] = 0.0
    for name, FF in (("full-rank F", F), ("rank-3 F (two free directions)", F_def)):
        wmax = []; capfrac = []; tot = []; okk = True
        for s in range(SEEDS):
            th, ok, d = traj("eq", 1.0, H, FF, a, b, s); okk = okk and ok
            w = np.array(d["w"]); wmax.append(w.max()); capfrac.append(float(np.mean(w >= om.WCAP)))
            tot.append(sum(om.losses(H, FF, a, b, th)) if ok else np.nan)
        # analytic: fixed points need g_p antiparallel to g_q; max |theta - a| in range(F) that needs H(theta - a) in range(F), which
the present optimum a satisfies only if the null-space components of theta - a vanish
        print(f"{name:32s} max w {np.nanmax(wmax):.4g}, fraction of steps at the cap {np.mean(capfrac):.4f}, finite {okk},
total loss {np.nanmean(tot):.4f}")
        # where does the trajectory go in the free directions?
        drift = []
        for s in range(SEEDS):
            th, ok, _ = traj("eq", 1.0, H, F_def, a, b, s)
            if ok: drift.append(np.abs(th[3:] - a[3:]).max())
        print(f"free directions under rank-3 F: max |theta - a| over the two free coordinates at the end = {np.mean(drift):.4f}
(the present optimum is reached there: the constraint says nothing about them, and the rule's w is driven by the constrained
directions only)")
        w_full = [np.median(traj('eq', 1.0, H, F, a, b, s)[2]['w']) for s in range(SEEDS)]; w_def = [np.median(traj('eq', 1.0, H,
F_def, a, b, s)[2]['w']) for s in range(SEEDS)]
        set_breaks = abs(np.mean(w_def) - np.mean(w_full)) > 0.05 * np.mean(w_full) or np.mean(capfrac) > 0
        print(f"median derived w: full-rank {np.mean(w_full):.4f}, rank-3 {np.mean(w_def):.4f} -> {'the set-valued minimum
moves the derived weight: a candidate mechanism' if set_breaks else 'NO: on a quadratic constraint the free directions relax to
the present optimum and the derived weight is unchanged; the quadratic model supplies no mechanism for the two-signed LwF
control, which stays an empirical statement (EQ3-4) with the softmax non-identifiability argument as the open candidate'}")

```

```

print()
# [4] the poisoned batch: a sweep over factor and run length; the failure threshold is read off, not chosen
print("[4] Poisoned present batches (gradient x factor for `run` consecutive steps from step 2000): which rule fails first,
and whether EQ-B (present gradient clipped at kappa x the largest kept length of its recent batches) survives")
t1 = tuned("fixed", H, F, a, b)
print(f"    clean reference: tuned fixed lambda = {t1[0]:g}, total {t1[1]:.4f}; clean totals: rule {evaluate('eq', 1.0, H,
F, a, b)[0]:.4f}, EQ-B {evaluate('eqb', 1.0, H, F, a, b)[0]:.4f}")
arms = (("fixed", "fixed", t1[0]), ("fixed+clip", "fixedclip", t1[0])), ("rule", "eq", 1.0), ("EQ-B", "eqb", 1.0))
first_fail = {name: None for name, _, _ in arms}; table = []
print(f"    {'factor':>7s} {'run':>4s} | " + " | ".join(f"{name:>22s}" for name, _, _ in arms) + "    (total after 4000
steps; DIVERGED if any seed is non-finite; max w of the rule arms)")
for factor in (10.0, 100.0, 1000.0, 10000.0):
    for run_len in (1, 5):
        cells = []
        for name, meth, knob in arms:
            tot, ok, _ = evaluate(meth, knob, H, F, a, b, poison=(POISON_STEP, factor, run_len))
            wmax = max(max(traj(meth, knob, H, F, a, b, s, poison=(POISON_STEP, factor, run_len))[2]["w"]) for s in
range(SEEDS))
            cells.append(f"[tot:10.4f] w<={wmax:8.3g}" if ok else f"{'DIVERGED':>10s} w<={wmax:8.3g}")
            if not ok and first_fail[name] is None: first_fail[name] = (factor, run_len)
            table.append((factor, run_len, cells))
        print(f"    {'factor':7g} {'run_len':4d} | " + " | ".join(f"{c:>22s}" for c in cells))
    print("    computed: first (factor, run) at which each arm diverges, in sweep order: " + "; ".join(f"{k}: {v if v else
'never'}" for k, v in first_fail.items()))
    clean_tot = {name: evaluate(meth, knob, H, F, a, b)[0] for name, meth, knob in arms}
    first_degrade = {name: None for name, _, _ in arms}
    for factor, run_len, cells in table:
        for (name, _, _), cell in zip(arms, cells):
            bad = cell.strip().startswith("DIVERGED") or float(cell.split()[0]) > 2 * clean_tot[name]
            if bad and first_degrade[name] is None: first_degrade[name] = (factor, run_len)
    print("    computed: first (factor, run) at which each arm's total exceeds twice its clean total (or diverges): " + ";
.join(f"{k}: {v if v else 'never'}" for k, v in first_degrade.items()))
    rule_deg = first_degrade["rule"]; fixed_deg = first_degrade["fixed"]
    print(f"    computed: the registered rule degrades before the tuned fixed weight does: {rule_deg is not None and (fixed_deg
is None or rule_deg < fixed_deg)}; EQ-B degrades in the sweep: {first_degrade['EQ-B'] is not None}")
    rule_fails = first_fail["rule"] is not None; eqb_ok = first_fail["EQ-B"] is None; fixed_fails = first_fail["fixed"] is not
None
    print(f"    computed: the registered rule has a poison threshold in this sweep: {rule_fails}; EQ-B survives the whole
sweep: {eqb_ok}; the tuned fixed weight has a threshold: {fixed_fails}"
    + (f"; the rule fails where the fixed weight does not: {rule_fails and (not fixed_fails or first_fail['rule'] <
first_fail['fixed'])}" if rule_fails else ""))
    eqb_c = evaluate('eqb', 1.0, H, F, a, b)[0]; eq_c = evaluate('eq', 1.0, H, F, a, b)[0]
    print(f"    computed: on the clean stream EQ-B is within 1 % of the registered rule: {abs(eqb_c - eq_c) <= 0.01 * eq_c}
(totals {eqb_c:.4f} vs {eq_c:.4f}); the bound is {'idle on the clean stream' if abs(eqb_c - eq_c) <= 0.01 * eq_c else 'NOT idle
on the clean stream'})")
    print()
# [5] the Fisher-norm ratio (H-EQ as written) has the same fixed-point set
print("[5] The Fisher-norm ratio of H-EQ as written ( $\lfloor \cdot \rfloor_F^{-1}$  on both gradients) has the same fixed-point set as the
Euclidean one: any positive scalar w needs  $g_p$  antiparallel to  $g_q$ , i.e. theta on the Pareto curve")
Finv = np.linalg.inv(F)
def fnorm(g): return float(np.sqrt(g @ Finv @ g))
ends = []
for s in range(SEEDS):
    rng = np.random.default_rng(s); th = b.copy(); ema_p = ema_q = None
    for t in range(STEPS):
        g_p = H @ (th - a) + NOISE * rng.standard_normal(5); g_q = F @ (th - b) + NOISE * rng.standard_normal(5)
        ema_p = g_p if ema_p is None else om.SMOOTH * ema_p + (1 - om.SMOOTH) * g_p
        ema_q = g_q if ema_q is None else om.SMOOTH * ema_q + (1 - om.SMOOTH) * g_q
        w = min(fnorm(ema_p) / max(fnorm(ema_q), om.DEN_FLOOR), om.WCAP)
        th = th - LR * (g_p + w * g_q)
    ends.append(th)
lams = np.logspace(-3, 3, 601)
def lam_of(th):
    j = int(np.argmax([np.linalg.norm(th - om.pareto(H, F, a, b, l)) for l in lams])); return lams[j], np.linalg.norm(th -
om.pareto(H, F, a, b, lams[j]))
eu = [lam_of(traj("eq", 1.0, H, F, a, b, s)[0]) for s in range(SEEDS)]; fi = [lam_of(th) for th in ends]
print(f"    Euclidean ratio: nearest Pareto lambda {np.mean([e in eu] for e in eu):.4f} at distance {np.mean([e[1] for e in
eu]):.4f} | Fisher ratio: lambda {np.mean([f[0] for f in fi]):.4f} at distance {np.mean([f[1] for f in fi]):.4f}")
print(f"    computed: both endpoints sit within 0.2 of the curve: {all([e[1] < 0.2 for e in eu]) and all([f[1] < 0.2 for f in
fi])}; the metric moves the stopping point along the curve, it does not change the set (A1's metric is a choice of stopping
point, not of mechanism)")
print()
# [6] Adam
print("[6] Under Adam (the rule upstream of the preconditioner) the divergence edge disappears for every method and the
scale test becomes a question of endpoint quality")
tA = tuned("fixed", H, F, a, b, optimizer="adam"); tA16 = tuned("fixed", H, 16 * F, a, b, optimizer="adam")
print(f"    tuned fixed lambda under Adam: {tA[0]:g} (total {tA[1]:.4f}) at F; {tA16[0]:g} (total {tA16[1]:.4f}) at 16F")
res = {}
for name, meth, knob in (("fixed lambda (tuned at F)", "fixed", tA[0]), ("rule Omega = 1", "eq", 1.0), ("EQ-B Omega = 1",
"eqb", 1.0)):
    r1 = evaluate(meth, knob, H, F, a, b, optimizer="adam"); r16 = evaluate(meth, knob, H, 16 * F, a, b, optimizer="adam")
    res[name] = (r1[0] / tA[1], r16[0] / tA16[1], r1[1], r16[1])
    print(f"    {name:26s} F: total {r1[0]:.4f} (/tuned {r1[0] / tA[1]:.3f}, finite {r1[1]}) | 16F, same knob: total
{r16[0]:.4f} (/tuned {r16[0] / tA16[1]:.3f}, finite {r16[1]})")
    robust = [k for k, v in res.items() if v[2] and v[3] and v[0] <= 1.10 and v[1] <= 1.10]
    adam_fixed_robust = "fixed lambda (tuned at F)" in robust
    print(f"    computed: scale-robust under Adam (within 10 % of the Adam-tuned fixed weight at both scales): {robust}; every
arm finite: {all([v[2] and v[3] for v in res.values()])}")
    print(f"    computed: {'the fixed weight is scale-robust under Adam too: the rule has no scale advantage under Adam in this
model, its property is an SGD property' if adam_fixed_robust else 'the fixed weight is not scale-robust under Adam; the rule"}

```

```

keeps its advantage'})")
    print()
    print("[7] What changes in H-EQ (proposal for the v3.2 list; theory/CRR.md is not edited here), each line conditional on the
numbers above:")
    print(f"      (a) scope: {'a set-valued past minimum moves the derived weight ([3]): constraints are outside the clause by
mechanism' if set_breaks else 'the quadratic model gives NO mechanism for excluding constraint terms ([3]); the narrowing of
H-EQ to losses on the past rests on EQ3-4 (empirical, two-signed) and is stated as an empirical scope, not a derived one'};")
    print(f"      (b) the bound is the mechanism ([2]): the claim is 'the past pull never exceeds (1 + Omega) x the present pull
in the smoothed gradients', not 'equal pull is optimal' (omega_sweeps.txt [1]-[2]);")
    print(f"      (c) present bound: {'the registered rule degrades at poison ' + str(rule_deg) + ' (diverges at ' +
str(first_fail['rule']) + ') where EQ-B ' + ('survives the whole sweep' if first_degrade['EQ-B'] is None else 'degrades too') +
' ([4]); EQ-B is the enhancement to test, with the poisoned stream as its positive control and the clean stream as its negative
control (the bound must be idle there)' if rule_deg is not None else 'no poison in the sweep degrades the registered rule ([4]);
EQ-B has no positive control in this model and would need one before any prereg'};")
    print(f"      (d) the metric is a stopping point on the Pareto curve, not a mechanism ([5]); (e) the optimiser is a
registered parameter ([6]){', and under Adam the fixed weight is scale-robust too, so the scale claim is SGD-only' if
adam_fixed_robust else ''}."
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

Appendix B. The pinned outputs

B.1 EQ3 scoring (`runs/eq3/score.txt`)

runs/eq3/score.txt

```
=====
EQ3 scoring – per-seed values in brackets (seeds 0-4); thresholds from prereg/eq3/PREREG.md
=====

[fars] resolvable step = max(1.0, 2*SE) = 3.3508 (tuned-lambda SE 1.6754)
EWC fixed-lambda grid: 0.1:9.28 0.3:9.28 1:9.28 3:9.28 10:9.28 30:9.26 35.3357:9.26 50:9.26 70.922:9.96 100:10.06
141:10.42 200:12.56 283:13.00 300:9.58 1000:0.00 3000:0.00 10000:0.00
EWC tuned lambda = 283: 13.0000 [18.00 15.70 9.20 12.00 10.10]
EWC rule  $\Omega=0.25$ : 9.2600 [9.20 9.30 9.20 9.30 9.30]  $\Omega$  – tuned: -3.7400
EWC rule  $\Omega=0.35$ : 9.2600 [9.20 9.30 9.20 9.30 9.30]  $\Omega$  – tuned: -3.7400
EWC rule  $\Omega=0.5$ : 9.2600 [9.20 9.30 9.20 9.30 9.30]  $\Omega$  – tuned: -3.7400
EWC rule  $\Omega=0.71$ : 9.2600 [9.20 9.30 9.20 9.30 9.30]  $\Omega$  – tuned: -3.7400
EWC rule  $\Omega=1.0$ : 9.2600 [9.20 9.30 9.20 9.30 9.30]  $\Omega$  – tuned: -3.7400
EWC rule  $\Omega=1.41$ : 9.2600 [9.20 9.30 9.20 9.30 9.30]  $\Omega$  – tuned: -3.7400
EWC rule  $\Omega=2.0$ : 9.2800 [9.20 9.30 9.20 9.40 9.30]  $\Omega$  – tuned: -3.7200
EWC rule  $\Omega=2.83$ : 9.2600 [9.20 9.30 9.20 9.30 9.30]  $\Omega$  – tuned: -3.7400
EWC rule  $\Omega=4.0$ : 9.2600 [9.20 9.30 9.20 9.30 9.30]  $\Omega$  – tuned: -3.7400
EWC reduction arm (fixed w = median derived 426.287): 6.8400 [0.00 0.00 9.60 13.90 10.70] rule( $\Omega=1$ ) – reduction: +2.4200
EWC Bayes arm (Laplace weight 0.5 on the task-size-weighted Fisher): 9.2800 [9.30 9.30 9.20 9.30 9.30] Bayes – tuned:
-3.7200; rule( $\Omega=1$ ) – Bayes: -0.0200
sensitivity ( $\Omega=1$  – tuned): cap10/sm0.8:-3.72 cap10/sm0.9:-3.72 cap10/sm0.98:-3.72 cap100/sm0.8:-3.74 cap100/sm0.9:-3.74
cap100/sm0.98:-3.74 cap10000/sm0.8:-3.74 cap10000/sm0.9:-3.74 cap10000/sm0.98:-3.72
EWC baseline agem : 9.2800 [9.30 9.30 9.20 9.30 9.30] rule( $\Omega=1$ ) – baseline: -0.0200
EWC baseline gradnorm: 16.3600 [18.30 10.00 9.20 23.40 20.90] rule( $\Omega=1$ ) – baseline: -7.1000
EWC baseline mega : 9.2800 [9.30 9.30 9.20 9.30 9.30] rule( $\Omega=1$ ) – baseline: -0.0200
ER-sum grid: 0.5:52.02 1:58.98 2:64.42 4:69.14; rule  $\Omega=1$ : 64.4600 [64.20 58.30 65.50 68.00 66.30] rule – best fixed:
-4.6800; reduction arm: 59.8400 [59.80 54.20 62.00 62.60 60.60] rule – reduction: +4.6200
derpp tuned w=0.3: 58.8800 [60.30 52.70 61.10 62.80 57.50]; rule best  $\Omega=1.41$ : 66.2600 [66.30 67.90 67.70 65.70 63.70] best- $\Omega$ 
– tuned: +7.3800;  $\Omega=1$  – tuned: +3.1600; finite-grid span 6.3200 -> INERT (excluded from EQ3-4)
lwf tuned w=10: 41.6800 [41.70 41.70 41.70 41.60 41.70]; rule best  $\Omega=1.41$ : 41.7000 [41.70 41.70 41.70 41.70 41.70] best- $\Omega$ 
– tuned: +0.0200;  $\Omega=1$  – tuned: -0.0400; finite-grid span 32.4000 -> load-bearing
si tuned w=0.1: 11.6800 [8.30 11.40 12.20 15.50 11.00]; rule  $\Omega=1$ : 10.4400 [12.20 9.30 10.40 10.30 10.00]  $\Omega=1$  – tuned:
-1.2400
mas tuned w=0.3: 41.3400 [33.60 43.60 47.70 48.90 32.90]; rule  $\Omega=1$ : 44.4800 [35.10 43.80 53.50 51.40 38.60]  $\Omega=1$  – tuned:
+3.1400
x4 regime: tuned lambda=3: 10.4200 [9.40 9.40 8.90 14.90 9.50]; rule  $\Omega=1$ : 9.5000 [9.50 9.40 9.80 10.00 8.80]  $\Omega=1$  – tuned:
-0.9200; reduction: 7.1800 [16.80 0.00 8.90 0.00 10.20]

[krkopt] resolvable step = max(1.0, 2*SE) = 8.9180 (tuned-lambda SE 4.4590)
EWC fixed-lambda grid: 0.1:11.27 0.3:12.13 1:15.58 1.06007:15.72 1.5:13.59 2.12766:14.57 3:16.90 4.23:13.15 6:0.00
8.49:0.00 10:0.00 30:0.00 100:0.00 300:0.00 1000:0.00 3000:0.00 10000:0.00
EWC tuned lambda = 3: 16.9031 [26.37 18.58 20.98 18.58 0.00]
EWC rule  $\Omega=0.25$ : 15.6044 [15.18 15.38 17.38 15.98 14.09]  $\Omega$  – tuned: -1.2987
EWC rule  $\Omega=0.35$ : 16.8432 [15.38 16.58 18.88 17.98 15.38]  $\Omega$  – tuned: -0.0599
EWC rule  $\Omega=0.5$ : 17.8222 [16.48 18.18 19.08 18.48 16.88]  $\Omega$  – tuned: +0.9191
EWC rule  $\Omega=0.71$ : 17.5225 [15.48 18.68 20.48 16.38 16.58]  $\Omega$  – tuned: +0.6194
EWC rule  $\Omega=1.0$ : 17.8821 [16.98 15.98 21.48 19.88 15.08]  $\Omega$  – tuned: +0.9790
EWC rule  $\Omega=1.41$ : 18.6613 [16.58 19.08 21.78 20.08 15.78]  $\Omega$  – tuned: +1.7582
EWC rule  $\Omega=2.0$ : 18.9411 [16.88 19.58 21.68 20.48 16.08]  $\Omega$  – tuned: +2.0380
EWC rule  $\Omega=2.83$ : 19.0809 [15.58 19.08 21.68 20.28 18.78]  $\Omega$  – tuned: +2.1778
EWC rule  $\Omega=4.0$ : 19.6204 [16.98 19.08 22.88 20.38 18.78]  $\Omega$  – tuned: +2.7173
EWC reduction arm (fixed w = median derived 4.76098): 10.1898 [0.00 0.00 27.17 23.78 0.00] rule( $\Omega=1$ ) – reduction: +7.6923
EWC Bayes arm (Laplace weight 0.5 on the task-size-weighted Fisher): 18.1618 [16.78 17.48 21.08 17.88 17.58] Bayes – tuned:
+1.2587; rule( $\Omega=1$ ) – Bayes: -0.2797
sensitivity ( $\Omega=1$  – tuned): cap10/sm0.8:+2.14 cap10/sm0.9:+1.32 cap10/sm0.98:+0.66 cap100/sm0.8:+2.18 cap100/sm0.9:+1.26
cap100/sm0.98:+0.12 cap10000/sm0.8:+1.48 cap10000/sm0.9:+0.98 cap10000/sm0.98:+2.20
EWC baseline agem : 12.6274 [12.69 12.59 14.29 14.29 9.29] rule( $\Omega=1$ ) – baseline: +5.2547
EWC baseline gradnorm: 0.0000 [0.00 0.00 0.00 0.00 0.00] rule( $\Omega=1$ ) – baseline: +17.8821
EWC baseline mega : 12.3077 [11.49 13.19 13.29 14.19 9.39] rule( $\Omega=1$ ) – baseline: +5.5744
ER-sum grid: 0.5:38.44 1:45.09 2:50.45 4:52.85; rule  $\Omega=1$ : 48.4715 [47.55 43.96 46.95 51.65 52.25] rule – best fixed:
-4.3756; reduction arm: 45.3746 [41.56 41.16 42.26 51.75 50.15] rule – reduction: +3.0969
derpp tuned w=1: 44.5954 [43.66 39.76 43.06 46.95 49.55]; rule best  $\Omega=1.0$ : 44.2557 [40.76 42.66 43.16 45.95 48.75] best- $\Omega$ 
– tuned: -0.3397;  $\Omega=1$  – tuned: -0.3397; finite-grid span 6.7932 -> INERT (excluded from EQ3-4)
lwf tuned w=1: 24.9950 [25.47 27.07 25.17 26.07 21.18]; rule best  $\Omega=1.0$ : 22.9570 [23.28 23.38 22.88 23.68 21.58] best- $\Omega$ 
– tuned: -2.0380;  $\Omega=1$  – tuned: -2.0380; finite-grid span 15.7043 -> INERT (excluded from EQ3-4)
si tuned w=0.01: 15.5844 [15.88 13.89 14.69 15.98 17.48]; rule  $\Omega=1$ : 13.9860 [13.79 10.49 13.59 14.89 17.18]  $\Omega=1$  – tuned:
-1.5984
mas tuned w=0.3: 21.3187 [23.88 25.17 0.00 22.48 35.06]; rule  $\Omega=1$ : 26.6933 [26.97 27.87 24.08 22.28 32.27]  $\Omega=1$  – tuned:
+5.3746
x4 regime: tuned lambda=0.1: 15.8841 [14.39 10.59 17.88 15.18 21.38]; rule  $\Omega=1$ : 18.4615 [16.88 13.69 26.67 17.98 17.08]  $\Omega=1$ 
– tuned: +2.5774; reduction: 3.8961 [0.00 0.00 0.00 19.48 0.00]

[led24] resolvable step = max(1.0, 2*SE) = 4.3633 (tuned-lambda SE 2.1816)
EWC fixed-lambda grid: 0.1:15.74 0.3:15.81 1:15.90 3:16.06 10:17.03 10.6007:17.18 15:17.53 21.2766:18.03 30:18.59
42.3:17.09 60:9.33 84.9:4.79 100:4.76 300:0.00 1000:0.00 3000:0.00 10000:0.00
EWC tuned lambda = 30: 18.5915 [22.38 10.64 19.72 22.54 17.68]
EWC rule  $\Omega=0.25$ : 18.1847 [20.66 10.49 19.87 22.22 17.68]  $\Omega$  – tuned: -0.4069
EWC rule  $\Omega=0.35$ : 19.2175 [22.54 10.80 20.50 24.10 18.15]  $\Omega$  – tuned: +0.6260
EWC rule  $\Omega=0.5$ : 19.4679 [22.85 10.80 21.44 24.10 18.15]  $\Omega$  – tuned: +0.8764
EWC rule  $\Omega=0.71$ : 19.6557 [23.00 11.11 21.91 24.10 18.15]  $\Omega$  – tuned: +1.0642
```

```

EWC rule  $\Omega=1.0$  : 19.2801 [22.07 10.80 21.60 23.63 18.31]  $\Omega$  - tuned: +0.6886
EWC rule  $\Omega=1.41$ : 20.2191 [22.38 11.58 22.85 24.26 20.03]  $\Omega$  - tuned: +1.6275
EWC rule  $\Omega=2.0$  : 20.2817 [23.32 10.80 22.54 23.79 20.97]  $\Omega$  - tuned: +1.6901
EWC rule  $\Omega=2.83$ : 19.0297 [23.47 10.80 22.07 20.66 18.15]  $\Omega$  - tuned: +0.4382
EWC rule  $\Omega=4.0$  : 19.9687 [21.28 11.11 22.69 24.73 20.03]  $\Omega$  - tuned: +1.3772
EWC reduction arm (fixed w = median derived 31.6972): 18.6541 [22.38 10.64 19.72 22.69 17.84] rule( $\Omega=1$ ) - reduction:
+0.6260
EWC Bayes arm (Laplace weight 0.5 on the task-size-weighted Fisher): 15.8372 [16.59 10.33 17.84 17.53 16.90] Bayes - tuned:
-2.7543; rule( $\Omega=1$ ) - Bayes: +3.4429
sensitivity ( $\Omega=1$  - tuned): cap10/sm0.8:-1.56 cap10/sm0.9:-1.56 cap10/sm0.98:-1.56 cap100/sm0.8:+1.53 cap100/sm0.9:+1.19
cap100/sm0.98:+0.94 cap10000/sm0.8:+1.28 cap10000/sm0.9:+0.69 cap10000/sm0.98:+0.63
EWC baseline agem : 16.0563 [17.21 10.33 17.84 18.00 16.90] rule( $\Omega=1$ ) - baseline: +3.2238
EWC baseline gradnorm: 14.3662 [21.91 0.00 15.65 15.18 19.09] rule( $\Omega=1$ ) - baseline: +4.9139
EWC baseline mega : 15.7746 [16.59 10.33 17.68 17.37 16.90] rule( $\Omega=1$ ) - baseline: +3.5055
ER-sum grid: 0.5:41.53 1:46.76 2:48.67 4:50.17; rule  $\Omega=1$ : 48.9202 [37.40 45.85 61.82 44.13 55.40] rule - best fixed:
-1.2520; reduction arm: 48.4194 [34.43 51.17 61.97 40.53 53.99] rule - reduction: +0.5008
derpp tuned w=3: 48.6385 [48.67 51.49 56.34 52.58 34.12]; rule best  $\Omega=1.41$ : 54.3349 [54.46 56.49 54.15 52.74 53.83] best- $\Omega$  -
tuned: +5.6964;  $\Omega=1$  - tuned: +3.9750; finite-grid span 6.2598 -> INERT (excluded from EQ3-4)
lwf tuned w=1: 26.0720 [22.38 29.42 26.29 26.76 25.51]; rule best  $\Omega=1.0$ : 20.2504 [23.63 18.94 18.47 20.81 19.41] best- $\Omega$  -
tuned: -5.8216;  $\Omega=1$  - tuned: -5.8216; finite-grid span 8.5133 -> INERT (excluded from EQ3-4)
si tuned w=0.03: 20.4695 [19.09 18.00 21.44 20.81 23.00]; rule  $\Omega=1$ : 17.7152 [17.84 17.21 15.65 18.00 19.87]  $\Omega=1$  - tuned:
-2.7543
mas tuned w=0.3: 41.8153 [30.83 35.05 48.83 45.07 49.30]; rule  $\Omega=1$ : 41.7840 [31.77 35.84 46.64 46.64 48.04]  $\Omega=1$  - tuned:
-0.0313
x4 regime: tuned lambda=0.3: 15.8685 [16.43 10.95 17.53 17.06 17.37]; rule  $\Omega=1$ : 17.4022 [19.41 17.84 16.59 16.90 16.28]  $\Omega=1$ 
- tuned: +1.5336; reduction: 12.8951 [16.28 0.00 14.40 16.43 17.37]

[led7] resolvable step = max(1.0, 2*SE) = 3.2031 (tuned-lambda SE 1.6015)
EWC fixed-lambda grid: 0.1:21.81 0.106007:21.81 0.15:21.81 0.212766:21.97 0.3:21.97 0.423:21.97 0.6:17.38 0.849:18.34
1:18.34 3:9.78 10:10.28 30:0.00 100:0.00 300:0.00 1000:0.00 3000:0.00
EWC tuned lambda = 0.212766: 21.9688 [18.12 24.69 18.28 25.78 22.97]
EWC rule  $\Omega=0.25$ : 24.6250 [23.91 24.69 25.78 25.78 22.97]  $\Omega$  - tuned: +2.6562
EWC rule  $\Omega=0.35$ : 24.9688 [25.62 24.69 25.78 25.78 22.97]  $\Omega$  - tuned: +3.0000
EWC rule  $\Omega=0.5$  : 25.8125 [25.47 29.06 25.78 25.78 22.97]  $\Omega$  - tuned: +3.8438
EWC rule  $\Omega=0.71$ : 26.0000 [25.62 29.06 25.62 25.78 23.91]  $\Omega$  - tuned: +4.0312
EWC rule  $\Omega=1.0$  : 26.0312 [25.62 29.06 25.78 25.78 23.91]  $\Omega$  - tuned: +4.0625
EWC rule  $\Omega=1.41$ : 26.8125 [30.00 29.69 25.62 25.78 22.97]  $\Omega$  - tuned: +4.8438
EWC rule  $\Omega=2.0$  : 26.1562 [25.62 29.69 25.62 25.78 24.06]  $\Omega$  - tuned: +4.1875
EWC rule  $\Omega=2.83$ : 28.9688 [30.00 29.84 30.16 30.94 23.91]  $\Omega$  - tuned: +7.0000
EWC rule  $\Omega=4.0$  : 27.9062 [30.31 29.84 25.62 30.94 22.81]  $\Omega$  - tuned: +5.9375
EWC reduction arm (fixed w = median derived 2.77719): 9.7812 [23.91 0.00 25.00 0.00 0.00] rule( $\Omega=1$ ) - reduction: +16.2500
EWC Bayes arm (Laplace weight 0.5 on the task-size-weighted Fisher): 21.9688 [18.12 24.69 18.28 25.78 22.97] Bayes - tuned:
+0.0000; rule( $\Omega=1$ ) - Bayes: +4.0625
sensitivity ( $\Omega=1$  - tuned): cap10/sm0.8:+3.00 cap10/sm0.9:+3.00 cap10/sm0.98:+3.00 cap100/sm0.8:+3.16 cap100/sm0.9:+2.66
cap100/sm0.98:+2.69 cap10000/sm0.8:+3.16 cap10000/sm0.9:+4.06 cap10000/sm0.98:+6.03
EWC baseline agem : 21.8125 [18.12 24.69 18.28 25.00 22.97] rule( $\Omega=1$ ) - baseline: +4.2188
EWC baseline gradnorm: 0.0000 [0.00 0.00 0.00 0.00 0.00] rule( $\Omega=1$ ) - baseline: +26.0312
EWC baseline mega : 17.3750 [18.12 24.69 18.28 25.78 0.00] rule( $\Omega=1$ ) - baseline: +8.6562
ER-sum grid: 0.5:59.69 1:64.06 2:66.28 4:67.34; rule  $\Omega=1$ : 63.8438 [62.97 60.47 70.94 64.69 60.16] rule - best fixed:
-3.5000; reduction arm: 64.6562 [63.75 62.50 70.78 65.31 60.94] rule - reduction: -0.8125
derpp tuned w=1: 70.9688 [71.09 67.19 72.34 69.06 75.16]; rule best  $\Omega=1.0$ : 70.3750 [71.41 66.88 70.94 68.75 73.91] best- $\Omega$  -
tuned: -0.5938;  $\Omega=1$  - tuned: -0.5938; finite-grid span 10.8125 -> load-bearing
lwf tuned w=1: 32.5000 [26.09 37.66 30.63 32.66 35.47]; rule best  $\Omega=1.0$ : 25.3125 [25.16 20.78 30.94 26.25 23.44] best- $\Omega$  -
tuned: -7.1875;  $\Omega=1$  - tuned: -7.1875; finite-grid span 14.7500 -> load-bearing
si tuned w=0.01: 25.8750 [31.09 25.00 23.12 25.78 24.38]; rule  $\Omega=1$ : 26.4062 [29.22 25.00 28.75 26.41 22.66]  $\Omega=1$  - tuned:
+0.5312
mas tuned w=0.1: 48.3125 [51.25 48.44 36.56 53.75 51.56]; rule  $\Omega=1$ : 55.2188 [60.62 53.44 46.88 62.19 52.97]  $\Omega=1$  - tuned:
+6.9062
x4 regime: tuned lambda=0.1: 23.4375 [38.59 23.12 30.00 25.47 0.00]; rule  $\Omega=1$ : 24.7500 [25.16 25.31 23.91 31.87 17.50]  $\Omega=1$  -
tuned: +1.3125; reduction: 6.3438 [0.00 0.00 31.72 0.00 0.00]

[mfeat_factors] resolvable step = max(1.0, 2*SE) = 4.5371 (tuned-lambda SE 2.2685)
EWC fixed-lambda grid: 0.1:41.95 0.3:41.95 1:42.00 3:42.25 10:43.30 30:45.35 100:49.30 106.007:49.45 150:50.95
212.766:52.45 300:54.45 423:43.80 600:44.55 849:34.55 1000:34.65 3000:23.50 10000:11.95
EWC tuned lambda = 300: 54.4500 [46.75 55.25 58.75 52.50 59.00]
EWC rule  $\Omega=0.25$ : 55.6000 [52.50 62.00 57.25 47.25 59.00]  $\Omega$  - tuned: +1.1500
EWC rule  $\Omega=0.35$ : 56.0000 [55.25 61.00 56.50 47.50 59.75]  $\Omega$  - tuned: +1.5500
EWC rule  $\Omega=0.5$  : 57.1500 [57.50 61.25 58.50 49.50 59.00]  $\Omega$  - tuned: +2.7000
EWC rule  $\Omega=0.71$ : 57.9000 [58.75 60.00 57.25 54.75 58.75]  $\Omega$  - tuned: +3.4500
EWC rule  $\Omega=1.0$  : 58.3500 [58.75 60.25 57.75 54.25 60.75]  $\Omega$  - tuned: +3.9000
EWC rule  $\Omega=1.41$ : 58.0000 [59.00 59.75 55.75 54.75 60.75]  $\Omega$  - tuned: +3.5500
EWC rule  $\Omega=2.0$  : 58.3500 [57.75 59.75 56.25 54.75 63.25]  $\Omega$  - tuned: +3.9000
EWC rule  $\Omega=2.83$ : 57.5500 [58.50 59.75 51.75 56.25 61.50]  $\Omega$  - tuned: +3.1000
EWC rule  $\Omega=4.0$  : 56.6000 [57.25 59.75 51.50 56.25 58.25]  $\Omega$  - tuned: +2.1500
EWC reduction arm (fixed w = median derived 1065.84): 34.8000 [53.00 61.50 0.00 0.00 59.50] rule( $\Omega=1$ ) - reduction: +23.5500
EWC Bayes arm (Laplace weight 0.5 on the task-size-weighted Fisher): 41.9500 [36.50 45.50 46.25 35.75 45.75] Bayes - tuned:
-12.5000; rule( $\Omega=1$ ) - Bayes: +16.4000
sensitivity ( $\Omega=1$  - tuned): cap10/sm0.8:-11.15 cap10/sm0.9:-11.15 cap10/sm0.98:-11.15 cap100/sm0.8:-5.25
cap100/sm0.9:-5.20 cap100/sm0.98:-5.15 cap10000/sm0.8:+3.65 cap10000/sm0.9:+3.90 cap10000/sm0.98:+2.70
EWC baseline agem : 47.5500 [44.00 53.25 52.25 37.25 51.00] rule( $\Omega=1$ ) - baseline: +10.8000
EWC baseline gradnorm: 0.0000 [0.00 0.00 0.00 0.00 0.00] rule( $\Omega=1$ ) - baseline: +58.3500
EWC baseline mega : 41.8500 [36.50 45.50 45.75 35.75 45.75] rule( $\Omega=1$ ) - baseline: +16.5000
ER-sum grid: 0.5:94.70 1:95.60 2:95.60 4:95.10; rule  $\Omega=1$ : 95.4000 [94.75 95.50 95.50 95.50 95.50] rule - best fixed:
-0.2000; reduction arm: 95.6500 [95.50 96.50 96.50 95.75 94.00] rule - reduction: -0.2500
derpp tuned w=0.3: 96.3500 [96.25 96.50 97.00 96.00 96.00]; rule best  $\Omega=1.0$ : 96.2500 [95.75 96.25 96.25 97.00 96.00] best- $\Omega$  -
tuned: -0.1000;  $\Omega=1$  - tuned: -0.1000; finite-grid span 1.0500 -> INERT (excluded from EQ3-4)
lwf tuned w=1: 68.0000 [60.75 70.00 73.25 72.50 63.50]; rule best  $\Omega=1.0$ : 48.7000 [48.50 48.00 39.50 56.75 50.75] best- $\Omega$  -
tuned: -19.3000;  $\Omega=1$  - tuned: -19.3000; finite-grid span 57.5000 -> load-bearing
si tuned w=0.03: 60.4500 [55.00 67.75 58.25 56.50 64.75]; rule  $\Omega=1$ : 54.8500 [53.25 58.75 53.00 49.75 59.50]  $\Omega=1$  - tuned:
-5.6000

```

```

mas tuned w=0.1: 79.7500 [80.00 75.50 84.75 82.00 76.50]; rule  $\Omega$ =1: 81.3500 [78.25 78.25 87.75 83.25 79.25]  $\Omega$ =1 - tuned:
+1.6000
x4 regime: tuned lambda=10: 27.2000 [21.75 27.50 26.25 32.50 28.00]; rule  $\Omega$ =1: 27.6000 [24.25 27.25 26.50 32.00 28.00]  $\Omega$ =1 -
tuned: +0.4000; reduction: 20.9000 [24.75 25.25 26.50 0.00 28.00]

[mfeat_morphological] resolvable step = max(1.0, 2*SE) = 4.8719 (tuned-lambda SE 2.4359)
EWC fixed-lambda grid: 0.1:25.20 0.3:26.00 0.353357:26.25 0.5:26.60 0.70922:26.95 1:27.65 1.41:28.75 2:33.25
2.83:34.20 3:26.20 10:23.10 30:7.85 100:0.00 300:0.00 1000:0.00 3000:0.00 10000:0.00
EWC tuned lambda = 2.83: 34.2000 [37.00 40.75 29.75 27.50 36.00]
EWC rule  $\Omega$ =0.25: 31.5000 [35.00 29.00 29.75 30.25 33.50]  $\Omega$  - tuned: -2.7000
EWC rule  $\Omega$ =0.35: 36.2000 [40.00 31.25 36.75 36.25 36.75]  $\Omega$  - tuned: +2.0000
EWC rule  $\Omega$ =0.5 : 37.0000 [39.25 32.50 39.75 36.50 37.00]  $\Omega$  - tuned: +2.8000
EWC rule  $\Omega$ =0.71: 37.3000 [41.25 30.50 39.75 38.25 36.75]  $\Omega$  - tuned: +3.1000
EWC rule  $\Omega$ =1.0 : 37.9000 [43.25 29.25 41.00 35.50 40.50]  $\Omega$  - tuned: +3.7000
EWC rule  $\Omega$ =1.41: 39.6500 [42.00 36.50 40.75 37.25 41.75]  $\Omega$  - tuned: +5.4500
EWC rule  $\Omega$ =2.0 : 38.9000 [42.75 34.25 39.00 37.75 40.75]  $\Omega$  - tuned: +4.7000
EWC rule  $\Omega$ =2.83: 41.6500 [42.75 39.75 41.00 43.25 41.50]  $\Omega$  - tuned: +7.4500
EWC rule  $\Omega$ =4.0 : 40.5000 [42.00 41.25 40.75 38.50 40.00]  $\Omega$  - tuned: +6.3000
EWC reduction arm (fixed w = median derived 14.6744): 23.5000 [42.50 0.00 39.50 35.50 0.00] rule( $\Omega$ =1) - reduction: +14.4000
EWC Bayes arm (Laplace weight 0.5 on the task-size-weighted Fisher): 26.6000 [26.00 30.50 24.75 24.00 27.75] Bayes - tuned:
-7.6000; rule( $\Omega$ =1) - Bayes: +11.3000
sensitivity ( $\Omega$ =1 - tuned): cap10/sm0.8:+3.65 cap10/sm0.9:+2.75 cap10/sm0.98:+3.85 cap100/sm0.8:+4.10 cap100/sm0.9:+2.70
cap100/sm0.98:+3.35 cap10000/sm0.8:+5.20 cap10000/sm0.9:+3.70 cap10000/sm0.98:+6.20
EWC baseline agem : 27.7500 [30.25 26.25 28.25 25.50 28.50] rule( $\Omega$ =1) - baseline: +10.1500
EWC baseline gradnorm: 0.0000 [0.00 0.00 0.00 0.00 0.00] rule( $\Omega$ =1) - baseline: +37.9000
EWC baseline mega : 26.4500 [26.00 30.00 24.50 24.25 27.50] rule( $\Omega$ =1) - baseline: +11.4500
ER-sum grid: 0.5:73.60 1:73.30 2:72.45 4:73.05; rule  $\Omega$ =1: 75.5000 [75.75 75.25 74.25 75.50 76.75] rule - best fixed:
+1.9000; reduction arm: 72.5500 [65.75 75.50 72.25 71.50 77.75] rule - reduction: +2.9500
derpp tuned w=0.03: 73.8500 [73.75 73.00 72.00 75.00 75.50]; rule best  $\Omega$ =0.71: 74.2000 [74.25 72.75 71.75 76.00 76.25]
best- $\Omega$  - tuned: +0.3500;  $\Omega$ =1 - tuned: -4.4000; finite-grid span 5.0000 -> INERT (excluded from EQ3-4)
lwf tuned w=0.3: 24.2000 [24.25 30.50 20.50 25.00 20.75]; rule best  $\Omega$ =0.71: 36.8000 [37.50 34.50 37.25 37.25 37.50] best- $\Omega$ 
- tuned: +12.6000;  $\Omega$ =1 - tuned: -6.1000; finite-grid span 10.2500 -> load-bearing
si tuned w=0.01: 21.7000 [27.75 30.00 26.50 24.25 0.00]; rule  $\Omega$ =1: 27.4000 [26.50 28.25 28.00 25.00 29.25]  $\Omega$ =1 - tuned:
+5.7000
mas tuned w=0.1: 51.8000 [52.50 46.50 54.50 55.75 49.75]; rule  $\Omega$ =1: 54.4000 [58.00 53.75 54.25 58.50 47.50]  $\Omega$ =1 - tuned:
+2.6000
x4 regime: tuned lambda=3: 27.7500 [41.25 36.25 27.00 0.00 34.25]; rule  $\Omega$ =1: 34.6000 [42.50 39.50 30.50 25.50 35.00]  $\Omega$ =1 -
tuned: +6.8500; reduction: 20.5000 [0.00 34.75 28.25 0.00 39.50]

-----
EQ3-0 precondition: tuned lambda per carrier {'fars': 283.0, 'krkopt': 3.0, 'led24': 30.0, 'led7': 0.212766, 'mfeat_factors':
300.0, 'mfeat_morphological': 2.83}, span 1410.00x -> DECIDABLE
EQ3-1 H-EQ2:  $\Omega$ =1 - tuned per carrier fars:-3.7400 (step 3.35; seeds not behind 3/5), krkopt:+0.9790 (step 8.92; seeds not behind
4/5), led24:+0.6886 (step 4.36; seeds not behind 5/5), led7:+4.0625 (step 3.20; seeds not behind 5/5), mfeat_factors:+3.9000
(step 4.54; seeds not behind 5/5), mfeat_morphological:+3.7000 (step 4.87; seeds not behind 4/5); best single lambda=1 loses
fars:3.7200, krkopt:1.3187, led24:2.6917, led7:3.6250, mfeat_factors:12.4500, mfeat_morphological:6.5500; not behind on 5/6 ->
FAIL
EQ3-2 reduction: rule - fixed@median per carrier fars:+2.4200, krkopt:+7.6923, led24:+0.6260, led7:+16.2500,
mfeat_factors:+23.5500, mfeat_morphological:+14.4000 -> DOES NOT REDUCE: normalised-gradient method
EQ3-3 control ER-sum: rule - best fixed fars:-4.6800, krkopt:-4.3756, led24:-1.2520, led7:-3.5000, mfeat_factors:-0.2000,
mfeat_morphological:+1.9000; rule - reduction fars:+4.6200, krkopt:+3.0969, led24:+0.5008, led7:-0.8125, mfeat_factors:-0.2500,
mfeat_morphological:+2.9500 -> VIOLATED
EQ3-4 control DER++/LWF (load-bearing cells only): best- $\Omega$  - tuned fars/derpp:+7.3800 (inert), fars/lwf:+0.0200,
krkopt/derpp:-0.3397 (inert), krkopt/lwf:-2.0380 (inert), led24/derpp:+5.6964 (inert), led24/lwf:-5.8216 (inert),
led7/derpp:-0.5938, led7/lwf:-7.1875, mfeat_factors/derpp:-0.1000 (inert), mfeat_factors/lwf:-19.3000,
mfeat_morphological/derpp:+0.3500 (inert), mfeat_morphological/lwf:+12.6000; load-bearing 5/12, inert 7 -> VIOLATED
EQ3-5 diagnostic SI/MAS:  $\Omega$ =1 - tuned fars/si:-1.2400, fars/mas:+3.1400, krkopt/si:-1.5984, krkopt/mas:+5.3746, led24/si:-2.7543,
led24/mas:-0.0313, led7/si:+0.5312, led7/mas:+6.9062, mfeat_factors/si:-5.6000, mfeat_factors/mas:+1.6000,
mfeat_morphological/si:+5.7000, mfeat_morphological/mas:+2.6000 (report only)
EQ3-6  $\Omega$  plateau: best  $\Omega$  per carrier {'fars': 2.0, 'krkopt': 4.0, 'led24': 2.0, 'led7': 2.83, 'mfeat_factors': 1.0,
'mfeat_morphological': 2.83} -> FAIL
EQ3-7 published baselines vs rule( $\Omega$ =1): fars/agem:+0.0200, fars/gradnorm:+7.1000, fars/mega:+0.0200, krkopt/agem:-5.2547,
krkopt/gradnorm:-17.8821, krkopt/mega:-5.5744, led24/agem:-3.2238, led24/gradnorm:-4.9139, led24/mega:-3.5055,
led7/agem:-4.2188, led7/gradnorm:-26.0312, led7/mega:-8.6562, mfeat_factors/agem:-10.8000, mfeat_factors/gradnorm:-58.3500,
mfeat_factors/mega:-16.5000, mfeat_morphological/agem:-10.1500, mfeat_morphological/gradnorm:-37.9000,
mfeat_morphological/mega:-11.4500 -> no published baseline is ahead of the rule by a step on every carrier
EQ3-5 sensitivity: EQ3-1 'not behind' flips in 6 of 54 cells -> FRAGILE
EQ3-8 diagnostic x4 regime:  $\Omega$ =1 - tuned fars:-0.9200 (tuned 0.1), krkopt:+2.5774 (tuned 0.1), led24:+1.5336 (tuned 0.3),
led7:+1.3125 (tuned 0.1), mfeat_factors:+0.4000 (tuned 10), mfeat_morphological:+6.8500 (tuned 3) (report only)
controls: a control is VIOLATED: the mechanism statement is falsified
EQ3-C cap axis ( $\Omega$ =1 - tuned at each cap; nb = not behind by a step; predicted nb iff cap >= tuned lambda):
fars/cap10:-3.72(behind; predicted behind), fars/cap100:-3.74(behind; predicted behind), fars/cap10000:-3.74(behind; predicted
nb), krkopt/cap10:+1.32(nb; predicted nb), krkopt/cap100:+1.26(nb; predicted nb), krkopt/cap10000:+0.98(nb; predicted nb),
led24/cap10:-1.56(nb; predicted behind), led24/cap100:+1.19(nb; predicted nb), led24/cap10000:+0.69(nb; predicted nb),
led7/cap10:+3.00(nb; predicted nb), led7/cap100:+2.66(nb; predicted nb), led7/cap10000:+4.06(nb; predicted nb),
mfeat_factors/cap10:-11.15(behind; predicted behind), mfeat_factors/cap100:-5.20(behind; predicted behind),
mfeat_factors/cap10000:+3.90(nb; predicted nb), mfeat_morphological/cap10:+2.75(nb; predicted nb),
mfeat_morphological/cap100:+2.70(nb; predicted nb), mfeat_morphological/cap10000:+3.70(nb; predicted nb) -> FAIL
EQ3-D empty-present diagnostic: fars: late-w median 324.9 vs whole-run median 426.3 (ratio 0.762; seeds with late-w < 1: 0/5);
krkopt: late-w median 5.211 vs whole-run median 4.761 (ratio 1.095; seeds with late-w < 1: 0/5); led24: late-w median 45.43 vs
whole-run median 31.7 (ratio 1.433; seeds with late-w < 1: 0/5); led7: late-w median 2.873 vs whole-run median 2.777 (ratio
1.035; seeds with late-w < 1: 2/5); mfeat_factors: late-w median 875.1 vs whole-run median 1066 (ratio 0.821; seeds with late-w
< 1: 0/5); mfeat_morphological: late-w median 18.29 vs whole-run median 14.67 (ratio 1.246; seeds with late-w < 1: 0/5) (report
only)
EQ3-A capacity x epochs on mfeat_factors: hidden256/ep3: tuned 300 -> 54.45, rule 58.35, diff +3.90 (step 4.54) nb;
hidden64/ep1: tuned 100 -> 63.55, rule 62.40, diff -1.15 (step 1.11) behind; hidden64/ep3: tuned 300 -> 62.20, rule 59.15, diff
-3.05 (step 1.88) behind; hidden256/ep1: tuned 100 -> 53.25, rule 55.90, diff +2.65 (step 5.09) nb -> not behind in 2/4 cells ->
FAIL
EQ3-I lr x batch invariance on mfeat_factors: lr0.05/bs10: tuned 300 -> 54.45, rule 58.35, diff +3.90 (step 4.54) nb;
lr0.0125/bs10: tuned 300 -> 61.35, rule 64.35, diff +3.00 (step 2.68) nb; lr0.2/bs10: tuned 3000 -> 29.70, rule 31.05, diff

```

```
+1.35 (step 2.42) nb; lr0.05/bs5: tuned 3000 -> 55.35, rule 56.20, diff +0.85 (step 4.63) nb; lr0.05/bs20: tuned 300 -> 60.10,
rule 60.55, diff +0.45 (step 3.26) nb -> not behind in 5/5 cells -> PASS
EQ3-B Bayes arm (Laplace weight 1/2): Bayes - tuned fars:-3.7200, krkopt:+1.2587, led24:-2.7543, led7:+0.0000,
mfeat_factors:-12.5000, mfeat_morphological:-7.6000; rule(Ω=1) - Bayes fars:-0.0200, krkopt:-0.2797, led24:+3.4429,
led7:+4.0625, mfeat_factors:+16.4000, mfeat_morphological:+11.3000; Bayes not behind tuned on 3/6 (report only)
EQ3-P plateau width on the nine-point grid: fars: 9 of 9 within a step of the best (0.25..4, contiguous); krkopt: 9 of 9 within
a step of the best (0.25..4, contiguous); led24: 9 of 9 within a step of the best (0.25..4, contiguous); led7: 7 of 9 within a
step of the best (0.5..4, contiguous); mfeat_factors: 9 of 9 within a step of the best (0.25..4, contiguous);
mfeat_morphological: 7 of 9 within a step of the best (0.5..4, contiguous); median width 9 (report only)
replication summary: EQ3-1 FAIL (not behind on 5/6); fragile: True; controls: ER-sum violated, DER++/LwF violated; cap axis:
FAIL
```

B.2 EQ2 scoring (`runs/eq2/score.txt`)

runs/eq2/score.txt

```
=====
EQ2 scoring - per-seed values in brackets (seeds 0-4); thresholds from prereg/eq2/PREREG.md
=====

[mfeat_fourier] resolvable step = max(1.0, 2*SE) = 1.0000 (tuned-lambda SE 0.4650)
EWC fixed-lambda grid: 0.1:24.35 0.3:24.35 1:24.35 3:24.75 10:25.75 10.6007:25.70 15:25.95 21.2766:25.95 30:26.10
42.3:20.95 60:20.90 84.9:20.90 100:20.90 300:5.25 1000:0.00 3000:0.00 10000:0.00
EWC tuned lambda = 30: 26.1000 [26.50 24.75 27.50 25.50 26.25]
EWC rule Ω=0.5 : 29.7000 [30.25 26.75 31.75 30.00 29.75] Ω - tuned: +3.6000
EWC rule Ω=0.71: 28.7500 [30.00 26.75 31.75 28.00 27.25] Ω - tuned: +2.6500
EWC rule Ω=1.0 : 29.4000 [31.00 26.75 32.00 29.25 28.00] Ω - tuned: +3.3000
EWC rule Ω=1.41: 28.9000 [30.00 27.00 30.25 30.00 27.25] Ω - tuned: +2.8000
EWC rule Ω=2.0 : 29.0000 [30.50 26.75 30.00 30.25 27.50] Ω - tuned: +2.9000
EWC reduction arm (fixed w = median derived 1336.54): 0.0000 [0.00 0.00 0.00 0.00 0.00] rule(Ω=1) - reduction: +29.4000
sensitivity (Ω=1 - tuned): cap10/sm0.8:-0.35 cap10/sm0.9:-0.35 cap10/sm0.98:-0.35 cap100/sm0.8:+0.05 cap100/sm0.9:+0.15
cap100/sm0.98:+0.05 cap10000/sm0.8:+2.80 cap10000/sm0.9:+3.30 cap10000/sm0.98:+3.15
EWC baseline agem : 25.2000 [25.00 24.50 26.50 25.00 25.00] rule(Ω=1) - baseline: +4.2000
EWC baseline gradnorm: 0.0000 [0.00 0.00 0.00 0.00 0.00] rule(Ω=1) - baseline: +29.4000
EWC baseline mega : 24.2500 [24.50 22.50 26.00 23.50 24.75] rule(Ω=1) - baseline: +5.1500
ER-sum grid: 0.5:72.95 1:74.85 2:76.10 4:76.20; rule Ω=1: 75.6000 [75.25 74.25 76.25 74.00 78.25] rule - best fixed:
-0.6000; reduction arm: 76.1000 [75.50 74.75 76.75 74.25 79.25] rule - reduction: -0.5000
derpp tuned w=1: 76.5000 [75.25 76.75 75.50 76.25 78.75]; rule best Ω=1.0: 74.9500 [75.75 72.50 75.25 74.00 77.25] best-Ω -
tuned: -1.5500; Ω=1 - tuned: -1.5500
lwf tuned w=1: 42.4000 [45.25 33.25 44.50 41.25 47.75]; rule best Ω=0.71: 31.0500 [32.00 30.25 31.00 32.00 30.00] best-Ω -
tuned: -11.3500; Ω=1 - tuned: -23.3000
si tuned w=0.03: 26.8000 [28.00 26.00 26.75 27.50 25.75]; rule Ω=1: 26.7000 [25.50 26.50 27.00 28.50 26.00] Ω=1 - tuned:
-0.1000
mas tuned w=0.3: 64.5000 [70.00 63.75 62.25 63.25 63.25]; rule Ω=1: 65.8000 [68.00 66.75 64.25 65.50 64.50] Ω=1 - tuned:
+1.3000
x4 regime: tuned lambda=3: 32.0000 [30.00 32.50 31.75 36.75 29.00]; rule Ω=1: 31.5500 [32.25 28.50 32.00 35.75 29.25] Ω=1 -
tuned: -0.4500; reduction: 0.0000 [0.00 0.00 0.00 0.00 0.00]

[mfeat_pixel] resolvable step = max(1.0, 2*SE) = 3.5121 (tuned-lambda SE 1.7561)
EWC fixed-lambda grid: 0.1:33.05 0.3:33.05 1:33.05 3:33.10 10:33.45 30:34.60 100:36.65 300:41.40 353.357:42.05
500:43.95 709.22:46.60 1000:48.35 1410:50.05 2000:40.85 2830:41.80 3000:42.00 10000:0.00
EWC tuned lambda = 1410: 50.0500 [49.50 49.50 46.75 47.75 56.75]
EWC rule Ω=0.5 : 50.7500 [47.00 51.00 48.75 48.50 58.50] Ω - tuned: +0.7000
EWC rule Ω=0.71: 50.4000 [42.25 51.75 50.25 49.00 58.75] Ω - tuned: +0.3500
EWC rule Ω=1.0 : 52.1500 [47.75 53.25 52.25 47.75 59.75] Ω - tuned: +2.1000
EWC rule Ω=1.41: 50.7000 [38.75 54.00 52.75 48.50 59.50] Ω - tuned: +0.6500
EWC rule Ω=2.0 : 52.6000 [51.00 51.00 52.75 48.25 60.00] Ω - tuned: +2.5500
EWC reduction arm (fixed w = median derived 3183.87): 42.1500 [0.00 52.25 50.75 48.00 59.75] rule(Ω=1) - reduction:
+10.0000
sensitivity (Ω=1 - tuned): cap10/sm0.8:-16.60 cap10/sm0.9:-16.60 cap10/sm0.98:-16.60 cap100/sm0.8:-13.45
cap100/sm0.9:-13.45 cap10000/sm0.98:-13.40 cap10000/sm0.8:+2.90 cap10000/sm0.9:+2.10 cap10000/sm0.98:+1.35
EWC baseline agem : 38.0500 [31.00 38.75 38.25 38.25 44.00] rule(Ω=1) - baseline: +14.1000
EWC baseline gradnorm: 0.0000 [0.00 0.00 0.00 0.00 0.00] rule(Ω=1) - baseline: +52.1500
EWC baseline mega : 33.0500 [27.25 34.00 31.50 33.75 38.75] rule(Ω=1) - baseline: +19.1000
ER-sum grid: 0.5:93.85 1:95.00 2:95.60 4:95.50; rule Ω=1: 95.3500 [96.25 95.50 95.50 93.75 95.75] rule - best fixed:
-0.2500; reduction arm: 95.6000 [96.75 95.50 95.75 94.50 95.50] rule - reduction: -0.2500
derpp tuned w=0.3: 96.2000 [96.25 95.25 96.75 95.75 97.00]; rule best Ω=0.71: 95.5000 [95.50 95.00 95.50 95.00 96.50] best-Ω
- tuned: -0.7000; Ω=1 - tuned: -0.8000
lwf tuned w=1: 60.1500 [57.25 67.75 58.00 58.50 59.25]; rule best Ω=1.0: 50.7500 [47.00 50.75 49.50 55.75 50.75] best-Ω -
tuned: -9.4000; Ω=1 - tuned: -9.4000
si tuned w=0.03: 53.3500 [56.75 55.75 50.00 51.00 53.25]; rule Ω=1: 45.4500 [45.00 44.50 44.25 43.75 49.75] Ω=1 - tuned:
-7.9000
mas tuned w=0.3: 86.4500 [83.00 88.25 88.00 82.50 90.50]; rule Ω=1: 84.7000 [81.50 89.00 86.25 76.75 90.00] Ω=1 - tuned:
-1.7500
x4 regime: tuned lambda=10000: 25.4500 [22.75 32.25 22.25 24.75 25.25]; rule Ω=1: 25.3000 [22.50 32.00 22.00 24.75 25.25]
Ω=1 - tuned: -0.1500; reduction: 25.4500 [22.75 32.25 22.25 24.75 25.25]

[texture] resolvable step = max(1.0, 2*SE) = 3.6120 (tuned-lambda SE 1.8060)
EWC fixed-lambda grid: 0.1:44.60 0.3:44.86 1:45.30 3:45.80 10:47.10 30:48.00 35.3357:48.08 50:48.18 70.922:48.56
100:48.56 141:40.98 200:39.36 283:20.20 300:14.78 1000:0.00 3000:0.00 10000:0.00
EWC tuned lambda = 100: 48.5600 [47.80 52.40 48.20 42.40 52.00]
EWC rule Ω=0.5 : 49.8200 [47.40 51.90 51.20 45.90 52.70] Ω - tuned: +1.2600
EWC rule Ω=0.71: 51.1000 [46.90 53.70 54.40 49.60 50.90] Ω - tuned: +2.5400
EWC rule Ω=1.0 : 51.3400 [48.60 53.30 54.50 47.80 52.50] Ω - tuned: +2.7800
EWC rule Ω=1.41: 51.5800 [50.00 53.70 53.00 47.60 53.60] Ω - tuned: +3.0200
EWC rule Ω=2.0 : 46.5200 [47.90 47.00 48.20 40.30 49.20] Ω - tuned: -2.0400
EWC reduction arm (fixed w = median derived 110.561): 48.5600 [47.50 52.40 48.30 42.40 52.20] rule(Ω=1) - reduction:
+2.7800
sensitivity (Ω=1 - tuned): cap10/sm0.8:-1.02 cap10/sm0.9:-1.26 cap10/sm0.98:-1.46 cap100/sm0.8:+0.30 cap100/sm0.9:+0.16
```

```

cap100/sm0.98:+0.04  cap10000/sm0.8:+0.98  cap10000/sm0.9:+2.78  cap10000/sm0.98:-2.18
EWC baseline agem      : 46.3600 [46.30 45.60 48.30 45.80 45.80]  rule(Ω=1) - baseline: +4.9800
EWC baseline gradnorm: 20.0000 [20.00 20.00 20.00 20.00 20.00]  rule(Ω=1) - baseline: +31.3400
EWC baseline mega      : 44.5000 [44.60 44.40 45.30 43.20 45.00]  rule(Ω=1) - baseline: +6.8400
ER-sum grid: 0.5:98.38  1:98.94  2:99.06  4:99.16; rule Ω=1: 98.5000 [97.70 98.80 98.90 98.50 98.60]  rule - best fixed:
-0.6600; reduction arm: 98.8600 [98.60 99.30 98.90 98.80 98.70]  rule - reduction: -0.3600
derpp tuned w=0.01: 98.6800 [98.50 98.90 98.80 98.50 98.70]; rule best Ω=1.0: 99.2600 [99.40 99.50 99.10 99.00 99.30]  best-Ω
- tuned: +0.5800; Ω=1 - tuned: +0.5800
lwf tuned w=0.3: 71.6200 [71.50 70.40 70.40 71.50 74.30]; rule best Ω=0.71: 53.0000 [54.50 52.30 59.40 46.30 52.50]  best-Ω
- tuned: -18.6200; Ω=1 - tuned: -58.6800
si tuned w=0.01: 49.1000 [48.60 53.30 47.20 46.40 50.00]; rule Ω=1: 49.2600 [47.90 51.90 47.60 51.40 47.50]  Ω=1 - tuned:
+0.1600
mas tuned w=0.1: 78.8400 [75.00 82.70 81.00 67.70 87.80]; rule Ω=1: 80.5200 [78.10 80.10 80.50 74.20 89.70]  Ω=1 - tuned:
+1.6800
x4 regime: tuned lambda=30: 46.5600 [58.60 64.50 53.20 19.60 36.90]; rule Ω=1: 29.9200 [24.60 28.60 20.00 33.00 43.40]  Ω=1 -
tuned: -16.6400; reduction: 43.7400 [54.00 52.90 45.30 29.70 36.80]

-----
EQ2-0 precondition: tuned lambda per carrier {'mfeat_fourier': 30.0, 'mfeat_pixel': 1410.0, 'texture': 100.0}, span 47.00x ->
DECIDABLE
EQ2-1 H-EQ2: Ω=1 - tuned per carrier mfeat_fourier:+3.3000 (step 1.00; seeds not behind 5/5), mfeat_pixel:+2.1000 (step 3.51;
seeds not behind 5/5), texture:+2.7800 (step 3.61; seeds not behind 5/5); best single lambda=30 loses mfeat_fourier:0.0000,
mfeat_pixel:15.4500, texture:0.5600 -> PASS
EQ2-2 reduction: rule - fixed@median per carrier mfeat_fourier:+29.4000, mfeat_pixel:+10.0000, texture:+2.7800 -> DOES NOT
REDUCE: normalised-gradient method
EQ2-3 control ER-sum: rule - best fixed mfeat_fourier:-0.6000, mfeat_pixel:-0.2500, texture:-0.6600; rule - reduction
mfeat_fourier:-0.5000, mfeat_pixel:-0.2500, texture:-0.3600 -> holds
EQ2-4 control DER++/LwF: best-Ω - tuned mfeat_fourier/derpp:-1.5500, mfeat_fourier/lwf:-11.3500, mfeat_pixel/derpp:-0.7000,
mfeat_pixel/lwf:-9.4000, texture/derpp:+0.5800, texture/lwf:-18.6200 -> VIOLATED
EQ2-5 diagnostic SI/MAS: Ω=1 - tuned mfeat_fourier/si:-0.1000, mfeat_fourier/mas:+1.3000, mfeat_pixel/si:-7.9000,
mfeat_pixel/mas:-1.7500, texture/si:+0.1600, texture/mas:+1.6800 (report only)
EQ2-6 Ω plateau: best Ω per carrier {'mfeat_fourier': 0.5, 'mfeat_pixel': 2.0, 'texture': 1.41} -> FAIL
EQ2-7 published baselines vs rule(Ω=1): mfeat_fourier/agem:-4.2000, mfeat_fourier/gradnorm:-29.4000, mfeat_fourier/mega:-5.1500,
mfeat_pixel/agem:-14.1000, mfeat_pixel/gradnorm:-52.1500, mfeat_pixel/mega:-19.1000, texture/agem:-4.9800,
texture/gradnorm:-31.3400, texture/mega:-6.8400 -> no published baseline is ahead of the rule by a step on every carrier
sensitivity: EQ2-1 'not behind' flips in 6 of 27 cells -> FRAGILE
EQ2-8 diagnostic x4 regime: Ω=1 - tuned mfeat_fourier:-0.4500 (tuned 3), mfeat_pixel:-0.1500 (tuned 10000), texture:-16.6400
(tuned 30) (report only)
controls: a control is VIOLATED: the mechanism statement is falsified

```

B.3 The mathematics check (`theory/checks/omega\_sweeps.txt`)

theory/checks/omega_sweeps.txt

```

Omega = 1 on existing mathematics (2026-09-22, prompt-log entry 69): Bayes, the two-task quadratic, the Kalman filter

[1] Bayes: a present batch (n_p samples, mean m_p) and a past batch (n_q samples, mean m_q) of a Gaussian mean with unit noise,
flat prior
    log posterior = -n_p (theta - m_p)^2 / 2 - n_q (theta - m_q)^2 / 2; MAP theta_B = (n_p m_p + n_q m_q) / (n_p + n_q)
    with batch-MEAN gradients g_p = theta - m_p, g_q = theta - m_q, the combination g_p + w g_q has fixed point (m_p + w
m_q)/(1 + w):
    it is the MAP iff w = n_q / n_p (each sample counted once: summed log-likelihoods, weight 1 each)
    n_q/n_p = 0.25: theta_B = 0.2000 (Bayes weight w_B = 0.25); rule endpoints (exact gradients, registered estimator, lr 0.05,
20000 steps):
        Omega = 0.5: from theta_0 = 0.1 -> 0.0000, 0.5 -> 0.0000, 0.9 -> 0.0000
        Omega = 1: from theta_0 = 0.1 -> 0.1000, 0.5 -> 0.5000, 0.9 -> 0.9000  (|end - theta_B| = 0.1000, 0.3000, 0.7000)
        Omega = 2: from theta_0 = 0.1 -> 1.0973, 0.5 -> 1.0973, 0.9 -> 1.0973
    n_q/n_p = 1: theta_B = 0.5000 (Bayes weight w_B = 1); rule endpoints (exact gradients, registered estimator, lr 0.05, 20000
steps):
        Omega = 0.5: from theta_0 = 0.1 -> 0.0000, 0.5 -> 0.0000, 0.9 -> 0.0000
        Omega = 1: from theta_0 = 0.1 -> 0.1000, 0.5 -> 0.5000, 0.9 -> 0.9000  (|end - theta_B| = 0.4000, 0.0000, 0.4000)
        Omega = 2: from theta_0 = 0.1 -> 1.0973, 0.5 -> 1.0973, 0.9 -> 1.0973
    n_q/n_p = 4: theta_B = 0.8000 (Bayes weight w_B = 4); rule endpoints (exact gradients, registered estimator, lr 0.05, 20000
steps):
        Omega = 0.5: from theta_0 = 0.1 -> 0.0000, 0.5 -> 0.0000, 0.9 -> 0.0000
        Omega = 1: from theta_0 = 0.1 -> 0.1000, 0.5 -> 0.5000, 0.9 -> 0.9000  (|end - theta_B| = 0.7000, 0.3000, 0.1000)
        Omega = 2: from theta_0 = 0.1 -> 1.0973, 0.5 -> 1.0973, 0.9 -> 1.0973
    computed: at Omega = 1 the endpoint is the starting point (every theta between the means is a fixed point): True; Omega =
0.5 ends at the present mean (within 1e-3): True; Omega = 2 ends within 0.2 of the past mean (it chatters there, where g_q
vanishes and the ratio hits the floor): True
    so Omega = 1 is the Bayes optimum only when the trajectory happens to stop at theta_B, and the rule contains nothing that
selects it; with equal counts the Bayes weight is w = 1 (ER-sum), the constant EQX found the rule reduced to

[2] Two-task quadratic model: L_p = (theta - a)' H (theta - a)/2 (present), L_q = (theta - b)' F (theta - b)/2 (past penalty), d
= 5, F 16x H (the gate's Fisher-scale mismatch); the learner starts at the past optimum b
    eigenvalues: H in [0.525, 1.720], F in [21.047, 30.442]; |a - b| = 2.2192
    (i) fixed points of g_p + Omega (|g_p|/|g_q|) g_q = 0 need g_p and g_q antiparallel, i.e. theta on the Pareto curve
theta(lambda) = (H + lambda F)^-1 (H a + lambda F b); there the update has norm |1 - Omega| |g_p| exactly:
        Omega = 0.9: |update|/|g_p| over 13 points of the curve: min 0.100000, max 0.100000 (|1 - Omega| = 0.1)
        Omega = 1: |update|/|g_p| over 13 points of the curve: min 0.000000, max 0.000000 (|1 - Omega| = 0.0)
        Omega = 1.1: |update|/|g_p| over 13 points of the curve: min 0.100000, max 0.100000 (|1 - Omega| = 0.1)
    computed: at Omega = 1 every point of the curve is stationary (a continuum of equilibria, one for every fixed lambda);
off Omega = 1 none is
    (ii) a fixed weight lambda diverges under gradient descent at lr 0.05 when lr (h_max + lambda f_max) > 2: edge lambda* =
1.2575
        fixed lambda = 0.01: converged L_present 0.0994 L_past 47.5001 sum 47.5996
        fixed lambda = 0.03: converged L_present 0.4867 L_past 26.5851 sum 27.0718
        fixed lambda = 0.1: converged L_present 1.5160 L_past 7.6535 sum 9.1695
        fixed lambda = 0.3: converged L_present 2.5139 L_past 1.4279 sum 3.9418

```

```

fixed lambda = 1: converged L_present 3.1168 L_past 0.1602 sum 3.2770
fixed lambda = 1.5: DIVERGED L_present nan L_past nan sum nan
fixed lambda = 3: DIVERGED L_present nan L_past nan sum nan
fixed lambda = 10: DIVERGED L_present nan L_past nan sum nan
(iii) the rule with exact gradients and instantaneous norms (smooth 0), from the past optimum, 4000 steps; L_present at b =
3.4456, L_past at a = 69.6621:
Omega = 0.25: L_present 0.0000 L_past 69.6621 sum 69.6621; nearest fixed-lambda equilibrium lambda_eff = 0.0001
(distance 5.25e-03); median w 2.277e-17 -> on the curve at lambda_eff 0.0001
Omega = 0.35: L_present 0.0000 L_past 69.6621 sum 69.6621; nearest fixed-lambda equilibrium lambda_eff = 0.0001
(distance 5.25e-03); median w 3.639e-17 -> on the curve at lambda_eff 0.0001
Omega = 0.5: L_present 0.0000 L_past 69.6621 sum 69.6621; nearest fixed-lambda equilibrium lambda_eff = 0.0001
(distance 5.25e-03); median w 6.584e-17 -> on the curve at lambda_eff 0.0001
Omega = 0.71: L_present 0.0000 L_past 69.6621 sum 69.6621; nearest fixed-lambda equilibrium lambda_eff = 0.0001
(distance 5.25e-03); median w 1.778e-16 -> on the curve at lambda_eff 0.0001
Omega = 1: L_present 2.9396 L_past 0.3905 sum 3.3301; nearest fixed-lambda equilibrium lambda_eff = 0.6237 (distance
5.33e-04); median w 0.6215 -> on the curve at lambda_eff 0.624
Omega = 1.41: L_present 3.5034 L_past 0.0047 sum 3.5080; nearest fixed-lambda equilibrium lambda_eff = 1e+06 (distance
1.79e-02); median w 0.8296 -> present never learned (at the past optimum)
Omega = 2: L_present 3.2032 L_past 0.4056 sum 3.6088; nearest fixed-lambda equilibrium lambda_eff = 1.318 (distance
1.43e-01); median w 1.28 -> present never learned (at the past optimum)
Omega = 2.83: L_present 2.4327 L_past 3.5955 sum 6.0281; nearest fixed-lambda equilibrium lambda_eff = 0.2851 (distance
3.71e-01); median w 1.216 -> off the curve (oscillating)
Omega = 4: L_present 2.5587 L_past 1.5840 sum 4.1428; nearest fixed-lambda equilibrium lambda_eff = 0.3311 (distance
1.48e-01); median w 1.127 -> off the curve (oscillating)
computed: Omega 0.25: on the curve at lambda_eff 0.0001; Omega 0.35: on the curve at lambda_eff 0.0001; Omega 0.5: on
the curve at lambda_eff 0.0001; Omega 0.71: on the curve at lambda_eff 0.0001; Omega 1: on the curve at lambda_eff 0.624; Omega
1.41: present never learned (at the past optimum); Omega 2: present never learned (at the past optimum); Omega 2.83: off the
curve (oscillating); Omega 4: off the curve (oscillating)
computed: at Omega = 1 the learner stops on the curve at lambda_eff = 0.6237, which is below the stability edge 1.2575
of a fixed weight
(iii b) the same with the registered estimator (EMA 0.9 of the gradient vectors, cap 1e4):
Omega = 0.25: L_present 0.0000 L_past 69.6621 sum 69.6621; lambda_eff 0.0001 (distance 5.25e-03); median w 2.277e-17 ->
on the curve at lambda_eff 0.0001
Omega = 0.35: L_present 0.0000 L_past 69.6621 sum 69.6621; lambda_eff 0.0001 (distance 5.25e-03); median w 3.639e-17 ->
on the curve at lambda_eff 0.0001
Omega = 0.5: L_present 0.0000 L_past 69.6621 sum 69.6621; lambda_eff 0.0001 (distance 5.25e-03); median w 6.584e-17 ->
on the curve at lambda_eff 0.0001
Omega = 0.71: L_present 0.0000 L_past 69.6621 sum 69.6621; lambda_eff 0.0001 (distance 5.25e-03); median w 3.205e-12 ->
on the curve at lambda_eff 0.0001
Omega = 1: L_present 0.7603 L_past 19.1150 sum 19.8753; lambda_eff 0.04467 (distance 2.79e-03); median w 0.04444 ->
on the curve at lambda_eff 0.0447
Omega = 1.41: L_present 6.7741 L_past 11.8277 sum 18.6018; lambda_eff 1e+06 (distance 8.82e-01); median w 1.114 -> off
the curve (oscillating)
Omega = 2: L_present 8.3523 L_past 22.6705 sum 31.0227; lambda_eff 1e+06 (distance 1.22e+00); median w 1.162 -> off
the curve (oscillating)
Omega = 2.83: L_present 11.1180 L_past 46.0227 sum 57.1408; lambda_eff 1e+06 (distance 1.74e+00); median w 1.125 -> off
the curve (oscillating)
Omega = 4: L_present 0.9585 L_past 46.6333 sum 47.5919; lambda_eff 0.03055 (distance 9.82e-01); median w 1.022 -> off
the curve (oscillating)
(iv) mini-batch noise (Gaussian, sd 'noise' on both gradients), registered estimator unless stated, 4000 steps, 5 seeds;
total loss L_present + L_past per Omega (the equal-count Bayes objective); plateau = Omegas within 10 % of the best total:
noise 0 smooth 0.9 cap 10000: total loss 0.25:69.662 0.35:69.662 0.5:69.662 0.71:69.662 1:19.875 1.41:18.602 2:31.023
2.83:57.141 4:47.592 | best Omega 1.41; plateau [1.0, 1.41]; w at cap in 2.50e-04 of steps
noise 0 smooth 0.8 cap 10000: total loss 0.25:69.662 0.35:69.662 0.5:69.662 0.71:69.662 1:3.377 1.41:15.478 2:3.387
2.83:9.294 4:70.480 | best Omega 1; plateau [1.0, 2.0]; w at cap in 2.50e-04 of steps
noise 0 smooth 0.98 cap 10000: total loss 0.25:69.662 0.35:69.662 0.5:69.662 0.71:24.222 1:36.147 1.41:59.610 2:108.012
2.83:211.579 4:452.020 | best Omega 0.71; plateau [0.71]; w at cap in 2.50e-04 of steps
noise 0 smooth 0.9 cap 100: total loss 0.25:69.662 0.35:69.662 0.5:69.662 0.71:69.662 1:19.875 1.41:18.602 2:31.023
2.83:57.141 4:47.592 | best Omega 1.41; plateau [1.0, 1.41]; w at cap in 2.50e-04 of steps
noise 0 smooth 0.9 cap 10: total loss 0.25:69.662 0.35:69.662 0.5:69.662 0.71:69.662 1:19.875 1.41:18.602 2:31.023
2.83:57.141 4:47.592 | best Omega 1.41; plateau [1.0, 1.41]; w at cap in 4.17e-04 of steps
noise 0.5 smooth 0.9 cap 10000: total loss 0.25:68.513 0.35:67.307 0.5:65.262 0.71:61.265 1:3.479 1.41:11.146 2:11.010
2.83:56.787 4:101.379 | best Omega 1; plateau [1.0]; w at cap in 0.00e+00 of steps
noise 0.5 smooth 0.8 cap 10000: total loss 0.25:66.965 0.35:65.115 0.5:62.009 0.71:56.057 1:3.510 1.41:6.189 2:4.930
2.83:6.803 4:212.481 | best Omega 1; plateau [1.0]; w at cap in 0.00e+00 of steps
noise 0.5 smooth 0.98 cap 10000: total loss 0.25:70.513 0.35:70.152 0.5:69.521 0.71:58.183 1:20.664 1.41:50.262 2:68.047
2.83:121.374 4:252.668 | best Omega 1; plateau [1.0]; w at cap in 0.00e+00 of steps
noise 0.5 smooth 0.9 cap 100: total loss 0.25:68.513 0.35:67.307 0.5:65.262 0.71:61.265 1:3.479 1.41:11.146 2:11.010
2.83:56.787 4:101.379 | best Omega 1; plateau [1.0]; w at cap in 0.00e+00 of steps
noise 0.5 smooth 0.9 cap 10: total loss 0.25:68.513 0.35:67.307 0.5:65.262 0.71:61.265 1:3.479 1.41:9.794 2:9.168
2.83:63.724 4:84.469 | best Omega 1; plateau [1.0]; w at cap in 1.28e-04 of steps
noise 2 smooth 0.9 cap 10000: total loss 0.25:71.861 0.35:66.991 0.5:59.231 0.71:45.890 1:4.412 1.41:4.525 2:4.647
2.83:8.766 4:31.483 | best Omega 1; plateau [1.0, 1.41, 2.0]; w at cap in 0.00e+00 of steps
noise 2 smooth 0.8 cap 10000: total loss 0.25:65.735 0.35:58.769 0.5:48.154 0.71:31.624 1:4.311 1.41:4.648 2:5.741
2.83:6.389 4:6.715 | best Omega 1; plateau [1.0, 1.41]; w at cap in 0.00e+00 of steps
noise 2 smooth 0.98 cap 10000: total loss 0.25:80.323 0.35:78.703 0.5:75.903 0.71:70.373 1:4.126 1.41:8.712 2:135.829
2.83:95.854 4:215.979 | best Omega 1; plateau [1.0]; w at cap in 0.00e+00 of steps
noise 2 smooth 0.9 cap 100: total loss 0.25:71.861 0.35:66.991 0.5:59.231 0.71:45.890 1:4.412 1.41:4.525 2:4.647
2.83:8.766 4:31.483 | best Omega 1; plateau [1.0, 1.41, 2.0]; w at cap in 0.00e+00 of steps
noise 2 smooth 0.9 cap 10: total loss 0.25:71.861 0.35:66.991 0.5:59.231 0.71:45.890 1:4.412 1.41:4.525 2:8.828
2.83:11.952 4:30.252 | best Omega 1; plateau [1.0, 1.41]; w at cap in 5.00e-05 of steps
computed: plateau width (number of grid Omegas within 10 % of the best) by noise at the registered smoothing and cap:
noise 0: 2 (best 1.41), noise 0.5: 1 (best 1), noise 2: 3 (best 1); best Omega is 1 in 11 of 15 cells; the cap binds in more
than 1 % of steps in 0 of 15 cells (median w here is of order 1; the EQ2 regime, tuned lambda in the hundreds and the cap
load-bearing, is not this model's)

[3] Kalman filter, scalar random walk (process variance q, observation variance r),  $v = \sqrt{q/r}$  the Fisher speed of P4
tuned (Bayes-optimal) gain  $K(v) = (v/2)/(\sqrt{v^2 + 4} - v)$ ; fixed rules from the two readings of Omega = 1: Fisher speed 1
->  $K(1) = 1/\phi$ ; equal pull (prior and datum with equal precision) ->  $K = 1/2$ 
 $K(1) = 0.618034$  ( $1/\phi = 0.618034$ );  $K(v) = 1/2$  at  $v = 0.707107$  ( $K(0.7071) = 0.500000$ )

```

```

the normalised-gradient rule applied to the filter update (innovation pull and prior pull scaled to equal norm) is  $K = 1/2$ 
at every  $v$ : a fixed gain, not an adaptive one
steady-state MSE ratio fixed / tuned (closed form),  $P(K) = ((1-K)^2 q + K^2 r) / (1 - (1-K)^2)$ ,  $r = 1$ ,  $q = v^2$ :
v = 0.01: K_tuned 0.0100 MSE 0.0100; K = 1/phi: ratio 44.9472; K = 1/2: ratio 33.5038
v = 0.03: K_tuned 0.0296 MSE 0.0296; K = 1/phi: ratio 15.1376; K = 1/2: ratio 11.2892
v = 0.1: K_tuned 0.0951 MSE 0.0951; K = 1/phi: ratio 4.7193; K = 1/2: ratio 3.5392
v = 0.3: K_tuned 0.2584 MSE 0.2584; K = 1/phi: ratio 1.7905; K = 1/2: ratio 1.4063
v = 0.5: K_tuned 0.3904 MSE 0.3904; K = 1/phi: ratio 1.2550; K = 1/2: ratio 1.0673
v = 0.707: K_tuned 0.4999 MSE 0.4999; K = 1/phi: ratio 1.0653; K = 1/2: ratio 1.0000
v = 1: K_tuned 0.6180 MSE 0.6180; K = 1/phi: ratio 1.0000; K = 1/2: ratio 1.0787
v = 1.414: K_tuned 0.7320 MSE 0.7320; K = 1/phi: ratio 1.0775; K = 1/2: ratio 1.3658
v = 2: K_tuned 0.8284 MSE 0.8284; K = 1/phi: ratio 1.3646; K = 1/2: ratio 2.0118
v = 3: K_tuned 0.9083 MSE 0.9083; K = 1/phi: ratio 2.1849; K = 1/2: ratio 3.6698
v = 10: K_tuned 0.9902 MSE 0.9902; K = 1/phi: ratio 17.7028; K = 1/2: ratio 34.0000
v = 30: K_tuned 0.9989 MSE 0.9989; K = 1/phi: ratio 154.3567; K = 1/2: ratio 300.6667
v = 100: K_tuned 0.9999 MSE 0.9999; K = 1/phi: ratio 1708.8220; K = 1/2: ratio 3334.0000
computed: the 1/phi rule is within 5 % of the tuned filter for  $v$  in [1.0] and within 25 % for  $v$  in [0.707, 1.0, 1.414]; the
1/2 rule within 5 % for  $v$  in [0.707] and within 25 % for  $v$  in [0.5, 0.707, 1.0]
Monte Carlo (200000 steps, seeds 0 and 1, first 10 % discarded) against the closed form:
v = 0.3 K = tuned (0.2584): closed form 0.2584, Monte Carlo 0.2591 0.2574
v = 0.3 K = 1/phi (0.6180): closed form 0.4626, Monte Carlo 0.4641 0.4600
v = 0.3 K = 1/2 (0.5000): closed form 0.3633, Monte Carlo 0.3646 0.3614
v = 1 K = tuned (0.6180): closed form 0.6180, Monte Carlo 0.6206 0.6153
v = 1 K = 1/phi (0.6180): closed form 0.6180, Monte Carlo 0.6206 0.6153
v = 1 K = 1/2 (0.5000): closed form 0.6667, Monte Carlo 0.6690 0.6639
v = 3 K = tuned (0.9083): closed form 0.9083, Monte Carlo 0.9122 0.9043
v = 3 K = 1/phi (0.6180): closed form 1.9846, Monte Carlo 1.9906 1.9785
v = 3 K = 1/2 (0.5000): closed form 3.3333, Monte Carlo 3.3377 3.3204
the tuned gain needs  $q/r$ ; an innovation-based estimator (Mehra 1970, named, not fetched) supplies it from the data; the
fixed rules do not, and lose everywhere but at one speed each

```

B.4 The gate (`prereg/eq3/gate\_EQ2.txt`)

prereg/eq3/gate_EQ2.txt

```

gate for EQ2
must FAIL on ['S-R convex replay, constant label scale (adaptivity idle)', 'S-X constraint learner, tuned weight a small
fraction of present norm', 'S-Y/LwF softmax MLP, distillation constraint (input scale 4)']
must PASS on ['S-Y softmax MLP, online EWC, 16x Fisher-scale mismatch (input scale 4)']

S-R convex replay, constant label scale (adaptivity idle) FAIL EQ( $\Omega$ =1)=1.4416 tuned w=1.0:1.2250 (w_med of EQ=0.607) |
behind-by +0.177 (seeds>step: 5/5) | best  $\Omega$ =2.0 landscape 0.5:1.661 0.71:1.582 1.0:1.442 1.41:1.257 2.0:1.226 | reduction arm
w=0.607:1.2799
S-V convex replay, 16x label-scale swings (adaptivity load-bearing) FAIL EQ( $\Omega$ =1)=1.1992 tuned w=0.0625:1.1024 (w_med of
EQ=0.599) | behind-by +0.088 (seeds>step: 3/5) | best  $\Omega$ =0.5 landscape 0.5:1.059 0.71:1.088 1.0:1.199 1.41:1.663 2.0:3.397 |
reduction arm w=0.599:3.9393
S-W EWC-Laplace convex replay, 16x curvature-scale mismatch FAIL EQ( $\Omega$ =1)=0.0654 tuned w=0.0625:0.0570 (w_med of EQ=1.409) |
behind-by +0.156 (seeds>step: 5/5) | best  $\Omega$ =0.71 landscape 0.5:0.055 0.71:0.053 1.0:0.065 1.41:0.065 2.0:0.065 | reduction arm
w=1.409:0.0654
S-X constraint learner, tuned weight a small fraction of present norm FAIL EQ( $\Omega$ =1)=0.9420 tuned w=0.25:0.8803 (w_med of
EQ=1.284) | behind-by +0.071 (seeds>step: 4/5) | best  $\Omega$ =1.0 landscape 0.5:1.478 0.71:1.182 1.0:0.942 1.41:1.072 2.0:1.414 |
reduction arm w=1.284:0.9430
S-Y softmax MLP, online EWC, 16x Fisher-scale mismatch (input scale 4) PASS EQ( $\Omega$ =1)=0.1740 tuned w=256.0:0.1977 (w_med of
EQ=1833.531) | behind-by -0.075 (seeds>step: 1/5) | best  $\Omega$ =0.5 landscape 0.5:0.128 0.71:0.168 1.0:0.174 1.41:0.213 2.0:0.215 |
reduction arm w=1833.531:0.8257
S-Y softmax MLP, online EWC, unit input scale (informational) FAIL EQ( $\Omega$ =1)=0.0544 tuned w=128.0:0.0394 (w_med of EQ=559.599)
| behind-by +0.634 (seeds>step: 3/5) | best  $\Omega$ =1.41 landscape 0.5:0.036 0.71:0.037 1.0:0.054 1.41:0.036 2.0:0.053 | reduction arm
w=559.599:0.2152
S-Y softmax MLP, online EWC, scale 4, cap=100 (informational) FAIL EQ( $\Omega$ =1)=0.2814 tuned w=256.0:0.1977 (w_med of EQ=84.093) |
behind-by +0.438 (seeds>step: 5/5) | best  $\Omega$ =2.0 landscape 0.5:0.287 0.71:0.285 1.0:0.281 1.41:0.277 2.0:0.273 | reduction arm
w=84.093:0.2651
S-Y softmax MLP, online EWC, scale 4, smooth=0.8 (informational) PASS EQ( $\Omega$ =1)=0.1484 tuned w=256.0:0.1977 (w_med of
EQ=575.167) | behind-by -0.182 (seeds>step: 1/5) | best  $\Omega$ =0.71 landscape 0.5:0.138 0.71:0.137 1.0:0.148 1.41:0.180 2.0:0.200 |
reduction arm w=575.167:0.4224
S-Y softmax MLP, online EWC, scale 4, smooth=0.98 (informational) FAIL EQ( $\Omega$ =1)=0.2470 tuned w=256.0:0.1977 (w_med of
EQ=3304.236) | behind-by +0.490 (seeds>step: 3/5) | best  $\Omega$ =0.5 landscape 0.5:0.211 0.71:0.250 1.0:0.247 1.41:0.312 2.0:0.276 |
reduction arm w=3304.236:1.0000
S-Y/LwF softmax MLP, distillation constraint (input scale 4) FAIL EQ( $\Omega$ =1)=0.7305 tuned w=0.0625:0.2106 (w_med of EQ=1.678) |
behind-by +2.782 (seeds>step: 5/5) | best  $\Omega$ =0.5 landscape 0.5:0.292 0.71:0.395 1.0:0.730 1.41:0.879 2.0:0.903 | reduction arm
w=1.678:0.7347

GATE OPEN

```

B.5 The figure numbers (`Continuous\_Learning/figures/figures.txt`)

Continuous_Learning/figures/figures.txt

```

Continuous_Learning figures (deterministic; numbers drawn on the figures)
[F1] Omega 0.5: w = 0.1667, net pull +0.5000
[F1] Omega 1: w = 0.3333, net pull +0.0000
[F1] Omega 2: w = 0.6667, net pull -1.0000
[figure] F01_tug_of_war.png
[F2] Omega 0.7: [update]/[g_p] on the curve at lambda 0.1, 1, 10 = 0.300000, 0.300000, 0.300000
[F2] Omega 1: [update]/[g_p] on the curve at lambda 0.1, 1, 10 = 0.000000, 0.000000, 0.000000
[F2] Omega 1.4: [update]/[g_p] on the curve at lambda 0.1, 1, 10 = 0.400000, 0.400000, 0.400000

```

```

[figure] F02_pareto_field.png
[F3] exact rule (instantaneous norms): Omega 0.25 endpoint (2.0000, 1.0000) L_present 0.0000 L_past 43.9999 lambda_eff 0.0001
distance to curve 5.74e-03
[F3] exact rule (instantaneous norms): Omega 0.35 endpoint (2.0000, 1.0000) L_present 0.0000 L_past 43.9996 lambda_eff 0.0001
distance to curve 5.73e-03
[F3] exact rule (instantaneous norms): Omega 0.5 endpoint (2.0000, 0.9999) L_present 0.0000 L_past 43.9963 lambda_eff 0.0001
distance to curve 5.64e-03
[F3] exact rule (instantaneous norms): Omega 0.71 endpoint (1.9981, 0.9962) L_present 0.0000 L_past 43.8458 lambda_eff 0.0001
distance to curve 1.56e-03
[F3] exact rule (instantaneous norms): Omega 1 endpoint (0.0411, 0.0069) L_present 2.1652 L_past 0.0141 lambda_eff 2.985
distance to curve 6.78e-05
[F3] exact rule (instantaneous norms): Omega 1.41 endpoint (0.0335, -0.0133) L_present 2.1902 L_past 0.0111 lambda_eff 4.074
distance to curve 1.87e-02
[F3] exact rule (instantaneous norms): Omega 2 endpoint (0.0190, 0.0438) L_present 2.1908 L_past 0.0259 lambda_eff 4.786
distance to curve 4.01e-02
[F3] exact rule (instantaneous norms): Omega 2.83 endpoint (0.0128, 0.0680) L_present 2.1918 L_past 0.0567 lambda_eff 5.248
distance to curve 6.49e-02
[F3] exact rule (instantaneous norms): Omega 4 endpoint (0.0397, -0.0510) L_present 2.1975 L_past 0.0438 lambda_eff 4.074
distance to curve 5.69e-02
[F3] registered estimator (EMA 0.9 of the gradients, cap 1e4): Omega 0.25 endpoint (2.0000, 1.0000) L_present 0.0000 L_past
43.9998 lambda_eff 0.0001 distance to curve 5.74e-03
[F3] registered estimator (EMA 0.9 of the gradients, cap 1e4): Omega 0.35 endpoint (2.0000, 1.0000) L_present 0.0000 L_past
43.9993 lambda_eff 0.0001 distance to curve 5.73e-03
[F3] registered estimator (EMA 0.9 of the gradients, cap 1e4): Omega 0.5 endpoint (1.9999, 0.9998) L_present 0.0000 L_past
43.9920 lambda_eff 0.0001 distance to curve 5.52e-03
[F3] registered estimator (EMA 0.9 of the gradients, cap 1e4): Omega 0.71 endpoint (1.9949, 0.9902) L_present 0.0000 L_past
43.6043 lambda_eff 0.0001905 distance to curve 1.20e-03
[F3] registered estimator (EMA 0.9 of the gradients, cap 1e4): Omega 1 endpoint (0.0339, 0.0057) L_present 2.1799 L_past 0.0096
lambda_eff 3.631 distance to curve 5.47e-05
[F3] registered estimator (EMA 0.9 of the gradients, cap 1e4): Omega 1.41 endpoint (0.0153, -0.1659) L_present 2.3094 L_past
0.3322 lambda_eff 1e+06 distance to curve 1.67e-01
[F3] registered estimator (EMA 0.9 of the gradients, cap 1e4): Omega 2 endpoint (0.0127, -0.2313) L_present 2.3537 L_past 0.6432
lambda_eff 1e+06 distance to curve 2.32e-01
[F3] registered estimator (EMA 0.9 of the gradients, cap 1e4): Omega 2.83 endpoint (0.0117, -0.3237) L_present 2.4147 L_past
1.2586 lambda_eff 1e+06 distance to curve 3.24e-01
[F3] registered estimator (EMA 0.9 of the gradients, cap 1e4): Omega 4 endpoint (0.0113, -0.4544) L_present 2.5063 L_past 2.4789
lambda_eff 1e+06 distance to curve 4.55e-01
[figure] F03_trajectories.png
[F4] noise 0: total loss per Omega 0.25:69.662 0.35:69.662 0.5:69.662 0.71:69.662 1:19.875 1.41:18.602 2:31.023 2.83:57.141
4:47.592
[F4] noise 0.5: total loss per Omega 0.25:68.513 0.35:67.307 0.5:65.262 0.71:61.265 1:3.479 1.41:11.146 2:11.010 2.83:56.787
4:101.379
[F4] noise 2: total loss per Omega 0.25:71.861 0.35:66.991 0.5:59.231 0.71:45.890 1:4.412 1.41:4.525 2:4.647 2.83:8.766 4:31.483
[figure] F04_plateau.png
[F5] edge lambda* = 1.2575; fixed converged for lambda <= 1.1253; first divergent lambda 1.3594; rule Omega=1 lambda_eff 0.04467
total loss 19.8753 median w 0.04444
[figure] F05_stability_edge.png
[F6] v = 0.1: ratio 1/phi 4.7193, ratio 1/2 3.5392
[F6] v = 1: ratio 1/phi 1.0000, ratio 1/2 1.0787
[F6] v = 10: ratio 1/phi 17.7028, ratio 1/2 34.0000
[figure] F06_kalman.png
[F7] EQ2 mfeat_fourier: tuned lambda 30 mean 26.1000 step 1.0000; rule per Omega 0.5:29.7000 0.71:28.7500 1:29.4000 1.41:28.9000
2:29.0000
[F7] EQ2 mfeat_pixel: tuned lambda 1410 mean 50.0500 step 3.5121; rule per Omega 0.5:50.7500 0.71:50.4000 1:52.1500 1.41:50.7000
2:52.6000
[F7] EQ2 texture: tuned lambda 100 mean 48.5600 step 3.6120; rule per Omega 0.5:49.8200 0.71:51.1000 1:51.3400 1.41:51.5800
2:46.5200
[F7] EQ3 fars: tuned lambda 283 mean 13.0000 step 3.3508; rule per Omega 0.25:9.2600 0.35:9.2600 0.5:9.2600 0.71:9.2600 1:9.2600
1.41:9.2600 2:9.2800 2.83:9.2600 4:9.2600
[F7] EQ3 krkopt: tuned lambda 3 mean 16.9031 step 8.9180; rule per Omega 0.25:15.6044 0.35:16.8432 0.5:17.8222 0.71:17.5225
1:17.8821 1.41:18.6613 2:18.9411 2.83:19.0809 4:19.6204
[F7] EQ3 led24: tuned lambda 30 mean 18.5915 step 4.3633; rule per Omega 0.25:18.1847 0.35:19.2175 0.5:19.4679 0.71:19.6557
1:19.2801 1.41:20.2191 2:20.2817 2.83:19.0297 4:19.9687
[F7] EQ3 led7: tuned lambda 0.212766 mean 21.9688 step 3.2031; rule per Omega 0.25:24.6250 0.35:24.9688 0.5:25.8125 0.71:26.0000
1:26.0312 1.41:26.8125 2:26.1562 2.83:28.9688 4:27.9062
[F7] EQ3 mfeat_factors: tuned lambda 300 mean 54.4500 step 4.5371; rule per Omega 0.25:55.6000 0.35:56.0000 0.5:57.1500
0.71:57.9000 1:58.3500 1.41:58.0000 2:58.3500 2.83:57.5500 4:56.6000
[F7] EQ3 mfeat_morphological: tuned lambda 2.83 mean 34.2000 step 4.8719; rule per Omega 0.25:31.5000 0.35:36.2000 0.5:37.0000
0.71:37.3000 1:37.9000 1.41:39.6500 2:38.9000 2.83:41.6500 4:40.5000
[figure] F07_omega_profiles.png
[F8] EQ2 mfeat_fourier: tuned 30 (26.1000); rule median w 1337 accuracy 29.4000; first fixed lambda above tuned with mean under
half the best: 300.0
[F8] EQ2 mfeat_pixel: tuned 1410 (50.0500); rule median w 3184 accuracy 52.1500; first fixed lambda above tuned with mean under
half the best: 10000.0
[F8] EQ2 texture: tuned 100 (48.5600); rule median w 110.6 accuracy 51.3400; first fixed lambda above tuned with mean under half
the best: 283.0
[F8] EQ3 fars: tuned 283 (13.0000); rule median w 426.3 accuracy 9.2600; first fixed lambda above tuned with mean under half the
best: 1000.0
[F8] EQ3 krkopt: tuned 3 (16.9031); rule median w 4.761 accuracy 17.8821; first fixed lambda above tuned with mean under half
the best: 6.0
[F8] EQ3 led24: tuned 30 (18.5915); rule median w 31.7 accuracy 19.2801; first fixed lambda above tuned with mean under half the
best: 84.9
[F8] EQ3 led7: tuned 0.212766 (21.9688); rule median w 2.777 accuracy 26.0312; first fixed lambda above tuned with mean under
half the best: 3.0
[F8] EQ3 mfeat_factors: tuned 300 (54.4500); rule median w 1066 accuracy 58.3500; first fixed lambda above tuned with mean under
half the best: 3000.0
[F8] EQ3 mfeat_morphological: tuned 2.83 (34.2000); rule median w 14.67 accuracy 37.9000; first fixed lambda above tuned with
mean under half the best: 30.0
[figure] F08_lambda_grids.png
[F9] EQ2 mfeat_fourier: Omega=1 - tuned by cap x smooth cap10/sm0.8:-0.35; cap10/sm0.9:-0.35; cap10/sm0.98:-0.35;

```

```

cap100/sm0.8:+0.05; cap100/sm0.9:+0.15; cap100/sm0.98:+0.05; cap10000/sm0.8:+2.80; cap10000/sm0.9:+3.30; cap10000/sm0.98:+3.15
[F9] EQ2 mfeat_pixel: Omega=1 - tuned by cap x smooth cap10/sm0.8:-16.60; cap10/sm0.9:-16.60; cap10/sm0.98:-16.60;
cap100/sm0.8:-13.45; cap100/sm0.9:-13.45; cap100/sm0.98:-13.40; cap10000/sm0.8:+2.90; cap10000/sm0.9:+2.10;
cap10000/sm0.98:+1.35
[F9] EQ2 texture: Omega=1 - tuned by cap x smooth cap10/sm0.8:-1.02; cap10/sm0.9:-1.26; cap10/sm0.98:-1.46; cap100/sm0.8:+0.30;
cap100/sm0.9:+0.16; cap100/sm0.98:+0.04; cap10000/sm0.8:+0.98; cap10000/sm0.9:+2.78; cap10000/sm0.98:-2.18
[F9] EQ3 fars: Omega=1 - tuned by cap x smooth cap10/sm0.8:-3.72; cap10/sm0.9:-3.72; cap10/sm0.98:-3.72; cap100/sm0.8:-3.74;
cap100/sm0.9:-3.74; cap100/sm0.98:-3.74; cap10000/sm0.8:-3.74; cap10000/sm0.9:-3.74; cap10000/sm0.98:-3.72
[F9] EQ3 krko: Omega=1 - tuned by cap x smooth cap10/sm0.8:+2.14; cap10/sm0.9:+1.32; cap10/sm0.98:+0.66; cap100/sm0.8:+2.18;
cap100/sm0.9:+1.26; cap100/sm0.98:+0.12; cap10000/sm0.8:+1.48; cap10000/sm0.9:+0.98; cap10000/sm0.98:+2.20
[F9] EQ3 led24: Omega=1 - tuned by cap x smooth cap10/sm0.8:-1.56; cap10/sm0.9:-1.56; cap10/sm0.98:-1.56; cap100/sm0.8:+1.53;
cap100/sm0.9:+1.19; cap100/sm0.98:+0.94; cap10000/sm0.8:+1.28; cap10000/sm0.9:+0.69; cap10000/sm0.98:+0.63
[F9] EQ3 led7: Omega=1 - tuned by cap x smooth cap10/sm0.8:+3.00; cap10/sm0.9:+3.00; cap10/sm0.98:+3.00; cap100/sm0.8:+3.16;
cap100/sm0.9:+2.66; cap100/sm0.98:+2.69; cap10000/sm0.8:+3.16; cap10000/sm0.9:+4.06; cap10000/sm0.98:+6.03
[F9] EQ3 mfeat_factors: Omega=1 - tuned by cap x smooth cap10/sm0.8:-11.15; cap10/sm0.9:-11.15; cap10/sm0.98:-11.15;
cap100/sm0.8:-5.25; cap100/sm0.9:-5.20; cap100/sm0.98:-5.15; cap10000/sm0.8:+3.65; cap10000/sm0.9:+3.90; cap10000/sm0.98:+2.70
[F9] EQ3 mfeat_morphological: Omega=1 - tuned by cap x smooth cap10/sm0.8:+3.65; cap10/sm0.9:+2.75; cap10/sm0.98:+3.85;
cap100/sm0.8:+4.10; cap100/sm0.9:+2.70; cap100/sm0.98:+3.35; cap10000/sm0.8:+5.20; cap10000/sm0.9:+3.70; cap10000/sm0.98:+6.20
[figure] F09_sensitivity.png
[F10] rule at  $\Omega = 1$ : minus tuned lambda per carrier fars:-3.7400, krko: +0.9790, led24:+0.6886, led7:+4.0625,
mfeat_factors:+3.9000, mfeat_morphological:+3.7000
[F10] Bayes (Laplace 1/2) arm: minus tuned lambda per carrier fars:-3.7200, krko: +1.2587, led24:-2.7543, led7:+0.0000,
mfeat_factors:-12.5000, mfeat_morphological:-7.6000
[F10] reduction arm (fixed w = rule's median w): minus tuned lambda per carrier fars:-6.1600, krko: -6.7133, led24:+0.0626,
led7:-12.1875, mfeat_factors:-19.6500, mfeat_morphological:-10.7000
[figure] F10_eq3_arms.png
[F11] optdigits fisher: rule per Omega 0.5:81.6370 0.71:85.4626 1:89.4128 1.41:88.7900 2:87.0463
[F11] optdigits euclid: rule per Omega 1:90.8719
[F11] optdigits fixed: 0.2:82.8648 0.5:88.1139 1:91.0498 2:89.4306 4:75.0712
[F11] pendigits fisher: rule per Omega 0.5:86.6212 0.71:93.1878 1:92.0418 1.41:86.2483 2:78.9632
[F11] pendigits euclid: rule per Omega 1:94.3156
[F11] pendigits fixed: 0.2:89.0314 0.5:92.5057 1:91.1960 2:90.0591 4:86.1301
[F11] letter fisher: rule per Omega 0.5:53.3083 0.71:62.8836 1:67.1264 1.41:59.6802 2:48.5507
[F11] letter euclid: rule per Omega 1:69.0255
[F11] letter fixed: 0.2:58.6807 0.5:65.3973 1:67.5962 2:68.8756 4:63.6282
[figure] F11_eqx_reduction.png
[F12] ratio 0.25: theta_B 0.2000; Omega=1 fixed-point interval [0, 1]
[F12] ratio 1: theta_B 0.5000; Omega=1 fixed-point interval [0, 1]
[F12] ratio 4: theta_B 0.8000; Omega=1 fixed-point interval [0, 1]
[figure] F12_bayes_line.png

```

B.6 The cross-verification battery (`theory/checks/omega\_vs\_methods.txt`)

theory/checks/omega_vs_methods.txt

Omega = 1 against the rules other methods use, on the two-task quadratic of omega_sweeps.py (F = 16 H; noise 0.5; lr 0.05; 4000 steps; 5 seeds; start at the old-task optimum)
stability edge of a fixed weight: lambda* = 1.2575 at F, 0.0786 at 16F; tuned fixed weight (in-sample, grid (0.003, 0.01, 0.03, 0.1, 0.3, 0.6, 1.0, 1.2, 1.5, 3.0)): lambda = 1 total 3.3168 at F, lambda = 0.03 total 14.7298 at 16F

method	knob	L_present	L_past	total	/tuned	16F: total	/tuned	knob x10	label
fixed	1	3.1169	0.1999	3.3168	1.000	nan	inf	DIVERGES	not scale-robust; has a divergence
edge in its knob									
omega	1	2.8203	0.6585	3.4789	1.049	9.7260	0.660	finite	scale-robust; no edge found at knob
x10									
gradnorm	0	2.8010	0.7296	3.5306	1.064	nan	inf	n/a	not scale-robust; no knob
mega	0	2.6479	1.0830	3.7309	1.125	nan	inf	n/a	not scale-robust; no knob
agem	0	0.2066	40.1317	40.3382	12.162	639.0390	43.384	n/a	not scale-robust; no knob
softproj	0.5	0.0215	65.2291	65.2506	19.672	1043.5128	70.843	finite	not scale-robust; no edge found at
knob x10									
subspace	2	2.9920	15.7519	18.7439	5.651	255.0219	17.313	finite	not scale-robust; no edge found at
knob x10									
klctrl	1	2.6652	1.0382	3.7035	1.117	nan	inf	finite	not scale-robust; no edge found at
knob x10									
select	0.8	3.1797	0.1855	3.3652	1.015	nan	inf	n/a	not scale-robust; no knob

computed:

- fixed [fixed weight (EWC/L2-SP/KL penalty); lambda = 1 is also the Bayes/ER-sum weight in this model; needs a sweep of lambda] -> within 10 % of the tuned fixed weight at F: True, at 16F with the same knob: False; scale-robust: False; divergence edge: True
- omega [normalised gradient (the rule under test); needs smoothing, cap, floor] -> within 10 % of the tuned fixed weight at F: True, at 16F with the same knob: True; scale-robust: True; divergence edge: False
- gradnorm [learned task weights (GradNorm alpha = 0); needs a weight learning rate] -> within 10 % of the tuned fixed weight at F: True, at 16F with the same knob: False; scale-robust: False; divergence edge: False
- mega [loss ratio (MEGA-I); needs smoothing] -> within 10 % of the tuned fixed weight at F: False, at 16F with the same knob: False; scale-robust: False; divergence edge: False
- agem [conflict projection (A-GEM); needs nothing] -> within 10 % of the tuned fixed weight at F: False, at 16F with the same knob: False; scale-robust: False; divergence edge: False
- softproj [soft conflict projection (PCR-style); needs a fraction c] -> within 10 % of the tuned fixed weight at F: False, at 16F with the same knob: False; scale-robust: False; divergence edge: False
- subspace [protected subspace, no past term (OGD / OGPSA / SafeAnchor-style); needs a rank k] -> within 10 % of the tuned fixed weight at F: False, at 16F with the same knob: False; scale-robust: False; divergence edge: False
- klctrl [target controller on the past loss (adaptive-KL-style); needs a target] -> within 10 % of the tuned fixed weight at F: False, at 16F with the same knob: False; scale-robust: False; divergence edge: False
- select [gradient-based sample selection at fixed lambda = 1; needs a quantile] -> within 10 % of the tuned fixed weight at F: True, at 16F with the same knob: False; scale-robust: False; divergence edge: False
- scale-robust methods (within 10 % of the tuned fixed weight at both F and 16F with one knob setting): ['omega']

methods that diverge at 16F with the knob that worked at F: ['fixed', 'gradnorm', 'mega', 'klctrl', 'select']
 methods that forget (L_{past} more than 10 x the tuned fixed weight's 0.1999) at F: ['agem', 'softproj', 'subspace']
 methods with a divergence edge in their own knob: ['fixed']
 reading (from the lists above): the rules that weight the past pull by a fixed number, a learned number or a loss ratio carry that number's units and fail when the past curvature changes scale; the rules that project rather than weight never diverge but drop the past wherever the two gradients do not conflict; the rule that scales the past pull by the present pull is the one in this list whose quality survives the change of scale, because its step is bounded by the present step

B.7 The reprocessed mathematics (`theory/checks/omega\_reprocessed.txt`)

theory/checks/omega_reprocessed.txt

H-EQ reprocessed (2026-09-22, prompt-log entry 74): the two-task quadratic of omega_sweeps.py (d = 5, F 16x H, noise 0.5, lr 0.05, 4000 steps, 5 seeds, start at the past optimum b)

[1] The rule is invariant to the SCALE of the past term (exactly, when the past term's noise scales with it) and not to its SHAPE
 max over steps and seeds of $|\theta_t(F) - \theta_t(16F)|$ under the rule, past noise scaled with the term: 0.000e+00 -> identical trajectories: $w(cF) = w(F)/c$ exactly, the EMA being linear in g_q
 the same with the past noise left at 0.5 (not scaled): 4.252e+00 -> the invariance is to the term, not to a noise of fixed size added to it
 mean endpoint distance between F and F with its diagonal reversed (same eigenvalues, other coordinates): 0.0412 -> the shape of F changes the endpoint; the rule does not see shape
 consequence: every method that changes the importance estimate (which coordinate is protected how much) composes with the rule and is untouched by it; the 16F test of omega_vs_methods.py measures scale only

[2] The step bound $|g_p + w g_q| \leq (1 + \Omega) |g_p|$: on which gradients does it hold, and what does EQ-B add to it?
 rule smoothed: $\max |e_{p,q} + w e_{q,q}| / |e_{p,q}| = 1.9778$, fraction above 2: 0.0000 -> bound holds
 rule raw: $\max |g_p + w g_q| / |g_p| = 38.6579$, fraction above 2: 0.0089, median 0.4903; $\max |g_p + w g_q| = 84.4515$
 EQ-B smoothed: $\max |e_{p,q} + w e_{q,q}| / |e_{p,q}| = 1.9778$, fraction above 2: 0.0000 -> bound holds
 EQ-B raw: $\max |g_p + w g_q| / |g_p| = 38.6579$, fraction above 2: 0.0089, median 0.4903; $\max |g_p + w g_q| = 84.4515$
 computed: the ratio bound is a bound on the smoothed pull; the raw ratio exceeds 2 on a fraction of steps for both rules (a small $|g_p|$ in the denominator), and EQ-B does not lower the largest raw step (84.4515 vs 84.4515): what EQ-B bounds is the present gradient's LENGTH (κ x the largest kept length of its recent batches), hence the absolute step, not the ratio

[3] Does a past term whose minimum is a SET (rank-deficient F: a constraint on a subspace) break the rule's premise that $|g_q|$ measures distance from the past? (the candidate mechanism for the two-signed LwF control of EQ3-4)
 full-rank F $\max w$ 6.815, fraction of steps at the cap 0.0000, finite True, total loss 3.4789
 rank-3 F (two free directions) $\max w$ 12.24, fraction of steps at the cap 0.0000, finite True, total loss 1.2541
 free directions under rank-3 F: $\max |\theta_t - a|$ over the two free coordinates at the end = 0.1178 (the present optimum is reached there: the constraint says nothing about them, and the rule's w is driven by the constrained directions only)
 median derived w : full-rank 0.5516, rank-3 0.5409 -> NO: on a quadratic constraint the free directions relax to the present optimum and the derived weight is unchanged; the quadratic model supplies no mechanism for the two-signed LwF control, which stays an empirical statement (EQ3-4) with the softmax non-identifiability argument as the open candidate

[4] Poisoned present batches (gradient x factor for 'run' consecutive steps from step 2000): which rule fails first, and whether EQ-B (present gradient clipped at κ x the largest kept length of its recent batches) survives
 clean reference: tuned fixed $\lambda = 1$, total 3.3168; clean totals: rule 3.4789, EQ-B 3.4789

factor	run	fixed	fixed+clip	rule	EQ-B	(total)
after 4000 steps; DIVERGED if any seed is non-finite; max w of the rule arms)						
10	1	3.3168 w<=	1	3.3168 w<=	1	3.4767 w<= 6.82 3.4786 w<= 6.82
10	5	3.3168 w<=	1	3.3168 w<=	1	3.4409 w<= 6.82 3.4307 w<= 6.82
100	1	3.3168 w<=	1	3.3168 w<=	1	3.4554 w<= 8.04 3.4786 w<= 6.82
100	5	3.3168 w<=	1	3.3168 w<=	1	17.2000 w<= 8.04 3.4244 w<= 6.82
1000	1	3.3168 w<=	1	3.3168 w<=	1	3.4457 w<= 75.4 3.4786 w<= 6.82
1000	5	DIVERGED w<=	1	3.3168 w<=	1	DIVERGED w<= 75.4 3.4244 w<= 6.82
10000	1	3.3168 w<=	1	3.3168 w<=	1	3.4445 w<= 749 3.4786 w<= 6.82
10000	5	DIVERGED w<=	1	3.3168 w<=	1	DIVERGED w<= 749 3.4244 w<= 6.82

computed: first (factor, run) at which each arm diverges, in sweep order: fixed: (1000.0, 5); fixed+clip: never; rule: (1000.0, 5); EQ-B: never
 computed: first (factor, run) at which each arm's total exceeds twice its clean total (or diverges): fixed: (1000.0, 5); fixed+clip: never; rule: (100.0, 5); EQ-B: never
 computed: the registered rule degrades before the tuned fixed weight does: True; EQ-B degrades in the sweep: False
 computed: the registered rule has a poison threshold in this sweep: True; EQ-B survives the whole sweep: True; the tuned fixed weight has a threshold: True; the rule fails where the fixed weight does not: False
 computed: on the clean stream EQ-B is within 1 % of the registered rule: True (totals 3.4789 vs 3.4789); the bound is idle on the clean stream

[5] The Fisher-norm ratio of H-EQ as written ($| \cdot |_F^{-1}$ on both gradients) has the same fixed-point set as the Euclidean one: any positive scalar w needs g_p antiparallel to g_q , i.e. θ on the Pareto curve
 Euclidean ratio: nearest Pareto $\lambda = 0.4926$ at distance 0.0558 | Fisher ratio: $\lambda = 0.4763$ at distance 0.0566
 computed: both endpoints sit within 0.2 of the curve: True; the metric moves the stopping point along the curve, it does not change the set (AI's metric is a choice of stopping point, not of mechanism)

[6] Under Adam (the rule upstream of the preconditioner) the divergence edge disappears for every method and the scale test becomes a question of endpoint quality
 tuned fixed λ under Adam: 1 (total 3.2859) at F; 1.2 (total 3.4399) at 16F
 fixed λ (tuned at F) F: total 3.2859 (/tuned 1.000, finite True) | 16F, same knob: total 3.4403 (/tuned 1.000, finite True)
 rule $\Omega = 1$ F: total 3.3370 (/tuned 1.016, finite True) | 16F, same knob: total 3.4812 (/tuned 1.012, finite True)
 EQ-B $\Omega = 1$ F: total 3.3370 (/tuned 1.016, finite True) | 16F, same knob: total 3.4812 (/tuned 1.012, finite True)
 computed: scale-robust under Adam (within 10 % of the Adam-tuned fixed weight at both scales): ['fixed λ (tuned at F)', 'rule $\Omega = 1$ ', 'EQ-B $\Omega = 1$ ']; every arm finite: True
 computed: the fixed weight is scale-robust under Adam too: the rule has no scale advantage under Adam in this model, its property is an SGD property

[7] What changes in H-EQ (proposal for the v3.2 list; theory/CRR.md is not edited here), each line conditional on the numbers above:

(a) scope: the quadratic model gives NO mechanism for excluding constraint terms ([3]); the narrowing of H-EQ to losses on the past rests on EQ3-4 (empirical, two-signed) and is stated as an empirical scope, not a derived one;

(b) the bound is the mechanism ([2]): the claim is 'the past pull never exceeds $(1 + \Omega)$ x the present pull in the smoothed gradients', not 'equal pull is optimal' (omega_sweeps.txt [1]-[2]);

(c) present bound: the registered rule degrades at poison (100.0, 5) (diverges at (1000.0, 5)) where EQ-B survives the whole sweep ([4]); EQ-B is the enhancement to test, with the poisoned stream as its positive control and the clean stream as its negative control (the bound must be idle there);

(d) the metric is a stopping point on the Pareto curve, not a mechanism ([5]); (e) the optimiser is a registered parameter ([6]), and under Adam the fixed weight is scale-robust too, so the scale claim is SGD-only.

B.8 The EQ-B gates (`prereg/eq4/gate\_EQB.txt`, `prereg/eq4/gate\_EQBM.txt`)

prereg/eq4/gate_EQB.txt

```
gate for EQB
  must FAIL on ['S-R convex replay, constant label scale (adaptivity idle)', 'S-X constraint learner, tuned weight a small
fraction of present norm', 'S-Y/LwF softmax MLP, distillation constraint (input scale 4)']
  must PASS on ['S-Y softmax MLP, online EWC, 16x Fisher-scale mismatch (input scale 4)', 'S-Y softmax MLP, online EWC, scale 4,
poisoned batches (x1000, 5 per task)']

S-R convex replay, constant label scale (adaptivity idle) FAIL EQB(Ω=1)=1.4416 rule(Ω=1)=1.4416 tuned w=1.0:1.2250 (w_med of
EQB=0.607) | behind-by +0.177 on the seed means (seeds behind: 5/5) | best Ω=2.0 landscape 0.5:1.661 0.71:1.582 1.0:1.442
1.41:1.257 2.0:1.226 | reduction arm w=0.607:1.2799
S-V convex replay, 16x label-scale swings (adaptivity load-bearing) FAIL EQB(Ω=1)=1.1992 rule(Ω=1)=1.1992 tuned
w=0.0625:1.1024 (w_med of EQB=0.599) | behind-by +0.088 on the seed means (seeds behind: 5/5) | best Ω=0.5 landscape 0.5:1.059
0.71:1.088 1.0:1.199 1.41:1.663 2.0:3.397 | reduction arm w=0.599:3.9393
S-W EWC-Laplace convex replay, 16x curvature-scale mismatch FAIL EQB(Ω=1)=0.0654 rule(Ω=1)=0.0654 tuned w=0.0625:0.0570
(w_med of EQB=1.409) | behind-by +0.149 on the seed means (seeds behind: 5/5) | best Ω=0.71 landscape 0.5:0.055 0.71:0.053
1.0:0.065 1.41:0.065 2.0:0.065 | reduction arm w=1.409:0.0654
S-X constraint learner, tuned weight a small fraction of present norm FAIL EQB(Ω=1)=0.9420 rule(Ω=1)=0.9420 tuned
w=0.25:0.8803 (w_med of EQB=1.284) | behind-by +0.070 on the seed means (seeds behind: 5/5) | best Ω=1.0 landscape 0.5:1.478
0.71:1.182 1.0:0.942 1.41:1.072 2.0:1.414 | reduction arm w=1.284:0.9430
S-Y softmax MLP, online EWC, 16x Fisher-scale mismatch (input scale 4) PASS EQB(Ω=1)=0.1745 rule(Ω=1)=0.1740 tuned
w=256.0:0.1977 (w_med of EQB=1833.531) | behind-by -0.117 on the seed means (seeds behind: 1/5) | best Ω=0.5 landscape 0.5:0.127
0.71:0.168 1.0:0.175 1.41:0.187 2.0:0.233 | reduction arm w=1833.531:0.8257
S-Y softmax MLP, online EWC, unit input scale (informational) FAIL EQB(Ω=1)=0.0544 rule(Ω=1)=0.0544 tuned w=128.0:0.0394
(w_med of EQB=559.599) | behind-by +0.380 on the seed means (seeds behind: 3/5) | best Ω=1.41 landscape 0.5:0.036 0.71:0.038
1.0:0.054 1.41:0.036 2.0:0.053 | reduction arm w=559.599:0.2152
S-Y softmax MLP, online EWC, scale 4, cap=100 (informational) FAIL EQB(Ω=1)=0.2755 rule(Ω=1)=0.2814 tuned w=256.0:0.1977
(w_med of EQB=84.093) | behind-by +0.393 on the seed means (seeds behind: 5/5) | best Ω=2.0 landscape 0.5:0.283 0.71:0.280
1.0:0.275 1.41:0.271 2.0:0.267 | reduction arm w=84.093:0.2651
S-Y softmax MLP, online EWC, scale 4, smooth=0.8 (informational) PASS EQB(Ω=1)=0.1482 rule(Ω=1)=0.1484 tuned w=256.0:0.1977
(w_med of EQB=575.167) | behind-by -0.250 on the seed means (seeds behind: 1/5) | best Ω=0.71 landscape 0.5:0.138 0.71:0.137
1.0:0.148 1.41:0.184 2.0:0.190 | reduction arm w=575.167:0.4224
S-Y softmax MLP, online EWC, scale 4, smooth=0.98 (informational) PASS EQB(Ω=1)=0.1968 rule(Ω=1)=0.2470 tuned w=256.0:0.1977
(w_med of EQB=3282.673) | behind-by -0.005 on the seed means (seeds behind: 2/5) | best Ω=1.0 landscape 0.5:0.219 0.71:0.204
1.0:0.197 1.41:0.214 2.0:0.312 | reduction arm w=3282.673:1.0000
S-Y/LwF softmax MLP, distillation constraint (input scale 4) FAIL EQB(Ω=1)=0.7305 rule(Ω=1)=0.7305 tuned w=0.0625:0.2106
(w_med of EQB=1.678) | behind-by +2.468 on the seed means (seeds behind: 5/5) | best Ω=0.5 landscape 0.5:0.293 0.71:0.405
1.0:0.730 1.41:0.879 2.0:0.903 | reduction arm w=1.678:0.7347
S-Y softmax MLP, online EWC, scale 4, poisoned batches (x1000, 5 per task) PASS EQB(Ω=1)=0.6287 rule(Ω=1)=0.8101 tuned
w=0.0625:0.7289 (w_med of EQB=3861.126) | behind-by -0.137 on the seed means (seeds behind: 1/5) | best Ω=0.5 landscape
0.5:0.527 0.71:0.620 1.0:0.629 1.41:0.551 2.0:0.602 | reduction arm w=3861.126:1.0000

GATE OPEN
```

prereg/eq4/gate_EQBM.txt

```
gate for EQBM
  must FAIL on ['S-R convex replay, constant label scale (adaptivity idle)', 'S-Y softmax MLP, online EWC, 16x Fisher-scale
mismatch (input scale 4)']
  must PASS on ['S-Y softmax MLP, online EWC, scale 4, poisoned batches (x1000, 5 per task)']

S-R convex replay, constant label scale (adaptivity idle) FAIL rule(Ω=1)=1.4416 EQB(Ω=1)=1.4416 | rel.gain of EQB +0.000
(seeds>0: 0/5) | per seed +0.000 +0.000 +0.000 +0.000 +0.000
S-V convex replay, 16x label-scale swings (adaptivity load-bearing) FAIL rule(Ω=1)=1.1992 EQB(Ω=1)=1.1992 | rel.gain of EQB
+0.000 (seeds>0: 0/5) | per seed +0.000 +0.000 +0.000 +0.000 +0.000
S-W EWC-Laplace convex replay, 16x curvature-scale mismatch FAIL rule(Ω=1)=0.0654 EQB(Ω=1)=0.0654 | rel.gain of EQB +0.000
(seeds>0: 0/5) | per seed +0.000 +0.000 +0.000 +0.000 +0.000
S-X constraint learner, tuned weight a small fraction of present norm FAIL rule(Ω=1)=0.9420 EQB(Ω=1)=0.9420 | rel.gain of EQB
+0.000 (seeds>0: 0/5) | per seed +0.000 +0.000 +0.000 +0.000 +0.000
S-Y softmax MLP, online EWC, 16x Fisher-scale mismatch (input scale 4) FAIL rule(Ω=1)=0.1740 EQB(Ω=1)=0.1745 | rel.gain of
EQB -0.002 (seeds>0: 0/5) | per seed +0.000 +0.000 -0.009 -0.002 +0.000
S-Y softmax MLP, online EWC, unit input scale (informational) FAIL rule(Ω=1)=0.0544 EQB(Ω=1)=0.0544 | rel.gain of EQB +0.000
(seeds>0: 0/5) | per seed +0.000 +0.000 +0.000 +0.000 +0.000
S-Y softmax MLP, online EWC, scale 4, cap=100 (informational) FAIL rule(Ω=1)=0.2814 EQB(Ω=1)=0.2755 | rel.gain of EQB +0.011
(seeds>0: 2/5) | per seed +0.000 +0.002 +0.000 +0.055 +0.000
S-Y softmax MLP, online EWC, scale 4, smooth=0.8 (informational) FAIL rule(Ω=1)=0.1484 EQB(Ω=1)=0.1482 | rel.gain of EQB
+0.002 (seeds>0: 1/5) | per seed +0.000 +0.000 +0.009 +0.000 +0.000
S-Y softmax MLP, online EWC, scale 4, smooth=0.98 (informational) FAIL rule(Ω=1)=0.2470 EQB(Ω=1)=0.1968 | rel.gain of EQB
+0.118 (seeds>0: 1/5) | per seed +0.727 +0.000 +0.000 -0.001 -0.136
S-Y/LwF softmax MLP, distillation constraint (input scale 4) FAIL rule(Ω=1)=0.7305 EQB(Ω=1)=0.7305 | rel.gain of EQB +0.000
(seeds>0: 0/5) | per seed +0.000 +0.000 +0.000 +0.000 +0.000
S-Y softmax MLP, online EWC, scale 4, poisoned batches (x1000, 5 per task) PASS rule(Ω=1)=0.8101 EQB(Ω=1)=0.6287 | rel.gain
```

B.9 The exploratory development run on the seen EQ3 carriers (`runs/eq4\_dev/summary.txt`)

runs/eq4_dev/summary.txt

```
EXPLORATORY (seen carriers; no verdict): EQ-B kappa choice and the poisoned regime
[fars] step 2.41; tuned lambda 30: 12.32; rule  $\Omega=1$ : 9.24 (-3.08) | EQ-B k=2: 9.24 (-3.08; clip 0.001) | EQ-B k=4: 9.24 (-3.08; clip 0.000) | EQ-B k=8: 9.24 (-3.08; clip 0.000)
  POISON: fixed tuned 0.1: 0.00; rule  $\Omega=1$ : 8.82 (finite 5/5, max w median 1e+04) | fixed+clip k=2 tuned 10: 9.88 | fixed+clip k=4 tuned 0.1: 10.18 | EQ-B k=2: 11.46 (rule - EQ-B -2.64; clip 0.006) | EQ-B k=4: 9.02 (rule - EQ-B -0.20; clip 0.005) | EQ-B k=8: 9.26 (rule - EQ-B -0.44; clip 0.005)
[krkopt] step 1.00; tuned lambda 1: 6.25; rule  $\Omega=1$ : 5.69 (-0.56) | EQ-B k=2: 5.75 (-0.50; clip 0.000) | EQ-B k=4: 5.69 (-0.56; clip 0.000) | EQ-B k=8: 5.69 (-0.56; clip 0.000)
  POISON: fixed tuned 0.1: 0.00; rule  $\Omega=1$ : 4.69 (finite 5/5, max w median 1e+04) | fixed+clip k=2 tuned 1: 9.28 | fixed+clip k=4 tuned 0.1: 0.00 | EQ-B k=2: 11.36 (rule - EQ-B -6.67; clip 0.009) | EQ-B k=4: 7.03 (rule - EQ-B -2.34; clip 0.009) | EQ-B k=8: 6.81 (rule - EQ-B -2.12; clip 0.009)
[led24] step 1.94; tuned lambda 3: 19.62; rule  $\Omega=1$ : 23.79 (+4.16) | EQ-B k=2: 23.82 (+4.19; clip 0.001) | EQ-B k=4: 23.79 (+4.16; clip 0.000) | EQ-B k=8: 23.79 (+4.16; clip 0.000)
  POISON: fixed tuned 0.1: 0.00; rule  $\Omega=1$ : 15.49 (finite 5/5, max w median 1e+04) | fixed+clip k=2 tuned 30: 21.60 | fixed+clip k=4 tuned 3: 20.41 | EQ-B k=2: 24.26 (rule - EQ-B -8.76; clip 0.011) | EQ-B k=4: 21.31 (rule - EQ-B -5.82; clip 0.011) | EQ-B k=8: 15.56 (rule - EQ-B -0.06; clip 0.010)
[led7] step 1.00; tuned lambda 3: 23.66; rule  $\Omega=1$ : 22.44 (-1.22) | EQ-B k=2: 22.66 (-1.00; clip 0.000) | EQ-B k=4: 22.44 (-1.22; clip 0.000) | EQ-B k=8: 22.44 (-1.22; clip 0.000)
  POISON: fixed tuned 0.1: 0.00; rule  $\Omega=1$ : 16.69 (finite 5/5, max w median 1e+04) | fixed+clip k=2 tuned 3: 25.16 | fixed+clip k=4 tuned 3: 20.97 | EQ-B k=2: 23.44 (rule - EQ-B -6.75; clip 0.009) | EQ-B k=4: 22.28 (rule - EQ-B -5.59; clip 0.008) | EQ-B k=8: 17.81 (rule - EQ-B -1.12; clip 0.009)
[mfeat_factors] step 4.54; tuned lambda 300: 54.45; rule  $\Omega=1$ : 58.35 (+3.90) | EQ-B k=2: 58.35 (+3.90; clip 0.000) | EQ-B k=4: 58.35 (+3.90; clip 0.000) | EQ-B k=8: 58.35 (+3.90; clip 0.000)
  POISON: fixed tuned 0.1: 13.50; rule  $\Omega=1$ : 19.75 (finite 5/5, max w median 1e+04) | fixed+clip k=2 tuned 100: 31.55 | fixed+clip k=4 tuned 100: 29.45 | EQ-B k=2: 39.75 (rule - EQ-B -20.00; clip 0.008) | EQ-B k=4: 30.05 (rule - EQ-B -10.30; clip 0.007) | EQ-B k=8: 25.15 (rule - EQ-B -5.40; clip 0.007)
[mfeat_morphological] step 2.76; tuned lambda 1: 27.65; rule  $\Omega=1$ : 37.90 (+10.25) | EQ-B k=2: 37.90 (+10.25; clip 0.000) | EQ-B k=4: 37.90 (+10.25; clip 0.000) | EQ-B k=8: 37.90 (+10.25; clip 0.000)
  POISON: fixed tuned 0.1: 11.10; rule  $\Omega=1$ : 17.90 (finite 5/5, max w median 1e+04) | fixed+clip k=2 tuned 1: 28.75 | fixed+clip k=4 tuned 1: 32.15 | EQ-B k=2: 38.65 (rule - EQ-B -20.75; clip 0.008) | EQ-B k=4: 37.80 (rule - EQ-B -19.90; clip 0.008) | EQ-B k=8: 32.95 (rule - EQ-B -15.05; clip 0.008)
```